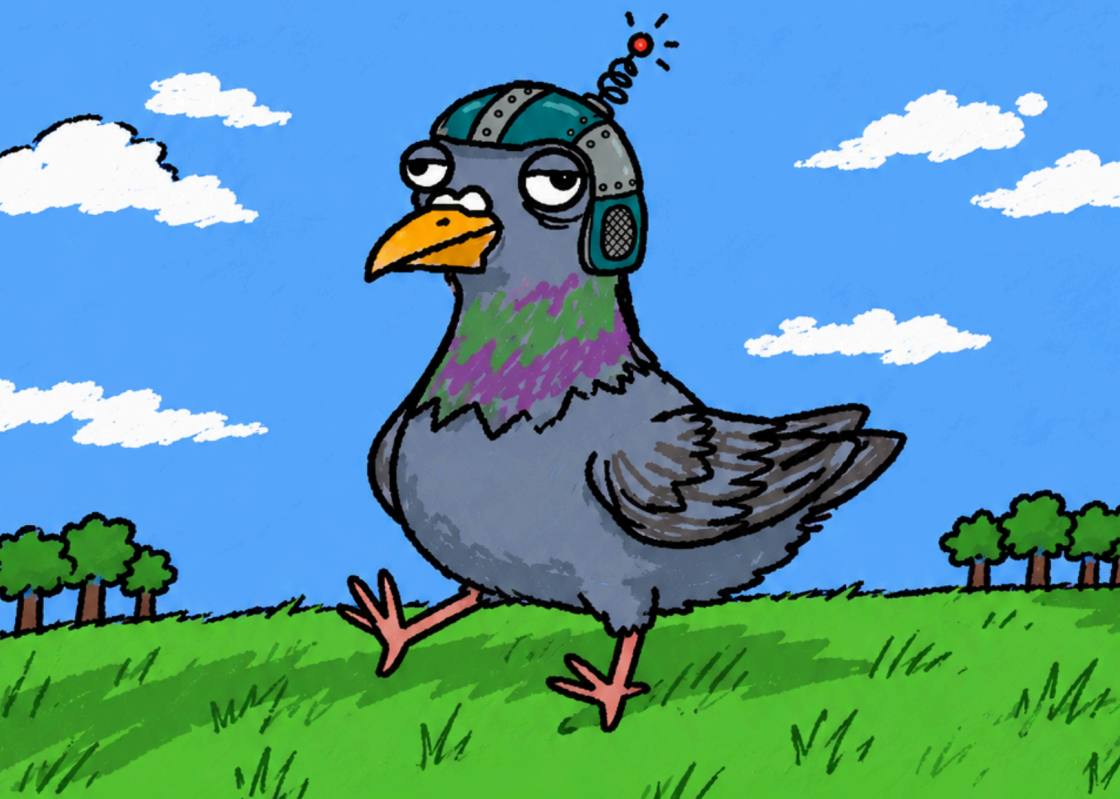


مبانی کامپیوتر و برنامه سازی

Issa Bagheryan



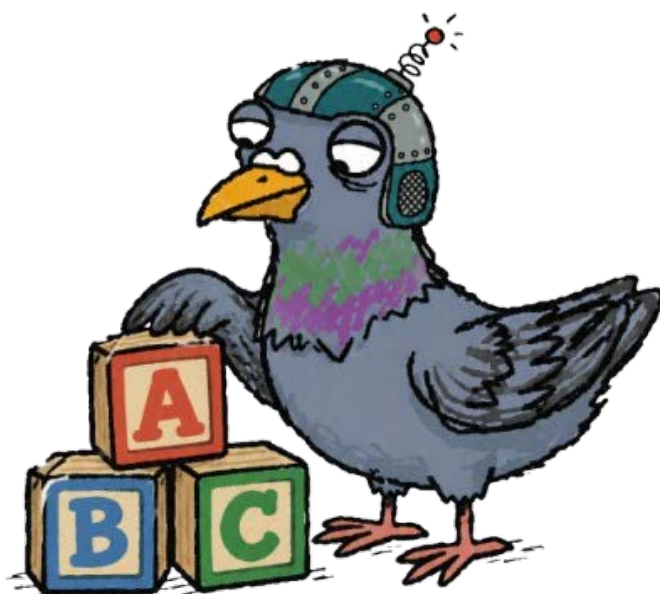
۱	۱. آشنایی با کامپیوتر
۴۰	۲. سیستم های عددی
۷۲	۳. الگوریتم و فلوچارت
۱۱۵	۴. عملگرها
۱۶۵	۵. شرط
۱۹۹	۶. حلقه و تکرار
۲۳۸	۷. حل مسئله و تفکر الگوریتمی
۲۸۲	۸. ساختمان داده ها
۳۲۲	۹. مرتب سازی و جستجو
۳۸۳	۱۰. توابع
۴۳۴	۱۱. زبان C

<https://github.com/issa-bagheryan>

با تشکر از اسپانسر این جزوه، تیم گفتارگاه

آشنایی با کامپیوتر

فصل اول



۱. مقدمه

امروزه کامپیوترها تقریباً در همه‌ی حوزه‌های زندگی و علم حضور دارند. از انجام محاسبات ساده و ذخیره اطلاعات گرفته تا طراحی هواپیما، تحلیل داده‌های پزشکی، شبیه‌سازی پدیده‌های علمی، اجرای موتورهای جست‌وجو و توسعه‌ی سامانه‌های هوش مصنوعی، بخش بزرگی از فعالیت‌های مدرن به نوعی به سیستم‌های کامپیوتری وابسته است. یک تصور رایج این است که کامپیوتر دستگاهی «هوشمند» است که خودش می‌داند چه کاری باید انجام دهد. در واقع چنین نیست. کامپیوتر به‌طور ذاتی نمی‌داند کاربر چه می‌خواهد. آنچه کامپیوتر انجام می‌دهد، اجرای مجموعه‌ای از دستورهای دقیق و مشخص است. برای مثال، اگر از کامپیوتر بخواهیم میانگین سه عدد را محاسبه کند، کامپیوتر به‌خودی‌خود مفهوم «میانگین» را مانند یک انسان درک نمی‌کند. ما باید دستورالعمل لازم را در قالب یک الگوریتم و سپس یک برنامه در اختیار آن قرار دهیم:

۱. عدد اول را دریافت کن.

۲. عدد دوم را دریافت کن.

۳. عدد سوم را دریافت کن.

۴. سه عدد را با هم جمع کن.

۵. حاصل را بر ۳ تقسیم کن.

۶. نتیجه را نمایش بده.

بنابراین می‌توانیم بگوییم:

کامپیوتر یک سیستم قابل‌برنامه‌ریزی است که داده‌ها را دریافت می‌کند، آن‌ها را طبق دستورالعمل‌های مشخص پردازش می‌کند و نتیجه تولید می‌کند.

این توانایی ساده، در مقیاس‌های بسیار بزرگ می‌تواند به سیستم‌هایی بسیار پیچیده منجر

شود.

نمونه‌هایی از نقش کامپیوتر

گسترش استفاده از کامپیوترها در چند دهه‌ی اخیر باعث شکل‌گیری فناوری‌ها و دستاوردهای بزرگی شده است. برای مثال:

- **اینترنت:** امکان ارتباط میان سیستم‌های کامپیوتری و دسترسی به اطلاعات و خدمات موجود در نقاط مختلف جهان را فراهم می‌کند.
 - **شبکه‌های کامپیوتری:** چندین کامپیوتر را به یکدیگر متصل می‌کنند تا بتوانند داده و منابع را به اشتراک بگذارند.
 - **هوش مصنوعی (AI):** حوزه‌ای از علوم کامپیوتر که به ساخت سیستم‌هایی می‌پردازد که می‌توانند وظایفی را انجام دهند که معمولاً به هوش انسانی نیاز دارند.
 - **مدل‌های زبان بزرگ (LLM):** مدل‌های هوش مصنوعی بزرگی که با آموزش روی حجم عظیمی از داده‌های متنی، توانایی درک و تولید زبان طبیعی را به دست می‌آورند.
 - **پروژه‌ی ژنوم انسان:** نمونه‌ای از پروژه‌های علمی بزرگی است که پردازش و مدیریت حجم عظیمی از داده‌های زیستی در آن نقش مهمی داشته است.
- نکته‌ی مهم این است که قدرت کامپیوتر فقط به سرعت آن محدود نمی‌شود. توانایی آن در ذخیره‌سازی، پردازش و انتقال حجم بسیار زیادی از داده‌ها نیز بخش مهمی از اهمیت آن است.

۲. تاریخچه و نسل‌های مختلف کامپیوتر

کامپیوترهای امروزی حاصل چندین دهه پیشرفت در مهندسی برق، الکترونیک و علوم کامپیوتر هستند. نخستین کامپیوترهای الکترونیکی با سیستم‌های امروزی تفاوت بسیار زیادی داشتند. آن‌ها بسیار بزرگ، پرمصرف و گران بودند و توان پردازشی آن‌ها نیز در مقایسه با کامپیوترهای امروزی بسیار محدود بود.

برای توضیح روند تاریخی این پیشرفت، معمولاً کامپیوترها را به چند نسل تقسیم می‌کنند. هر نسل با یک فناوری مهم در ساخت یا طراحی کامپیوترها شناخته می‌شود.

نسل	دوره‌ی تقریبی	فناوری کلیدی	ویژگی‌های اصلی
نسل اول	دهه‌ی ۱۹۴۰ و اوایل ۱۹۵۰	لامپ خلأ	ابعاد بسیار بزرگ، مصرف انرژی زیاد و تولید حرارت بالا
نسل دوم	اواخر دهه‌ی ۱۹۵۰ و دهه‌ی ۱۹۶۰	ترانزیستور	کاهش اندازه و مصرف انرژی و افزایش قابلیت اطمینان
نسل سوم	دهه‌ی ۱۹۶۰ به بعد	مدار مجتمع (IC)	قرارگیری چندین قطعه‌ی الکترونیکی روی یک تراشه
نسل چهارم	از دهه‌ی ۱۹۷۰	مدارهای مجتمع با تراکم بسیار بالا و ریزپردازنده	ظهور کامپیوترهای شخصی و گسترش استفاده‌ی عمومی
نسل پنجم	اصطلاحی برای نسل‌های نوین	هوش مصنوعی و پردازش هوشمند	تلاش برای ساخت سیستم‌هایی با توانایی‌های پیشرفته‌تر در پردازش و استنتاج

۳. سیستم کامپیوتری و مفاهیم اولیه

حالا که با روند کلی تکامل کامپیوترها آشنا شدیم، باید ببینیم یک سیستم کامپیوتری دقیقاً چگونه کار می‌کند. یک سیستم کامپیوتری را می‌توان مجموعه‌ای از سخت‌افزار و نرم‌افزار دانست که برای دریافت داده، پردازش آن و تولید نتیجه با یکدیگر همکاری می‌کنند. یک مدل ساده از عملکرد کامپیوتر چنین است:



این مدل یکی از مهم‌ترین مدل‌های پایه در علوم کامپیوتر است:

Input → Processing → Output

یعنی:

۱. چیزی وارد سیستم می‌شود.
 ۲. سیستم روی آن عملیاتی انجام می‌دهد.
 ۳. نتیجه تولید می‌شود.
- برای مثال، فرض کنید می‌خواهیم مجموع دو عدد را محاسبه کنیم.

- ورودی: دو عدد ۱۰ و ۲۰
- پردازش: جمع کردن دو عدد
- خروجی: عدد ۳۰

البته سیستم‌های واقعی بسیار پیچیده‌تر از این مثال هستند و ممکن است در کنار ورودی، پردازش و خروجی، داده‌ها را ذخیره و دوباره بازیابی کنند.

۳.۱. داده (Data)

داده به مقادیر خامی گفته می‌شود که سیستم می‌تواند آن‌ها را دریافت، ذخیره یا پردازش کند. داده می‌تواند شکل‌های مختلفی مانند عدد، حرف، متن، تصویر، صدا، ویدئو یا اطلاعات حسگرها داشته باشند. برای مثال، عدد 25 به تنهایی می‌تواند فقط یک داده باشد. اگر بدانیم این عدد «دمای هوا بر حسب درجه‌ی سانتی‌گراد» است، معنای مشخص‌تری پیدا می‌کند.

۳.۲. پردازش (Processing)

پردازش به عملیات‌هایی گفته می‌شود که روی داده‌ها انجام می‌شوند. این عملیات می‌توانند بسیار ساده باشند؛ مثلاً:

$$20 + 10$$

یا بسیار پیچیده باشند؛ مثلاً پردازش میلیون‌ها تصویر برای آموزش یک مدل یادگیری ماشین. پردازش می‌تواند شامل مواردی مانند محاسبه، مقایسه، مرتب‌سازی، جست‌وجو، تبدیل، فیلتر کردن یا تحلیل داده باشد.

۳.۳. اطلاعات (Information)

وقتی داده‌ها به شکلی پردازش شوند که **معنا و کاربرد مشخصی پیدا کنند**، می‌توان به آنها اطلاعات گفت. برای مثال:

18 , 20 , 22 , 24

اگر این اعداد دمای چهار روز متوالی باشند، محاسبه‌ی میانگین آن‌ها می‌تواند اطلاعات مفیدی تولید کند:

میانگین دما = 21 درجه

بنابراین می‌توان گفت:

داده، ماده‌ی خام است و پردازش روی داده می‌تواند اطلاعات معنادار تولید کند.

البته در مباحث پیشرفته‌تر علوم اطلاعات و داده، مرز دقیق میان «داده» و «اطلاعات» می‌تواند پیچیده‌تر باشد؛ اما این تعریف برای درک مبانی کامپیوتر مناسب است.

۴. الگوریتم، برنامه و زبان برنامه‌سازی

سه اصطلاح الگوریتم، برنامه و زبان برنامه‌سازی از مهم‌ترین مفاهیم پایه‌ی برنامه‌نویسی هستند و نباید آن‌ها را با یکدیگر اشتباه گرفت.

۴.۱. الگوریتم چیست؟

الگوریتم (Algorithm) مجموعه‌ای از دستورالعمل‌های دقیق و مرحله‌به‌مرحله برای حل یک مسئله یا انجام یک کار است. الگوریتم الزاماً با زبان برنامه‌نویسی نوشته نمی‌شود. برای مثال، فرض کنید می‌خواهیم دو عدد را پیدا کنیم. می‌توان الگوریتم را به فارسی بنویسیم:

۱. عدد اول را دریافت کن.

۲. عدد دوم را دریافت کن.

۳. اگر عدد اول بزرگ‌تر از عدد دوم بود، عدد اول را نمایش بده.

۴. در غیر این صورت، عدد دوم را نمایش بده.

۵. پایان.

این یک الگوریتم است، حتی اگر هیچ خط کدی در آن وجود نداشته باشد.

ویژگی مهم الگوریتم: پایان‌پذیری

یک الگوریتم باید برای رسیدن به نتیجه، بالاخره متوقف شود!

برای مثال:

1. عدد را دریافت کن.
2. عدد را دو برابر کن.
3. نتیجه را نمایش بده.
4. پایان.

پایان مشخصی دارد.

اما دستور زیر یک الگوریتم کامل برای حل مسئله محسوب نمی‌شود:

1. اعداد را بررسی کن.
2. بهترین عدد را پیدا کن.
3. دوباره بررسی کن.
4. دوباره بررسی کن.

زیرا مشخص نیست چه زمانی کار تمام می‌شود.

۴.۲. الگوریتم فقط برای کامپیوتر نیست

یکی از سوءبرداشت‌های رایج این است که الگوریتم را چیزی مخصوص کامپیوتر بدانیم. در حقیقت، مفهوم الگوریتم بسیار عمومی‌تر است. برای مثال، دستور پخت یک غذا نیز می‌تواند نوعی دستورالعمل مرحله‌ای باشد:

۱. مواد را آماده کن.
۲. پیاز را خرد کن.
۳. روغن را داغ کن.

۴. پیاز را سرخ کن.

۵. مواد بعدی را اضافه کن.

۶. غذا را در بشقاب بریز.

۷. پایان.

تفاوت اصلی این است که در برنامه‌نویسی، دستورالعمل‌ها باید برای کامپیوتر به‌اندازه‌ی کافی دقیق و قابل‌اجرا باشند. ما این پیش‌فرض را داریم که برای پخت غذا نیاز به روشن کردن اجاق داریم، اما برای کامپیوتر چنین مسئله‌ای مطرح نیست.

۴.۳. برنامه‌ی کامپیوتری چیست؟

برنامه (Program) پیاده‌سازی یک الگوریتم با استفاده از یک زبان برنامه‌سازی است.

برای مثال، الگوریتم قبلی می‌گفت:

دو عدد را بگیر و مجموع آن‌ها را نمایش بده.

همین الگوریتم را می‌توان با زبان C به شکل زیر پیاده‌سازی کرد:

۴.۴. زبان برنامه‌سازی چیست؟

زبان برنامه‌سازی (Programming Language) مجموعه‌ای از قواعد و دستورهای است که به برنامه‌نویس اجازه می‌دهد الگوریتم‌ها را به شکلی قابل‌اجرا برای کامپیوتر بیان کند. نمونه‌هایی از زبان‌های برنامه‌سازی عبارت‌اند از C, C++, JavaScript, Python و... هر زبان قواعد خودش را دارد. برای مثال، در زبان C می‌توان نوشت:

```
int x = 10;
```

اما نمی‌توان دستورها را کاملاً دلخواه نوشت؛ زبان دارای **نحو (Syntax)** و قواعد مشخصی است.

به همین دلیل رابطه‌ی این سه مفهوم را می‌توان چنین خلاصه کرد:

۴.۵. زبان‌های سطح پایین و سطح بالا

یکی از نکات مهم این فصل، تفاوت میان زبان‌های سطح پایین و سطح بالا است.

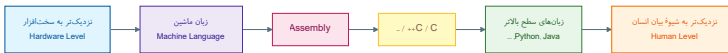
زبان سطح پایین

به سخت‌افزار نزدیک‌تر است. مثل Assembly و Machine Language

زبان سطح بالا

جزئیات بیشتری از سخت‌افزار را از برنامه‌نویس پنهان می‌کند و نوشتن برنامه را ساده‌تر می‌سازد. مثل C و Python.

در یک نگاه ساده:



البته «سطح بالا» به معنای «کند» و «سطح پایین» به معنای «سریع» نیست. سرعت واقعی یک برنامه به عوامل بسیار بیشتری مانند زبان، کامپایلر، الگوریتم، سخت‌افزار و نحوه‌ی اجرای برنامه بستگی دارد.

مزیت اصلی زبان‌های سطح پایین، کنترل بیشتر و نزدیکی بیشتر به سخت‌افزار است؛ در مقابل، زبان‌های سطح بالا معمولاً بهره‌وری برنامه‌نویس و خوانایی بیشتری فراهم می‌کنند.

۵. اجزای کامپیوتر: سخت افزار و نرم افزار

اکنون که درباره‌ی برنامه و زبان برنامه‌سازی صحبت کردیم، باید ببینیم برنامه در چه چیزی اجرا می‌شود.

هر سیستم کامپیوتری را می‌توان از دو بخش کلی بررسی کرد:

۱. سخت افزار (Hardware)

۲. نرم افزار (Software)

۵.۱. سخت افزار

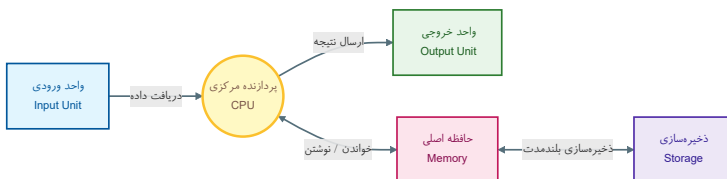
سخت افزار به اجزای فیزیکی کامپیوتر گفته می‌شود؛ یعنی چیزهایی که از نظر فیزیکی وجود دارند و می‌توان آن‌ها را لمس کرد. مثل: CPU، RAM، SSD، مانیتور، ماوس و کیبورد.

۵.۲. نرم افزار

نرم افزار مجموعه‌ای از برنامه‌ها و دستورالعمل‌هایی است که به سخت افزار می‌گویند چه کاری انجام دهد. مثل: سیستم عامل، مرورگر وب و بازی. بدون نرم افزار، سخت افزار یک سیستم قابل استفاده‌ی عمومی نخواهد بود؛ و نرم افزار نیز برای اجرا به سخت افزار نیاز دارد.

۵.۳. معماری کلی سخت‌افزار

برای درک نحوه‌ی کار یک کامپیوتر، می‌توان اجزای اصلی آن را به چند بخش تقسیم کرد:



اجزای اصلی عبارت‌اند از:

- واحد ورودی
- واحد خروجی
- واحد پردازش مرکزی
- حافظه
- ذخیره‌سازی

۵.۴. واحد ورودی (Input Unit)

واحد ورودی وظیفه دارد داده‌ها و دستورهای موردنیاز سیستم را از محیط بیرون دریافت کند. نمونه‌های آن دوربین، اسکنر، صفحه‌نمایش لمسی، ماوس و کیبورد و میکروفون هستند. مثلاً وقتی کلید **A** را روی صفحه‌کلید فشار می‌دهید، یک رویداد ورودی ایجاد می‌شود و سیستم آن را دریافت و پردازش می‌کند.

ورودی فقط به معنی «نوشتن متن» نیست. یک تصویر دوربین، صدای میکروفون یا اطلاعات یک حسگر نیز می‌تواند ورودی سیستم باشد.

۵.۵. واحد خروجی (Output Unit)

واحد خروجی وظیفه دارد نتیجه‌ی پردازش را به شکلی قابل استفاده برای کاربر یا یک سیستم دیگر ارائه کند. نمونه‌های آن مانیتور، پرینتر، هدفون و اسپیکر و پلاتر هستند. برای مثال، وقتی یک برنامه عدد 42 را روی صفحه نمایش می‌دهد، داده‌ای که در داخل کامپیوتر پردازش شده، از طریق سیستم خروجی به شکل قابل مشاهده برای کاربر ارائه می‌شود.

۵.۶. واحد پردازش مرکزی (CPU)

CPU (Central Processing Unit) یا واحد پردازش مرکزی، یکی از مهم‌ترین اجزای سیستم کامپیوتری است. CPU دستورهای برنامه را اجرا می‌کند و عملیات مختلف را روی داده‌ها انجام می‌دهد. می‌توان CPU را محل اصلی اجرای دستورهای برنامه دانست. CPU از بخش‌های مختلفی تشکیل شده است که سه مورد مهم عبارت‌اند از:

- ALU
- CU
- Registers

۵.۶.۱. واحد محاسبه و منطق (ALU)

ALU (Arithmetic Logic Unit) واحد محاسبه و منطق است. این بخش عملیات ریاضی و منطقی مختلفی را انجام می‌دهد. عملیات ریاضی می‌توانند شامل مواردی مانند:

جمع، تفریق، ضرب

و عملیات منطقی می‌توانند شامل مواردی مانند:

مقایسه، بررسی برابر بودن، بزرگتر یا کوچکتر بودن

باشند.

مثلاً اگر برنامه بخواهد بررسی کند:

$$5 < 10$$

یکی از بخش‌های پردازنده باید این مقایسه را انجام دهد.

۵.۶.۲. واحد کنترل (CU)

CU (Control Unit) مسئول هماهنگ کردن عملیات مختلف پردازنده است. می‌توان آن را مانند بخشی در نظر گرفت که تعیین می‌کند کدام عملیات، در چه زمانی و روی چه داده‌ای انجام شود. برای مثال، اجرای یک دستور ممکن است به صورت ساده شامل این مراحل باشد:

۱. دستور از حافظه دریافت شود.
۲. دستور تفسیر شود.
۳. داده‌ی موردنیاز آماده شود.
۴. عملیات موردنظر انجام شود.
۵. نتیجه ذخیره شود.

واحد کنترل در هماهنگ کردن این مراحل نقش مهمی دارد.

۵.۶.۳. ثبات‌ها (Registers)

Registerها حافظه‌های بسیار کوچک و بسیار سریع داخل پردازنده هستند. CPU برای انجام عملیات مختلف به مکان‌هایی نیاز دارد که بتواند داده‌ها و نتایج موقت را در آن‌ها نگه دارد. برای مثال، هنگام انجام یک محاسبه، برخی مقادیر ممکن است برای مدت بسیار کوتاهی در رجیستر CPU قرار بگیرند. رجیستر معمولاً بسیار سریع‌تر از حافظه‌ی اصلی هستند، اما تعداد و ظرفیت آن‌ها بسیار محدود است.

۵.۶.۴. سرعت پردازنده و هرتز

یکی از واحدهایی که در ارتباط با پردازنده می‌بینیم **هرتز (Hz)** است. یک هرتز یعنی یک چرخه در ثانیه. واحدهای رایج‌تر عبارت‌اند از:

MHz = مگاهرتز = میلیون هرتز

GHz = گیگاهرتز = میلیارد هرتز

برای مثال:

$$3 \text{ GHz} = 3,000,000,000 \text{ Hz}$$

فرکانس CPU به‌تنهایی معیار کاملی برای مقایسه‌ی عملکرد دو پردازنده نیست.

دو پردازنده با فرکانس یکسان ممکن است به دلیل تفاوت در معماری، تعداد هسته‌ها، حافظه‌ی Cache، طراحی داخلی و سایر عوامل، عملکرد متفاوتی داشته باشند. بنابراین نباید تصور کنیم که یک پردازنده ۴ گیگاهرتزی همیشه از یک پردازنده ۳ گیگاهرتزی بهتر است.

۵.۷. حافظه‌ی نهان (Cache)

بین CPU و حافظه‌ی اصلی، مفهومی به نام **Cache** اهمیت زیادی دارد. **Cache** حافظه‌ای با ظرفیت نسبتاً کم اما سرعت بسیار بالا است که برای کاهش زمان دسترسی CPU به داده‌های موردنیاز استفاده می‌شود. فرض کنید CPU دائماً به یک داده‌ی خاص نیاز دارد. اگر مجبور باشد هر بار آن را از حافظه‌ی اصلی دریافت کند، زمان بیشتری صرف خواهد شد. اگر داده‌ی موردنظر در **Cache** موجود باشد، CPU می‌تواند آن را سریع‌تر دریافت کند.

به این ایده می‌توان به شکل ساده نگاه کرد:



هرچه به CPU نزدیک‌تر می‌شویم، معمولاً ظرفیت کمتر و سرعت بیشتر می‌شود. در کامپیوترهای قدیمی‌تر، حافظه‌ی **Cache** معمولاً یک قطعه‌ی جدا از CPU بود و به صورت خارجی در کنار پردازنده قرار می‌گرفت. با پیشرفت فناوری، برای افزایش سرعت دسترسی به داده‌ها و کاهش تأخیر، حافظه‌ی کش به داخل خود CPU منتقل شد. امروزه بیشتر پردازنده‌ها دارای چند سطح حافظه‌ی کش داخلی مانند **L۱**، **L۲** و **L۳** هستند که مستقیماً در نزدیکی هسته‌های پردازشی قرار دارند.

۵.۸. حافظه (Memory) و انواع آن

کامپیوتر عموماً نیاز دارد داده‌ها و دستورالعمل‌ها را نگهداری کند. حافظه‌ها را می‌توان به شکل‌های مختلفی دسته‌بندی کرد. در اینجا دو دسته‌ی مهم را بررسی می‌کنیم:

۱. حافظه‌ی اصلی (Main Memory)

۲. حافظه‌ی جانبی یا ذخیره‌سازی ثانویه (Secondary Storage)

تفاوت اساسی آن‌ها را می‌توان این‌گونه خلاصه کرد:

ویژگی	حافظه اصلی	حافظه جانبی
سرعت	بیشتر	معمولاً کمتر
ظرفیت	محدودتر	معمولاً بیشتر
کاربرد	کار جاری سیستم	نگهداری بلندمدت داده
نمونه	RAM	SSD, HDD

۵.۸.۱. حافظه‌ی اصلی (Main Memory)

حافظه‌ی اصلی حافظه‌ای است که سیستم برای نگهداری داده‌ها و دستورالعمل‌هایی که در حال استفاده هستند، از آن استفاده می‌کند. %۹۰ یکی از مهم‌ترین انواع حافظه‌ی اصلی است.

۵.۸.۲. RAM

RAM (Random Access Memory) حافظه‌ای است که سیستم‌عامل و

برنامه‌های در حال اجرا از آن برای نگهداری داده‌ها و دستورالعمل‌های موردنیاز استفاده

می‌کنند. برای مثال، وقتی یک برنامه را باز می‌کنید، بخش‌هایی از داده‌ها و کد مورد نیاز آن باید در حافظه‌ی مناسب قرار بگیرند تا پردازنده بتواند با آن‌ها کار کند. فرض کنید یک برنامه‌ی سنگین را اجرا کرده‌اید و چند برنامه‌ی دیگر نیز هم‌زمان باز هستند. هرچه داده‌های مورد نیاز برنامه‌ها بیشتر شود، سیستم به حافظه‌ی بیشتری نیاز دارد. اگر RAM کافی نباشد، سیستم ممکن است مجبور شود از روش‌های دیگری برای مدیریت داده‌های فعال استفاده کند که معمولاً عملکرد را کاهش می‌دهد.

RAM معمولاً یک حافظه‌ی **فرّار (Volatile)** است. یعنی داده‌های موجود در آن با قطع برق پایدار نمی‌مانند. به همین دلیل، وقتی کامپیوتر را خاموش می‌کنید، اطلاعات موقتی موجود در RAM از بین می‌روند. مثلاً اگر فایلی را در Microsoft Word باز کرده باشید ولی هنوز آن را ذخیره نکرده باشید، اطلاعات آن ممکن است در حافظه‌ی موقت قرار داشته باشد و با خاموش شدن سیستم از دست برود.

عبارت **Random Access** به این معنی است که سیستم می‌تواند به مکان‌های مختلف حافظه با دسترسی مستقیم دسترسی پیدا کند و لازم نیست برای رسیدن به یک خانه، همه‌ی خانه‌های قبلی را به ترتیب طی کند. این مفهوم با «تصادفی بودن داده‌ها» اشتباه نشود.

۵.۸.۳. ROM

ROM (Read Only Memory) نوعی حافظه‌ی غیرفرّار است؛ یعنی اطلاعات آن بدون برق نیز باقی می‌ماند. ROM و فناوری‌های خانواده‌ی آن در سیستم‌های کامپیوتری برای نگهداری اطلاعاتی که باید پس از خاموش شدن دستگاه نیز باقی بمانند استفاده شده‌اند. در سیستم‌های مدرن، چیزی که در گفتار عمومی «ROM» نامیده می‌شود همیشه به معنای یک حافظه‌ی کاملاً غیرقابل تغییر نیست. انواع مختلفی از حافظه‌های غیرفرّار قابل برنامه‌ریزی یا بازنویسی وجود دارند.

۵.۸.۴. حافظه‌ی جانبی و ذخیره‌سازی ثانویه

در کنار RAM، سیستم به نوعی حافظه نیاز دارد که اطلاعات را برای مدت طولانی نگهداری کند. مثل HDD, SSD, Flash Drive, Memory Card, CD, DVD. این تجهیزات را معمولاً **Storage** یا ذخیره‌سازی می‌نامیم. مثلاً وقتی یک فایل Word را ذخیره می‌کنید، انتظار دارید پس از خاموش و روشن کردن کامپیوتر همچنان وجود داشته باشد. بنابراین فایل باید روی یک رسانه‌ی ذخیره‌سازی غیرفرا قرار بگیرد، نه صرفاً در RAM.

حالا می‌دانیم وقتی که برنامه‌ی Word را باز می‌کنیم، فایل از فضای ذخیره‌سازی خوانده می‌شود، بخش‌های موردنیاز آن در RAM قرار می‌گیرند و CPU دستورهای برنامه را اجرا می‌کند.

۶. واحدهای اندازه گیری داده

حالا با مفاهیم مربوط به حافظه آشنا شده ایم، اما باید بدانیم که داده ها چگونه در حافظه ذخیره و نمایش داده می شوند. کامپیوترها داده ها را در سطح بنیادی با بیت ها نمایش می دهند.

۶.۱. بیت (Bit)

Bit کوتاه شده ی Binary Digit است. هر بیت می تواند یکی از دو مقدار 0 و 1 را داشته باشد. دلیل استفاده از بیت این است که کامپیوترها در سطح سخت افزار با دو حالت ساده کار می کنند؛ مثل روشن و خاموش بودن یک کلید الکتریکی. بنابراین هر بیت می تواند فقط یکی از دو مقدار 0 یا 1 را نشان دهد. با کنار هم قرار گرفتن تعداد زیادی بیت، کامپیوتر می تواند اطلاعات پیچیده تر مانند اعداد، متن، تصویر و صدا را ذخیره و پردازش کند. این دو مقدار با مفهوم دودویی ارتباط دارند. بیت کوچک ترین واحد بنیادی اطلاعات دیجیتال در این مدل است.

۶.۲. نیبل (Nibble)

هر ۴ بیت یک نیبل را تشکیل می دهد:

1010

بنابراین:

$\text{Nibble} = 4 \text{ Bits}$

۶.۳. بایت (Byte)

هر ۸ بیت برابر با یک بایت است:

$$\text{Byte} = 8 \text{ Bits}$$

برای مثال:

10110101

یک بایت است. بایت یکی از مهم‌ترین واحدهای اندازه‌گیری داده و ظرفیت ذخیره‌سازی است.

۶.۴. واحدهای بزرگ‌تر ذخیره‌سازی

حتما میدانید که برای بیان ظرفیت حافظه و ذخیره‌سازی، از واحدهایی مانند KB، MB، GB و TB استفاده می‌کنیم. در محاسبات دودویی سنتی:

$$\text{KB} = 1024 \text{ Bytes}$$

$$\text{MB} = 1024 \text{ KB}$$

$$\text{GB} = 1024 \text{ MB}$$

$$\text{TB} = 1024 \text{ GB}$$

به صورت توان‌های ۲:

$$2^{10} = 1024$$

بنابراین:

$$\text{KB} = 2^{10} \text{ Bytes}$$

$$\text{MB} = 2^{20} \text{ Bytes}$$

$$\text{GB} = 2^{30} \text{ Bytes}$$

$$\text{TB} = 2^{40} \text{ Bytes}$$

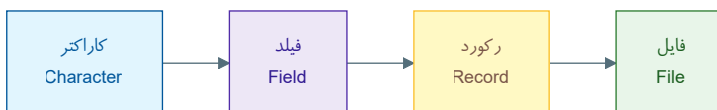
واحدهای بزرگ‌تر نیز عبارت‌اند از:

واحد	نماد	رابطه‌ی دودویی سنتی
کیلوبایت	KB	1024 B
مگابایت	MB	1024 KB
گیگابایت	GB	1024 MB
ترابایت	TB	1024 GB
پتابایت	PB	1024 TB
اگزابایت	EB	1024 PB
زتابایت	ZB	1024 EB
یوتابایت	YB	1024 ZB

۷. ساختار داده‌ها در حافظه و سازمان‌دهی اطلاعات

تا اینجا با بیت و بایت آشنا شدیم. اما اگر بخواهیم یک سیستم اطلاعاتی بزرگ را مدیریت کنیم، نمی‌توانیم فقط با مجموعه‌ای عظیم از صفر و یک‌ها کار کنیم. برای انسان و نرم‌افزار، داده‌ها معمولاً در ساختارهای منطقی سازمان‌دهی می‌شوند.

یک مدل ساده‌ی سلسله‌مراتبی چنین است:



۷.۱. کاراکتر (Character)

Character می‌تواند یک حرف، رقم یا نماد باشد. برای مثال **A** , **@** , یا **7**. کامپیوتر برای ذخیره‌ی کاراکترها از **کدهای عددی** استفاده می‌کند. یعنی یک حرف مانند **A** در سطح ذخیره‌سازی به صورت خودِ مفهوم «حرف **A**» نگهداری نمی‌شود؛ بلکه با یک نمایش عددی/بیتی مشخص ذخیره می‌شود.

۷.۱.۲. ASCII

یکی از استانداردهای معروف برای نمایش کاراکترها **ASCII** است. ASCII کلاسیک از ۷ بیت برای نمایش ۱۲۸ کاراکتر استفاده می‌کند و مقادیر ۰ تا ۱۲۷ را شامل می‌شود. برای مثال:

A → 65, B → 66, C → 67

و برخی کاراکترهای دیگر نیز کدهای مخصوص خود را دارند.

۷.۲. فیلد (Field)

Field مجموعه‌ای از داده است که یک ویژگی مشخص را بیان می‌کند. فرض کنید اطلاعات یک دانشجو را باید ذخیره کنیم:

نام: علی

نام خانوادگی: احمدی

سن: 22

در اینجا:

مقدار	فیلد
علی	فیلد نام
احمدی	فیلد نام خانوادگی
۲۲	فیلد سن

پس هر فیلد یک نوع اطلاعات مشخص را نگهداری می‌کند.

۷.۳. رکورد (Record)

Record مجموعه‌ای از فیلدهای مرتبط است که در کنار هم اطلاعات یک موجودیت مشخص را تشکیل می‌دهند. مثلاً یک رکورد دانشجو می‌تواند شامل موارد زیر باشد:

شماره دانشجویی

نام

نام خانوادگی

رشته

سال تولد

در واقع:

رکورد دانشجو

— شماره دانشجویی

— نام

— نام خانوادگی

— رشته

— سال تولد

۷.۴. فایل (File)

File مجموعه‌ای از داده‌ها یا رکوردهای مرتبط است که به صورت یک واحد ذخیره می‌شود. برای مثال، اگر یک دانشگاه اطلاعات ۱۰ هزار دانشجو را ذخیره کند، می‌توان تصور کرد که مجموعه‌ای از رکوردهای دانشجویان در یک ساختار داده‌ای یا فایل ذخیره شده است.

البته در سیستم‌های مدرن، پایگاه‌های داده و سیستم‌های فایل بسیار پیچیده‌تر از این مدل ساده هستند، اما این سلسله‌مراتب برای درک اولیه‌ی سازمان‌دهی داده‌ها مفید است.

۸. نرم افزار (Software)

تا اینجا بیشتر درباره‌ی سخت افزار صحبت کردیم. اما سخت افزار به تنهایی نمی تواند نیازهای کاربر را برآورده کند. برای مثال، CPU، RAM و SSD خودشان نمی دانند که کاربر می خواهد یک فایل متنی برای پایان نامه اش ایجاد کند، موسیقی مورد علاقه اش را پخش کند یا بازی Minesweeper را اجرا کند! این وظیفه بر عهده‌ی **نرم افزار** است. نرم افزار را می توان به چند گروه کلی تقسیم کرد

۸.۱. نرم افزارهای کاربردی (Application Software)

نرم افزارهای کاربردی برای انجام کارهایی طراحی می شوند که مستقیماً با نیاز کاربر نهایی ارتباط دارند. مثل نرم افزار های ویرایش متن (Microsoft Word)، مرورگر های وب (Google Chrome)، نرم افزار های ویرایش تصویر (Adobe Photoshop)، بازی ها (Grand Theft Auto IV).

۸.۲. نرم افزارهای سیستمی (System Software)

نرم افزارهای سیستمی وظیفه دارند بستر لازم برای اجرای برنامه ها و مدیریت منابع سیستم را فراهم کنند. مهم ترین دسته های مورد نظر در این درس عبارت اند از:

۱. سیستم عامل (Operating System)

نرم افزاری است که بین کاربر و سخت افزار ارتباط برقرار می کند و منابع سیستم مانند پردازنده، حافظه و فایل ها را مدیریت می کند.

مثال: Windows, Linux, macOS, Android

۲. برنامه‌های کمکی (Utility Programs)

برنامه‌هایی هستند که برای نگهداری، مدیریت و بهینه‌سازی سیستم استفاده می‌شوند.

مثال: آنتی‌ویروس‌ها، برنامه‌های فشرده‌سازی مانند WinRAR، ابزار پاک‌سازی

دیسک (Disk Cleanup)

۳. مترجم‌ها (Language Translators)

برنامه‌هایی هستند که کد نوشته‌شده توسط برنامه‌نویس را به زبان قابل فهم برای

کامپیوتر تبدیل می‌کنند.

مثال: کامپایلر زبان C مانند GCC، مفسر Python، یا ماشین مجازی Java

(JVM)

۳.۲.۸. سیستم‌عامل (Operating System)

سیستم‌عامل (OS) یکی از مهم‌ترین نرم‌افزارهای یک کامپیوتر است. سیستم‌عامل بین

سخت‌افزار، برنامه‌ها و کاربر نقش واسطه دارد. برای مثال، یک برنامه‌ی کاربردی لازم نیست

برای ذخیره‌ی یک فایل، مستقیماً دستورهای سخت‌افزاری پیچیده‌ای به SSD ارسال کند.

برنامه می‌تواند از خدمات سیستم‌عامل استفاده کند و سیستم‌عامل عملیات لازم را مدیریت

کند. Microsoft Windows, MacOS, Android، و Linux از نمونه‌های

سیستم‌عامل‌های معروف هستند.

می‌توان این رابطه را چنین نشان داد:



سیستم عامل مسئولیت های بسیار زیادی دارد، از جمله:

- مدیریت پردازنده
- مدیریت حافظه
- مدیریت فایل ها
- مدیریت دستگاه های ورودی و خروجی
- مدیریت اجرای برنامه ها
- کنترل دسترسی به منابع

به همین دلیل سیستم عامل یکی از مهم ترین اجزای نرم افزاری سیستم است. تصور کنید سیستم عامل وجود نداشت و هر برنامه باید مستقیماً با سخت افزار ارتباط برقرار می کرد. در چنین شرایطی برنامه نویسی باید جزئیات بسیار زیادی درباره ی سخت افزار بداند و برنامه ی خود را برای کنترل مستقیم آن بنویسد. سیستم عامل این پیچیدگی را تا حد زیادی پنهان می کند. برای مثال، وقتی در یک برنامه فایلی را باز می کنید، معمولاً لازم نیست بدانید فایل دقیقاً در کدام سکتور یا سلول فیزیکی SSD قرار دارد. سیستم عامل است که این جزئیات را مدیریت می کند.

یک نکته ی کاربردی درباره ی سیستم عامل این است که **حذف یک برنامه یا پوشه ی مربوط به یک برنامه، الزاماً معادل حذف کامل آن برنامه نیست**. ممکن است فایل ها و تنظیمات برنامه در پوشه ها و رجیستری های مختلف سیستم عامل باقی بماند و Uninstall کردن یک برنامه آن را کامل از سیستم حذف نمی کند!

۸.۳. برنامه‌های کمکی (Utilities)

Utility ها برنامه‌هایی هستند که برای نگهداری، مدیریت، بررسی یا بهینه‌سازی سیستم استفاده می‌شوند.

نمونه‌ها می‌توانند شامل ابزارهایی برای:

- مدیریت دیسک
- پشتیبان‌گیری
- فشرده‌سازی فایل‌ها
- امنیت
- مدیریت فایل

باشند.

برای مثال، یک ابزار مدیریت دیسک می‌تواند اطلاعات مربوط به پارتیشن‌ها و فضای ذخیره‌سازی را نمایش دهد.

۸.۴. مترجم‌ها (Translators)

تا اینجا گفتیم که برنامه‌نویس معمولاً برنامه را با یک زبان سطح بالا می‌نویسد. اما پردازنده در نهایت باید دستورهای را اجرا کند که با معماری و دستورهای ماشین آن سازگار باشند. بنابراین میان کدی که انسان می‌نویسد و دستورهای که پردازنده اجرا می‌کند به یک فرایند تبدیل یا ترجمه نیاز داریم. دو مفهوم مهم در این زمینه عبارت‌اند از:

- Compiler
- Interpreter

۸.۴.۱. کامپایلر (Compiler)

Compiler برنامه‌ای است که کد یک زبان برنامه‌سازی را به شکل دیگری تبدیل می‌کند تا بتوان آن را اجرا کرد. در مدل کلاسیک، کامپایلر برنامه‌ی منبع را بررسی کرده و آن را به کد ماشین یا یک شکل میانی قابل اجرا تبدیل می‌کند.

برای مثال:



یکی از مزایای مدل کامپایل‌شده این است که پس از انجام فرایند ترجمه، نتیجه‌ی ترجمه‌شده می‌تواند برای اجرا مورد استفاده قرار گیرد. البته جزئیات واقعی بسیار پیچیده‌تر هستند. در بسیاری از سیستم‌ها مراحل مختلفی مانند، preprocessing، compilation، assembly و linking وجود دارد.

۸.۴.۲. مفسر (Interpreter)

در مدل سنتی **Interpreter**، برنامه هنگام اجرا توسط یک نرم‌افزار مفسر بررسی و اجرا می‌شود.



این مدل در زبان‌ها و محیط‌های مختلف شکل‌های متفاوتی دارد و نباید آن را صرفاً «خط اول را ترجمه کن، اجرا کن، سپس خط دوم را ترجمه کن» در نظر گرفت. برای مثال، برخی زبان‌های مدرن از ماشین مجازی، JIT compilation، bytecode یا ترکیبی از

روش‌های تفسیر و کامپایل استفاده می‌کنند. بنابراین تفاوت Interpreter و Compiler در دنیای واقعی کاملاً به این سادگی نیست که یکی «کل برنامه» و دیگری همیشه «خط‌به‌خط» کار کند؛ این توضیح بیشتر یک مدل مقدماتی برای درک تفاوت آن‌هاست.

۸.۴.۳. مقایسه‌ی Interpreter و Compiler

ویژگی	Compiler	Interpreter
ایده‌ی اصلی	تبدیل برنامه پیش از اجرا	پردازش و اجرای برنامه در محیط اجرا
خروجی ترجمه‌شده	معمولاً تولید می‌شود	لزوماً به شکل فایل اجرایی مستقل تولید نمی‌شود
تشخیص برخی خطاها	می‌تواند پیش از اجرا انجام شود	ممکن است برخی خطاها هنگام رسیدن به بخش مربوطه مشخص شوند
مدل اجرا	معمولاً برنامه‌ی ترجمه‌شده اجرا می‌شود	محیط اجرا در اجرای برنامه نقش دارد

در عمل، زبان‌ها و محیط‌های مدرن ممکن است از ترکیبی از این روش‌ها استفاده کنند.

۹. میان‌افزار (Firmware)

Firmware نوعی نرم‌افزار است که به‌صورت نزدیک و مستقیم با سخت‌افزار یک دستگاه ارتباط دارد و وظیفه‌ی کنترل یا راه‌اندازی آن را بر عهده می‌گیرد. می‌توان **Firmware** را در نقطه‌ای میان سخت‌افزار و نرم‌افزارهای سطح بالاتر تصور کرد. برای مثال، یک دستگاه الکترونیکی ممکن است **Firmware** مخصوص خود را داشته باشد که وظیفه‌ی کنترل اولیه‌ی سخت‌افزار را انجام می‌دهد.

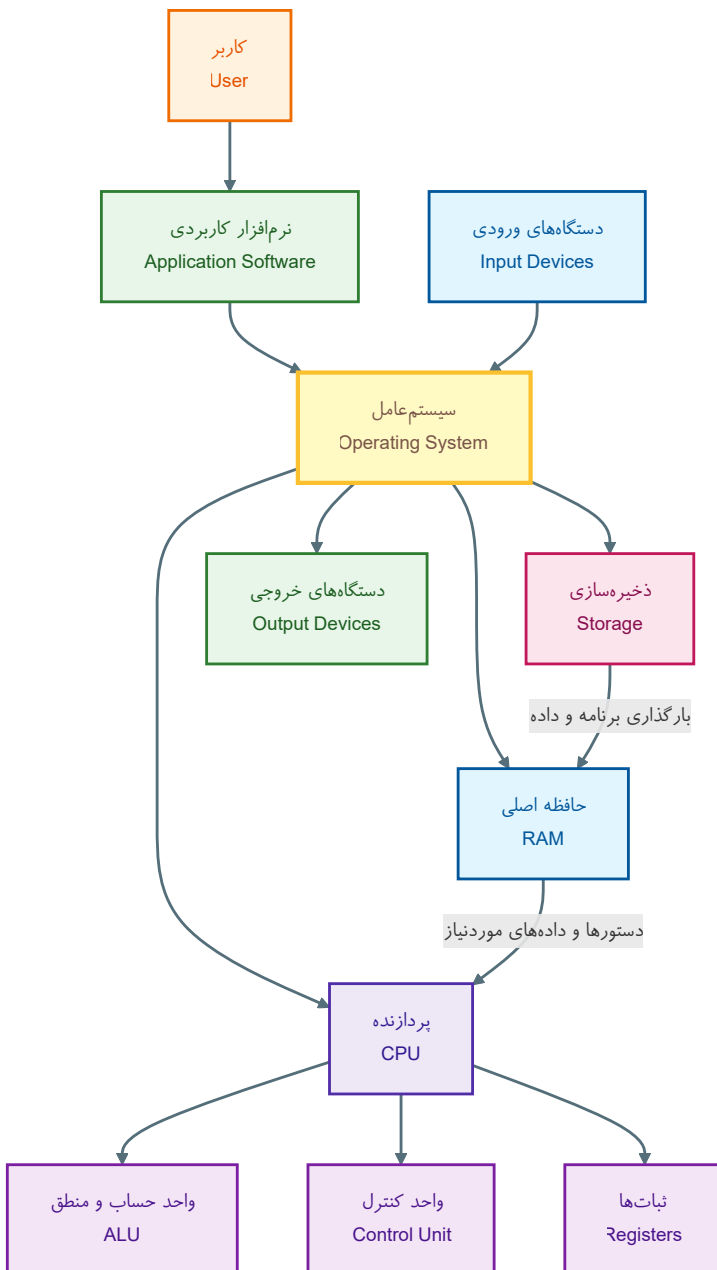
نمونه‌های آشنا:

- **Firmware** یک روتر
- **Firmware** یک SSD
- **Firmware** یک مادربرد
- **Firmware** یک دستگاه IoT

Firmware معمولاً در حافظه‌ی غیرقرّار ذخیره می‌شود تا پس از خاموش شدن دستگاه باقی بماند. در سیستم‌های مدرن، **Firmware** لزوماً در ROM سنتی ذخیره نمی‌شود و می‌تواند در حافظه‌هایی مانند Flash قرار داشته باشد.

۱۰. نگاه کلی به کل سیستم

اکنون می‌توانیم مفاهیم اصلی این فصل را در یک تصویر کلی کنار هم قرار دهیم:



این نمودار یک مدل ساده شده است، اما چند ارتباط مهم را نشان می دهد:

۱. کاربر معمولاً با نرم افزارهای کاربردی کار می کند.
۲. نرم افزار کاربردی برای استفاده از منابع سیستم به سیستم عامل متکی است.
۳. سیستم عامل منابعی مانند CPU، RAM و ذخیره سازی را مدیریت می کند.
۴. CPU دستورهای برنامه را اجرا می کند.
۵. RAM داده ها و دستورالعمل های مورد نیاز در حال اجرا را نگهداری می کند.
۶. Storage داده ها را برای نگهداری بلندمدت حفظ می کند.
۷. دستگاه های ورودی اطلاعات را وارد سیستم می کنند.
۸. دستگاه های خروجی نتایج را به کاربر یا محیط بیرون ارائه می دهند.

۱۰.۱. یک مثال کامل: اجرای یک برنامه

برای اینکه ارتباط مفاهیم این فصل روشن تر شود، فرض کنید برنامه ای داریم که دو عدد را از کاربر می گیرد و مجموع آن ها را نمایش می دهد.

برنامه از کاربر ورودی دریافت می کند:

عدد اول: 10

عدد دوم: 20

سپس عملیات جمع انجام می شود:

$$30 = 20 + 10$$

و در نهایت نتیجه نمایش داده می شود:

حالا این فرایند را از دید سیستم بررسی کنیم.

مرحله ی اول: ورودی

کاربر اعداد را با صفحه کلید وارد می کند. صفحه کلید یک **دستگاه ورودی** است.

مرحله ی دوم: برنامه

برنامه دستورهای دارد که مشخص می کنند:

۱. ورودی را دریافت کن.

۲. مقدارها را نگهداری کن.

۳. آن ها را جمع کن.

۴. نتیجه را نمایش بده.

این دستورها در یک **زبان برنامه سازی** نوشته شده اند.

مرحله ی سوم: اجرای برنامه

برای اجرای برنامه، کد باید به شکلی درآید که محیط اجرا و در نهایت پردازنده بتواند آن را اجرا کند.

مرحله ی چهارم: حافظه

داده ها و بخش های مورد نیاز برنامه در حافظه قرار می گیرند.

مرحله ی پنجم: CPU

CPU دستورها را اجرا می کند و عملیات محاسباتی لازم انجام می شود.

ALU در انجام عملیات محاسباتی نقش دارد و واحد کنترل اجرای مراحل مختلف را هماهنگ می کند.

نتیجه‌ی 30 از طریق سیستم خروجی، مثلاً مانیتور، به کاربر نمایش داده می‌شود.

پس حتی یک برنامه‌ی بسیار ساده نیز مجموعه‌ای از مفاهیم گفته شده این فصل را درگیر می‌کند!

۱۱. جمع‌بندی فصل

در این فصل با مفاهیم پایه‌ای آشنا شدیم که تقریباً تمام مباحث بعدی علوم کامپیوتر بر آن‌ها بنا می‌شوند. ابتدا دیدیم که کامپیوتر یک سیستم قابل برنامه‌ریزی است که داده را دریافت و پردازش می‌کند و نتیجه تولید می‌کند. سپس تاریخچه‌ی کلی نسل‌های کامپیوتر را بررسی کردیم و دیدیم که فناوری‌هایی مانند لامپ خلأ، ترانزیستور، مدار مجتمع و ریزپردازنده چگونه باعث کوچک‌تر، سریع‌تر و ارزان‌تر شدن سیستم‌های کامپیوتری شدند. بعد با مفاهیم بنیادی زیر آشنا شدیم:

- داده
- پردازش
- اطلاعات
- الگوریتم
- برنامه
- زبان برنامه‌سازی
- سخت‌افزار
- نرم‌افزار

سپس درباره‌ی تفاوت زبان‌های سطح پایین و سطح بالا صحبت کردیم و در بخش سخت‌افزار، اجزای مهم سیستم را بررسی کردیم (Input, CPU, Memory, Storage).

و همچنین رجیسترها آشنا CU و ALU با مفاهیم CPU در بخش مربوط به (Output).
و (RAM, ROM, Storage) شدیم در ادامه تفاوت میان انواع حافظه را مرور کردیم
همچنین واحدهای اندازه‌گیری داده را شناختیم. در نهایت، با ساختار ساده‌ی سازمان‌دهی
داده‌ها آشنا شدیم و بخش نرم‌افزار را با معرفی انواع آن (کاربردی، سیستم عامل) به پایان
رساندیم.

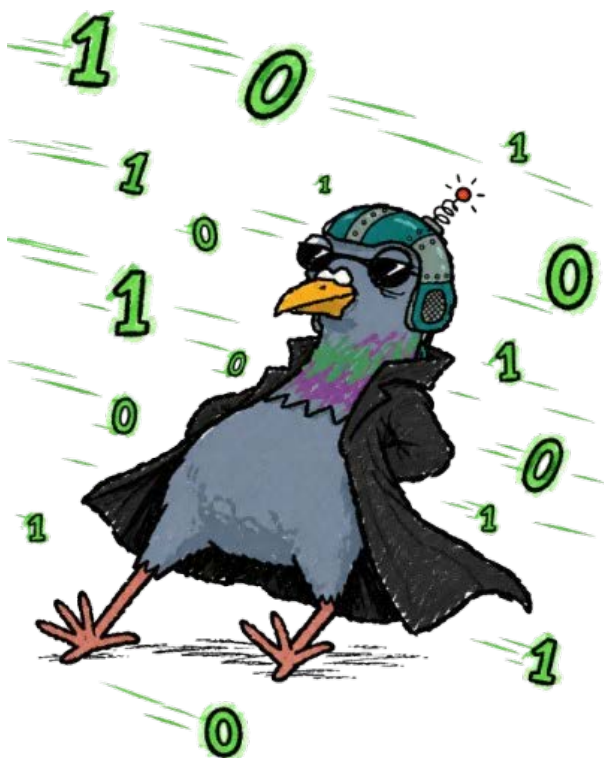
اگر بخواهیم تمام فصل را در یک مدل ذهنی خلاصه کنیم، می‌توانیم بگوییم:

1
1
1
1
1
1

این مدل ذهنی پایه‌ای است که در فصل‌های بعدی، هنگام ورود به الگوریتم‌ها، فلوچارت،
متغیرها، عملگرها، شرط‌ها، حلقه‌ها، آرایه‌ها و برنامه‌نویسی، مرتب به آن برمی‌گردیم.

سیستم های عددی

فصل دوم



۱. مقدمه

ما در زندگی روزمره تقریباً همیشه از سیستم عددی دهدهی (Decimal) استفاده می‌کنیم. از دوران کودکی یاد می‌گیریم که برای شمارش از ده رقم 0 تا 9 استفاده کنیم و با مفاهیمی مانند یکان، دهگان، صدگان و هزارگان آشنا می‌شویم. برای مثال، وقتی عدد 98065 را می‌بینیم، می‌دانیم که رقم 5 در جایگاه یکان، 6 در جایگاه دهگان، 0 در جایگاه صدگان، 8 در جایگاه هزارگان و 9 در جایگاه ده‌هزارگان قرار دارد. اما این موضوع به خودی خود یک ویژگی ریاضیاتی مهم‌تر را نشان می‌دهد: ارزش هر رقم به جایگاه آن وابسته است.

در سیستم دهدهی، ارزش جایگاه‌ها بر اساس توان‌های عدد 10 تعیین می‌شود:

$$98065 = (9 \times 10^4) + (8 \times 10^3) + (0 \times 10^2) + (6 \times 10^1) + (5 \times 10^0)$$

بنابراین:

$$98065 = 90000 + 8000 + 0 + 60 + 5$$

در اینجا عدد 10 را مبنای سیستم عددی می‌نامیم.

سیستم‌های عددی فقط به مبنای ۱۰ محدود نیستند. در علوم کامپیوتر، مبنای دیگری مانند ۲، ۸ و ۱۶ اهمیت بسیار زیادی دارند؛ زیرا کامپیوتر در سطح سخت‌افزار با مقادیر دودویی، یعنی صفر و یک، کار می‌کند.

۲. مفهوم مبنا در سیستم‌های عددی

یک سیستم عددی با مبنا b از تعدادی رقم مشخص برای نمایش اعداد استفاده می‌کند. در یک سیستم با مبنا b رقم‌های مجاز عبارت‌اند از:

$$0, 1, 2, \dots, b-1$$

بنابراین:

- در مبنا ۱۰ ، ده رقم 0 تا 9 داریم.
- در مبنا ۲ ، دو رقم 0 و 1 داریم.
- در مبنا ۸ ، هشت رقم 0 تا 7 داریم.
- در مبنا ۱۶ ، شانزده نماد از 0 تا 9 و سپس A تا F داریم.
- نکته مهم این است که رقمی که از مبنا سیستم بزرگ‌تر یا مساوی با مبنا باشد، در آن سیستم مجاز نیست. برای مثال، در مبنا ۷ فقط این رقم‌ها مجاز هستند:

$$\{0, 1, 2, 3, 4, 5, 6\}$$

بنابراین عددی مانند $(526)_7$ معتبر است، اما $(527)_7$ معتبر نیست؛ زیرا رقم 7 در مبنا 7 وجود ندارد. برای مشخص کردن مبنا یک عدد، عدد مبنا را به صورت زیرنویس در سمت راست عدد می‌نویسیم؛ مانند $(526)_7$.

۲.۱. ارزش مکانی در هر مبنا

همان‌طور که در سیستم ده‌دهی جایگاه‌های عدد بر اساس توان‌های ۱۰ تعیین می‌شوند، در یک سیستم با مبنای b ، جایگاه‌ها بر اساس توان‌های b تعیین می‌شوند. برای مثال، عدد زیر را در مبنای ۷ در نظر بگیرید:

$$(2421)_7$$

از سمت راست، توان‌های مبنا از صفر شروع می‌شوند:

رقم	جایگاه	ارزش مکانی
۱	7^0	۱
۲	7^1	۷
۴	7^2	۴۹
۲	7^3	۳۴۳

بنابراین:

$$(2421)_7 = 2(7^3) + 4(7^2) + 2(7^1) + 1(7^0)$$

یعنی:

$$= 2(343) + 4(49) + 2(7) + 1$$

پس مفهوم اصلی سیستم‌های عددی بسیار ساده است:

هر رقم در یک جایگاه مشخص قرار دارد و ارزش آن جایگاه با توان‌های مبنا تعیین می‌شود. این اصل تقریباً تمام تبدیل‌های مبنا را توضیح می‌دهد.

۳. تبدیل بین سیستم‌های عددی

در برنامه‌نویسی و علوم کامپیوتر، لازم است بتوانیم یک عدد را از یک مبنای دیگر تبدیل کنیم. دو تبدیل پایه عبارت‌اند از:

۱. تبدیل از مبنای ۱۰ به مبنای دیگر

۲. تبدیل از یک مبنای دیگر به مبنای ۱۰

سپس می‌توانیم از رابطه میان مبناهای ۲، ۸ و ۱۶ برای تبدیل سریع‌تر استفاده کنیم.

۳.۱. تبدیل از مبنای ۱۰ به مبنای دلخواه

برای تبدیل یک عدد صحیح از مبنای ۱۰ به مبنای b ، از روش **تقسیمات متوالی** استفاده می‌کنیم.

مراحل کار عبارت‌اند از:

۱. عدد را بر مبنای جدید b تقسیم می‌کنیم.

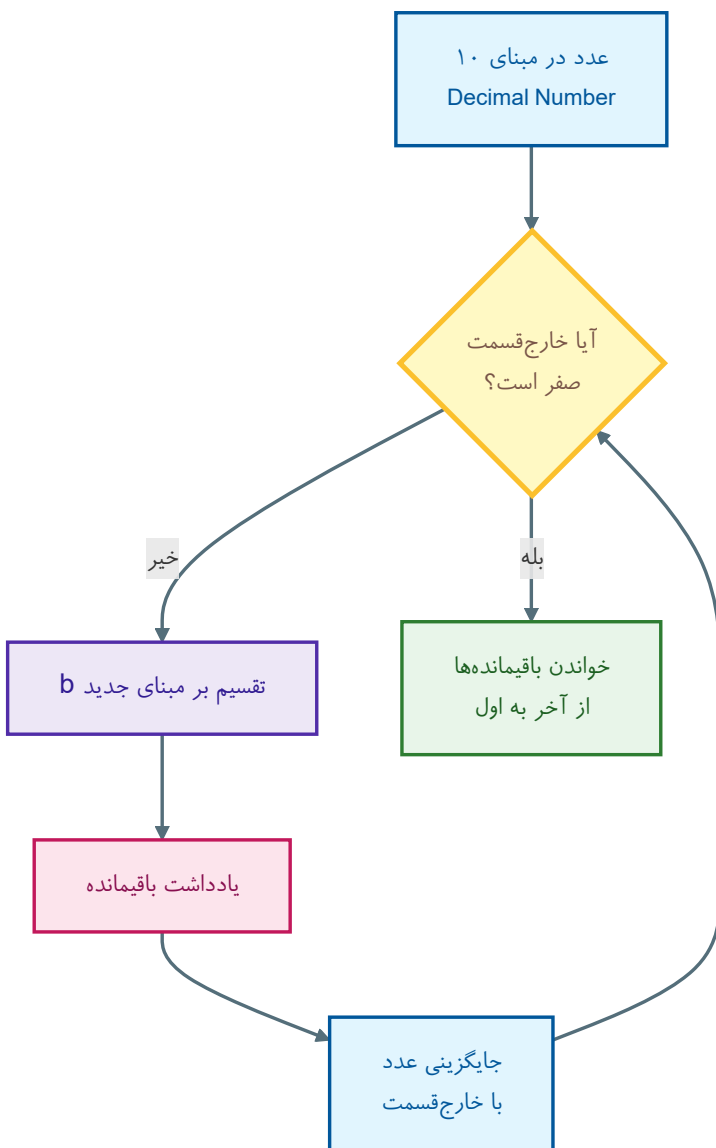
۲. باقیمانده تقسیم را یادداشت می‌کنیم.

۳. خارج‌قسمت را دوباره بر b تقسیم می‌کنیم.

۴. این کار را تا زمانی ادامه می‌دهیم که خارج‌قسمت صفر شود.

۵. در پایان، باقیمانده‌ها را از آخر به اول می‌خوانیم.

دلیل خواندن معکوس باقیمانده‌ها این است که اولین تقسیم، رقم مربوط به کم‌ارزش‌ترین جایگاه را به دست می‌دهد و آخرین تقسیم، رقم مربوط به پرارزش‌ترین جایگاه را مشخص می‌کند.



مثال: تبدیل 897_{10} به مبنای ۷

عدد 897 را به صورت متوالی بر 7 تقسیم می‌کنیم:

$$\begin{array}{r|l} 897 & \div 7 = 128 \text{ remainder } 1 \\ 128 & \div 7 = 18 \text{ remainder } 2 \\ 18 & \div 7 = 2 \text{ remainder } 4 \\ 2 & \div 7 = 0 \text{ remainder } 2 \end{array}$$

باقیمانده‌ها از بالا به پایین عبارت‌اند از:

$1, 2, 4, 2$

اما باید آن‌ها را از پایین به بالا بخوانیم:

$2, 4, 2, 1$

بنابراین:

$$897_{10} = (2421)_7$$

چرا اولین باقیمانده سمت راست عدد است؟

به رابطه زیر توجه کنید:

$$897 = 7 \times 128 + 1897 = 7 \times 128 + 1$$

این 1 در واقع همان رقم مربوط به 707^0 است؛ یعنی کم‌ارزش‌ترین رقم.

سپس:

$$128 = 7 \times 18 + 2128 = 7 \times 18 + 2$$

رقم بعدی 2 مربوط به 717^1 است.

به همین ترتیب، هر تقسیم یکی از جایگاه‌های عدد را از سمت راست به سمت چپ مشخص می‌کند. به همین دلیل ترتیب نهایی باقیمانده‌ها معکوس می‌شود.

۳.۲. تبدیل از مبنای دلخواه به مبنای ۱۰

تبدیل در جهت مخالف ساده‌تر است. برای تبدیل یک عدد از مبنای b به دهدهی، کافی است:

۱. هر رقم را در توان مناسب مبنا ضرب کنیم.

۲. تمام حاصل‌ها را با یکدیگر جمع کنیم.

برای مثال:

$$(2421)_7$$

دارای چهار رقم است. بنابراین از سمت راست، توان‌های ۰ تا ۳ را داریم:

$$(2421)_7 = 2(7^3) + 4(7^2) + 2(7^1) + 1(7^0)$$

حالا محاسبه می‌کنیم:

$$= 2(343) + 4(49) + 2(7) + 1 = 686 + 196 + 14 + 1 = 897$$

پس:

$$(2421)_7 = (897)_{10}$$

این نتیجه همچنین تبدیل قبلی ما را تأیید می‌کند.

نکته مهم

در این روش، شماره‌گذاری توان‌ها همیشه از سمت راست و از صفر شروع می‌شود.

برای مثال:

$$(5312)_b$$

به شکل زیر تفسیر می‌شود:

$$5b^3 + 3b^2 + 1b^1 + 2b^0$$

نه اینکه اولین رقم را در b^1 ضرب کنیم.

۴. مبناهای مهم در علوم کامپیوتر

اگرچه می‌توان از هر مبنای دلخواهی استفاده کرد، سه مبنای ۲، ۸ و ۱۶ در علوم کامپیوتر اهمیت ویژه‌ای دارند. این سه مبنا به دلیل رابطه بسیار ساده‌ای که با سیستم دودویی دارند، در برنامه‌نویسی، معماری کامپیوتر، نمایش حافظه، مدارهای دیجیتال و بسیاری از حوزه‌های دیگر استفاده می‌شوند.

۴.۱. مبنای دو؛ Binary

سیستم دودویی یا **Binary** دارای مبنای ۲ است و فقط از دو رقم استفاده می‌کند: $\{0, 1\}$ همانطور که در فصل قبل خواندیم هر رقم دودویی را یک بیت (**Bit**) می‌نامیم. بنابراین یک بیت می‌تواند تنها یکی از ارقام ۰ یا ۱ باشد. این ویژگی با نحوه کار بسیاری از مدارهای الکترونیکی کامپیوتر سازگار است. در سطح سخت‌افزار، وضعیت‌های مختلف مدار را می‌توان به شکل دو حالت منطقی نمایش داد و این دو حالت معمولاً با ۰ و ۱ نشان داده می‌شوند. با کنار هم قرار گرفتن بیت‌ها می‌توانیم مقادیر بسیار بیشتری را نمایش دهیم.

برای مثال:

- ۱ بیت $\rightarrow$ ۲ حالت
- ۲ بیت $\rightarrow$ ۴ حالت
- ۳ بیت $\rightarrow$ ۸ حالت
- ۴ بیت $\rightarrow$ ۱۶ حالت
- ۸ بیت $\rightarrow$ ۲۵۶ حالت

به طور کلی، با n بیت می‌توان 2^n حالت متفاوت را نمایش داد.
برای مثال، با ۸ بیت $2^8 = 256$ حالت مختلف داریم.

مثال: تبدیل 486_{10} به مبنای ۲

عدد 486 را به صورت متوالی بر ۲ تقسیم می‌کنیم:

486	÷ 2	= 243	remainder 0
243	÷ 2	= 121	remainder 1
121	÷ 2	= 60	remainder 1
60	÷ 2	= 30	remainder 0
30	÷ 2	= 15	remainder 0
15	÷ 2	= 7	remainder 1
7	÷ 2	= 3	remainder 1
3	÷ 2	= 1	remainder 1
1	÷ 2	= 0	remainder 1

باقیمانده‌ها را از پایین به بالا می‌خوانیم: 111100110

بنابراین:

$$486_{10} = (111100110)_2$$

می‌توانیم برای اطمینان نتیجه را دوباره به دهدهی تبدیل کنیم:

$$\begin{aligned}
 (111100110)_2 &= 1(2^8) + 1(2^7) + 1(2^6) + 1(2^5) + 0(2^4) \\
 &\quad + 0(2^3) + 1(2^2) + 1(2^1) + 0(2^0) \\
 &= 256 + 128 + 64 + 32 + 4 + 2 \\
 &= 486
 \end{aligned}$$

پس تبدیل درست است.

۴.۲. مبنای هشت؛ Octal

سیستم هشت‌هشتی یا **Octal** مبنای ۸ دارد و از هشت رقم زیر استفاده می‌کند:

$$\{0, 1, 2, 3, 4, 5, 6, 7\}$$

رابطه مهم مبنای ۸ با مبنای ۲ از اینجا می‌آید:

$$8 = 2^3$$

به همین دلیل هر رقم هشت‌هشتی دقیقاً معادل سه بیت دودویی است.

برای مثال:

Octal	Binary
۰	۰۰۰
۱	۰۰۱
۲	۰۱۰
۳	۰۱۱
۴	۱۰۰
۵	۱۰۱
۶	۱۱۰
۷	۱۱۱

این رابطه باعث می‌شود تبدیل بین باینری و اکتال بسیار سریع باشد.

تبدیل باینری به اکتال

برای تبدیل یک عدد صحیح دودویی به مبنای ۸:

۱. از سمت راست شروع می‌کنیم.
۲. بیت‌ها را سه‌تا سه‌تا گروه‌بندی می‌کنیم.
۳. اگر گروه سمت چپ کمتر از سه بیت داشت، در سمت چپ آن صفر اضافه می‌کنیم.
۴. هر گروه سه‌بیتی را به رقم اکتال تبدیل می‌کنیم.

مثال

$(10011111)_2$

گروه‌بندی از سمت راست:

10 | 011 | 111

گروه اول فقط دو بیت دارد، بنابراین یک صفر در سمت چپ آن اضافه می‌کنیم:

010 | 011 | 111

حالا هر گروه را تبدیل می‌کنیم:

2 ← 010

3 ← 011

7 ← 111

پس:

$$(10011111)_2 = (237)_8$$

چرا گروه‌بندی از راست انجام می‌شود؟

زیرا رقم سمت راست در سیستم عددی کم‌ارزش‌ترین رقم است و هر رقم اکثلاً دقیقاً سه جایگاه دودویی را پوشش می‌دهد. بنابراین گروه‌بندی باید از سمت راست آغاز شود تا مرز جایگاه‌ها درست حفظ شود.

۴.۳. مبنای شانزده؛ Hexadecimal

سیستم شانزده‌شانزده‌ی یا **Hexadecimal** مبنای ۱۶ دارد. مشکل اینجاست که برای نمایش اعداد ۰ تا ۱۵ به ۱۶ نماد نیاز داریم، در حالی که فقط ده رقم ۰ تا ۹ در اختیار داریم. بنابراین برای شش مقدار باقی‌مانده از حروف انگلیسی استفاده می‌کنیم:

ده‌دهی	هگزادسیمال
۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸, ۹	۱, ۲, ۳, ۴, ۵, ۶, ۷, ۸, ۹
۱۰	A
۱۱	B
۱۲	C
۱۳	D
۱۴	E
۱۵	F

رابطه مهم هگزادسیمال با باینری نیز بسیار ساده است:

$$16 = 2^4$$

بنابراین هر رقم هگزادسیمال دقیقاً معادل چهار بیت دودویی است. این ویژگی باعث شده هگزادسیمال یکی از مهم‌ترین روش‌های خلاصه‌نویسی مقادیر دودویی باشد.

تبدیل باینری به هگزادسیمال

برای تبدیل:

۱. بیت‌ها را از سمت راست چهار تا چهار تا گروه‌بندی می‌کنیم.
۲. اگر گروه سمت چپ کمتر از چهار بیت داشت، صفر اضافه می‌کنیم.
۳. هر گروه را به یک رقم هگزادسیمال تبدیل می‌کنیم.

مثال

عدد $(1010011101)_2$ را در نظر بگیرید.

گروه‌بندی:

10 | 1001 | 1101

گروه سمت چپ را با صفر کامل می‌کنیم:

0010 | 1001 | 1101

حالا:

D ← 13 ← 1101

9 ← 1001

2 ← 0010

پس:

$$(1010011101)_2 = (29D)_{16}$$

تبدیل اکتال و هگزادسیمال به باینری

در جهت مخالف، کار حتی ساده‌تر است. برای تبدیل اکتال به باینری، هر رقم را با سه بیت جایگزین می‌کنیم. برای تبدیل هگزادسیمال به باینری، هر رقم را با چهار بیت جایگزین می‌کنیم.

مثال: هگزادسیمال به باینری

عدد $(BF3)_{16}$ را در نظر بگیرید.

از جدول بالا:

B ← 1011

F ← 1111

3 ← 0011

بنابراین:

$$(BF3)_{16} = (101111110011)_2$$

مثال: اکتال به باینری

عدد:

$$(367)_8$$

را در نظر بگیرید:

$$011 \leftarrow 3$$

$$110 \leftarrow 6$$

$$111 \leftarrow 7$$

پس:

$$111 \mid 110 \mid 011$$

و در نتیجه:

$$(367)_8 = (011110111)_2$$

صفر ابتدایی را می‌توان در صورت نیاز حذف کرد:

$$(367)_8 = (11110111)_2$$

هر دو نمایش از نظر مقدار عددی یکسان‌اند.

۵. تبدیل اعداد دارای بخش اعشاری

تا اینجا با اعداد صحیح کار کردیم. اما اعداد می‌توانند دارای بخش اعشاری نیز باشند، مانند: **41.75**. در چنین حالتی، بخش صحیح و بخش اعشاری را **جداگانه** تبدیل می‌کنیم. برای بخش صحیح از تقسیمات متوالی استفاده می‌شود. اما برای بخش اعشاری، روش متفاوتی داریم: **ضرب‌های متوالی در مینا**.

۵.۱. تبدیل بخش اعشاری از مبنای ۱۰ به مبنای ۲

برای تبدیل یک بخش اعشاری از دهدهی به باینری:

۱. بخش اعشاری را در ۲ ضرب می‌کنیم.
۲. بخش صحیح حاصل را استخراج می‌کنیم.
۳. فقط بخش اعشاری باقی‌مانده را دوباره در ۲ ضرب می‌کنیم.
۴. این کار را ادامه می‌دهیم.
۵. ارقام صحیحی که در هر مرحله به دست آمده‌اند، از بالا به پایین بعد از ممیز قرار می‌گیرند.

مثال: تبدیل 0.75_{10} به باینری

مرحله اول:

$$0.75 \times 2 = 1.5$$

بخش صحیح **1** است.

بخش اعشاری باقی‌مانده:

$$0.5$$

مرحله دوم:

$$0.5 \times 2 = 1.0$$

دوباره بخش صحیح 1 است.

چون بخش اعشاری اکنون صفر شده، کار تمام است.

بنابراین:

$$0.11_2 = 0.75_{10}$$

مثال: تبدیل 41.75_{10} به باینری

ابتدا بخش صحیح را تبدیل می‌کنیم:

$$41_{10} = 101001_2$$

سپس بخش اعشاری:

$$0.75_{10} = 0.11_2$$

دو قسمت را کنار هم قرار می‌دهیم:

$$(41.75)_{10} = (101001.11)_2$$

چرا روش ضرب استفاده می‌شود؟

در بخش صحیح، تقسیم‌های متوالی با مبنا باعث می‌شوند رقم‌ها را از سمت راست استخراج کنیم.

در بخش اعشاری، ضرب‌های متوالی در مبنا باعث می‌شوند رقم‌های بعد از ممیز را از سمت چپ استخراج کنیم.

بنابراین می‌توان این دو روش را به صورت زیر به خاطر سپرد:

قسمت صحیح → تقسیم متوالی

قسمت اعشاری → ضرب متوالی

۵.۲. تبدیل عدد اعشاری باینری به دهدهی

برای تبدیل عددی که دارای بخش اعشاری است از باینری به دهدهی، همان مفهوم ارزش مکانی را ادامه می‌دهیم. تفاوت این است که بعد از ممیز، توان‌های مبنا منفی می‌شوند:

$$2^{-1}, 2^{-2}, 2^{-3}, \dots$$

برای مثال:

$$(0.101)_2$$

را در نظر بگیرید.

رقم اول بعد از ممیز در 2^{-1} ، رقم دوم در 2^{-2} و رقم سوم در 2^{-3} قرار دارد:

$$(0.101)_2 = 1(2^{-1}) + 0(2^{-2}) + 1(2^{-3})$$

بنایراین:

$$= 1 \left(\frac{1}{2} \right) + 0 \left(\frac{1}{4} \right) + 1 \left(\frac{1}{8} \right) = 0.5 + 0 + 0.125 = 0.625$$

پس:

$$(0.101)_2 = (0.625)_{10}$$

۶. روش خط کش باینری

یکی دیگر از روش‌های ساده برای تبدیل یک عدد ده‌دهی به باینری، استفاده از توان‌های ۲ است.

در این روش، توان‌های ۲ را از بزرگ به کوچک می‌نویسیم و بررسی می‌کنیم که هر توان آیا در عدد باقی‌مانده جا می‌شود یا خیر.

برای مثال، برای اعداد تا ۸ بیت می‌توان نوشت:

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7
۱	۲	۴	۸	۱۶	۳۲	۶۴	۱۲۸

اگر یک مقدار در عدد موردنظر جا شود، ۱ می‌نویسیم و آن مقدار را کم می‌کنیم.

اگر جا نشود، ۰ می‌نویسیم.

مثال: تبدیل ۱۳ به باینری

ببزرگ‌ترین توان ۲ که در ۱۳ جا می‌شود، ۸ است:

$$13 - 8 = 5$$

پس برای 2^3 رقم ۱ داریم.

اکنون ۴ در ۵ جا می‌شود:

$$5 - 4 = 1$$

پس برای 2^2 نیز ۱ داریم.

عدد ۲ در ۱ جا نمی‌شود، پس برای 2^1 رقم ۰ داریم.

در نهایت 1 در ۱ جا می‌شود، پس برای 2^0 رقم 1 داریم.

بنابراین:

توان	2^3	2^2	2^1	2^0
مقدار	۸	۴	۲	۱
باینری	۱	۱	۰	۱

پس:

$$(13)_{10} = (1101)_2$$

این روش درک خوبی از مفهوم باینری ایجاد می‌کند، زیرا به جای انجام تقسیمات متوالی، مستقیماً می‌بینیم که عدد از چه توان‌هایی از ۲ تشکیل شده است:

$$13 = 8 + 4 + 1$$

و به همین دلیل نمایش باینری آن 1101 است.

۷. اعداد اعشاری در مبنای ۸ و ۱۶

رابطه بین باینری، اکتال و هگزادسیمال فقط برای اعداد صحیح نیست؛ برای اعداد دارای بخش اعشاری نیز برقرار است.

تفاوت اصلی در نحوه گروه‌بندی بخش بعد از ممیز است.

برای بخش صحیح، گروه‌بندی را از **راست به چپ** انجام می‌دهیم.

برای بخش اعشاری، گروه‌بندی را از **چپ به راست** انجام می‌دهیم.

مثال: اکتال به باینری

عدد زیر را در نظر بگیرید:

$$(547.632)_8$$

برای بخش صحیح:

$$101 \rightarrow 5$$

$$100 \rightarrow 4$$

$$111 \rightarrow 7$$

برای بخش اعشاری نیز:

$$110 \rightarrow 6$$

$$011 \rightarrow 3$$

$$010 \rightarrow 2$$

بنابراین:

$$(547.632)_8 = (101100111.110011010)_2$$

مثال: هگزادسیمال به باینری

$$(F4E.8C)_{16}$$

هر رقم هگزادسیمال را به چهار بیت تبدیل می‌کنیم:

$$F \rightarrow 1111$$

$$0100 \rightarrow 4$$

$$E \rightarrow 1110$$

$$1000 \rightarrow 8$$

$$C \rightarrow 1100$$

در نتیجه:

$$(F4E.8C)_{16} = (111101001110.10001100)_2$$

مثال: باینری به اکتال

$$(1011.1011)_2$$

بخش صحیح را از راست سه‌تا سه‌تا و بخش اعشاری را از چپ سه‌تا سه‌تا گروه‌بندی

می‌کنیم:

$$100 \mid 101 \mid 011 \mid 001$$

سپس:

1 → 001

3 → 011

5 → 101

4 → 100

بنابراین:

$$(1011.1011)_2 = (13.54)_8$$

مثال: باینری به هگزادسیمال

$$(10111.10111)_2$$

گروه‌بندی چهار بیتی:

$$1000 \mid 1011 \mid 0111 \mid 0001$$

بنابراین:

1 → 0001

7 → 0111

B → 1011

8 → 1000

در نتیجه:

$$(10111.10111)_2 = (17.B8)_{16}$$

نکته مهم درباره صفرهای اضافه

صفرهایی که برای کامل کردن گروه‌ها در سمت چپ بخش صحیح یا سمت راست بخش اعشاری اضافه می‌کنیم، مقدار عدد را تغییر نمی‌دهند.

برای مثال:

1011

9

001011

از نظر مقدار برابرند.

همین‌طور:

0.1011

9

0.101100

از نظر مقدار برابرند.

این صفرها فقط برای کامل کردن گروه‌های سه‌بیتی یا چهاربیتی استفاده می‌شوند.

۸. چرا هگزادسیمال در کامپیوتر مهم است؟

ممکن است این سؤال مطرح شود که اگر کامپیوتر در نهایت با صفر و یک کار می‌کند، چرا اصلاً از اعداد هگزادسیمال استفاده می‌کنیم؟ پاسخ این است که نمایش رشته‌های طولانی باینری برای انسان بسیار دشوار است.

برای مثال:

```
111111101010110011001101
```

خواندن و بررسی چنین رشته‌ای دشوار است و احتمال اشتباه نیز زیاد است.

اما چون هر رقم هگزادسیمال دقیقاً چهار بیت را نمایش می‌دهد، می‌توان همان مقدار را بسیار فشرده‌تر نوشت:

```
FEACCD
```

این دو نمایش از نظر اطلاعاتی یکسان‌اند؛ فقط نمایش دوم برای انسان کوتاه‌تر و خواناتر است.

به همین دلیل هگزادسیمال در حوزه‌هایی مانند:

- نمایش آدرس‌های حافظه
 - برنامه‌نویسی سطح پایین
 - دیباگ کردن
 - معماری کامپیوتر
 - نمایش رنگ‌ها در برخی محیط‌های نرم‌افزاری
- زیاد دیده می‌شود.

۹. کدگذاری کاراکترها؛ ASCII

تا اینجا درباره نمایش اعداد صحبت کردیم. اما کامپیوتر فقط عدد ذخیره نمی‌کند. متن نیز باید به شکلی در حافظه ذخیره شود.

برای انسان، تفاوت میان این دو عبارت کاملاً واضح است:

ABC

9

123

اما سخت‌افزار در نهایت با الگوهایی از بیت‌ها کار می‌کند. بنابراین برای نمایش حروف و علائم باید یک روش استاندارد داشته باشیم که مشخص کند هر کاراکتر با چه عددی نمایش داده شود.

این کار را کدگذاری کاراکتر (Character Encoding) انجام می‌دهد.

۹.۱. ASCII چیست؟

یکی از استانداردهای تاریخی و بسیار مهم در این زمینه ASCII است:

American Standard Code for Information Interchange

ASCII در شکل اصلی خود از ۷ بیت برای نمایش هر کاراکتر استفاده می‌کند. از آنجا که با ۷ بیت می‌توان

$$2^7 = 128$$

حالت مختلف ایجاد کرد، ASCII اصلی می‌تواند ۱۲۸ کد مختلف داشته باشد.

این کدها برای مواردی مانند حروف انگلیسی بزرگ و کوچک، ارقام 0 تا 9، علائم نگارشی، فاصله و برخی کاراکترهای کنترلی استفاده می‌شوند.

۹.۲. هر کاراکتر در واقع یک عدد است

برای مثال، حرف بزرگ: L در ASCII دارای مقدار دهدهی 76 است. بنابراین می‌توانیم مقدار آن را در مبناهای مختلف نمایش دهیم:

$$L = 76_{10}$$

9:

$$76_{10} = 01001100_2$$

همچنین:

$$76_{10} = 4C_{16}$$

بنابراین می‌توان گفت:



این نکته مهم است: کامپیوتر برای ذخیره یک حرف، خودِ شکل گرافیکی آن حرف را ذخیره نمی‌کند؛ بلکه یک کد عددی متناظر با آن را ذخیره می‌کند.

برای مثال، اگر یک برنامه بخواهد حرف **L** را پردازش کند، در سطح پایین با مقدار عددی متناظر با آن سروکار دارد.

۹.۳. ASCII و یک بایت

ASCII اصلی ۷ بیتی است، اما در بسیاری از سیستم‌های کامپیوتری، داده‌ها در واحدهایی مانند **بایت (۸ بیت)** ذخیره و پردازش می‌شوند. بنابراین یک کاراکتر ASCII معمولاً در یک بایت قرار می‌گیرد و بیت اضافی می‌تواند در برخی زمینه‌های قدیمی برای مقاصدی مانند تشخیص خطا یا در برخی مجموعه کاراکترهای توسعه‌یافته مورد استفاده قرار گیرد. البته باید دقت کرد که **ASCII اصلی با یک کدگذاری ۸ بیتی یکسان نیست.** ASCII استاندارد پایه همان مجموعه ۷ بیتی با ۱۲۸ کد است.

۹.۴. آیا امروزه همه متن‌ها با ASCII ذخیره می‌شوند؟

خیر.

ASCII برای زبان انگلیسی و مجموعه محدودی از علائم بسیار مناسب است، اما برای زبان‌هایی مانند فارسی، عربی، چینی، ژاپنی و بسیاری از زبان‌های دیگر کافی نیست. برای حل این مشکل، استانداردهای گسترده‌تری مانند **Unicode** ایجاد شدند. یکی از مهم‌ترین روش‌های کدگذاری مبتنی بر **UTF-۸، UTF، Unicode** است. **UTF-۸** می‌تواند کاراکترهای ASCII را نیز بدون تغییر نمایش دهد؛ به همین دلیل **ASCII و UTF-۸** ارتباط بسیار نزدیکی با یکدیگر دارند، اما این دو یکی نیستند. در **UTF-۸**، کاراکترهای مختلف می‌توانند با تعداد متفاوتی از بایت‌ها ذخیره شوند. بنابراین وقتی یک فایل متنی را روی کامپیوتر ذخیره می‌کنیم، علاوه بر خود متن، روش کدگذاری آن نیز اهمیت دارد؛ زیرا نرم‌افزار دریافت‌کننده باید بداند بیت‌ها و بایت‌ها چگونه باید به کاراکتر تبدیل شوند.

۱۰. جمع‌بندی

در این فصل با مفهوم سیستم‌های عددی و نقش آن‌ها در علوم کامپیوتر آشنا شدیم. ابتدا دیدیم که هر سیستم عددی بر اساس یک مبنا (Base) تعریف می‌شود و رقم‌های مجاز در آن از ۰ تا $b-1$ هستند. سپس با مفهوم ارزش مکانی آشنا شدیم و دیدیم که ارزش هر رقم بر اساس جایگاه آن و توان‌های مبنا تعیین می‌شود. در ادامه روش‌های تبدیل اعداد بین مبناهای مختلف را بررسی کردیم؛ از جمله تقسیمات متوالی برای تبدیل بخش صحیح از دهدهی و ضرب‌های متوالی برای تبدیل بخش اعشاری، همچنین تبدیل مستقیم اعداد به دهدهی با استفاده از توان‌های مثبت و منفی مبنا. سپس سیستم‌های دودویی، اکتال و هگزادسیمال را بررسی کردیم و ارتباط میان آن‌ها را شناختیم؛ به‌طوری‌که هر رقم اکتال معادل سه بیت و هر رقم هگزادسیمال معادل چهار بیت است. در بخش پایانی نیز با مفاهیم بیت، بایت، ASCII، Unicode، UTF-۸، BCD به‌عنوان روش‌های نمایش و کدگذاری داده آشنا شدیم. در نهایت، مهم‌ترین نکته‌ی این فصل این است که سیستم‌های عددی مجموعه‌ای از قواعد حفظی نیستند، بلکه همگی بر یک مفهوم اساسی یعنی ارزش مکانی رقم‌ها بر اساس توان‌های مبنا بنا شده‌اند.

الگوریتم و فلوچارت

فصل سوم



۱. مقدمه

یکی از مهم‌ترین مفاهیم در علوم کامپیوتر، مفهوم الگوریتم (Algorithm) است. تقریباً هر برنامه‌ای که با کامپیوتر اجرا می‌شود، در پشت خود مجموعه‌ای از مراحل منطقی دارد که مشخص می‌کنند چه کاری، با چه ترتیبی و تحت چه شرایطی باید انجام شود. در ساده‌ترین بیان، الگوریتم یعنی روش مرحله‌به‌مرحله برای حل یک مسئله یا انجام یک کار.

برای مثال، اگر بخواهیم به شخصی آموزش دهیم چگونه یک چای درست کند، می‌توانیم مراحل را مانند این را بیان کنیم:

۱. آب را داخل کتری بریز.
۲. کتری را روشن کن.
۳. صبر کن تا آب بجوشد.
۴. چای را داخل قوری قرار بده.
۵. آب جوش را به قوری اضافه کن.
۶. مدتی صبر کن.
۷. چای را در فنجان بریز.

این مراحل یک دستورالعمل برای انجام یک کار هستند. در برنامه‌سازی نیز اساس کار مشابه است؛ با این تفاوت که دستورالعمل‌ها باید بسیار دقیق‌تر باشند، زیرا کامپیوتر نمی‌تواند منظور ما را حدس بزند.

۲. الگوریتم

واژه **Algorithm** در زبان‌های اروپایی از شکل لاتینی‌شده نام **ابوجعفر محمد بن موسی خوارزمی**، دانشمند و ریاضی‌دان ایرانی، گرفته شده است. آثار خوارزمی در گسترش روش‌های محاسباتی و ریاضی در جهان تأثیر زیادی داشته‌اند و نام او در شکل‌گیری واژه **Algorithm** نیز باقی مانده است.

۲.۱. چرا به الگوریتم نیاز داریم؟

انسان در بسیاری از موقعیت‌ها می‌تواند دستورهای ناقص یا مبهم را با استفاده از دانش و تجربه خود تفسیر کند.

فرض کنید به یک انسان بگوییم:

«دو عدد را با هم جمع کن.»

اگر دو عدد در اختیار او قرار گرفته باشد، احتمالاً بدون مشکل متوجه منظور ما می‌شود.

اما کامپیوتر چنین درکی ندارد. باید مشخص کنیم:

- دو عدد از کجا دریافت شوند؟
- هر عدد در کجا ذخیره شود؟
- عملیات جمع دقیقاً چگونه انجام شود؟
- نتیجه در کجا قرار گیرد؟
- نتیجه چگونه به کاربر نمایش داده شود؟

بنابراین، در برنامه‌سازی باید مسئله را به مجموعه‌ای از دستورهای دقیق و قابل اجرا تبدیل کنیم. به همین دلیل، الگوریتم را می‌توان پلی میان مسئله و برنامه دانست:

مسئله



تحلیل مسئله



طراحی الگوریتم



کدنویسی



برنامه

نکته مهم این است که الگوریتم با برنامه یکسان نیست. الگوریتم منطق و روش حل مسئله را مشخص می‌کند، در حالی که برنامه، همان منطق را با استفاده از یک زبان برنامه‌نویسی مانند C، Python یا Java به شکلی بیان می‌کند که کامپیوتر بتواند آن را اجرا کند.

۲.۲. تعریف دقیق الگوریتم

می‌توان الگوریتم را چنین تعریف کرد:

الگوریتم مجموعه‌ای متناهی از دستورالعمل‌های دقیق و بدون ابهام است که با ترتیب مشخص، برای حل یک مسئله یا انجام یک کار اجرا می‌شوند و در نهایت به نتیجه‌ای مشخص می‌رسند.

در این تعریف چند نکته مهم وجود دارد:

- دستورها باید دقیق باشند.
- دستورها نباید مبهم باشند.

- ترتیب اجرای مراحل اهمیت دارد.
- مراحل باید قابل اجرا باشند.
- اجرای الگوریتم باید در نهایت به پایان برسد.
- الگوریتم باید نتیجه یا خروجی موردنظر را تولید کند.

۲.۳. ویژگی‌های اساسی یک الگوریتم

برای اینکه یک مجموعه دستور را بتوانیم الگوریتم بنامیم، معمولاً ویژگی‌های مشخصی برای آن در نظر گرفته می‌شود.

۲.۳.۱ ورودی (Input)

الگوریتم ممکن است یک یا چند **ورودی** داشته باشد یا حتی در بعضی موارد بدون ورودی باشد. ورودی همان داده‌ای است که الگوریتم برای انجام کار خود دریافت می‌کند. برای مثال، در الگوریتم محاسبه میانگین سه عدد، سه عدد ورودی داریم:

$$A = 15$$

$$B = 18$$

$$C = 12$$

این اعداد داده‌های اولیه‌ای هستند که الگوریتم بر اساس آن‌ها محاسبه را انجام می‌دهد. البته هر الگوریتم الزاماً ورودی ندارد. برای مثال، الگوریتمی که فقط پیام «سلام» را چاپ می‌کند، می‌تواند بدون دریافت داده از کاربر اجرا شود.

۲.۳.۲ خروجی (Output)

الگوریتم باید نتیجه‌ای تولید کند که بتوان آن را به عنوان **خروجی** در نظر گرفت. برای

مثال، اگر الگوریتم میانگین سه عدد را محاسبه کند:

$$A = 15$$

$$B = 18$$

$$C = 12$$

خروجی می‌تواند برابر باشد با:

15

زیرا:

$$\frac{15 + 18 + 12}{3} = 15$$

بنابراین، ورودی و خروجی را می‌توان به صورت ساده چنین در نظر گرفت:

ورودی‌ها → الگوریتم → خروجی

۲.۳.۳ قطعیت و عدم ابهام (Definiteness)

هر دستور در یک الگوریتم باید دقیق و بدون ابهام باشد. برای مثال، دستور:

«یک مقدار مناسب انتخاب کن.»

دستور مناسبی برای یک الگوریتم نیست، زیرا مشخص نیست «مناسب» دقیقاً به چه معناست.

اما دستور:

دقیق‌تر است، زیرا مشخص می‌کند چه داده‌ای باید دریافت شود.

این ویژگی در برنامه‌سازی اهمیت بسیار زیادی دارد. انسان می‌تواند بسیاری از دستورهای مبهم را بر اساس شرایط تفسیر کند، اما کامپیوتر باید دستور را به شکلی دریافت کند که تنها یک برداشت منطقی از آن وجود داشته باشد.

۲.۳.۴ جزئیات کافی و قابل‌اجرا بودن دستورها

هر مرحله باید به اندازه‌ای دقیق باشد که مجری الگوریتم بتواند آن را انجام دهد. مثلاً اگر بخواهیم الگوریتمی برای پیدا کردن بزرگ‌ترین عدد داشته باشیم، دستور «بزرگ‌ترین عدد را پیدا کن» به تنهایی یک الگوریتم کامل نیست؛ زیرا روش انجام این کار را مشخص نکرده است. باید دستورهای مانند این داشته باشیم:

```
Max ← A
```

```
If B > Max, then Max ← B
```

```
If C > Max, then Max ← C
```

در این حالت، روش پیدا کردن بزرگ‌ترین مقدار به مراحل مشخص و قابل اجرا شکسته شده است.

۲.۳.۵ ترتیب مراحل (Sequence)

دستورهای الگوریتم باید ترتیب منطقی داشته باشند. برای مثال، برای محاسبه میانگین سه عدد، ابتدا باید عددها را دریافت کنیم و سپس مجموع آن‌ها را محاسبه کنیم:

را بخوان A ، B ، C

↓

$\text{Sum} \leftarrow A + B + C$

↓

$\text{Avg} \leftarrow \text{Sum} / 3$

↓

را چاپ کن Avg

اگر قبل از دریافت اعداد بخواهیم مجموع آن‌ها را محاسبه کنیم، اطلاعات مورد نیاز هنوز در اختیار الگوریتم نیست. بنابراین، ترتیب اجرای مراحل بخشی از منطق الگوریتم است.

۲.۳.۶ خاتمه‌پذیری (Finiteness)

یک الگوریتم باید پس از تعداد محدودی مرحله به پایان برسد. این ویژگی به‌خصوص هنگام استفاده از حلقه‌ها (Loops) اهمیت پیدا می‌کند. برای مثال، اگر الگوریتمی داشته باشیم که دائماً دستور زیر را تکرار کند:

عدد را بخوان

عدد را چاپ کن

دوباره عدد را بخوان

دوباره چاپ کن

...

و هیچ شرطی برای توقف آن وجود نداشته باشد، الگوریتم ممکن است هرگز به پایان نرسد. چنین وضعیتی را معمولاً **حلقه بی‌نهایت (Infinite Loop)** می‌نامیم. البته در نرم‌افزارهای واقعی گاهی برنامه‌ها عمداً برای مدت نامحدود اجرا می‌شوند؛ برای مثال یک

سیستم عامل یا سرویس شبکه ممکن است تا زمان خاموش شدن سیستم فعال باقی بماند. بنابراین، منظور از خاتمه پذیری در تعریف کلاسیک الگوریتم، وجود یک پایان مشخص برای فرایند حل مسئله است.

۲.۳.۷ کارایی و اثربخشی (Effectiveness)

دستورهای الگوریتم باید از نظر منطقی و عملی قابل انجام باشند. برای مثال، دستور زیر برای یک الگوریتم کامپیوتری مناسب نیست:

«تا آخرین رقم اعشار عدد π را محاسبه کن.»

زیرا نمایش اعشاری کامل π پایان ندارد.

در مقابل، دستور:

« π را تا ۱۰ رقم اعشار محاسبه کن.»

یک کار مشخص و قابل انجام است.

۳. مسئله

پیش از آنکه الگوریتم بنویسیم، باید بدانیم دقیقاً چه مسئله‌ای را می‌خواهیم حل کنیم. یک مسئله در برنامه‌سازی معمولاً شامل سه بخش اصلی است:

۱. ورودی: چه اطلاعاتی در اختیار داریم؟

۲. پردازش: چه عملیاتی باید روی این اطلاعات انجام دهیم؟

۳. خروجی: در پایان چه نتیجه‌ای می‌خواهیم؟

برای مثال، مسئله «محاسبه میانگین سه عدد» را در نظر بگیرید.

ورودی

سه عدد:

A, B, C

پردازش

ابتدا آن‌ها را با هم جمع می‌کنیم و سپس حاصل را بر ۳ تقسیم می‌کنیم:

$$Average = \frac{A+B+C}{3}$$

خروجی

میانگین سه عدد.

پس مسئله به شکل زیر خلاصه می‌شود:

A, B, C

↓

Sum the three numbers

↓

Divide by 3

↓

Average

این روش نگاه کردن به مسئله، یکی از ساده‌ترین و مهم‌ترین مهارت‌های اولیه در برنامه‌سازی است.

۳.۱. مراحل حل یک مسئله با کامپیوتر

حل یک مسئله فقط به نوشتن کد محدود نمی‌شود. معمولاً باید چند مرحله پشت سر هم طی شود.

مرحله اول: تعریف دقیق مسئله

ابتدا باید مشخص کنیم دقیقاً چه چیزی می‌خواهیم حل کنیم. برای مثال، به جای اینکه بگوییم:

«برنامه‌ای برای کار با عددها بنویس.»

باید مسئله را دقیق‌تر تعریف کنیم:

«برنامه‌ای بنویسید که سه عدد را دریافت کند و بزرگ‌ترین آن‌ها را نمایش دهد.»

در این حالت ورودی و خروجی مشخص‌تر هستند.

مرحله دوم: بررسی راه‌حل‌ها

ممکن است یک مسئله بیش از یک راه‌حل داشته باشد. برای مثال، برای پیدا کردن بزرگ‌ترین عدد بین چند عدد، می‌توان روش‌های مختلفی طراحی کرد.

بنابراین، پیش از کدنویسی باید بررسی کنیم:

- چه روش‌هایی برای حل مسئله وجود دارد؟
- هر روش چه مراحل دارد؟
- کدام روش ساده‌تر است؟
- آیا روش انتخاب‌شده برای داده‌های مختلف درست عمل می‌کند؟

مرحله سوم: طراحی الگوریتم

پس از تحلیل مسئله، روش حل را به مجموعه‌ای از مراحل دقیق تبدیل می‌کنیم. در این مرحله هنوز الزاماً یک زبان برنامه‌نویسی خاص را نمی‌نویسیم. ممکن است الگوریتم را با زبان طبیعی، شبه‌کد یا فلوجارت بیان کنیم.

مرحله چهارم: ردیابی الگوریتم (Trace)

یکی از روش‌های ساده برای بررسی درست بودن الگوریتم، اجرای ذهنی یا دستی آن با چند داده فرضی است. به این کار **Trace** یا ردیابی الگوریتم گفته می‌شود. فرض کنید الگوریتمی برای پیدا کردن بزرگ‌ترین عدد بین سه عدد داریم و ورودی‌ها عبارت‌اند از:

$$A = 7$$

$$B = 12$$

$$C = 9$$

می‌توانیم مرحله به مرحله بررسی کنیم:

```
Max = A
```

```
Max = 7
```

```
Is B > Max?
```

```
12 > 7 → Yes
```

```
Max = 12
```

```
Is C > Max?
```

```
9 > 12 → No
```

```
Result: Max = 12
```

اگر الگوریتم برای نمونه‌های مختلف نتیجه درست بدهد، احتمال وجود خطای منطقی کاهش می‌یابد.

مرحله پنجم: کدنویسی

پس از اینکه منطق حل مسئله مشخص شد، الگوریتم را با یک زبان برنامه‌نویسی پیاده‌سازی می‌کنیم. برای مثال، ممکن است الگوریتمی که طراحی کرده‌ایم را در زبان C یا Python بنویسیم. در این مرحله باید قواعد زبان برنامه‌نویسی را نیز رعایت کنیم.

مرحله ششم: آزمایش و رفع خطا

برنامه را اجرا می‌کنیم و بررسی می‌کنیم که آیا نتیجه درست است یا خیر. خطاها می‌توانند

انواع مختلفی داشته باشند.

خطای نحوی (Syntax Error)

برنامه قواعد زبان را به درستی رعایت نکرده است.

خطای زمان اجرا (Runtime Error)

برنامه از نظر نحوی قابل اجراست، اما هنگام اجرا با مشکلی مواجه می‌شود.

خطای منطقی (Logical Error)

برنامه اجرا می‌شود، اما نتیجه اشتباه است.

این نوع خطا به خصوص در طراحی الگوریتم اهمیت دارد؛ زیرا ممکن است کد کاملاً درست اجرا شود ولی الگوریتم از ابتدا اشتباه طراحی شده باشد.

مرحله هفتم: استقرار و استفاده

پس از آزمایش و رفع خطاها، نرم‌افزار می‌تواند در محیط واقعی مورد استفاده قرار گیرد. در پروژه‌های واقعی، کار معمولاً در همین‌جا تمام نمی‌شود؛ نرم‌افزار ممکن است بعداً نیاز به نگهداری، اصلاح خطا، به‌روزرسانی و توسعه داشته باشد.

۳.۷ شبه کد (Pseudo-code)

یکی از روش‌های متداول برای بیان الگوریتم، استفاده از شبه کد است. شبه کد نه یک زبان برنامه‌نویسی واقعی است و نه قرار است مستقیماً توسط کامپیوتر اجرا شود. هدف آن این است که منطق الگوریتم را به شکلی ساده و قابل فهم بیان کند. برای مثال، می‌توان الگوریتم میانگین سه عدد را به این صورت نوشت:

Start

Read A, B, and C

$Sum \leftarrow A + B + C$

$Avg \leftarrow Sum / 3$

Print Avg

End

در شبه‌کد می‌توان از زبان طبیعی در کنار نمادهای ریاضی استفاده کرد. نکته مهم این است که شبه‌کد استاندارد واحد و اجباری مانند زبان‌هایی مثل C یا Python ندارد. ممکن است در کتاب‌ها و دانشگاه‌های مختلف، کلمات و قالب‌های متفاوتی برای آن استفاده شود. بنابراین، قواعدی مانند «حتماً خط اول شروع و خط آخر پایان باشد» بیشتر یک قرارداد آموزشی هستند تا قانون جهانی شبه‌کد.

۴. متغیرها در الگوریتم

هنگام اجرای یک الگوریتم معمولاً به مکانی برای نگهداری داده‌ها و نتایج موقت نیاز داریم. برای این منظور از متغیر (Variable) استفاده می‌کنیم.

برای مثال:

```
A ← 10
```

یعنی مقدار ۱۰ را در متغیری با نام A قرار بده.

اگر بعداً بنویسیم:

```
A ← 25
```

مقدار جدید A برابر ۲۵ خواهد شد. به همین دلیل به آن متغیر گفته می‌شود؛ زیرا مقدار آن می‌تواند در طول اجرای الگوریتم تغییر کند. می‌توان متغیر را به صورت یک ظرف یا محل نام‌گذاری شده در حافظه تصور کرد:



نام A به ما اجازه می‌دهد به مقدار موجود در آن محل مراجعه کنیم.

۵. عملگر انتساب

برای قرار دادن یک مقدار در متغیر از عملگر انتساب (Assignment Operator) استفاده می‌کنیم. در شبه‌کد معمولاً از پیکان زیر استفاده می‌شود:

←

برای مثال:

```
A ← 5
```

یعنی:

مقدار ۵ را در متغیر *A* قرار بده.

یا:

```
Sum ← A + B
```

یعنی:

۱. مقدار *A* و *B* را محاسبه کن.

۲. آن‌ها را با هم جمع کن.

۳. نتیجه را در *Sum* قرار بده.

نکته بسیار مهم

علامت انتساب با علامت مساوی در ریاضیات دقیقاً یک مفهوم ندارد. وقتی می‌نویسیم:

$$A \leftarrow A + 1$$

در ریاضیات چنین عبارتی ممکن است عجیب به نظر برسد؛ اما در برنامه‌سازی معنی آن این است:

مقدار فعلی A را بگیر، یک واحد به آن اضافه کن و نتیجه را دوباره داخل A قرار بده.

اگر:

$$A = 5$$

باشد، پس از اجرای:

$$A \leftarrow A + 1$$

خواهیم داشت:

$$A = 6$$

این مفهوم بعداً در بحث حلقه‌ها و شمارنده‌ها اهمیت زیادی پیدا می‌کند.

مثال اول: محاسبه میانگین سه عدد

فرض کنید می‌خواهیم الگوریتمی بنویسیم که سه عدد را دریافت کند و میانگین آن‌ها را نمایش دهد.

تحلیل مسئله:

ورودی

سه عدد:

A, B, C

پردازش

ابتدا مجموع را محاسبه می‌کنیم:

$$Sum = A + B + C$$

سپس:

$$Average = \frac{Sum}{3}$$

خروجی

مقدار **Average**.

الگوریتم

1. Start
2. Read A, B, and C.
3. $Sum \leftarrow A + B + C$
4. $Avg \leftarrow Sum / 3$
5. Print Avg.

6. End

فرض کنید کاربر این مقادیر را وارد کند:

$$A = 10$$

$$B = 15$$

$$C = 20$$

آنگاه:

$$\text{Sum} = 10 + 15 + 20$$

$$= 45$$

9:

$$\text{Avg} = 45 / 3$$

$$= 15$$

پس خروجی الگوریتم برابر ۱۵ است.

مثال دوم: تشخیص زوج یا فرد بودن عدد

برای تشخیص زوج یا فرد بودن یک عدد، از باقی‌مانده تقسیم استفاده می‌کنیم. اگر عددی بر ۲ تقسیم شود و باقی‌مانده آن صفر باشد، عدد زوج است. در غیر این صورت، عدد فرد است.

در شبه‌کد:

1. Start
2. Read N.
3. $R \leftarrow N \% 2$
4. If $R = 0$, print: "The number is even"
5. Otherwise, print: "The number is odd"
6. End

در اینجا $\%$ عملگر باقی‌مانده تقسیم است.

برای مثال، اگر:

$N = 14$

باشد:

$R = 14 \% 2$

$R = 0$

پس عدد زوج است.

اما اگر:

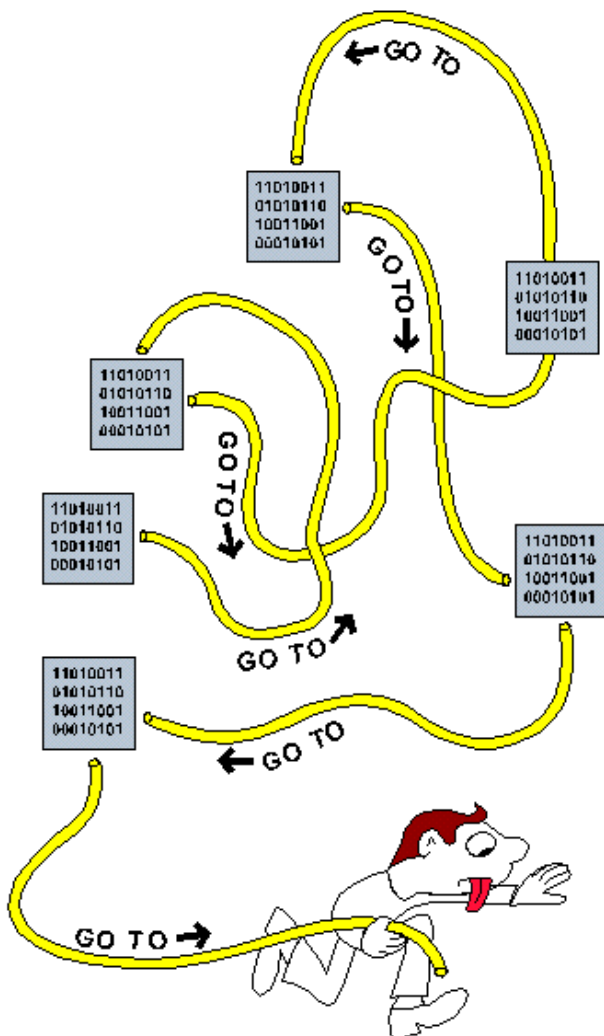
$N = 17$

باشد:

$R = 17 \% 2$

$$R = 1$$

پس عدد فرد است. این مثال برای نخستین بار مفهوم تصمیم‌گیری یا شرط (Condition) را وارد الگوریتم می‌کند.



۶. شرط و تصمیم‌گیری

در بسیاری از مسائل، الگوریتم نمی‌تواند همیشه یک مسیر ثابت را طی کند. گاهی لازم است ابتدا یک وضعیت را بررسی کنیم و بر اساس نتیجه، یکی از چند مسیر ممکن را انتخاب کنیم.

ساختار کلی چنین الگوریتمی این است:

```
اگر شرط برقرار بود
    مسیر اول
در غیر این صورت
    مسیر دوم
```

مثلاً:

```
If Age ≥ 18
    Print: "Adult"
Otherwise
    Print: "Under 18"
```

مثال سوم: بررسی امکان تشکیل مثلث

سه پاره‌خط زمانی می‌توانند یک مثلث تشکیل دهند که مجموع طول هر دو ضلع از ضلع سوم بزرگ‌تر باشد.

اگر طول ضلع‌ها a ، b و c باشند، باید هر سه شرط زیر برقرار باشند:

$$a + b > c$$

$$a + c > b$$

$$b + c > a$$

بنابراین الگوریتم می‌تواند چنین باشد:

```
1. Start
2. Read a, b, and c.
3. If:
    (a < b + c)
    and (b < a + c)
    and (c < a + b)
    is true, go to Step 4;
    otherwise, go to Step 5.
4. Print: "A triangle can be formed" and go to Step
6.
5. Print: "A triangle cannot be formed."
6. End
```

در این مثال با یک شرط مرکب روبه‌رو هستیم؛ یعنی برای تصمیم‌گیری باید چند شرط به صورت هم‌زمان بررسی شوند. عملگر و (AND) زمانی برقرار است که همه شرط‌های موردنظر درست باشند.

مثال چهارم: ریشه‌های معادله درجه دوم

معادله درجه دوم به صورت زیر نوشته می‌شود:

$$ax^2 + bx + c = 0$$

که در آن معمولاً $a \neq 0$ است.

برای بررسی ریشه‌های حقیقی معادله، ابتدا ممیز یا دلتا (**Discriminant**) را محاسبه می‌کنیم:

$$\Delta = b^2 - 4ac$$

مقدار دلتا تعیین می‌کند چند ریشه حقیقی داریم:

- اگر $\Delta < 0$ باشد، معادله ریشه حقیقی ندارد.
- اگر $\Delta = 0$ باشد، یک ریشه حقیقی تکراری دارد.
- اگر $\Delta > 0$ باشد، دو ریشه حقیقی دارد.

برای حالت دوم:

$$x = \frac{-b}{2a}$$

و برای حالت سوم:

$$x_1 = \frac{-b + \sqrt{\Delta}}{2a}$$

$$x_2 = \frac{-b - \sqrt{\Delta}}{2a}$$

بنابراین الگوریتم:

1. Start
2. Read the coefficients a, b, and c.
3. $\Delta \leftarrow (b \times b) - (4 \times a \times c)$

```

4. If  $\Delta < 0$ :
    Print: "The equation has no real roots"
    and go to Step 8.
5. If  $\Delta = 0$ :
     $X \leftarrow -b / (2 \times a)$ 
    Print X
    and go to Step 8.
6. If  $\Delta > 0$ :
     $X1 \leftarrow (-b + \sqrt{\Delta}) / (2 \times a)$ 
     $X2 \leftarrow (-b - \sqrt{\Delta}) / (2 \times a)$ 
7. Print X1 and X2.
8. End

```

این مثال نشان می‌دهد که یک الگوریتم می‌تواند چندین تصمیم پشت سر هم داشته باشد. همچنین نشان می‌دهد که پیش از نوشتن کد، بهتر است منطق مسئله را کاملاً مشخص کنیم. در غیر این صورت، احتمال اشتباه در پیاده‌سازی افزایش پیدا می‌کند.

۷. حلقه و تکرار

تا اینجا الگوریتم‌های ما عمدتاً هر دستور را یک بار اجرا می‌کردند. اما بسیاری از مسائل به تکرار نیاز دارند. برای مثال، فرض کنید می‌خواهیم مجموع ۱۰۰ عدد را محاسبه کنیم. نوشتن ۱۰۰ دستور جداگانه منطقی نیست:

```
عدد اول را بخوان  
عدد دوم را بخوان  
...  
عدد صدم را بخوان
```

به جای آن، می‌توانیم یک مجموعه دستور را در یک حلقه قرار دهیم و آن را چندین بار اجرا کنیم.

حلقه معمولاً سه بخش مهم دارد:

۱. مقداردهی اولیه

۲. شرط ادامه

۳. تغییر وضعیت یا شمارنده

مثال پنجم: محاسبه مجموع n عدد

فرض کنید کاربر ابتدا تعداد اعداد را مشخص می‌کند و سپس همان تعداد عدد را وارد می‌کند.

الگوریتم:

1. Start
2. Read n .
3. $Sum \leftarrow 0$
4. $Counter \leftarrow 1$
5. Read a number from the user and store it in A .
6. $Sum \leftarrow Sum + A$
7. $Counter \leftarrow Counter + 1$
8. If $Counter \leq n$, go back to Step 5.
9. Print Sum .
10. End

اجازه دهید این الگوریتم را با یک مثال بررسی کنیم.

فرض کنید:

$$n = 4$$

و چهار عدد زیر وارد شوند:

10

20

5

15

در ابتدا:

```
Sum = 0
```

```
Counter = 1
```

تکرار اول

عدد ۱۰ دریافت می‌شود:

```
Sum = 0 + 10 = 10
```

```
Counter = 2
```

چون:

```
2 ≤ 4
```

است، حلقه ادامه پیدا می‌کند.

تکرار دوم

عدد ۲۰:

```
Sum = 10 + 20 = 30
```

```
Counter = 3
```

تکرار سوم

عدد ۵:

```
Sum = 30 + 5 = 35
```

```
Counter = 4
```

تکرار چهارم

عدد ۱۵:

```
Sum = 35 + 15 = 50
```

```
Counter = 5
```

اکنون:

```
5 ≤ 4
```

نادرست است، بنابراین حلقه متوقف می‌شود.

در نهایت:

```
Sum = 50
```

این مثال مفهوم مهمی را نشان می‌دهد: متغیر **Counter** تعداد دفعات اجرای حلقه را کنترل می‌کند.

۸ فلوچارت (Flowchart)

وقتی الگوریتم‌ها کوتاه هستند، نمایش متنی آن‌ها معمولاً کافی است. اما با پیچیده‌تر شدن الگوریتم، دنبال کردن مسیر اجرای دستورها در یک متن ممکن است دشوار شود. برای حل این مشکل می‌توان الگوریتم را به صورت **تصویری** نمایش داد. به این نمایش تصویری، **فلوچارت (Flowchart)** یا **روندنما** گفته می‌شود. فلوچارت در واقع مجموعه‌ای از شکل‌های استاندارد است که هر کدام معنای مشخصی دارند و با پیکان‌هایی به یکدیگر متصل می‌شوند. به این ترتیب می‌توان مسیر اجرای الگوریتم را مانند یک نقشه دنبال کرد.

۸.۱ نمادهای اصلی فلوچارت

شکل	نام	کاربرد
بیضی	Terminator	شروع یا پایان
متوازی‌الاضلاع	Input/Output	دریافت ورودی یا نمایش خروجی
مستطیل	Process	محاسبه، پردازش و انتساب
لوزی	Decision	بررسی شرط و انتخاب مسیر
پیکان	Flow Line	نشان دادن جهت اجرای الگوریتم
سند	Document	نمایش یک سند یا خروجی مستندی

بیضی: شروع و پایان

بیضی معمولاً برای نشان دادن نقطه شروع و پایان فرایند استفاده می‌شود.

متوازی‌الاضلاع: ورودی و خروجی

این شکل معمولاً برای عملیات ورودی و خروجی استفاده می‌شود.

برای مثال:

را بخوان N عدد

یا:

را چاپ کن N

مستطیل: پردازش

مستطیل نشان‌دهنده یک عملیات یا پردازش است.

برای مثال:

$Sum = A + B$

یا:

$Counter = Counter + 1$

لوزی: تصمیم‌گیری

لوزی برای بررسی یک شرط استفاده می‌شود.

معمولاً از لوزی دو مسیر خارج می‌شود:

- بله (Yes / True)

- خیر (No / False)

البته بسته به الگوریتم ممکن است تعداد مسیرها یا نحوه برجسب گذاری متفاوت باشد.

پیکان: جریان اجرا

پیکان‌ها نشان می‌دهند که اجرای الگوریتم از کدام مرحله به کدام مرحله حرکت می‌کند. این موضوع در الگوریتم‌هایی که دارای شرط و حلقه هستند اهمیت ویژه‌ای دارد.

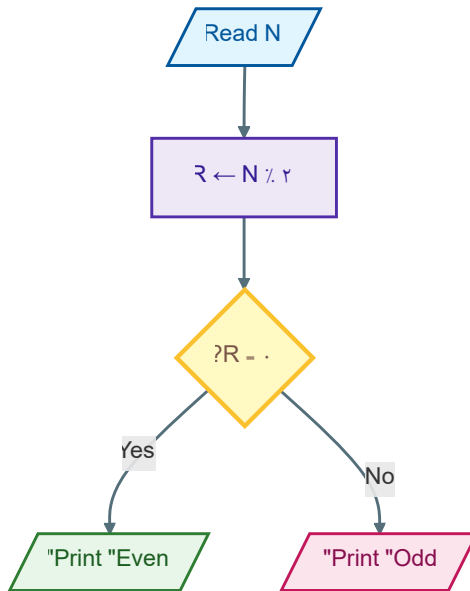
۷.۲. رابطه شبه‌کد و فلوچارت

شبه‌کد و فلوچارت دو روش متفاوت برای نمایش یک منطق واحد هستند. مثلاً الگوریتم زوج یا فرد را می‌توان به صورت متنی چنین نوشت:

```
Read N
R ← N % 2

If R = 0
    Print "Even"
Otherwise
    Print "Odd"
```

و همین منطق را می‌توان به صورت تصویری نمایش داد:

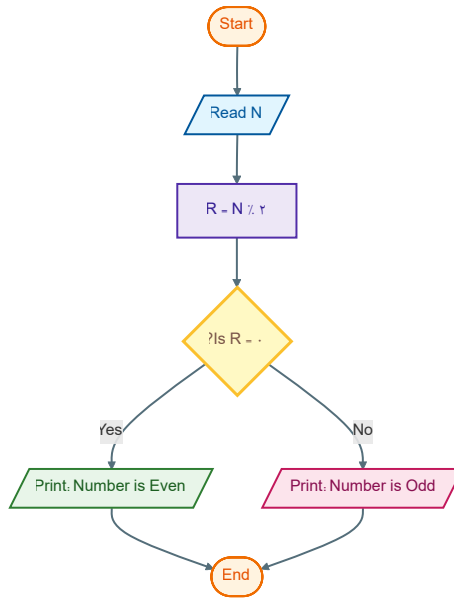


بنابراین، فلوچارت روش دیگری برای بیان همان الگوریتم است.

فلوچارت تشخیص زوج یا فرد

الگوریتم این مسئله چنین بود:

۱. عدد را دریافت کن.
۲. باقی‌مانده تقسیم بر ۲ را محاسبه کن.
۳. اگر باقی‌مانده صفر بود، عدد زوج است.
۴. در غیر این صورت، عدد فرد است.



هنگام خواندن این فلوچارت، باید از **Start** حرکت کنیم و پیکان‌ها را دنبال کنیم.

اگر مثلاً **N = 8** باشد:

N = 8

↓

R = 8 % 2

↓

R = 0

↓

بله

↓

عدد زوج است

اما اگر $N = 9$ باشد:

$N = 9$

↓

$R = 9 \% 2$

↓

$R = 1$

↓

خیر

↓

عدد فرد است

فلوچارت پیدا کردن بزرگ‌ترین عدد بین سه عدد

فرض کنید سه عدد A ، B و C داریم. یک روش ساده این است که ابتدا A را به عنوان بزرگ‌ترین مقدار فعلی در نظر بگیریم:

$Max \leftarrow A$

سپس بررسی کنیم آیا B از Max بزرگ‌تر است یا نه. اگر بله:

$Max \leftarrow B$

در مرحله بعد همین کار را برای C انجام می‌دهیم.

در پایان، Max بزرگ‌ترین مقدار خواهد بود.



نکته مهم این مثال، روش به‌روزرسانی تدریجی یک مقدار است. در ابتدا فقط A را می‌شناسیم و آن را بزرگ‌ترین مقدار فعلی فرض می‌کنیم. سپس با مشاهده مقادیر جدید، در صورت نیاز مقدار Max را تغییر می‌دهیم. این ایده در بسیاری از الگوریتم‌های دیگر نیز استفاده می‌شود.

فلوچارت چاپ اعداد زوج بین m و n

حال مسئله‌ای را در نظر بگیریم که در آن باید تمام اعداد زوج بین m و n چاپ شوند. اگر m خودش زوج باشد، می‌توانیم از همان‌جا شروع کنیم. اگر فرد باشد، باید یک واحد به

آن اضافه کنیم تا به اولین عدد زوج برسیم:

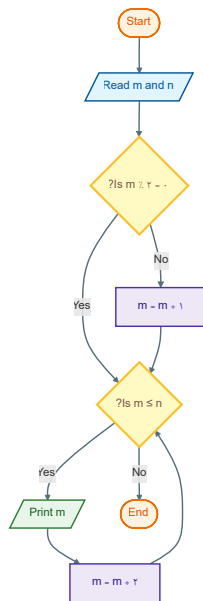
```
If  $m \% 2 \neq 0$   
     $m \leftarrow m + 1$ 
```

پس از پیدا کردن اولین عدد زوج، هر بار دو واحد به آن اضافه می‌کنیم:

```
 $m \leftarrow m + 2$ 
```

زیرا فاصله بین هر دو عدد زوج متوالی برابر ۲ است.

فلوچارت:



این مثال نمونه خوبی از ترکیب دو مفهوم مهم است:

- شرط
- حلقه

ابتدا با استفاده از شرط، نقطه شروع را اصلاح می‌کنیم و سپس با استفاده از حلقه، اعداد موردنظر را یکی پس از دیگری تولید می‌کنیم.

فلوچارت معکوس کردن یک عدد

یکی دیگر از مسائل کلاسیک الگوریتمی، معکوس کردن رقم‌های یک عدد است. برای مثال:

123 → 321

برای حل این مسئله باید بتوانیم آخرین رقم عدد را استخراج کنیم. برای این کار از باقی‌مانده تقسیم بر ۱۰ استفاده می‌کنیم.

اگر:

$N = 123$

باشد:

$123 \% 10 = 3$

بنابراین رقم آخر ۳ است. سپس با تقسیم صحیح بر ۱۰، رقم آخر را حذف می‌کنیم:

$123 / 10 = 12$

اکنون:

$$12 \% 10 = 2$$

و دوباره:

$$12 / 10 = 1$$

در مرحله بعد:

$$1 \% 10 = 1$$

و:

$$1 / 10 = 0$$

پس رقم‌ها را به ترتیب:

$$3 \rightarrow 2 \rightarrow 1$$

به دست آورده‌ایم.

اما برای ساختن عدد 321 باید رقم‌ها را در جای مناسب قرار دهیم.

از این رابطه استفاده می‌کنیم:

$$Reverse = Reverse \times 10 + Digit$$

در نتیجه:

$$Reverse = 0$$

Digit = 3

Reverse = $0 \times 10 + 3 = 3$

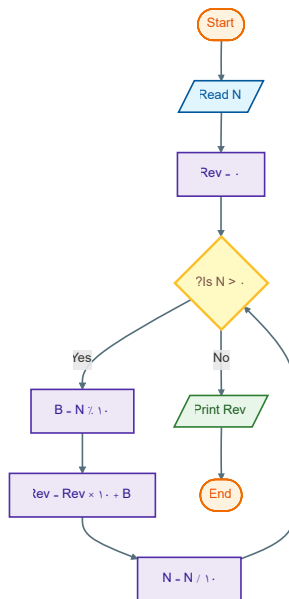
Digit = 2

Reverse = $3 \times 10 + 2 = 32$

Digit = 1

Reverse = $32 \times 10 + 1 = 321$

فلوچارت:



چرا فلوچارت‌ها مهم هستند؟

فلوچارت صرفاً یک روش تزئینی برای زیباتر کردن الگوریتم نیست. یکی از مزیت‌های اصلی آن این است که ساختار منطقی مسئله را قابل مشاهده می‌کند.

برای مثال، در یک فلوچارت می‌توان به سرعت دید:

- الگوریتم از کجا شروع می‌شود؟
- چه داده‌هایی دریافت می‌شوند؟
- چه پردازش‌هایی انجام می‌شوند؟
- تصمیم‌گیری در کجا اتفاق می‌افتد؟
- در چه نقطه‌ای حلقه ایجاد شده است؟
- هر مسیر در نهایت به کجا می‌رسد؟

این ویژگی برای آموزش برنامه‌نویسی و طراحی الگوریتم بسیار مفید است.

با این حال، برای برنامه‌های بسیار بزرگ و پیچیده، فلوچارت‌های عظیم می‌توانند خوانایی خود را از دست بدهند. به همین دلیل در پروژه‌های نرم‌افزاری مدرن، بسته به نوع مسئله، از ابزارها و روش‌های دیگری مانند شبه‌کد، نمودارهای UML، مستندات فنی و خود کد استفاده می‌شود.

۹ جمع‌بندی فصل

در این فصل با مفهوم الگوریتم و روش‌های مختلف نمایش و طراحی آن آشنا شدیم. ابتدا دیدیم که الگوریتم مجموعه‌ای از دستورهای دقیق و مرحله‌به‌مرحله برای حل یک مسئله است و باید دارای دستورهای مشخص، قابل اجرا و بدون ابهام باشد و در تعداد محدودی مرحله به پایان برسد. سپس با نحوه‌ی تعریف مسئله، تعیین ورودی و خروجی و طراحی راه‌حل آشنا شدیم و اهمیت بررسی الگوریتم با داده‌های نمونه (Trace) را پیش از کدنویسی دیدیم. در ادامه، شبه‌کد را به‌عنوان روشی ساده و مستقل از زبان برنامه‌نویسی برای بیان منطق الگوریتم بررسی کردیم و با مفاهیمی مانند متغیر، عملگر انتساب، شرط و حلقه آشنا شدیم. سپس فلوچارت را به‌عنوان نمایش تصویری الگوریتم شناختیم و کاربرد نمادهای مختلف آن مانند بیضی برای شروع و پایان، مستطیل برای پردازش، لوزی برای تصمیم‌گیری و متوازی‌الاضلاع برای ورودی و خروجی را بررسی کردیم. در نهایت، دیدیم که با ترکیب شرط‌ها و حلقه‌ها می‌توان الگوریتم‌هایی برای حل مسائل پیچیده‌تر طراحی کرد. در فصل‌های بعدی، همین الگوریتم‌ها را به‌صورت برنامه و کد قابل اجرا پیاده‌سازی خواهیم کرد.

عملگرها

فصل چهارم



۱. مقدمه

در فصل گذشته با مفهوم الگوریتم و فلوچارت آشنا شدیم و دیدیم که یک مسئله را چگونه می‌توان به مجموعه‌ای از مراحل دقیق و قابل اجرا تبدیل کرد. همچنین با مفاهیمی مانند شرط، حلقه، ورودی، خروجی و انتساب آشنا شدیم. اما برای نوشتن الگوریتم‌های واقعی، به ابزارهای بیشتری نیاز داریم. الگوریتم باید بتواند داده‌ها را نگهداری کند، روی آن‌ها محاسبه انجام دهد، مقادیر را با یکدیگر مقایسه کند و بر اساس نتیجه این مقایسه‌ها تصمیم بگیرد.

دو مفهوم اساسی که در این فصل بررسی می‌کنیم عبارت‌اند از:

- متغیرها (Variables) برای نگهداری داده‌ها
- عملگرها (Operators) برای انجام عملیات روی داده‌ها

در ادامه، این مفاهیم را با یکدیگر ترکیب می‌کنیم و چند مسئله کلاسیک را به صورت الگوریتم و فلوچارت حل خواهیم کرد.

۲. متغیرها (Variables)

کامپیوتر هنگام اجرای یک برنامه دائماً با داده‌ها کار می‌کند. این داده‌ها ممکن است عدد، متن، نتیجه یک محاسبه، وضعیت یک شرط یا اطلاعات دیگری باشند. برای اینکه بتوانیم در طول اجرای برنامه به این داده‌ها دسترسی داشته باشیم، باید آن‌ها را در حافظه نگهداری کنیم. یکی از ساده‌ترین راه‌ها برای تصور این موضوع، در نظر گرفتن حافظه به شکل مجموعه‌ای از محل‌های ذخیره‌سازی است. فرض کنید حافظه را مانند قفسه‌ای با تعداد زیادی کشو در نظر بگیریم. هر کشو می‌تواند مقداری را در خود نگه دارد و برای دسترسی راحت‌تر، می‌توانیم نامی برای آن در نظر بگیریم.

در برنامه‌نویسی، **متغیر (Variable)** نامی است که برای اشاره به یک مقدار ذخیره‌شده در برنامه استفاده می‌کنیم.

برای مثال:

```
age = 25
```

در اینجا تغییری به نام **age** داریم که مقدار آن **25** است.

یا:

```
total = 0
```

در اینجا **total** می‌تواند برای نگهداری مجموع اعداد استفاده شود. متغیرها فقط برای نگهداری مقدار اولیه نیستند؛ مقدار یک متغیر می‌تواند در طول اجرای برنامه تغییر کند. مثلاً:

```
x = 10
```

$$x = 20$$

پس از اجرای دستور دوم، مقدار فعلی x برابر ۲۰ است. به همین دلیل به آن متغیر گفته می‌شود؛ زیرا مقدار آن می‌تواند در طول اجرای برنامه تغییر کند.

۲.۱. متغیر به عنوان یک نام برای داده

برای درک بهتر متغیر، به این مثال توجه کنید:

$$\text{score} = 18$$

می‌توانیم این عبارت را به صورت مفهومی چنین بخوانیم:

مقدار ۱۸ را در متغیر score قرار بده.

بعداً می‌توانیم از نام score استفاده کنیم:

$$\text{score} + 2$$

در این حالت برنامه مقدار موجود در score را پیدا می‌کند و از آن در محاسبه استفاده می‌کند.

اگر:

$$\text{score} = 18$$

باشد، نتیجه عبارت:

```
score + 2
```

برابر با ۲۰ خواهد بود.

بنابراین متغیر باعث می‌شود لازم نباشد مقدار را هر بار مستقیماً در الگوریتم تکرار کنیم.

۲.۲ نام‌گذاری متغیرها

برای استفاده از متغیر باید یک نام برای آن انتخاب کنیم. قواعد دقیق نام‌گذاری متغیرها به زبان برنامه‌نویسی بستگی دارد؛ بنابراین نمی‌توان یک مجموعه قانون کاملاً یکسان را برای تمام زبان‌ها در نظر گرفت. با این حال، در بسیاری از زبان‌های رایج، قواعدی شبیه موارد زیر وجود دارد.

۲.۲.۱ نام معمولاً باید با حرف یا زیرخط شروع شود

برای مثال:

```
age  
score  
_total  
student_name
```

اما نامی مانند:

```
2students
```

در بسیاری از زبان‌های رایج نامعتبر است، زیرا با عدد شروع شده است.

۲.۲.۲. فاصله معمولاً مجاز نیست

برای مثال:

```
student name
```

نمی‌تواند در بسیاری از زبان‌ها یک نام متغیر معتبر باشد.

به جای آن معمولاً از روش‌هایی مانند زیر استفاده می‌شود:

```
student_name
```

یا:

```
studentName
```

روش اول را معمولاً **snake_case** و روش دوم را **camelCase** می‌نامند.

۲.۲.۳. حروف بزرگ و کوچک ممکن است متفاوت باشند

در بسیاری از زبان‌های برنامه‌نویسی، نام‌های زیر دو متغیر متفاوت محسوب می‌شوند:

```
sum
```

```
Sum
```

```
SUM
```

این ویژگی را حساسیت به بزرگی و کوچکی حروف (**Case Sensitivity**) می‌نامیم.

برای مثال اگر بنویسیم:

```
sum = 10
```

و سپس:

```
print(Sum)
```

ممکن است برنامه `Sum` را متغیری متفاوت از `sum` در نظر بگیرد.

۲.۲.۴. کلمات کلیدی قابل استفاده به عنوان نام متغیر نیستند

زبان‌های برنامه‌نویسی کلماتی دارند که معنای خاصی برای خود زبان دارند. برای مثال در زبان‌هایی مانند `C` و `Python` کلماتی مانند:

```
if  
while  
for
```

دارای معنای مشخصی هستند.

به چنین کلماتی **کلمات کلیدی (Keywords)** گفته می‌شود و معمولاً نمی‌توان از آن‌ها به عنوان نام متغیر استفاده کرد.

۲.۲.۵. انتخاب نام مناسب

علاوه بر رعایت قواعد نحوی، بهتر است نام متغیر معنی مشخصی داشته باشد.

مثلاً:

```
x = 25
```

از نظر فنی ممکن است کاملاً معتبر باشد، اما اگر این مقدار نشان‌دهنده سن باشد، نام:

```
age = 25
```

خوانایی بیشتری دارد.

یا به جای:

```
s = 0
```

اگر متغیر قرار است مجموع را نگهداری کند، بهتر است بنویسیم:

```
sum = 0
```

نام‌گذاری مناسب باعث می‌شود الگوریتم و کد برای خودمان و دیگران قابل فهم‌تر باشد.

۳. عملگرها (Operators)

متغیرها به ما اجازه می‌دهند داده‌ها را نگهداری کنیم؛ اما برای انجام کار روی این داده‌ها به **عملگرها (Operators)** نیاز داریم. عملگر نمادی است که نوع خاصی از عملیات را روی یک یا چند مقدار مشخص می‌کند.

برای مثال:

$$5 + 3$$

در این عبارت:

- 5 و 3 عملوند (Operand) هستند.
- + عملگر (Operator) است.

عملگرها می‌توانند برای محاسبات ریاضی، مقایسه مقادیر، ترکیب شرایط منطقی، عملیات بیتی و موارد دیگر استفاده شوند. در ادامه مهم‌ترین دسته‌های عملگر را بررسی می‌کنیم.

۳.۱ عملگرهای حسابی (Arithmetic Operators)

عملگرهای حسابی برای انجام محاسبات عددی استفاده می‌شوند.

عملگر	نام	مثال	نتیجه
+	جمع	$3 + 5$	8
-	تفریق	$3 - 5$	2
*	ضرب	$3 * 5$	15
/	تقسیم	$2 / 7$	وابسته به زبان: معمولاً 3.5 در تقسیم معمول

عملگر	نام	مثال	نتیجه
//	تقسیم صحیح	2 // 7	3 در زبان‌هایی که این عملگر را دارند
%	باقیمانده	2 % 7	1
**	توان	3 ** 2	8 در زبان‌هایی که این نماد را دارند

نکته مهم این است که همه زبان‌های برنامه‌نویسی دقیقاً از همین نمادها استفاده نمی‌کنند. جدول بالا بیشتر بر اساس نمادهای رایج در زبان‌هایی مانند Python تنظیم شده است. در زبان C، برای مثال، ** عملگر توان نیست و برای محاسبه توان معمولاً از تابعی مانند pow استفاده می‌شود.

۳.۱.۱ جمع، تفریق و ضرب

سه عملگر اصلی تقریباً در همه زبان‌های رایج به شکل زیر نوشته می‌شوند:

$a + b$

$a - b$

$a * b$

مثلاً:

$a = 10$

$b = 4$

آنگاه:

$a + b = 14$

$$a - b = 6$$

$$a * b = 40$$

۳.۱.۲ تقسیم

تقسیم کمی مهم‌تر است، زیرا نحوه رفتار آن به نوع داده و زبان برنامه‌نویسی بستگی دارد.

برای مثال:

$$7 / 2$$

در بعضی زبان‌ها یا در بعضی شرایط ممکن است نتیجه 3.5 باشد، در حالی که تقسیم صحیح ممکن است نتیجه 3 بدهد.

در Python:

$$7 / 2$$

نتیجه:

$$3.5$$

9:

$$7 // 2$$

نتیجه:

اما در زبان‌هایی مانند C، رفتار $/$ به نوع داده‌های دو طرف بستگی دارد. بنابراین هنگام مطالعه یک زبان برنامه‌نویسی خاص باید قواعد همان زبان را بررسی کرد.

۳.۱.۳. عملگر باقیمانده %

عملگر % یکی از مهم‌ترین عملگرها در الگوریتم‌نویسی است. این عملگر باقی‌مانده تقسیم را به دست می‌آورد.

برای مثال:

$$7 = 3 \times 2 + 1 \quad 7 = 3 \times 2 + 1$$

بنابراین:

$$7 \% 3 = 1$$

همچنین:

$$10 \% 2 = 0$$

زیرا ۱۰ دقیقاً بر ۲ بخش‌پذیر است. این ویژگی باعث می‌شود % در مسائل زیادی کاربرد داشته باشد.

تشخیص زوج و فرد

اگر:

$$N \% 2 = 0$$

باشد، N زوج است.

اگر:

$$N \% 2 \neq 0$$

باشد، N فرد است.

برای مثال:

$$18 \% 2 = 0$$

پس ۱۸ زوج است.

اما:

$$19 \% 2 = 1$$

پس ۱۹ فرد است.

استخراج آخرین رقم

برای اعداد صحیح مثبت، با استفاده از:

$$N \% 10$$

می‌توان رقم آخر را به دست آورد.

مثلاً:

$$123 \% 10 = 3$$

این تکنیک در فصل قبل هنگام بررسی الگوریتم معکوس کردن عدد نیز استفاده شد.

۳.۲. عملگرهای مقایسه‌ای (Relational Operators)

گاهی لازم نیست محاسبه‌ای انجام دهیم؛ بلکه می‌خواهیم بدانیم رابطه بین دو مقدار چیست.

مثلاً:

آیا a بزرگ‌تر از b است؟

برای این کار از عملگرهای مقایسه‌ای استفاده می‌کنیم.

عملگر	معنی	مثال	نتیجه با $a=5$ و $b=3$
$==$	برابر است با	$a == b$	False
$!=$	نا برابر است	$a \neq b$	True
$<$	بزرگ‌تر از	$a > b$	True
$>$	کوچک‌تر از	$a < b$	False
$==>$	بزرگ‌تر یا مساوی	$a \geq 5$	True
$=<$	کوچک‌تر یا مساوی	$a \leq 3$	False

نتیجه یک مقایسه معمولاً یک مقدار منطقی یا **Boolean** است:

True

یا:

False

۳.۲.۱ تفاوت `=` و `==`

این یکی از مهم‌ترین نکات ابتدایی برنامه‌نویسی است. در بسیاری از زبان‌ها:

`=`

برای انتساب استفاده می‌شود.

مثلاً:

`x = 5`

یعنی مقدار ۵ را در `x` قرار بده.

اما:

`==`

برای مقایسه برابری استفاده می‌شود.

مثلاً:

```
x == 5
```

یعنی:

آیا مقدار x برابر ۵ است؟

فرض کنید:

```
x = 5
```

اکنون:

```
x == 5
```

نتیجه:

```
True
```

اما:

```
x == 8
```

نتیجه:

```
False
```

می‌شود.

بنابراین می‌توان این دو را این‌گونه به خاطر سپرد:

= → مقدار قرار بده

== → سؤال بپرس

۳.۳. عملگرهای منطقی (Logical Operators)

گاهی یک شرط به تنهایی کافی نیست. فرض کنید می‌خواهیم بررسی کنیم آیا هر دو عدد مثبت هستند.

دو شرط داریم:

`a > 0`

`b > 0`

و باید هر دو درست باشند. برای ترکیب این شرط‌ها از عملگرهای منطقی استفاده می‌کنیم. سه عملگر اصلی عبارت‌اند از:

- AND
- OR
- NOT

نماد دقیق آن‌ها بسته به زبان متفاوت است.

برای مثال:

- در بسیاری از زبان‌ها مانند C از `&&` ، `||` و `!` استفاده می‌شود.
- در Python از `and` ، `or` و `not` استفاده می‌شود.

۳.۳.۱. عملگر AND

عملگر **AND** یا «و» تنها زمانی درست است که تمام شرط‌های موردنظر درست باشند. جدول زیر رفتار آن را نشان می‌دهد:

شرط اول	شرط دوم	AND
درست	درست	درست
درست	نادرست	نادرست
نادرست	درست	نادرست
نادرست	نادرست	نادرست

مثلاً:

$$(a > 0) \text{ AND } (b > 0)$$

فقط زمانی درست است که هم **a** مثبت باشد و هم **b**.

۳.۳.۲. عملگر OR

عملگر **OR** یا «یا» زمانی درست است که حداقل یکی از شرط‌ها درست باشد.

شرط اول	شرط دوم	OR
درست	درست	درست
درست	نادرست	درست

شرط اول	شرط دوم	OR
نادرست	درست	درست
نادرست	نادرست	نادرست

مثلاً:

$(a > 0) \text{ OR } (b > 0)$

اگر حتی یکی از دو عدد مثبت باشد، نتیجه درست خواهد بود.

۳.۳.۳. عملگر NOT

عملگر NOT نتیجه یک شرط را معکوس می‌کند.

اگر:

$a > 0$

درست باشد، آنگاه:

$\text{NOT } (a > 0)$

نادرست خواهد بود.

به صورت ساده:

$\text{NOT True} \rightarrow \text{False}$

NOT False $\rightarrow$ True

مثال شرط مرکب

فرض کنید می‌خواهیم بررسی کنیم آیا هر دو عدد مثبت هستند.

الگوریتم:

Start

Read a and b

If $a > 0$ and $b > 0$ then

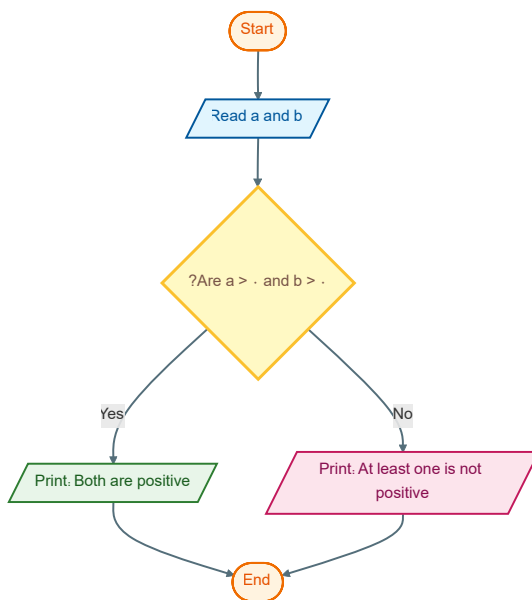
Print "Both are positive"

Otherwise

Print "At least one is not positive"

End

نمایش فلوجارت:



دقت کنید که عبارت «حداقل یکی منفی است» در متن اولیه کمی محدودتر از شرط واقعی بود؛ اگر یکی از دو عدد صفر باشد، شرط «هر دو مثبت هستند» نیز نادرست است، اما هیچ کدام الزاماً منفی نیستند. بنابراین عبارت دقیق تر، «حداقل یکی مثبت نیست» است.

۳.۴. عملگرهای یونری (Unary Operators)

برخی عملگرها برای کار کردن به دو عملوند نیاز دارند.

مثلاً:

$a + b$

در اینجا دو عملوند داریم: a و b .

اما عملگرهای یونری (Unary Operators) تنها روی یک مقدار یا متغیر عمل می‌کنند.

برای مثال:

$-x$

تنها روی x عمل می‌کند.

۳.۴.۱. عملگر منفی

عملگر $-$ در حالت یونری می‌تواند علامت یک عدد را تغییر دهد.
اگر:

$x = 5$

باشد:

$-x$

برابر با:

-5

است.

دقت کنید که این با تفریق تفاوت دارد:

```
a - b
```

در اینجا `-` یک عملگر دودویی است، زیرا دو عملوند دارد.

اما:

```
-x
```

یک عملگر یونری است.

۳.۴.۲. افزایش و کاهش: `++` و `--`

در زبان‌هایی مانند C، C++ و Java، عملگرهای زیر برای افزایش و کاهش یک واحدی استفاده می‌شوند:

```
i++
```

```
i--
```

`++` مقدار را یک واحد افزایش می‌دهد و `--` آن را یک واحد کاهش می‌دهد.

مثلاً:

```
i = 5
```

```
i++
```

پس از اجرای دستور، مقدار `i` برابر ۶ خواهد بود.

9:

```
i = 5
i--
```

مقدار را به ۴ تبدیل می‌کند.

نکته مهم درباره Python

C عملگرهای ++ و -- را به معنای افزایش و کاهش یک واحدی مانند Python معمولاً می‌نویسیم Python پشتیبانی نمی‌کند. در

```
i += 1
```

یا:

```
i -= 1
```

بنابراین هنگام مطالعه یک زبان خاص باید همیشه قواعد همان زبان را در نظر گرفت.

۳.۴.۳. پیشوند و پسوند

در زبان‌هایی که ++ را پشتیبانی می‌کنند، دو حالت وجود دارد:

```
++i
```

:9

```
i++
```


هر دو در نهایت مقدار **i** را یک واحد افزایش می‌دهند، اما در صورتی که درون یک عبارت بزرگ‌تر استفاده شوند، زمان استفاده از مقدار قدیمی و جدید متفاوت است.

فرض کنید:

```
i = 5
```

در:

```
x = ++i
```

ابتدا **i** افزایش می‌یابد:

```
i = 6
```

و سپس مقدار جدید در **x** قرار می‌گیرد:

```
x = 6
```

اما:

```
x = i++
```

ابتدا مقدار فعلی **i** در عبارت استفاده می‌شود:

```
x = 5
```

و سپس **i** افزایش پیدا می‌کند:

$$i = 6$$

به همین دلیل، برای جلوگیری از سردرگمی، در کدهای آموزشی بهتر است این تفاوت را با مثال‌های ساده تمرین کنیم.

۳.۴.۵. عملگرهای انتساب ترکیبی

گاهی می‌خواهیم مقدار یک متغیر را با استفاده از مقدار فعلی خودش تغییر دهیم.

برای مثال:

$$x = x + 5$$

یعنی ۵ واحد به مقدار فعلی x اضافه کن.

بسیاری از زبان‌ها شکل کوتاه‌تری برای این عملیات ارائه می‌کنند:

$$x += 5$$

این دو عبارت از نظر نتیجه معادل‌اند.

عملگر	شکل کوتاه	شکل معادل
$+=$	$x += 5$	$x = x + 5$
$-=$	$x -= 3$	$x = x - 3$
$*=$	$x *= 2$	$x = x * 2$
$/=$	$x /= 4$	$x = x / 4$
$\% =$	$x \% = 3$	$x = x \% 3$

برای مثال:

```
score = 10  
score += 5
```

نتیجه:

```
score = 15
```

یا:

```
counter = 10  
counter -= 2
```

نتیجه:

```
counter = 8
```

این عملگرها در حلقه‌ها بسیار پرکاربرد هستند.

۴. اولویت عملگرها (Operator Precedence)

وقتی چند عملگر در یک عبارت وجود داشته باشد، باید مشخص باشد که کدام عملیات زودتر انجام می‌شود.

برای مثال:

$$2 + 3 * 4$$

اگر ابتدا جمع را انجام دهیم، نتیجه می‌شود:

$$5 * 4 = 20$$

اما اگر ابتدا ضرب را انجام دهیم:

$$2 + 12 = 14$$

نتیجه صحیح بر اساس قواعد معمول اولویت عملگرها 14 است. بنابراین کامپیوتر باید ترتیب مشخصی برای ارزیابی عبارت‌ها داشته باشد.

۴.۱. پرانتز

پرانتز معمولاً بالاترین اولویت را دارد.

مثلاً:

$$(2 + 3) * 4$$

ابتدا:

$$2 + 3 = 5$$

و سپس:

$$5 * 4 = 20$$

بنابراین استفاده از پرانتز علاوه بر تغییر ترتیب محاسبات، باعث خوانایی بیشتر عبارت نیز می‌شود.

۴.۲. ترتیب کلی اولویت

ترتیب دقیق اولویت‌ها به زبان برنامه‌نویسی بستگی دارد، اما در بسیاری از زبان‌ها می‌توان یک ساختار کلی شبیه زیر مشاهده کرد:

سطح	گروه عملگرها
۱	پرانتزها و عبارت‌های درون آن‌ها
۲	عملگرهای یونی و نقیض
۳	توان، در زبان‌هایی که چنین عملگری دارند
۴	ضرب، تقسیم و باقیمانده
۵	جمع و تفریق
۶	مقایسه‌های $<$ ، $>$ ، $\leq$ ، $\geq$
۷	برابری و نابرابری $=$ و $\neq$

سطح	گروه عملگرها
۸	AND
۹	OR
۱۰	انتساب

این جدول را نباید به عنوان قانون یکسان برای تمام زبان‌ها حفظ کرد؛ زیرا جزئیات **Operator Precedence** میان زبان‌ها متفاوت است. در کدهای مهم و پیچیده، استفاده از پرانتز می‌تواند از بسیاری از اشتباه‌ها جلوگیری کند.

چند مثال برای اولویت عملگرها

فرض کنید:

$$x = 2 + 3 * 4$$

ابتدا ضرب انجام می‌شود:

$$3 * 4 = 12$$

سپس:

$$2 + 12 = 14$$

پس:

$$x = 14$$

اما اگر بنویسیم:

$$x = (2 + 3) * 4$$

ابتدا پرانتز:

$$2 + 3 = 5$$

و سپس:

$$5 * 4 = 20$$

بنابراین:

$$x = 20$$

مثال دیگر:

$$10 + 6 / 2$$

ابتدا تقسیم:

$$6 / 2 = 3$$

سپس:

$$10 + 3 = 13$$

۵. عملگرهای بیتی (Bitwise Operators)

یکی از موضوعات مهم در مبانی علوم کامپیوتر، عملیات بیتی (Bitwise Operations) است. در فصل مربوط به سیستم‌های عددی دیدیم که اطلاعات در سطح پایین کامپیوتر به صورت بیت‌هایی شامل 0 و 1 نمایش داده می‌شوند.

برای مثال عدد ده در مبنای دو برابر است با:

$$(1010)_2 = 10_{10}$$

بنابراین عدد ۱۰ را می‌توان به صورت چهار بیت زیر مشاهده کرد:

1010

عملگرهای بیتی به جای اینکه دو عدد را صرفاً به عنوان مقادیر معمولی با یکدیگر مقایسه یا محاسبه کنند، روی بیت‌های تشکیل‌دهنده آن‌ها عملیات انجام می‌دهند.

برای مثال، اگر دو عدد زیر را داشته باشیم:

$$A = 12 = 1100_2$$

$$B = 10 = 1010_2$$

می‌توانیم بیت‌های متناظر آن‌ها را با یکدیگر بررسی کنیم.

۵.۱ عملگر AND بیتی

عملگر AND بیتی معمولاً با نماد:

&

نشان داده می‌شود. قاعده آن بسیار ساده است:

اگر هر دو بیت 1 باشند، نتیجه 1 است؛ در غیر این صورت نتیجه 0 است.

جدول:

A AND B	B	A
0	0	0
0	1	0
0	0	1
1	1	1

مثلاً:

```
1100
& 1010

1000
``````text
12 & 10 = 8
```

پس:

```
12 & 10 = 8
```

زیرا:

$$1000_2 = 8_{10}$$

## ۵.۲. عملگر OR بیتی

عملگر OR بیتی معمولاً با:

|

نشان داده می‌شود. قاعده آن:

اگر حداقل یکی از دو بیت 1 باشد، نتیجه 1 است.

مثلاً:

1100

| 1010

1110

بنابراین:

$$12 \mid 10 = 14$$

زیرا:

$$1110_2 = 14_{10}$$

### ۵.۳. عملگر XOR

یکی دیگر از عملگرهای مهم بیتی:

^

است که XOR نام دارد.

در XOR، نتیجه زمانی 1 است که دو بیت با یکدیگر متفاوت باشند.

A	B	A XOR B
۰	۰	۰
۰	۱	۱
۱	۰	۱
۱	۱	۰

مثلاً:

1100  
^ 1010  
  
0110

بنابراین:

$$12 \text{ XOR } 10 = 6$$

زیرا:

$$0110_2 = 6_{10}$$

## ۵.۴. عملگر NOT بیتی

عملگر NOT بیتی با:

$\sim$

نشان داده می‌شود.

این عملگر هر بیت را معکوس می‌کند:

$$0 \rightarrow 1$$

$$1 \rightarrow 0$$

مثلاً اگر تنها چهار بیت را در نظر بگیریم:

1010

پس از NOT:

0101

البته در کامپیوترها اعداد معمولاً با تعداد بیت مشخص و روش‌هایی مانند مکمل دو (Two's Complement) نمایش داده می‌شوند؛ بنابراین تفسیر عددی  $x \sim$  به تعداد بیت‌ها و نحوه نمایش اعداد صحیح در زبان موردنظر وابسته است.

## ۵.۵. شیفت بیتی (Bit Shift)

دو عملگر مهم دیگر، شیفت به چپ و راست هستند. در بسیاری از زبان‌ها:

<<

برای شیفت به چپ و:

>>

برای شیفت به راست استفاده می‌شود.

فرض کنید:

$$8 = 1000_2$$

اگر یک بیت به چپ شیفت دهیم:

$$1000 \ll 1$$

در نمایش معمول عدد صحیح نتیجه:

$$10000$$

خواهد بود که برابر ۱۶ است.

بنابراین برای این مثال:

$$8 \ll 1 = 16$$

به طور مشابه:

$$1000 \gg 1$$

به:

$$0100$$

تبدیل می‌شود که برابر ۴ است.

در شرایط معمول، شیفت به چپ یک بیت تقریباً معادل ضرب در ۲ و شیفت به راست یک بیت تقریباً معادل تقسیم صحیح بر ۲ است؛ البته این توضیح ساده‌شده است و در مورد اعداد منفی و سرریز باید جزئیات نمایش عدد را نیز در نظر گرفت.

### ۵.۵.۱ کاربردهای عملگرهای بیتی

عملگرهای بیتی در مسائل مختلف علوم کامپیوتر کاربرد دارند، از جمله:

- کار با پرچم‌ها و وضعیت‌های دودویی
- مدیریت بیت‌های یک عدد
- فشرده‌سازی و پردازش داده
- برخی الگوریتم‌های سطح پایین
- برنامه‌نویسی سیستم‌ها

- کنترل سخت‌افزار
  - رمزنگاری و برخی روش‌های پردازش داده
  - ساختمان داده‌ها و الگوریتم‌های خاص
- برای برنامه‌نویسی مقدماتی، مهم‌ترین هدف این است که بدانیم این عملگرها مستقیماً با نمایش بیتی داده‌ها کار می‌کنند.

## مثال جامع اول: محاسبه فاکتوریل

اکنون چند مفهوم این فصل را با یکدیگر ترکیب می‌کنیم. فاکتوریل عدد  $n$  با نماد  $n!$  نشان داده می‌شود و برابر حاصل ضرب اعداد صحیح مثبت از ۱ تا  $n$  است.

برای مثال:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

همچنین طبق تعریف:

$$0! = 1$$

### تحلیل مسئله

ورودی:

$n$

پردازش:

$$1 \times 2 \times 3 \times \dots \times n$$

خروجی:

$n!$

برای نگهداری نتیجه به متغیری مانند `fact` نیاز داریم. چون قرار است ضرب را از مقدار اولیه شروع کنیم، مقدار اولیه `fact` را برابر ۱ قرار می‌دهیم. دلیل این کار آن است که ۱ عنصر خنثی ضرب است:



$$1 \times x = x$$

اگر **fact** را صفر قرار دهیم، تمام ضرب‌های بعدی نیز صفر خواهند شد.

## الگوریتم

1. Start
2. Read  $n$ .
3.  $\text{fact} \leftarrow 1$
4.  $i \leftarrow 1$
5. If  $i \leq n$ , go to Step 6; otherwise, go to Step 9.
6.  $\text{fact} \leftarrow \text{fact} \times i$
7.  $i \leftarrow i + 1$
8. Go back to Step 5.
9. Print  $\text{fact}$ .
10. End

## اجرای الگوریتم برای $n = 5$

ابتدا:

```
fact = 1
i = 1
```

تکرار اول:

```
fact = 1 × 1 = 1
```

```
i = 2
```

تکرار دوم:

```
fact = 1 × 2 = 2
```

```
i = 3
```

تکرار سوم:

```
fact = 2 × 3 = 6
```

```
i = 4
```

تکرار چهارم:

```
fact = 6 × 4 = 24
```

```
i = 5
```

تکرار پنجم:

```
fact = 24 × 5 = 120
```

```
i = 6
```

اکنون:

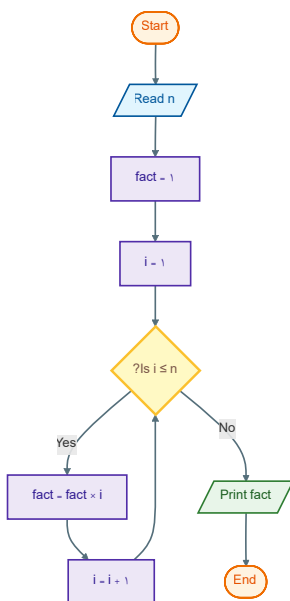
$$i \leq n$$

$$6 \leq 5$$

نادرست است؛ بنابراین حلقه پایان می‌یابد و مقدار **fact** چاپ می‌شود:

120

## فلوچارت



در این فلوچارت، یک حلقه داریم که تا زمانی که شرط  $i \leq n$  برقرار است، ادامه پیدا می‌کند.

## مثال جامع دوم: یافتن بزرگ‌ترین عدد بین سه عدد

فرض کنید سه عدد  $a$ ،  $b$  و  $c$  داریم و می‌خواهیم بزرگ‌ترین آن‌ها را پیدا کنیم.

یک روش، استفاده از شرط‌های تو در تو است.

ابتدا بررسی می‌کنیم:

$a \geq b$  ؟ آیا

اگر پاسخ مثبت باشد، فقط لازم است  $a$  و  $c$  را مقایسه کنیم.

اگر پاسخ منفی باشد، می‌دانیم  $b$  از  $a$  بزرگ‌تر است یا حداقل برابر نیست؛ بنابراین  $b$  را با  $c$  مقایسه می‌کنیم.

الگوریتم:

```
1. Start
2. Read a, b, and c.
3. If $a \geq b$ then:
 If $a \geq c$ then:
 $\text{max} \leftarrow a$
 Otherwise:
 $\text{max} \leftarrow c$
4. Otherwise:
 If $b \geq c$ then:
 $\text{max} \leftarrow b$
 Otherwise:
```

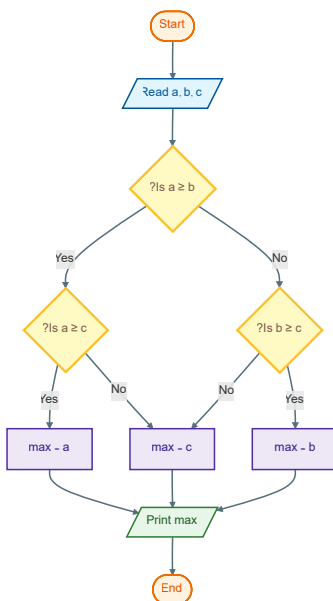
$\text{max} \leftarrow c$

5. Print max.

6. End

این مثال مفهوم شرط تو در تو (Nested Condition) را نشان می‌دهد؛ یعنی یک تصمیم‌گیری درون مسیر یکی از تصمیم‌گیری‌های دیگر قرار گرفته است.

## فلوچارت



## مثال جامع سوم: تشخیص عدد اول

عدد اول (Prime Number) عددی صحیح بزرگ‌تر از ۱ است که فقط دو مقسوم‌علیه مثبت دارد:

- ۱

- خودش

برای مثال، ۹ علاوه بر ۱ و خودش، بر ۳ نیز بخش‌پذیر است:  $9 = 3 \times 3$ . بنابراین اول نیست.

### روش ساده بررسی

برای یک روش ساده، می‌توانیم اعداد بین ۲ و  $n/2$  را بررسی کنیم. اگر یکی از این اعداد بتواند  $n$  را بدون باقی‌مانده تقسیم کند،  $n$  اول نیست. مثلاً برای:

$$n = 15$$

بررسی می‌کنیم:

$$15 \% 2$$

$$15 \% 3$$

...

در مرحله ۳ داریم:

$$15 \% 3 = 0$$

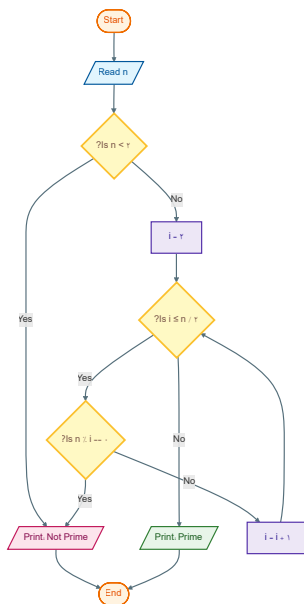
پس ۱۵ اول نیست.

## الگوریتم

1. Start
2. Read n.
3. If  $n < 2$ :  
    Print "Not prime"  
    End
4.  $i \leftarrow 2$
5. If  $i > n / 2$ :  
    Print "Prime"  
    End
6. If  $n \% i = 0$ :  
    Print "Not prime"  
    End
7.  $i \leftarrow i + 1$
8. Go back to Step 5.

## فلوچارت

---



## یک نکته مهم‌تر

روش بررسی تا  $n/2$  درست است، اما بهینه‌ترین روش نیست. در واقع برای تشخیص اول بودن یک عدد، کافی است مقسوم‌علیه‌های ممکن را تا  $\sqrt{n}$  بررسی کنیم.

دلیل آن این است که اگر  $n = a \times b$  باشد، حداقل یکی از دو عامل  $a$  یا  $b$  باید کوچک‌تر یا مساوی  $\sqrt{n}$  باشد.

برای مثال، در عدد  $36 = 6 \times 6$  و اگر یک عامل بزرگ‌تر از  $6 = \sqrt{36}$  باشد، عامل دیگر کوچک‌تر یا مساوی  $6$  خواهد بود.

بنابراین نسخه بهینه‌تر الگوریتم به جای:



$$i \leq n/2$$

از شرطی مانند:

$$i * i \leq n$$

استفاده می‌کند. این نکته نمونه خوبی از این موضوع است که یک مسئله ممکن است چند الگوریتم درست داشته باشد، اما هزینه اجرای آن‌ها متفاوت باشد.

## ۶. جمع‌بندی فصل

در این فصل با ابزارهای اصلی مورد استفاده برای ساخت الگوریتم‌های قابل اجرا آشنا شدیم. ابتدا مفهوم متغیر را بررسی کردیم و دیدیم که متغیر نامی برای دسترسی به داده‌ای است که در طول اجرای برنامه نگهداری می‌شود و مقدار آن می‌تواند تغییر کند. سپس با قواعد نام‌گذاری متغیرها و تفاوت حروف بزرگ و کوچک در بسیاری از زبان‌های برنامه‌نویسی آشنا شدیم. در ادامه، عملگرها و کاربرد آن‌ها را بررسی کردیم؛ از عملگرهای حسابی و % برای محاسبات و باقی‌مانده‌ی تقسیم گرفته تا عملگرهای مقایسه‌ای، انتساب، منطقی و یونری. همچنین تفاوت مهم میان = و ==، عملگرهای انتساب ترکیبی مانند += و \*= و محدودیت استفاده از ++ و -- در زبان‌هایی مانند Python را شناختیم. سپس مفهوم اولویت عملگرها و نقش پرانتز در تعیین صریح ترتیب محاسبات را بررسی کردیم و با عملگرهای بیتی مانند AND، OR، XOR، NOT و شیفت‌های بیتی آشنا شدیم. در نهایت، دیدیم که ترکیب متغیرها و عملگرها با ساختارهایی مانند شرط‌ها و حلقه‌ها امکان طراحی الگوریتم‌هایی برای حل مسائل مختلف، مانند محاسبه‌ی فاکتوریل، پیدا کردن بزرگ‌ترین مقدار و تشخیص عدد اول را فراهم می‌کند. در فصل‌های بعدی، همین ابزارها را در قالب ساختارهای اصلی برنامه‌نویسی مانند شرط‌ها، حلقه‌ها، توابع و آرایه‌ها به کار خواهیم گرفت و گام‌به‌گام از الگوریتم‌های ساده به سمت برنامه‌های واقعی حرکت خواهیم کرد.

# شرط

فصل پنجم



## ۱. مقدمه

تا اینجا یاد گرفتیم که الگوریتم مجموعه‌ای از دستورهای مشخص برای حل یک مسئله است. در ساده‌ترین حالت، دستورهای یک الگوریتم می‌توانند یکی پس از دیگری اجرا شوند:

دستور اول

↓

دستور دوم

↓

دستور سوم

↓

دستور چهارم

به این نوع اجرا، **اجرای ترتیبی (Sequential Execution)** گفته می‌شود. اما بسیاری از مسائل واقعی چنین ساختار ساده‌ای ندارند. در زندگی روزمره نیز تصمیم‌های مختلفی می‌گیریم:

اگر هوا بارانی بود، چتر برمی‌دارم؛

اگر بارانی نبود، چتر برنمی‌دارم.

یا:

اگر نمره دانشجو حداقل ۱۰ بود، قبول است؛

در غیر این صورت، مردود است.

یک برنامه کامپیوتری نیز باید بتواند چنین تصمیم‌هایی بگیرد. برای این کار از **ساختارهای شرطی (Conditional Statements)** استفاده می‌کنیم. ساختار شرطی به برنامه

اجازه می‌دهد بر اساس نتیجه یک عبارت منطقی، مسیر اجرای متفاوتی را انتخاب کند.

به عبارت ساده:

اگر یک شرط درست بود  
یک کار را انجام بده  
در غیر این صورت  
کار دیگری انجام بده

بنابراین شرط‌ها یکی از مهم‌ترین ابزارهای کنترل جریان اجرای برنامه (**Control**  
**Flow**) هستند.

## ۲. کنترل جریان برنامه

منظور از جریان اجرا، ترتیبی است که دستورهای برنامه در هنگام اجرای می‌کنند. در یک برنامه ساده، جریان معمولاً از بالا به پایین حرکت می‌کند:

دستور ۱

↓

دستور ۲

↓

دستور ۳

↓

دستور ۴

اما ساختارهای کنترلی می‌توانند این جریان را تغییر دهند. به طور کلی، سه مفهوم بسیار مهم در کنترل جریان عبارت‌اند از:

۱. توالی (Sequence) — اجرای دستورها به ترتیب

۲. انتخاب یا تصمیم‌گیری (Selection) — انتخاب یک مسیر بر اساس شرط

۳. تکرار (Iteration) — اجرای چندباره یک بخش از برنامه

ساختارهای شرطی مربوط به بخش دوم، یعنی انتخاب هستند و در عین حال نقش مهمی در کنترل بخش سوم، یعنی حلقه‌ها، دارند.

### ۳. شرط ساده: `if`

ساده‌ترین نوع ساختار شرطی، `if` است. در این ساختار، یک شرط بررسی می‌شود و اگر شرط درست باشد، مجموعه‌ای از دستورها اجرا می‌شوند. ساختار کلی آن در زبان‌هایی مانند C به صورت زیر است:

```
if (شرط) {
 // دستورهایی که در صورت درست بودن شرط اجرا
 می‌شوند
}
```

برای مثال:

```
if (age ≥ 18) {
 print("Adult")
}
```

منطق آن چنین است:

```
Is age ≥ 18?
|
Yes
↓
Print "Adult"
```

اگر شرط نادرست باشد، دستور داخل `if` اجرا نمی‌شود و برنامه به دستور بعدی می‌رود.

## مثال

فرض کنید عددی را از کاربر دریافت کرده‌ایم و می‌خواهیم اگر مثبت بود، پیام خاصی نمایش دهیم.

الگوریتم:

```
1. Read n.
2. If $n > 0$:
 Print "The number is positive."
3. End.
```

اگر:

```
n = 7
```

باشد، شرط:

```
n > 0
```

درست است و پیام نمایش داده می‌شود.

اما اگر:

```
n = -3
```

باشد، شرط نادرست است و برنامه بدون اجرای آن بخش ادامه پیدا می‌کند. نکته مهم این است که در `if` برای حالت نادرست الزاماً هیچ دستور خاصی نداریم.



## ۴. شرط دوگانه: `if-else`

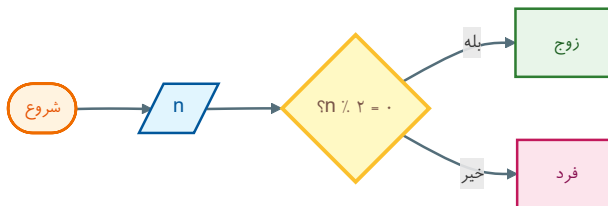
گاهی نمی‌خواهیم در صورت نادرست بودن شرط، برنامه هیچ کاری نکند. می‌خواهیم برای هر دو حالت، مسیر مشخصی داشته باشیم. در این شرایط از `if-else` استفاده می‌کنیم. ساختار کلی:

```
if (شرط) {
 // مسیر اول: شرط درست است
}
else {
 // مسیر دوم: شرط نادرست است
}
```

برای مثال، می‌توانیم زوج یا فرد بودن یک عدد را تشخیص دهیم:

```
if (n % 2 == 0) {
 print("زوج")
}
else {
 print("فرد")
}
```

در اینجا دقیقاً دو مسیر وجود دارد:



اگر شرط درست باشد، مسیر `if` اجرا می‌شود.

اگر شرط نادرست باشد، مسیر `else` اجرا می‌شود.



## ۵. شرط‌های چند گانه: `if - else if - else`

گاهی تنها دو حالت نداریم. برای مثال، فرض کنید می‌خواهیم بر اساس نمره دانشجو وضعیت او را تعیین کنیم:

- اگر نمره ۱۸ یا بیشتر باشد: عالی
- اگر نمره ۱۴ یا بیشتر باشد: خوب
- اگر نمره ۱۰ یا بیشتر باشد: قبول
- در غیر این صورت: مردود

در چنین مسئله‌ای چند شرط باید به ترتیب بررسی شوند.

ساختار کلی:

```
if (شرط اول) {
 // حالت اول
} else if (شرط دوم) {
 // حالت دوم
} else {
 هیچ‌کدام از شرط‌های قبلی درست نبودند
}
```

برای مثال:

```
if (score ≥ 18) {
 print("عالی")
}
```

```

} else if (score ≥ 14) {
 print("خوب")
} else if (score ≥ 10) {
 print("قبول")
} else {
 print("مردود")
}

```

نکته مهم این است که شرطها به ترتیب از بالا به پایین بررسی می‌شوند. به محض اینکه یکی از شرطها درست باشد، مسیر مربوط به آن اجرا می‌شود و شرطهای بعدی در همان ساختار دیگر بررسی نمی‌شوند.

## ۵.۱. ترتیب شرطها اهمیت دارد

فرض کنید می‌خواهیم همین دسته‌بندی نمره‌ها را انجام دهیم، اما ترتیب را اشتباه بنویسیم:

```

if (score ≥ 10) {
 print("قبول")
} else if (score ≥ 18) {
 print("عالی")
}

```

اگر **score** برابر ۱۹ باشد، شرط اول یعنی:

```
score ≥ 10
```

از قبل درست است. بنابراین برنامه همان ابتدا "قبول" را چاپ می‌کند و دیگر به شرط دوم نمی‌رسد. در نتیجه، در ساختارهای چندگانه باید به ترتیب و پوشش شرط‌ها دقت کنیم. این موضوع یکی از خطاهای رایج هنگام طراحی الگوریتم است.

## مثال: تشخیص مضرب ۲ یا ۳

اکنون یک مثال ترکیبی را بررسی کنیم.

### طرح مسئله

برنامه‌ای بنویسید که یک عدد را از کاربر دریافت کند:

- اگر عدد مضرب ۲ بود، پیام «مضرب ۲» را نمایش دهد.
  - اگر مضرب ۲ نبود اما مضرب ۳ بود، پیام «مضرب ۳» را نمایش دهد.
  - اگر هیچ‌کدام نبود، کاری انجام ندهد.
- برای بررسی مضرب بودن از عملگر  $\%$  استفاده می‌کنیم.

اگر:

$$n \% 2 == 0$$

باشد،  $n$  بر ۲ بخش‌پذیر است.

همچنین:

$$n \% 3 == 0$$

نشان می‌دهد که عدد بر ۳ بخش‌پذیر است.

## الگوریتم

```
1. Start
2. Read n.
3. If $n \% 2 == 0$:
 Print "Multiple of 2."
4. Otherwise, if $n \% 3 == 0$:
 Print "Multiple of 3."
5. End
```

در اینجا از ساختار:

```
if
else if
```

استفاده کرده‌ایم.

### نکته مهم درباره صورت مسئله

فرض کنید عدد:

```
n = 6
```

باشد. ۶ هم مضرب ۲ است و هم مضرب ۳. اما طبق الگوریتم بالا، چون شرط اول برقرار است، فقط:

## مضرب ۲

نمایش داده می‌شود.

اگر هدف مسئله این باشد که تمام ویژگی‌های عدد را گزارش کنیم، باید دو `if` مستقل داشته باشیم:

```
if (n % 2 == 0) {
 print("Multiple of 2");
}

if (n % 3 == 0) {
 print("Multiple of 3");
}
```

در این حالت برای ۶ هر دو پیام نمایش داده می‌شوند.

پس تفاوت میان:

```
if ... else if
```

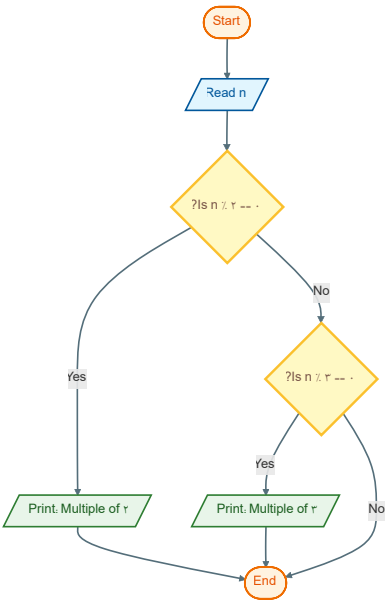
:

```
if
if
```

مهم است.

در ساختار اول، معمولاً فقط یکی از مسیرها انتخاب می‌شود؛ اما در ساختار دوم، هر شرط به صورت مستقل بررسی می‌شود.

## فلوچارت





## ۶. شرط‌ها و حلقه‌ها

تا اینجا شرط را برای تصمیم‌گیری مستقیم بررسی کردیم. اما شرط فقط برای انتخاب بین دو یا چند مسیر استفاده نمی‌شود. یکی از مهم‌ترین کاربردهای شرط، **کنترل حلقه‌ها** است. حلقه به برنامه اجازه می‌دهد یک مجموعه دستور را چندین بار اجرا کند.

اما یک سؤال اساسی وجود دارد:

*برنامه از کجا بفهمد چند بار باید حلقه را اجرا کند؟*

پاسخ معمولاً یک شرط است.

برای مثال:

```
While $i \leq M$:
 Print i
 Increment i by 1
```

شرط:

$i \leq M$

تعیین می‌کند که حلقه ادامه پیدا کند یا متوقف شود. بنابراین می‌توان گفت شرط در یک حلقه نقش کنترل‌کننده ادامه یا پایان تکرار را دارد.

**مثال: چاپ اعداد از ۱ تا  $M$**

طرح مسئله

عدد  $M$  را از کاربر دریافت کنید و تمام اعداد از ۱ تا  $M$  را به ترتیب چاپ کنید.

فرض کنید:

$$M = 5$$

خروجی باید باشد:

```
1
2
3
4
5
```

برای حل مسئله، یک متغیر شمارنده به نام  $i$  تعریف می‌کنیم.

ابتدا:

$$i = 1$$

سپس بررسی می‌کنیم:

$$i \leq M \text{ ؟}$$

اگر بله،  $i$  را چاپ می‌کنیم و بعد:

$$i = i + 1$$

را انجام می‌دهیم.

این فرایند تا زمانی ادامه پیدا می‌کند که شرط دیگر درست نباشد.

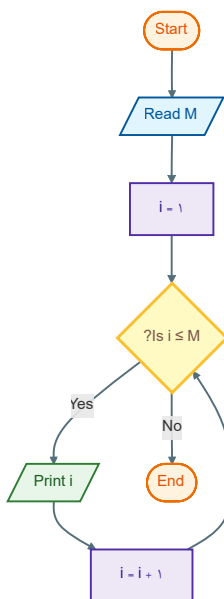
## اجرای مرحله به مرحله برای $M = 5$

مرحله	مقدار $i$	آیا $i \leq 5$ ؟	خروجی
۱	۱	بله	۱
۲	۲	بله	۲
۳	۳	بله	۳
۴	۴	بله	۴
۵	۵	بله	۵
۶	۶	خیر	—

وقتی  $i$  به ۶ می‌رسد:

$$6 \leq 5$$

نا درست است. در نتیجه حلقه متوقف می‌شود.



در این فلوچارت، سه قسمت بسیار مهم وجود دارد:

مقداردهی اولیه



بررسی شرط



تغییر شمارنده



اگر تغییر شمارنده را فراموش کنیم، ممکن است شرط همیشه درست باقی بماند و حلقه بی‌نهایت ایجاد شود.

## حلقه نزولی: چاپ اعداد از $M$ تا ۱

همان ایده را می‌توان در جهت معکوس اجرا کرد.

### طرح مسئله

عدد  $M$  را دریافت کنید و اعداد را از  $M$  تا ۱ چاپ کنید.

اگر:

$$M = 5$$

باشد:

5  
4  
3  
2  
1

در اینجا مقدار اولیه شمارنده:

$$i = M$$

است.

شرط:

```
i ≥ 1
```

است.

و در هر تکرار:

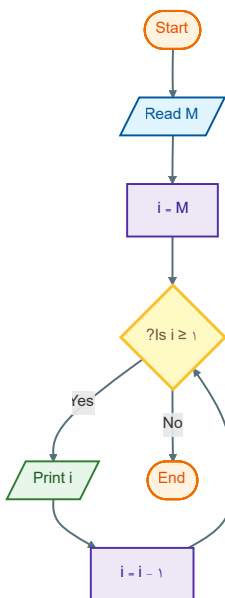
```
i = i - 1
```

انجام می‌شود.

## الگوریتم

---

1. Read M.
2.  $i \leftarrow M$
3. While  $i \geq 1$ :  
    Print i.  
     $i \leftarrow i - 1$
4. End



### ۶.۱. ساختار کلی یک حلقه شمارشی

دو مثال قبلی را می‌توان به یک الگوی عمومی تبدیل کرد.

#### حلقه صعودی

```
i ← Starting value
```

```
While $i \leq$ Ending value:
 Perform the desired operation
 $i \leftarrow i + 1$
```

### حلقه نزولی

```
 $i \leftarrow$ Starting value

While $i \geq$ Ending value:
 Perform the desired operation
 $i \leftarrow i - 1$
```

این الگو در بسیاری از مسائل الگوریتمی تکرار می‌شود.



## ۷. کنترل حلقه با شرط

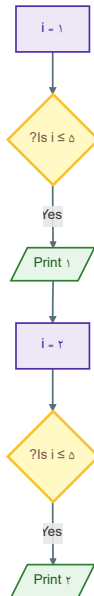
در یک حلقه، شرط معمولاً در ابتدای هر تکرار بررسی می‌شود.

برای مثال:

```
i = 1

While i ≤ 5:
 Print i
 i = i + 1
```

روند اجرا چنین است:



در نهایت:

```
i = 6
```

:9

```
6 ≤ 5
```

نادرست می‌شود. برنامه در این نقطه از حلقه خارج می‌شود. بنابراین شرط حلقه را می‌توان مانند یک دروازه در نظر گرفت که در هر دور مشخص می‌کند آیا اجرای حلقه اجازه ادامه دارد یا خیر.

## ۸. حلقه بی‌نهایت

اگر شرط یک حلقه هیچ‌وقت نادرست نشود، حلقه ممکن است برای همیشه ادامه پیدا کند.

برای مثال:

```
i = 1
```

```
While i ≤ 5:
```

```
 Print i
```

در این الگوریتم مقدار **i** هیچ‌وقت تغییر نمی‌کند. پس **i** همیشه ۱ باقی می‌ماند و شرط:

```
i ≤ 5
```

همیشه درست خواهد بود. این وضعیت می‌تواند یک **حلقه بی‌نهایت (Infinite Loop)**

ایجاد کند.

بنابراین هنگام طراحی هر حلقه باید بررسی کنیم:

۱. متغیرهای دخیل در شرط مقدار اولیه مناسب دارند؟

۲. شرط درست نوشته شده است؟

۳. در هر تکرار چیزی تغییر می‌کند که بتواند در نهایت شرط را نادرست کند؟

این سه سؤال از مهم‌ترین ابزارهای پیدا کردن خطا در حلقه‌ها هستند.

## مثال: چاپ مقسوم‌علیه‌های یک عدد

یکی از مثال‌های مناسب برای ترکیب شرط و حلقه، پیدا کردن مقسوم‌علیه‌های یک عدد است. یک عدد  $i$ ، مقسوم‌علیه  $n$  است اگر تقسیم  $n$  بر  $i$  بدون باقی‌مانده انجام شود. بنابراین شرط اصلی ما:

$$n \% i == 0$$

است.

مثلاً برای عدد:

$$n = 12$$

عددهای ۱ تا ۱۲ را بررسی می‌کنیم.

برای  $i = 1$ :

$$12 \% 1 = 0$$

پس ۱ مقسوم علیه ۱۲ است.

برای  $i = 2$  :

$$12 \% 2 = 0$$

پس ۲ نیز مقسوم علیه است.

اما برای  $i = 5$  :

$$12 \% 5 = 2$$

پس ۵ مقسوم علیه ۱۲ نیست.

در نهایت مقسوم علیه های ۱۲ عبارت اند از:

1, 2, 3, 4, 6, 12

## بررسی تمام اعداد تا $n$

ساده ترین الگوریتم این است که تمام اعداد از ۱ تا خود  $n$  را بررسی کنیم.

## الگوریتم

```
1. Read n.
2. $i \leftarrow 1$
3. While $i \leq n$:
 If $n \% i == 0$:
```

```
Print i.
i ← i + 1
4. End
```

نکته مهم این است که در هر تکرار دو تصمیم مختلف داریم:

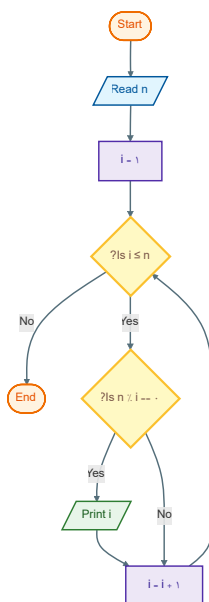
اول:

- آیا هنوز باید حلقه ادامه پیدا کند؟

و سپس، درون حلقه:

- آیا  $n$  بر  $i$  بخش پذیر است؟

بنابراین در این مثال، یک شرط برای کنترل حلقه و یک شرط برای انجام عملیات داخل حلقه داریم.



### تحلیل اجرای الگوریتم مقسوم علیه‌ها

فرض کنیم:

$n = 10$

باشد.

برنامه ابتدا:

```
i = 1
```

قرار می‌دهد.

سپس:

```
10 % 1 == 0
```

درست است، پس 1 چاپ می‌شود.

بعد:

```
i = 2
```

9:

```
10 % 2 == 0
```

درست است، پس 2 چاپ می‌شود.

برای:

```
i = 3
```

داریم:

```
10 % 3 = 1
```

بنابراین چیزی چاپ نمی‌شود.

در ادامه **i** به ۴، ۵، ۶ و ... می‌رسد.

در نهایت:

$$10 \% 10 = 0$$

و **10** نیز چاپ می‌شود.

خروجی:

1  
2  
5  
10

است.

## ایده بهینه‌سازی الگوریتم

یکی از مهارت‌های مهم در علوم کامپیوتر این است که فقط به درست بودن یک الگوریتم اکتفا نکنیم؛ بلکه بررسی کنیم آیا می‌توان آن را با هزینه کمتر اجرا کرد یا نه. فرض کنید عدد بسیار بزرگی داریم. در روش قبلی، برای پیدا کردن مقسوم‌علیه‌ها باید تمام اعداد:

1, 2, 3, ..., n

را بررسی کنیم.

اگر **n** بسیار بزرگ باشد، تعداد بررسی‌ها نیز بسیار زیاد خواهد بود. اما می‌توانیم از یک ویژگی ریاضی استفاده کنیم. اگر **i** مقسوم‌علیه **n** باشد، آنگاه  $\frac{n}{i}$  نیز مقسوم‌علیه **n**



است.

برای مثال:

$$36 = 4 \times 9$$

پس وقتی 4 را به عنوان یک مقسوم‌علیه پیدا می‌کنیم، 9 نیز یک مقسوم‌علیه است. همین موضوع باعث می‌شود بتوانیم جست‌وجو را به شکل کارآمدتری انجام دهیم.

### بررسی تا ریشه دوم

در روش بهینه‌تر، به جای بررسی تمام اعداد تا  $n$ ، می‌توانیم جفت مقسوم‌علیه‌ها را تا  $\sqrt{n}$  بررسی کنیم.

برای مثال:

$$36 = 1 \times 36$$

$$36 = 2 \times 18$$

$$36 = 3 \times 12$$

$$36 = 4 \times 9$$

$$36 = 6 \times 6$$

وقتی به:

$$\sqrt{36} = 6$$

رسیدیم، تمام جفت‌های ممکن پوشش داده شده‌اند. بنابراین اگر هدف صرفاً پیدا کردن مقسوم‌علیه‌ها باشد، الگوریتم می‌تواند تا ریشه دوم را بررسی کند و برای هر مقسوم‌علیه، جفت آن را نیز در نظر بگیرد. این روش نشان می‌دهد که شناخت ساختار ریاضی مسئله می‌تواند مستقیماً به بهبود الگوریتم منجر شود.

## درست بودن در برابر کارآمد بودن

دو الگوریتم ممکن است هر دو پاسخ صحیح تولید کنند، اما مقدار کاری که برای رسیدن به پاسخ انجام می‌دهند متفاوت باشد.

برای مثال:

روش اول:

را بررسی کن  $n$  تا 1.

در مقابل:

روش بهینه‌تر:

بررسی کن و از جفت مقسوم‌علیه‌ها استفاده کن  $\sqrt{n}$  تا

هر دو می‌توانند برای مسئله مقسوم‌علیه‌ها جواب درست بدهند، اما روش دوم از اطلاعات ریاضی بیشتری استفاده می‌کند و تعداد بررسی‌های بسیار کمتری نیاز دارد. این موضوع مقدمه‌ای برای مفهوم پیچیدگی الگوریتم (**Algorithmic Complexity**) است که در مباحث بعدی با جزئیات بیشتری بررسی خواهد شد.

## ۹. شرط تو در تو (Nested Conditions)

گاهی اجرای یک شرط وابسته به نتیجه شرط دیگری است. مثلاً در مثال بزرگ‌ترین عدد بین سه عدد، ابتدا بررسی می‌کردیم:

آیا  $a \geq b$  ؟

و تنها بعد از پاسخ «بله»، سؤال دیگری می‌پرسیدیم:

آیا  $a \geq c$  ؟

این ساختار را شرط تو در تو (Nested Condition) می‌نامیم.

به شکل ساده:

:اگر شرط اول درست بود

:اگر شرط دوم درست بود

A عملیات

:در غیر این صورت

B عملیات

شرط تو در تو زمانی مفید است که تصمیم دوم تنها در صورت تحقق وضعیت خاصی معنا داشته باشد. البته استفاده بیش از حد از شرط‌های تو در تو می‌تواند خوانایی الگوریتم را کاهش دهد؛ بنابراین باید ساختار شرط‌ها را تا حد امکان واضح طراحی کرد.

## ۱۰. جمع‌بندی فصل

در این فصل با ساختارهای شرطی و نقش آن‌ها در کنترل جریان اجرای برنامه آشنا شدیم. ابتدا دیدیم که برنامه‌ها در حالت عادی به صورت ترتیبی اجرا می‌شوند، اما با استفاده از شرط‌ها می‌توان بر اساس وضعیت یک عبارت، مسیرهای متفاوتی برای اجرای برنامه ایجاد کرد. سپس ساختارهای `if-else`، `if` و `if-else if-else` را بررسی کردیم و با تفاوت میان شرط‌های مستقل و زنجیره‌ای و همچنین اهمیت ترتیب بررسی شرط‌ها آشنا شدیم. در ادامه دیدیم که شرط‌ها علاوه بر تصمیم‌گیری، می‌توانند برای کنترل ادامه یا پایان حلقه‌ها نیز به کار روند و با مفاهیمی مانند مقداردهی اولیه، شرط ادامه و تغییر شمارنده در حلقه‌های شمارشی آشنا شدیم. همچنین بررسی کردیم که تغییر نکردن صحیح متغیر کنترل‌کننده می‌تواند باعث ایجاد حلقه بی‌نهایت شود و در مسائل پیچیده‌تر، شرط‌های تو در تو امکان تصمیم‌گیری‌های وابسته به یکدیگر را فراهم می‌کنند. در بخش الگوریتمی نیز دیدیم که تنها درست بودن نتیجه کافی نیست و باید به کارایی الگوریتم توجه کرد؛ برای مثال، استفاده از ویژگی‌های ریاضی در مسئله‌ی بررسی مقسوم‌علیه‌ها می‌تواند تعداد عملیات لازم را کاهش دهد و ما را با ایده‌ی اولیه‌ی پیچیدگی و بهینه‌سازی الگوریتم‌ها آشنا کند. در فصل‌های بعدی، همین ساختارهای کنترلی را در کنار حلقه‌ها، توابع و آرایه‌ها به کار خواهیم گرفت تا بتوانیم برنامه‌هایی با منطق پیچیده‌تر و توانایی پردازش داده‌های بیشتر طراحی کنیم.

# حلقه و تکرار

فصل ششم



## ۱. مقدمه

در فصل‌های گذشته با دو ساختار اساسی در طراحی الگوریتم آشنا شدیم: **توالی و انتخاب**. در ساختار توالی، دستورها به ترتیب و از بالا به پایین اجرا می‌شوند. در ساختار انتخاب یا شرط، برنامه بر اساس درست یا نادرست بودن یک شرط، مسیر مناسب را انتخاب می‌کند. اما بسیاری از مسائل برنامه‌نویسی فقط به اجرای یک دستور یا انتخاب بین چند مسیر محدود نمی‌شوند. در بسیاری از مسائل لازم است یک عملیات، بارها و بارها تکرار شود.

برای مثال:

- چاپ اعداد ۱ تا ۱۰۰؛
- محاسبه مجموع اعداد ۱ تا  $n$ ؛
- دریافت تعداد نامشخصی عدد از کاربر؛
- محاسبه فاکتوریل یک عدد؛
- بررسی تک‌تک عناصر یک مجموعه؛
- ساخت جدول ضرب؛
- پیمایش سطرها و ستون‌های یک جدول یا ماتریس.

اگر برای هر تکرار، دستورها را جداگانه بنویسیم، الگوریتم بسیار طولانی و غیرعملی خواهد شد. **حلقه تکرار (Loop)** این مشکل را حل می‌کند. حلقه ساختاری است که به برنامه اجازه می‌دهد یک یا چند دستور را تا زمانی که یک شرط مشخص برقرار است، یا برای تعداد مشخصی از دفعات، تکرار کند. در این فصل ابتدا با ساختار حلقه و انواع آن آشنا می‌شویم و سپس به سراغ **حلقه‌های تودرتو (Nested Loops)** می‌رویم؛ یعنی حالتی که یک حلقه درون حلقه دیگری قرار می‌گیرد.

## ۲. اجزای اصلی یک حلقه

هر حلقه بسته به نوع و مسئله ممکن است شکل متفاوتی داشته باشد، اما در طراحی الگوریتم معمولاً باید چهار جزء اصلی آن را مشخص کنیم:

۱. مقداردهی اولیه

۲. شرط حلقه

۳. بدنه حلقه

۴. به‌روزرسانی یا گام حلقه

### ۲.۱. مقداردهی اولیه (Initialization)

پیش از شروع حلقه، باید وضعیت اولیه متغیرهای مورد استفاده مشخص شود. برای مثال:

```
i = 1
```

در اینجا شمارنده **i** از مقدار ۱ شروع می‌شود.

### ۲.۲. شرط حلقه (Loop Condition)

شرط تعیین می‌کند که آیا اجرای حلقه باید ادامه پیدا کند یا خیر. مثلاً:

```
i ≤ n
```

اگر شرط درست باشد، بدنه حلقه اجرا می‌شود و اگر نادرست باشد، اجرای حلقه پایان می‌یابد.

## ۲.۳. بدنه حلقه (Loop Body)

بدنه شامل دستورهای است که باید در هر تکرار اجرا شوند. برای مثال:

```
print i
```

## ۲.۴. گام یا به‌روزرسانی (Step / Update)

در بسیاری از حلقه‌ها باید متغیر کنترل‌کننده تغییر کند تا حلقه به سمت پایان حرکت کند.

برای مثال:

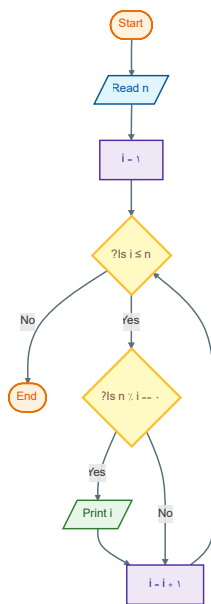
```
i = i + 1
```

اگر این تغییر انجام نشود، ممکن است شرط حلقه هیچ‌گاه نادرست نشود و برنامه در یک حلقه بی‌نهایت (Infinite Loop) گرفتار شود.

## الگوی کلی

می‌توان ساختار یک حلقه را به شکل زیر در نظر گرفت:





این الگویی از مهم‌ترین الگوهای ذهنی برای حل مسائل مربوط به حلقه‌هاست.

## ۳ انواع ساختارهای تکرار

سه ساختار رایج برای تکرار عبارت‌اند از:

- For
- While
- Do-While

تفاوت اصلی آن‌ها در نحوه کنترل تکرار و زمان بررسی شرط است.

### ۳.۱. حلقه For

حلقه For معمولاً زمانی استفاده می‌شود که تعداد تکرارها یا محدوده آن‌ها از قبل مشخص باشد. مثلاً:

اعداد ۱ تا ۱۰ را چاپ کن.

یا:

این عملیات را دقیقاً ۱۰۰ بار انجام بده.

الگوی کلی:

```
For i from M to N with a step of 1:
```

```
 Loop body statements
```

```
End loop
```

یا

```
for (i = M; i ≤ N; i++) {
 // Loop body statements
}
```

در این ساختار، محدوده شروع، پایان و گام تکرار مشخص است.

مثلاً:

```
For i = 1 to 5:
 Print i
```

روند اجرا:

```
i = 1 → 1 چاپ
i = 2 → 2 چاپ
i = 3 → 3 چاپ
i = 4 → 4 چاپ
i = 5 → 5 چاپ
i = 6 → پایان حلقه
```

## ۳.۲. حلقه While

در حلقه **While**، شرط قبل از اجرای بدنه بررسی می‌شود.

ساختار کلی:

```
While the condition is true:
 Loop body
 Update
```

مثلاً:

```
i = 1

While i ≤ 5:
 Print i
 i = i + 1
```

روند اجرا:

```
i = 1 → 1 چاپ
i = 2 → 2 چاپ
i = 3 → 3 چاپ
i = 4 → 4 چاپ
i = 5 → 5 چاپ
i = 6 → شرط نادرست → خروج
```

نکته مهم این است که چون شرط پیش از بدنه بررسی می‌شود، ممکن است بدنه حلقه صفر بار اجرا شود.

مثلاً:

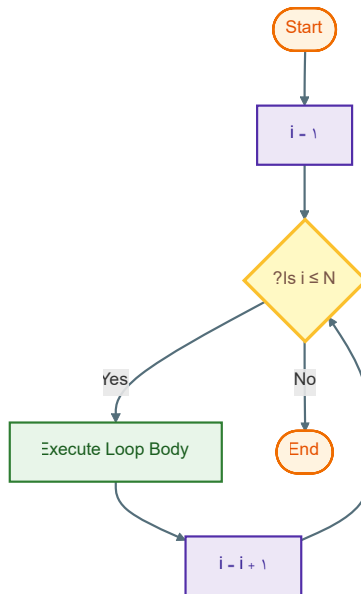
```
i = 10
```

```
While i ≤ 5:
```

```
 Print i
```

از همان ابتدا شرط نادرست است؛ بنابراین هیچ عددی چاپ نمی‌شود.

### فلوچارت While



### ۳.۳. حلقه Do-While

در **Do-While** ابتدا بدنه اجرا و سپس شرط بررسی می‌شود. بنابراین:

Execute the loop body

↓

Check the condition

↓

If true → Repeat

If false → Exit

از آنجا که شرط در انتها بررسی می‌شود، بدنه این حلقه حداقل یک بار اجرا خواهد شد. این ویژگی در مواردی مانند نمایش منو یا دریافت ورودی از کاربر کاربرد دارد؛ زیرا ممکن است بخواهیم عملیات دریافت یا نمایش حداقل یک بار انجام شود و بعد درباره ادامه آن تصمیم بگیریم.

مثلاً:

```
i = 1
```

```
Do:
```

```
 Print i
```

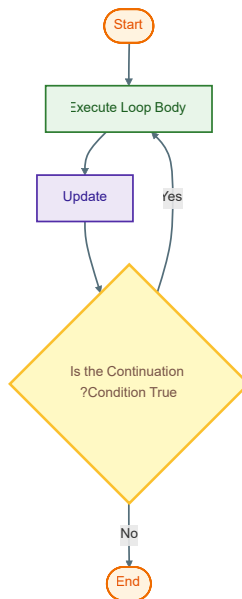
```
 i = i + 1
```

```
While i ≤ 5
```

روند اجرا:

i = 1 → 1 چاپ  
i = 2 → 2 چاپ  
i = 3 → 3 چاپ  
i = 4 → 4 چاپ  
i = 5 → 5 چاپ  
i = 6 → شرط نادرست → پایان حلقه

## فلوچارت



## ۴ مقایسه انواع حلقه‌ها

نوع حلقه	مشخص بودن تعداد تکرار	زمان بررسی شرط	حداقل اجرای بدنه	کاربرد معمول
For	معمولاً مشخص	قبل از بدنه	۰ بار	تکرار به تعداد مشخص
While	معمولاً نامشخص	قبل از بدنه	۰ بار	تکرار تا برقرار بودن یک شرط
Do- While	معمولاً نامشخص	بعد از بدنه	۱ بار	منوها و ورودی‌هایی که باید حداقل یک بار دریافت شوند

نکته: این تقسیم‌بندی یک راهنمای مفهومی است. در زبان‌های برنامه‌نویسی مختلف، امکانات و کاربردهای دقیق این ساختارها ممکن است متفاوت باشد.

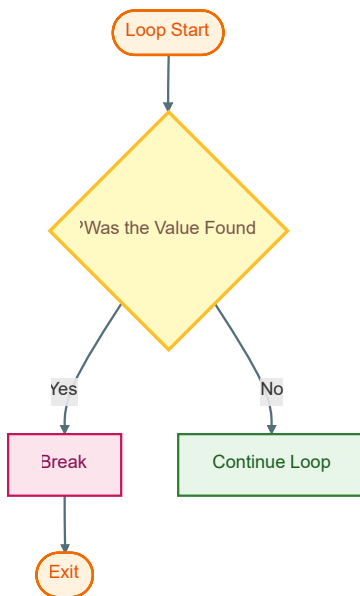


## ۵. کنترل اجرای حلقه با Break و Continue

گاهی لازم است رفتار عادی حلقه را در میانه اجرا تغییر دهیم. دو دستور مهم برای این کار Break و Continue هستند.

### ۵.۱ دستور Break

دستور Break باعث می‌شود اجرای حلقه به طور کامل متوقف شود و برنامه به نخستین دستور بعد از حلقه منتقل شود. مثلاً در جستجوی یک مقدار، اگر مقدار موردنظر پیدا شود، می‌توانیم حلقه را متوقف کنیم.



بنابراین:

**Break** یعنی خروج کامل از حلقه.

## ۵.۲. دستور Continue

دستور **Continue** باعث می‌شود ادامه بدنه در تکرار فعلی اجرا نشود و کنترل به مرحله بعدی حلقه منتقل شود. مثلاً اگر در حال پردازش اعداد باشیم و بخواهیم اعداد منفی را نادیده بگیریم، می‌توانیم در صورت مشاهده عدد منفی از **Continue** استفاده کنیم.

تفاوت اصلی:

عملکرد	دستور
خروج کامل از حلقه	<b>Break</b>
صرف‌نظر از ادامه تکرار فعلی	<b>Continue</b>

به زبان ساده:

**Break** = حلقه را تمام کن

**Continue** = این تکرار را رد کن

## ۶ خطاهای رایج در حلقه‌ها

### ۶.۱. حلقه بی‌نهایت

اگر شرط پایان حلقه هیچ‌گاه نادرست نشود، حلقه بی‌نهایت ایجاد می‌شود. مثلاً:

```
i = 1

While i ≤ 10:
 Print i
```

در اینجا **i** هیچ‌گاه تغییر نمی‌کند. بنابراین شرط همیشه درست باقی می‌ماند.  
نسخه صحیح:

```
i = 1

While i ≤ 10:
 Print i
 i = i + 1
```

### ۶.۲. خطای یکی کم یا یکی زیاد (Off-by-One Error)

گاهی حلقه به جای تعداد موردنظر، یک بار کمتر یا بیشتر اجرا می‌شود. برای مثال تفاوت:

```
i < 10
```

$$i \leq 10$$

باعث می‌شود در حالت دوم مقدار ۱۰ نیز پردازش شود.

برای جلوگیری از این خطا باید قبل از نوشتن حلقه مشخص کنیم:

- اولین مقدار چیست؟
- آخرین مقدار چیست؟
- آیا آخرین مقدار باید پردازش شود؟
- چند بار باید حلقه اجرا شود؟

## ۷ عناصر خنثی و مقداردهی اولیه در حلقه‌ها

یکی از نکات بسیار مهم در طراحی الگوریتم‌های تکراری، انتخاب مقدار اولیه مناسب برای متغیرهای محاسباتی است.

### عنصر خنثی جمع

عنصر خنثی جمع 0 است:

$$x + 0 = x$$

بنابراین برای محاسبه مجموع:

$$\text{sum} = 0$$

### عنصر خنثی ضرب

عنصر خنثی ضرب 1 است:

$$x \times 1 = x$$

بنابراین برای محاسبه حاصل ضرب:

$$\text{product} = 1$$

اگر **product** را از صفر شروع کنیم، اولین ضرب نیز صفر خواهد شد و حاصل ضرب همیشه صفر باقی می‌ماند.

عملیات	مقدار اولیه
مجموع	0
حاصل ضرب	1
شمارش	0
فاکتوریل	1

قاعده ساده:

برای جمع از صفر و برای ضرب از یک شروع می کنیم.

## ۸ شمارنده حلقه

متغیری که برای کنترل تعداد یا محدوده تکرار استفاده می‌شود، معمولاً شمارنده (Loop Counter) نام دارد. شمارنده می‌تواند صعودی باشد:

```
i = i + 1
```

نزولی باشد:

```
i = i - 1
```

یا با گام دیگری تغییر کند:

```
i = i + 2
```

مثلاً:

```
i = 1
```

با گام ۲ مقادیر زیر را تولید می‌کند:

```
1, 3, 5, 7, 9, ...
```

## مثال: مجموع و حاصل ضرب اعداد ۱ تا $n$

فرض کنید می‌خواهیم مجموع و حاصل ضرب اعداد ۱ تا  $n$  را محاسبه کنیم.

```
sum = 0
product = 1
i = 1

While i ≤ n:
 sum = sum + i
 product = product × i
 i = i + 1
```

برای  $n = 4$  :

```
sum = 1 + 2 + 3 + 4 = 10
```

؛9

```
product = 1 × 2 × 3 × 4 = 24
```

این مثال نشان می‌دهد که یک حلقه می‌تواند چند محاسبه مرتبط را همزمان انجام دهد.



## مثال: دریافت اعداد تا مقدار نگهبان

گاهی تعداد ورودی‌ها از قبل مشخص نیست. فرض کنید برنامه باید اعداد را از کاربر دریافت کند و آن‌ها را جمع بزند. هرگاه کاربر 0 وارد کرد، دریافت اعداد متوقف شود.

در اینجا 0 یک مقدار نگهبان (Sentinel Value) است.

الگوریتم:

```
sum = 0

Read n

While n ≠ 0: // تا وقتی که مقدار 0 نیست
 sum = sum + n
 Read n

Print sum
```

اگر ورودی‌ها چنین باشند:

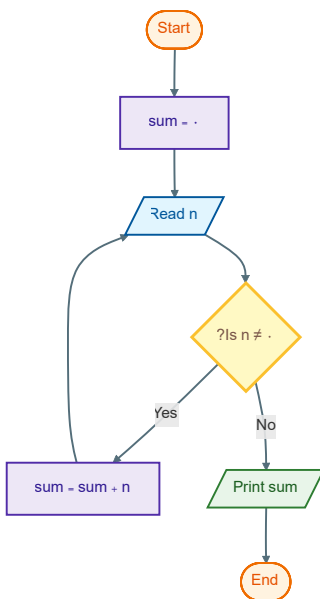
```
5
8
3
0
```

محاسبه به شکل زیر پیش می‌رود:

```
sum = 0
sum = 5
sum = 13
sum = 16
```

با ورود 0 شرط نادرست می‌شود و حلقه پایان می‌یابد.

## فلوچارت



## ۹. ورود به حلقه‌های تودرتو

تا اینجا هر حلقه را به صورت مستقل بررسی کردیم. اما گاهی یک حلقه به تنهایی کافی نیست. فرض کنید می‌خواهیم یک جدول را بررسی کنیم. برای هر سطر باید تمام ستون‌های آن را بررسی کنیم. در چنین شرایطی از حلقه تودرتو (Nested Loop) استفاده می‌کنیم. حلقه تودرتو یعنی:

یک حلقه درون بدنه حلقه دیگری قرار گرفته باشد.

مثلاً:

```
M از 1 تا i برای
N از 1 تا j برای
دستورات
```

در اینجا:

- $i$  شمارنده حلقه بیرونی است.
- $j$  شمارنده حلقه داخلی است.

### ۹.۱ نحوه اجرای حلقه‌های تودرتو

مهم‌ترین نکته این است:

به ازای هر بار اجرای حلقه بیرونی، حلقه داخلی از ابتدا تا انتها اجرا می‌شود.

مثلاً:

```
For i from 1 to 5:
 For j from 1 to 3:
 Print (i, j)
```

خروجی:

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
(4,1) (4,2) (4,3)
(5,1) (5,2) (5,3)
```

در اولین مرحله:

```
i = 1
```

و **j** تمام مقادیر خود را طی می‌کند:

```
1 → 2 → 3
```

سپس:

```
i = 2
```

و **j** دوباره از ۱ شروع می‌شود:

1 → 2 → 3

این روند تا پایان حلقه بیرونی ادامه دارد.

## ۹.۲ محاسبه تعداد تکرارها در حلقه‌های تودرتو

اگر حلقه بیرونی  $M$  بار و حلقه داخلی در هر بار  $N$  بار اجرا شود، بدنه حلقه داخلی در مجموع:

$M \times N$

بار اجرا می‌شود.

مثلاً:

$5 \times 3 = 15$

بنابراین در مثال قبل، دستور:

چاپ ( $i, j$ )

۱۵ بار اجرا می‌شود.

اگر سه حلقه داشته باشیم:

```
For i from 1 to A:
 For j from 1 to B:
```

For k from 1 to C:

Operation

در حالت ساده، عملیات :

$$A \times B \times C$$

بار اجرا خواهد شد. این مفهوم بعداً در بحث پیچیدگی زمانی الگوریتم‌ها اهمیت زیادی پیدا می‌کند.

## مثال: جدول ضرب $10 \times 10$

### صورت مسئله

الگوریتمی بنویسید که جدول ضرب  $10 \times 10$  را تولید کند.

برای این کار به دو حلقه نیاز داریم:

- حلقه بیرونی برای سطرها:
- حلقه داخلی برای ستون‌ها.

متغیرها:

```
i = شماره سطر
j = شماره ستون
```

برای هر خانه جدول:

```
i × j
```

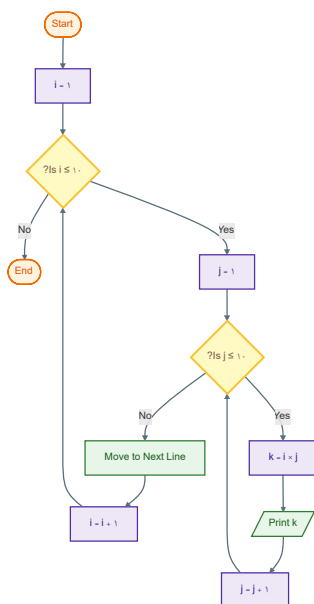
محاسبه می‌شود.

الگوریتم:

```
```text  
For i from 1 to 10:  
    For j from 1 to 10:  
        Print i × j  
    Go to the next line
```

دستور «رفتن به خط بعد» پس از پایان حلقه داخلی قرار دارد؛ زیرا در آن لحظه یک سطر کامل شده است.

فلوچارت



جدول ضرب با ابعاد دلخواه

حالا مسئله را عمومی تر می کنیم.

دو عدد a و b را دریافت کنید و جدولی با a سطر و b ستون تولید کنید.

```
Read a and b
```

```
i = 1
```

```
While i  $\leq$  a:
```

```
    j = 1
```

```
    While j  $\leq$  b:
```

```
        k = i  $\times$  j
```

```
        Print k
```

```
        j = j + 1
```

```
    Go to the next line
```

```
    i = i + 1
```

اگر:

```
a = 3
```

```
b = 4
```

خروجی:

1	2	3	4
2	4	6	8
3	6	9	12

نکته مهم

در این الگوریتم، j در ابتدای هر سطر دوباره مقداردهی می‌شود:

$j = 1$

دلیل آن این است که پس از پایان یک سطر، حلقه داخلی به پایان رسیده و برای سطر بعد باید دوباره از ستون اول شروع شود.

مسئله ترکیبی: پردازش عدد اول و مرکب

اکنون چند مفهوم مختلف را در یک مسئله ترکیب می‌کنیم.

صورت مسئله

عددی مانند n را دریافت کنید و:

۱. اگر $n = 1$ بود، پیام «نه اول است و نه مرکب» نمایش داده شود.
۲. اگر n اول بود، $n \times 3$ محاسبه و نمایش داده شود.
۳. اگر n مرکب بود، n^5 بدون استفاده از عملگر توان محاسبه و نمایش داده شود.

تحلیل مسئله

ابتدا حالت ویژه $n = 1$ را جدا می‌کنیم:

```
If n == 1:  
    Print "Neither prime nor composite"
```

اگر n برابر ۱ نباشد، باید اول بودن آن را بررسی کنیم. برای این کار از زیرالگوریتم `IsPrime` استفاده می‌کنیم.

اگر:

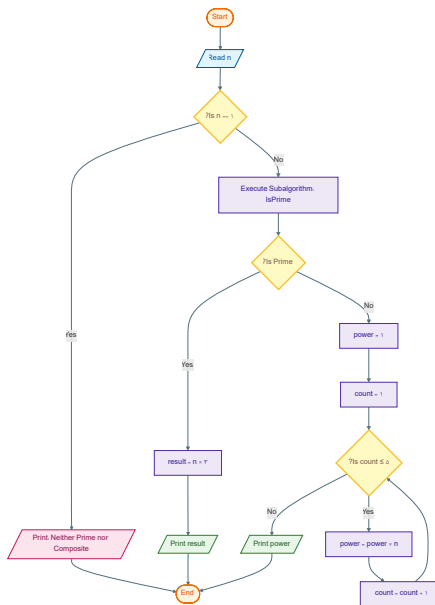
```
IsPrime(n) = True
```

باشد:

```
result = n × 3
```

در غیر این صورت، در چارچوب این مسئله عدد را مرکب در نظر می‌گیریم و توان پنجم آن را با یک حلقه محاسبه می‌کنیم.

فلوچارت مسئله اصلی



زیر الگوریتم تشخیص عدد اول (IsPrime)

برای تشخیص اول بودن n ، باید بررسی کنیم آیا عدد مقسوم‌علیه دیگری غیر از ۱ و خودش دارد یا خیر. در روش مورد استفاده در این فصل، مقادیر 2 تا $n/2$ بررسی می‌شوند.

اگر:

$$n \% i == 0$$

باشد، یعنی n بر i بخش‌پذیر است و بنابراین اول نیست.

الگوریتم:

```

IsPrime(n):

    If n < 2:
        Return False

    i = 2

    While i ≤ n / 2:

        If n % i == 0:
            Return False

        i = i + 1

    Return True

```

برای مثال:

```

7 % 2 = 1
7 % 3 = 1

```

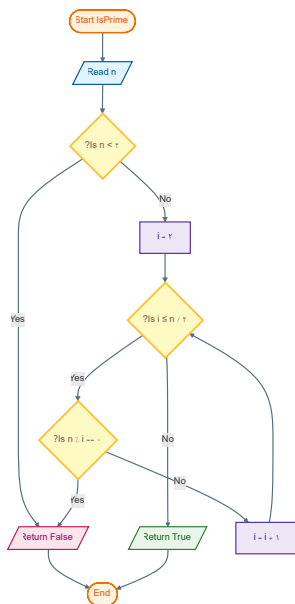
بنابراین مقسوم‌علیه‌ای در این بازه پیدا نمی‌شود و عدد ۷ اول است.

اما:

$$4 \% 2 = 0$$

بنابراین ۴ اول نیست.

فلوچارت IsPrime



اجرای الگوریتم (Trace Table)

$$n = 7$$

عدد ۷ اول است.

گام	مرحله	مقدار متغیر	نتیجه
۱	دریافت ورودی	$n = 7$	ورودی دریافت شد
۲	بررسی $n == 1$	$1 == 7$	نادرست
۳	شروع $IsPrime$	$i = 2$	بررسی آغاز شد
۴	$2 \% 7$	1	مقسوم‌علیه نیست
۵	$i = 3$	3	ادامه بررسی
۶	$3 \% 7$	1	مقسوم‌علیه نیست
۷	پایان بررسی	—	مقسوم‌علیه پیدا نشد
۸	نتیجه $IsPrime$	True	عدد اول است
۹	محاسبه	$21 = 3 \times 7$	نتیجه ۲۱
۱۰	خروجی	21	پایان

حالت دوم: $n = 4$

عدد ۴ مرکب است.

گام	مرحله	مقدار متغیر	نتیجه
۱	دریافت ورودی	$n = 4$	ورودی دریافت شد
۲	بررسی $n == 1$	$1 == 4$	نادرست
۳	شروع $IsPrime$	$i = 2$	بررسی آغاز شد
۴	$2 \% 4$	0	مقسوم‌علیه پیدا شد
۵	نتیجه $IsPrime$	False	عدد اول نیست

گام	مرحله	مقدار متغیر	نتیجه
۶	مقداردهی توان	$\text{power} = 1$	آماده محاسبه
۷	شمارنده	$\text{count} = 1$	شروع حلقه
۸	تکرار اول	$\text{power} = 1 \times 4 = 4$	$\text{count} = 2$
۹	تکرار دوم	$\text{power} = 4 \times 4 = 16$	$\text{count} = 3$
۱۰	تکرار سوم	$\text{power} = 16 \times 4 = 64$	$\text{count} = 4$
۱۱	تکرار چهارم	$\text{power} = 64 \times 4 = 256$	$\text{count} = 5$
۱۲	تکرار پنجم	$\text{power} = 256 \times 4 = 1024$	$\text{count} = 6$
۱۳	پایان حلقه	$\text{count} \leq 5$ نادرست	خروج
۱۴	خروجی	1024	پایان

در نتیجه:

$$4^5 = 1024$$

۱۰. ترکیب ساختارهای شرطی و تکراری

یکی از مهم‌ترین نکاتی که باید از این فصل یاد بگیریم این است که در مسائل واقعی، ساختارهای الگوریتمی معمولاً به صورت جدا از یکدیگر استفاده نمی‌شوند.

یک مسئله ممکن است همزمان شامل:

- شرط درون حلقه؛
- حلقه درون شرط؛
- شرط درون شرط؛
- حلقه درون حلقه؛
- یا چند مورد از این ساختارها به صورت همزمان

باشد.

برای مثال، در مسئله تشخیص عدد اول، یک حلقه داریم که در هر تکرار از یک شرط برای بررسی بخش‌پذیری استفاده می‌کند:

```
While  $i \leq n / 2$ :  
  
    If  $n \% i == 0$ :  
        ...
```

در مسئله جدول ضرب نیز یک حلقه درون حلقه دیگر قرار گرفته است:

```
For i:  
    For j:
```

...

بنابراین مهارت اصلی الگوریتم‌نویسی فقط حفظ کردن `if` یا `while` نیست؛ بلکه توانایی ترکیب درست این ساختارها برای حل مسئله است.



۱۱. جمع‌بندی فصل

در این فصل با یکی از مهم‌ترین ساختارهای الگوریتم‌نویسی، یعنی **حلقه‌ها**، آشنا شدیم. ابتدا دیدیم که حلقه برای تکرار یک یا چند دستور به کار می‌رود و اجزای اصلی آن شامل **مقداردهی اولیه**، **شرط**، **بدنه** و **به‌روزرسانی** است. سپس ساختارهای **While** ، **For** و **Do-While** را بررسی کردیم و تفاوت آن‌ها را از نظر نحوه‌ی بررسی شرط و تعداد دفعات اجرای بدنه شناختیم. در ادامه با دستورهای **Break** و **Continue** آشنا شدیم و دیدیم که چگونه می‌توانند جریان اجرای حلقه را تغییر دهند. همچنین خطاهای رایج مانند **حلقه‌ی بی‌نهایت** و خطای **Off-by-One** را بررسی کردیم و با مفاهیمی مانند مقدار نگهبان (**Sentinel**) و نحوه‌ی مقداردهی اولیه برای محاسبه‌ی مجموع و حاصل‌ضرب آشنا شدیم. سپس **حلقه‌های تودرتو** را بررسی کردیم و دیدیم که چگونه می‌توان از آن‌ها برای پردازش جدول‌ها، ماتریس‌ها و مسائل چندبعدی استفاده کرد و در حالت ساده، دو حلقه‌ی **M** و **N** تکراری می‌توانند باعث اجرای $M \times N$ باره‌ی بدنه‌ی حلقه‌ی داخلی شوند. در ادامه، ترکیب حلقه‌ها با شرط‌ها را در مسائلی مانند **تشخیص عدد اول** بررسی کردیم. در نهایت، حلقه را نه صرفاً به‌عنوان ابزاری برای تکرار، بلکه به‌عنوان یکی از پایه‌های اصلی پردازش داده‌ها، شمارش، جستجو، محاسبه و طراحی الگوریتم‌های پیچیده‌تر شناختیم.

حل مسئله و تفکر الگوریتمی

فصل هفتم



۱ مقدمه

تا اینجا با مفاهیم پایه الگوریتم، فلوچارت، متغیرها، شرطها و حلقه‌های تکرار آشنا شده‌ایم. در این فصل قرار است این مفاهیم را در مجموعه‌ای از مسائل مهم و بسیار رایج الگوریتمی به کار بگیریم. بخش بزرگی از مسائل مقدماتی برنامه‌نویسی در ظاهر متفاوت‌اند، اما در واقع از چند الگوی ساده و تکرارشونده ساخته شده‌اند. برای مثال، محاسبه مجموع چند عدد، پیدا کردن بزرگ‌ترین عدد، محاسبه فاکتوریل، بررسی ارقام یک عدد و تشخیص صعودی یا نزولی بودن یک دنباله، همگی بر چند ایده مشترک تکیه دارند:

- حلقه‌های تکرار
- شمارنده‌ها (Counter)
- انباشت‌گرها (Accumulator)
- متغیرهای وضعیت یا پرچم (Flag)
- مقدارهای نگهبان (Sentinel Value)
- مقایسه مقدار فعلی با مقدار قبلی
- مقداردهی اولیه صحیح
- ردیابی مرحله‌به‌مرحله (Tracing)

هدف این فصل فقط یادگیری چند الگوریتم جداگانه نیست؛ بلکه می‌خواهیم الگوهایی را یاد بگیریم که بتوانیم در مسائل جدید نیز آن‌ها را تشخیص دهیم و به کار ببریم.

۲ پردازش ارقام یک عدد

۲.۱. دسترسی به ارقام یک عدد

یکی از مسائل کلاسیک در الگوریتم نویسی، کار کردن با تک تک ارقام یک عدد صحیح است. فرض کنید عدد زیر را داریم:

1234

ممکن است بخواهیم:

- مجموع ارقام را حساب کنیم؛
- تعداد ارقام را به دست آوریم؛
- عدد را معکوس کنیم؛
- ارقام زوج یا فرد را بشماریم؛
- بزرگ ترین رقم را پیدا کنیم؛
- یا روی هر رقم عملیات خاصی انجام دهیم.

برای این کار لزوماً نیازی نداریم عدد را به رشته (String) تبدیل کنیم. می توانیم با استفاده از دو عملگر ریاضی، ارقام را مستقیماً پردازش کنیم:

۲.۲. گرفتن آخرین رقم با % 10

اگر عددی را بر ۱۰ تقسیم کنیم، باقیمانده تقسیم برابر با آخرین رقم آن خواهد بود. برای مثال:

```
1234 % 10 = 4
```

بنابراین:

```
digit = n % 10
```

آخرین رقم `n` را در متغیر `digit` قرار می‌دهد.

۲.۳. حذف آخرین رقم با تقسیم صحیح بر ۱۰

با تقسیم صحیح عدد بر ۱۰، آخرین رقم آن حذف می‌شود. مثلاً:

```
1234 / 10 = 123
```

```
123 / 10 = 12
```

```
12 / 10 = 1
```

```
1 / 10 = 0
```

بنابراین اگر داشته باشیم:

```
n = n / 10
```

در هر مرحله یک رقم از سمت راست عدد حذف می‌شود.

نکته: منظور از این دستور، تقسیم صحیح است. در زبانی مانند `C`، تقسیم دو عدد صحیح نتیجه صحیح می‌دهد. در زبان‌هایی مانند `Python`، اگر از `/` استفاده کنیم نتیجه ممکن است اعشاری باشد و برای این الگو باید از تقسیم صحیح (`//`) استفاده شود.

۳. الگوی عمومی پردازش ارقام

اکنون دو عملیات اصلی را در یک حلقه قرار می‌دهیم:

```
While n > 0:  
    digit = n % 10  
    Process digit  
    n = n / 10
```

این حلقه ارقام را از راست به چپ پردازش می‌کند.

برای 1234 ترتیب پردازش چنین خواهد بود:

4 → 3 → 2 → 1

دلیل این موضوع کاملاً روشن است: 10% همیشه آخرین رقم فعلی را استخراج می‌کند و تقسیم بر ۱۰ آن رقم را حذف می‌کند.

صورت مسئله

الگوریتمی طراحی کنید که یک عدد صحیح n را دریافت کرده و مجموع ارقام آن را محاسبه کند.

برای مثال:

1234 → 1 + 2 + 3 + 4 = 10

منطق حل مسئله

برای نگهداری مجموع به یک انباشت گر نیاز داریم.

چون عملیات ما جمع است، مقدار اولیه انباشت گر باید صفر باشد:

```
sum = 0
```

سپس در هر تکرار:

۱. آخرین رقم را استخراج می‌کنیم.

۲. آن را به `sum` اضافه می‌کنیم.

۳. آخرین رقم را از `n` حذف می‌کنیم.

شبه‌کد

```
Start
Read n
sum = 0

While n > 0:
    digit = n % 10
    sum = sum + digit
    n = n / 10

Print sum
```

اجرای الگوریتم برای `n = 1234`

مرحله	n	digit	sum
اولیه	۱۲۳۴	—	۰
تکرار ۱	۱۲۳	۴	۴
تکرار ۲	۱۲	۳	۷
تکرار ۳	۱	۲	۹
تکرار ۴	۰	۱	۱۰

در پایان، شرط $n > 0$ برقرار نیست و مقدار نهایی sum برابر ۱۰ است.

معکوس کردن یک عدد

مسئله دیگری که با همین الگو حل می‌شود، ساختن معکوس یک عدد است.

برای مثال:

```
1234 → 4321
```

در اینجا فقط استخراج رقم کافی نیست؛ باید هر رقم استخراج شده را به انتهای عدد جدید اضافه کنیم. برای این کار از رابطه زیر استفاده می‌کنیم:

```
reverse = reverse * 10 + digit
```

چرا ضرب در ۱۰؟

فرض کنید:

```
reverse = 43
```

```
digit = 2
```

اگر بخواهیم 2 را به انتهای 43 اضافه کنیم:

```
43 → 430 → 432
```

بنابراین:

```
43 * 10 + 2 = 432
```

الگوریتم

```
Read n
```

```
reverse = 0
```

```
While n > 0:
```

```
    digit = n % 10
```

```
    reverse = (reverse * 10) + digit
```

```
    n = n / 10
```

```
Print reverse
```

```
End
```

برای 1234 :

```
digit = 4 → reverse = 4  
digit = 3 → reverse = 43  
digit = 2 → reverse = 432  
digit = 1 → reverse = 4321
```

بنابراین خروجی:

4321

نکته درباره صفرهای انتهایی

اگر ورودی:

1200

باشد، معکوس عددی آن:

21

خواهد بود.

زیرا هنگام ساختن یک عدد، صفرهای ابتدای آن از نظر عددی معنا ندارند. بنابراین

21 و 0021 از نظر مقدار عددی یکسان‌اند.

۴. الگوهای مهم پردازش ارقام

الگو	هدف
<code>digit = n % 10</code>	گرفتن آخرین رقم
<code>n = n / 10</code>	حذف آخرین رقم
<code>new = new * 10 + digit</code>	ساختن عدد جدید
<code>count = count + 1</code>	شمارش ارقام
<code>sum = sum + digit</code>	جمع ارقام

این چند الگو پایه بسیاری از مسائل مربوط به ارقام هستند.

۵. ردیابی الگوریتم و فلوچارت

نوشتن الگوریتم یا رسم فلوچارت تنها بخشی از حل مسئله است. باید بتوانیم بررسی کنیم که الگوریتم واقعاً همان کاری را انجام می‌دهد که انتظار داریم. برای این کار می‌توانیم اجرای الگوریتم را بدون کامپیوتر و روی کاغذ شبیه‌سازی کنیم. به این کار ردیابی (Tracing) یا اجرای خشک (Dry Run) گفته می‌شود.

در ردیابی، مانند پردازنده عمل می‌کنیم:

۱. دستورها را به ترتیب اجرا می‌کنیم.

۲. مقدار متغیرها را ثبت می‌کنیم.

۳. شرط‌ها را ارزیابی می‌کنیم.

۴. مسیر اجرای فلوچارت را دنبال می‌کنیم.

۵. خروجی نهایی را به دست می‌آوریم.

این روش برای پیدا کردن خطاهای منطقی (Logic Errors) بسیار مهم است.

۵.۱. جدول ردیابی (Trace Table)

برای ردیابی منظم از جدولی استفاده می‌کنیم که وضعیت متغیرها را در مراحل مختلف نشان می‌دهد. یک جدول می‌تواند چنین ساختاری داشته باشد:

مرحله	شرط	متغیرها	خروجی	توضیح
...	True/False	مقادیر فعلی	مقدار چاپ‌شده	اتفاقی که افتاد

مهم‌ترین اصل این است که در هر مرحله بدانیم الگوریتم دقیقاً چه دستوری را اجرا کرده است.

قوانین مهم ردیابی

۱. برای متغیرهای مهم ستون جدا داشته باشید.

مثلاً اگر الگوریتم دارای `count` ، `sum` و `n` است، وضعیت آن‌ها باید قابل مشاهده باشد.

۲. شرط‌ها را صریحاً ارزیابی کنید.

مثلاً:

```
i ≤ n
```

باید به چیزی مانند:

```
3 ≤ 5 → True
```

تبدیل شود.

۳. چیزی را که الگوریتم نگفته است اجرا نکنید.

ردیابی شبیه اجرای واقعی برنامه است؛ نباید از نتیجه بعدی جلو بزنیم.

۴. تغییرات متغیرها را به ترتیب ثبت کنید.

اگر:

```
sum = sum + number
```

اجرا شده، ابتدا مقدار قبلی `sum` را در نظر می‌گیریم و سپس مقدار جدید را محاسبه

می‌کنیم.

ردیابی یک حلقه فاکتوریل

فرض کنید الگوریتم زیر را داریم:

```
fact = 1
i = 1

while i ≤ N:
    fact = fact * i
    i = i + 1
```

اگر:

$N = 4$

باشد:

مرحله	i	$fact$	شرط
اولیه	۱	۱	—
تکرار ۱	۱	۱	$True \rightarrow 4 \Rightarrow 1$
افزایش	۲	۱	—
تکرار ۲	۲	۲	$True \rightarrow 4 \Rightarrow 2$
افزایش	۳	۲	—

مرحله	i	fact	شرط
تکرار ۳	۳	۶	True → 4 ⇒ 3
افزایش	۴	۶	—
تکرار ۴	۴	۲۴	True → 4 ⇒ 4
افزایش	۵	۲۴	—
پایان	۵	۲۴	False → 4 ⇒ 5

بنابراین:

$$4! = 24$$

۵.۲ پیدا کردن خطا با Tracing

یکی از مهم‌ترین کاربردهای ردیابی، پیدا کردن خطاهای الگوریتم است.

خطای Off-by-One

فرض کنید به جای:

$$i \leq N$$

نوشته‌ایم:

$$i < N$$

برای $N = 4$ ، حلقه فقط برای:

1, 2, 3

اجرا می‌شود و مقدار ۴ پردازش نمی‌شود.

در فاکتوریل، نتیجه به جای:

24

برابر خواهد شد با:

6

Tracing این خطا را به سرعت آشکار می‌کند.

حلقه بی‌نهایت

اگر شمارنده تغییر نکند:

```
count = 1
```

```
while count ≤ 3:
```

```
...
```

شرط همیشه برقرار می‌ماند.

بنابراین حلقه پایان پیدا نمی‌کند.

مقداردهی اولیه اشتباه

اگر یک انباشت‌گر جمعی را با 1 شروع کنیم:

```
sum = 1
```

یک واحد اضافه‌تر از مقدار واقعی وارد محاسبه شده است.

اگر انباشت‌گر ضربی را با 0 شروع کنیم:

```
product = 0
```

تمام ضرب‌های بعدی نیز صفر خواهند بود.

۶. مجموع، میانگین و الگوهای انباشت

۶.۱. انباشت گر و شمارنده

در بسیاری از الگوریتم‌ها لازم است نتیجه‌ای را به صورت تدریجی بسازیم. این کار معمولاً با دو الگو انجام می‌شود:

انباشت گر (Accumulator)

متغیری که در هر تکرار تغییر می‌کند و نتیجه را جمع یا ضرب می‌کند.

مثلاً:

```
sum = sum + number
```

شمارنده (Counter)

متغیری که تعداد دفعات انجام یک عملیات را می‌شمارد:

```
count = count + 1
```

این دو الگو از مهم‌ترین الگوهای پایه‌ای در الگوریتم‌نویسی هستند.

۶.۲. محاسبه مجموع و میانگین

میانگین حسابی از رابطه زیر به دست می‌آید:

$$Average = \frac{Sum}{Count}$$

بنابراین برای محاسبه میانگین باید دو چیز را بدانیم:

۱. مجموع اعداد

۲. تعداد اعداد

حالت اول: تعداد ثابت داده‌ها

فرض کنید دقیقاً ۲۰ عدد داریم. در این حالت می‌توانیم حلقه را دقیقاً ۲۰ بار اجرا کنیم.

```
sum = 0

For 20 iterations:
    Read number
    sum = sum + number

average = sum / 20
```

در این روش تعداد داده‌ها از ابتدا مشخص است.

حالت دوم: تعداد داده‌ها از قبل مشخص می‌شود

گاهی ابتدا تعداد داده‌ها را دریافت می‌کنیم.

مثلاً:

```
n = 5
```

سپس دقیقاً پنج عدد دریافت می‌کنیم.

```
Read n
sum = 0

For n iterations:
    Read number
    sum = sum + number

average = sum / n
```

مزیت این روش این است که الگوریتم برای تعدادهای مختلف قابل استفاده است.

حالت سوم: استفاده از Sentinel

گاهی تعداد داده‌ها از قبل مشخص نیست. در این شرایط می‌توانیم یک مقدار خاص را به‌عنوان مقدار نگهبان (Sentinel Value) تعیین کنیم.

مثلاً:

```
-1
```

به معنای «ورودی تمام شده است» باشد.

در این صورت:

```
sum = 0
count = 0
Read number
```

```
While number  $\neq$  -1:  
    sum = sum + number  
    count = count + 1  
    Read number  
  
If count > 0:  
    average = sum / count
```

عدد 1- در این مثال داده واقعی نیست و نباید در مجموع یا شمارش وارد شود.

چرا $count > 0$ مهم است؟

اگر کاربر از همان ابتدا 1- وارد کند، هیچ عدد معتبری دریافت نشده است:

```
count = 0
```

بنابراین محاسبه:

```
sum / count
```

به معنای تقسیم بر صفر خواهد بود.

۷. فاکتوریل، توان و سری‌ها

۷.۱ فاکتوریل

فاکتوریل عدد n با نماد $n!$ نشان داده می‌شود و برابر حاصل ضرب اعداد ۱ تا n است. الگوی الگوریتمی:

```
fact = 1

For i from 1 to n:
    fact = fact * i
```

مقدار اولیه 1 است، زیرا انباشت‌گر ما ضربی است.

۷.۲ توان

توان نیز می‌تواند با یک حلقه محاسبه شود.

مثلاً:

$$2^4 = 2 \times 2 \times 2 \times 2 = 16$$

الگوریتم:

```
power = 1
```


n: از 1 تا i برای

```
power = power * 2
```

برای $n = 4$:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$

همچنین برای هر عدد غیر صفر:

$$a^0 = 1$$

۷.۳ محاسبه یک سری

حلقه‌ها ابزار بسیار مناسبی برای محاسبه سری‌های ریاضی هستند. فرض کنید می‌خواهیم مقدار زیر را محاسبه کنیم:

$$\sum_{i=0}^m (2^i \times i!)$$

یعنی برای هر i از صفر تا m ، مقدار:

$$2^i \times i!$$

را محاسبه کرده و همه آن‌ها را با هم جمع کنیم.

استفاده از وضعیت مرحله قبل

یک روش مستقیم این است که در هر مرحله توان و فاکتوریل را از ابتدا محاسبه کنیم؛ اما لازم نیست چنین کاری انجام دهیم. می‌توانیم از نتیجه مرحله قبل استفاده کنیم.

در مرحله i :

$$2^i = 2^{i-1} \times 2$$

9:

$$i! = (i - 1)! \times i$$

بنابراین می‌توانیم مقدارهای قبلی را نگه داریم و فقط آن‌ها را به‌روزرسانی کنیم.

```
sum = 0
power = 1
fact = 1

For i from 0 to m:
    sum = sum + (power * fact)
    power = power * 2
    fact = fact * (i + 1)
```

این روش باعث می‌شود برای محاسبه هر جمله، حلقه‌های اضافی ایجاد نکنیم.

ادغام دو وضعیت در یک متغیر

حتی می‌توانیم `power` و `fact` را در یک متغیر ادغام کنیم.

جمله سری:

$$2^i i!$$

را `term` در نظر می‌گیریم.

جمله بعدی از جمله فعلی چگونه به دست می‌آید؟

$$2^{i+1}(i+1)!$$

برابر است با:

$$(2^i i!) \times 2 \times (i+1)$$

بنابراین:

```
term = term * 2 * (i + 1)
```

می‌تواند جمله بعدی را بسازد.

این ایده نمونه‌ای از استفاده از رابطه بین دو مرحله متوالی برای ساده‌تر کردن الگوریتم است.

۷.۴ یک الگوی ترکیبی

گاهی الگوریتم علاوه بر یک محاسبه ضربی، یک انباشت‌گر جمعی نیز دارد.

مثلاً:

```
p = 1
```

```
sum = 0
```

```
For i from 1 to m:
```

```
    p = p * 2 * i
```

```
    sum = sum + 5
```

در این الگوریتم:

- `p` یک انباشت گر ضربی است.

- `sum` یک انباشت گر جمعی است.

از آنجا که در هر تکرار دقیقاً ۵ واحد به `sum` اضافه می‌شود:

$$sum = 5m$$

۸. تشخیص صعودی و نزولی بودن دنباله

مسئله تشخیص روند یک دنباله

فرض کنید کاربر تعدادی عدد وارد می‌کند و پایان ورودی با **1-** مشخص می‌شود. می‌خواهیم بدون ذخیره کل اعداد، مشخص کنیم دنباله:

- صعودی است؛
- نزولی است؛
- یا هیچ‌کدام.

در اینجا صعودی و نزولی را به‌صورت **یکنواخت** در نظر می‌گیریم:

صعودی

$$a_{i+1} \geq a_i$$

نزولی

$$a_{i+1} \leq a_i$$

بنابراین دنباله:

2, 5, 5, 8

صعودی محسوب می‌شود.

:9

9, 7, 7, 3

نزولی محسوب می‌شود.

اگر همه اعداد برابر باشند، هر دو رابطه برقرار هستند.

استفاده از Flag

برای حل این مسئله لازم نیست همه اعداد را ذخیره کنیم. دو متغیر وضعیت تعریف می‌کنیم:

```
isAscending = 1  
isDescending = 1
```

در ابتدا فرض می‌کنیم هر دو حالت ممکن‌اند. سپس هر بار عدد جدید را با عدد قبلی مقایسه می‌کنیم.

اگر:

```
num < prev
```

باشد، دیگر دنباله نمی‌تواند صعودی باشد:

```
isAscending = 0
```

اگر:

```
num > prev
```

باشد، دیگر دنباله نمی‌تواند نزولی باشد:

```
isDescending = 0
```

مشکل اولین عدد

برای مقایسه باید `prev` داشته باشیم؛ اما هنگام دریافت اولین عدد، هنوز عدد قبلی وجود ندارد. بنابراین یک پرچم دیگر تعریف می‌کنیم:

```
isFirst = 1
```

در اولین ورودی:

```
prev = num  
isFirst = 0
```

و از ورودی دوم به بعد، مقایسه انجام می‌شود.

الگوریتم تشخیص روند

```
isAscending = 1  
isDescending = 1  
isFirst = 1
```

```
Read num
```

```

While num  $\neq$  -1:

    If isFirst == 1:
        prev = num
        isFirst = 0

    Otherwise:
        If num < prev:
            isAscending = 0

        If num > prev:
            isDescending = 0

        prev = num

    Read the next num

```

پس از پایان حلقه، وضعیت پرچم‌ها نتیجه را مشخص می‌کند.

اجرای مثال

دنباله زیر را در نظر بگیرید:

```
3, 7, 5, 9, -1
```


isDescending	isAscending	مقایسه	prev	ورودی
۱	۱	اولین عدد	۳	۳
۰	۱	$3 < 7$	۳	۷
۰	۰	$7 > 5$	۷	۵
۰	۰	$5 < 9$	۵	۹
۰	۰	پایان	—	۱-

چون هر دو پرچم صفر شده‌اند، دنباله نه صعودی است و نه نزولی.

تفسیر نهایی پرچم‌ها

isDescending	isAscending	مقایسه	prev	ورودی
۱	۱	اولین عدد	۳	۳
۰	۱	$3 < 7$	۳	۷
۰	۰	$7 > 5$	۷	۵
۰	۰	$5 < 9$	۵	۹
۰	۰	پایان	—	۱-

۹. یافتن کمینه و بیشینه

مسئله کمینه و بیشینه

یکی از رایج‌ترین مسائل پردازش داده این است که از میان مجموعه‌ای از اعداد:

- کوچک‌ترین مقدار (Minimum):
- بزرگ‌ترین مقدار (Maximum)

را پیدا کنیم.

مثلاً برای:

```
12, 5, 23, 8, 19
```

داریم:

```
min = 5  
max = 23
```

چرا نباید min را صفر کنیم؟

یک اشتباه رایج این است که بنویسیم:

```
min = 0
```

فرض کنید همه ورودی‌ها مثبت باشند:

12, 5, 19

هیچ کدام از این اعداد از صفر کوچک تر نیستند؛ بنابراین `min` همان صفر باقی می ماند، در حالی که صفر اصلاً در داده ها وجود نداشته است. راه حل صحیح این است که اولین عدد ورودی را به عنوان مقدار اولیه کمینه و بیشینه قرار دهیم.

```
read first_num

min = first_num
max = first_num
```

در این لحظه، چون فقط یک عدد داریم، همان عدد هم کوچک ترین و هم بزرگ ترین مقدار موجود است.

الگوریتم یافتن کمینه و بیشینه

فرض کنید ۵۰۰ عدد داریم:

```
Read the first number

min = first_num
max = first_num

For the 2nd to 500th numbers:

    Read n
```

```
If n < min:
```

```
    min = n
```

```
If n > max:
```

```
    max = n
```

```
Print min and max
```

اجرای الگوریتم

برای سادگی، پنج عدد زیر را بررسی می‌کنیم:

12, 5, 23, 8, 19

توضیح	max	min	عدد
اولین عدد	۱۲	۱۲	۱۲
۵ کمینه جدید است	۱۲	۵	۵
۲۳ بیشینه جدید است	۲۳	۵	۲۳
بین دو مقدار فعلی قرار دارد	۲۳	۵	۸
رکوردی را تغییر نمی‌دهد	۲۳	۵	۱۹

نتیجه:

min = 5 و max = 23

۱۰. شمارش تکرار کمینه و بیشینه

۷.۳۴ نسخه پیشرفته تر مسئله

اکنون مسئله را کمی پیچیده تر می کنیم. علاوه بر پیدا کردن کمینه و بیشینه، می خواهیم بدانیم هر کدام چند بار در ورودی تکرار شده اند.

مثلاً:

```
5, 12, 5, 23, 5, 23, 8
```

نتیجه:

```
min = 5
Count of min = 3

max = 23
Count of max = 2
```

برای این کار به دو شمارنده نیاز داریم:

```
cmin
cmax
```

مقداردهی اولیه

اولین عدد را دریافت می‌کنیم:

```
min = first_num  
max = first_num  
cmin = 1  
cmax = 1
```

چرا شمارنده‌ها ۱ هستند؟ چون اولین عدد یک بار دیده شده است.

هنگام پیدا شدن کمینه جدید

اگر:

```
n < min
```

باشد، یک کمینه جدید پیدا کرده‌ایم.

پس:

```
min = n  
cmin = 1
```

چرا **cmin** دوباره ۱ می‌شود؟ چون این مقدار جدید را همین الان برای اولین بار به‌عنوان کمینه دیده‌ایم.

هنگام تکرار کمینه

اگر:

```
n == min
```

باشد، مقدار کمینه تغییر نکرده اما یک نمونه دیگر از آن پیدا شده است:

```
cmin = cmin + 1
```

همین منطق برای بیشینه نیز برقرار است.

. الگوریتم کامل

```
Read first_num
```

```
min = first_num
```

```
max = first_num
```

```
cmin = 1
```

```
cmax = 1
```

```
For the remaining numbers:
```

```
    Read n
```

```
    If n < min:
```

```

min = n
cmin = 1

Otherwise if n == min:
    cmin = cmin + 1

If n > max:
    max = n
    cmax = 1

Otherwise if n == max:
    cmax = cmax + 1

Print min and cmin
Print max and cmax

```

اجرای مثال

ورودی:

5, 12, 5, 23, 5, 23, 8

توضیح	cmax	max	cmin	min	n
مقدار اولیه	۱	۵	۱	۵	۵

توضیح	cmax	max	cmin	min	n
پیشینه جدید	۱	۱۲	۱	۵	۱۲
تکرار کمینه	۱	۱۲	۲	۵	۵
پیشینه جدید	۱	۲۳	۲	۵	۲۳
تکرار کمینه	۱	۲۳	۳	۵	۵
تکرار پیشینه	۲	۲۳	۳	۵	۲۳
بدون تغییر	۲	۲۳	۳	۵	۸

نتیجه نهایی:

min = 5

cmin = 3

max = 23

cmax = 2

۱۱. الگوهای عمومی و نکات مهم

۱۱.۱ عنصر خنثی و مقداردهی اولیه

یکی از مهم‌ترین الگوهایی که در تمام این مسائل مشاهده کردیم، انتخاب مقدار اولیه مناسب است.

مقدار اولیه باید با توجه به نوع عملیات انتخاب شود.

عملیات	انباشت‌گر	مقدار اولیه
جمع	<code>sum</code> , <code>total</code>	۰
ضرب	<code>product</code> , <code>fact</code> , <code>power</code>	۱
شمارش	<code>count</code>	۰
کمینه/بیشینه	<code>min</code> , <code>max</code>	اولین داده

دلیل دو مورد اول مفهوم عنصر خنثی است.

در جمع:

$$x + 0 = x$$

در ضرب:

$$x \times 1 = x$$

بنابراین صفر برای انباشت جمعی و یک برای انباشت ضربی مناسب است.

۱۱.۲ تفاوت Flag و Accumulator، Counter

این سه مفهوم ممکن است در ابتدا شبیه یکدیگر به نظر برسند، اما نقش متفاوتی دارند.

Accumulator

برای ساختن یک نتیجه تدریجی:

```
sum = sum + x
```

Counter

برای شمردن:

```
count = count + 1
```

Flag

برای نگهداری یک وضعیت:

```
isAscending = 0
```

معمولاً پرچم‌ها وضعیت‌هایی مانند «بله/خیر» یا «درست/نادرست» را نشان می‌دهند.

Sentinel Value ۱۱.۳

مقدار نگهبان، داده واقعی نیست؛ بلکه علامتی برای پایان ورود داده‌هاست.

مثلاً:

-1

اگر:

10, 20, 30, -1

وارد شود، 1- باید به عنوان فرمان پایان در نظر گرفته شود، نه یک عدد برای محاسبه.

بنابراین:

```
sum = 60
```

```
count = 3
```

است.

۱۱.۴ مقایسه مقدار فعلی با مقدار قبلی

بسیاری از مسائل مربوط به دنباله‌ها به همین الگو برمی‌گردند:

```
Read current
```

```
Compare current with previous
```

```
previous = current
```

این الگو برای مسائل مختلفی کاربرد دارد؛ از جمله:

- تشخیص صعودی یا نزولی بودن؛

- تشخیص تغییرات متوالی؛

- پیدا کردن نقاط تغییر؛

- بررسی تکرارهای متوالی.

نکته اصلی این است که **اولین داده وضعیت خاصی دارد**، زیرا هنوز مقدار قبلی برای مقایسه وجود ندارد.



۱۲ ردیابی به عنوان ابزار حل مسئله

Tracing فقط روشی برای پاسخ دادن به سؤال‌های کنکور نیست. این روش یک ابزار فکری برای بررسی الگوریتم است.

هنگام ردیابی باید مرتب از خودمان بپرسیم:

۱. مقدار اولیه هر متغیر چیست؟
۲. شرط حلقه در این لحظه چه مقداری دارد؟
۳. کدام شاخه شرط اجرا می‌شود؟
۴. کدام متغیر تغییر می‌کند؟
۵. بعد از این تغییر، شرط چگونه تغییر خواهد کرد؟
۶. آیا حلقه بالاخره تمام می‌شود؟
۷. خروجی دقیقاً چه زمانی تولید می‌شود؟

اگر بتوانیم این پرسش‌ها را روی کاغذ پاسخ دهیم، درک ما از اجرای الگوریتم بسیار دقیق‌تر خواهد شد.

۱۳. جمع‌بندی نهایی فصل

در این فصل مجموعه‌ای از الگوهای بنیادی حل مسئله و الگوریتم‌نویسی را بررسی کردیم. ابتدا یاد گرفتیم چگونه با استفاده از **% 10** آخرین رقم یک عدد را استخراج کنیم و با تقسیم صحیح بر ۱۰ آن را حذف کنیم و از ترکیب این دو عملیات برای حل مسائلی مانند مجموع ارقام و معکوس کردن عدد استفاده کردیم. سپس با مفهوم **Tracing** و جدول ردیابی آشنا شدیم و دیدیم که چگونه می‌توان اجرای یک الگوریتم را مرحله‌به‌مرحله شبیه‌سازی کرد و مقدار متغیرها، وضعیت شرط‌ها، تعداد تکرارها و خروجی را بررسی کرد تا خطاهای منطقی مانند **Off-by-One** و حلقه‌های بی‌نهایت شناسایی شوند. در ادامه، الگوهای **Accumulator** و **Counter** را بررسی کردیم و با استفاده از آن‌ها محاسبه‌ی مجموع و میانگین را در حالت‌های مختلف، از جمله تعداد ثابت داده‌ها، تعداد مشخص‌شده توسط کاربر و تعداد نامشخص با استفاده از **Sentinel** انجام دادیم. سپس همین الگوهای حلقه‌ای را برای محاسبه‌ی فاکتوریل، توان و سری‌های ریاضی به کار بردیم و دیدیم که با نگهداری وضعیت مرحله‌ی قبل می‌توان از محاسبات تکراری جلوگیری کرد. در بخش بعد، با استفاده از **Flag** و مقایسه‌ی هر داده با داده‌ی قبلی، الگوریتمی برای تشخیص صعودی یا نزولی بودن یک دنباله طراحی کردیم. در نهایت نیز مسئله‌ی پیدا کردن کمینه و بیشینه را بررسی کردیم و یاد گرفتیم که مقدار اولیه‌ی آن‌ها بهتر است از اولین داده گرفته شود؛ سپس با ترکیب این الگو با **Counter**، تعداد تکرار کمینه و بیشینه را نیز محاسبه کردیم. در مجموع، الگوهایی مانند پردازش ارقام، **Accumulator**، **Counter**، **Sentinel**، **Flag**، مقایسه با مقدار قبلی و به‌روزرسانی کمینه و بیشینه از ابزارهای پایه‌ای حل مسئله هستند و در بسیاری از مسائل جدید می‌توان به‌جای ساختن یک راه‌حل کاملاً جدید، مسئله را به همین الگوها تجزیه و آن‌ها را با یکدیگر ترکیب کرد.

ساختمان داده‌ها

فصل هشتم



۱ مقدمه

تا اینجا با مفاهیمی مانند الگوریتم، متغیر، عملگر، شرط و حلقه آشنا شدیم. این مفاهیم به ما اجازه می‌دهند برنامه‌هایی بنویسیم که بتوانند محاسبات و عملیات مختلفی را انجام دهند. اما با بزرگ‌تر شدن مسئله‌ها، یک سؤال مهم مطرح می‌شود:

داده‌های برنامه را چگونه نگهداری و سازمان‌دهی کنیم؟

فرض کنید برنامه‌ای داریم که باید نمرات ۱۰۰۰ دانشجو را دریافت و پردازش کند. اگر برای هر نمره یک متغیر جداگانه تعریف کنیم، مجبور خواهیم بود متغیرهایی مانند:

```
score1  
score2  
score3  
...  
score1000
```

داشته باشیم. چنین روشی نه تنها نوشتن برنامه را دشوار می‌کند، بلکه انجام عملیاتی مانند جستجو، مرتب‌سازی، محاسبه میانگین یا پیدا کردن بیشترین مقدار را نیز بسیار پیچیده خواهد کرد.

اینجاست که مفهوم **ساختمان داده (Data Structure)** اهمیت پیدا می‌کند.

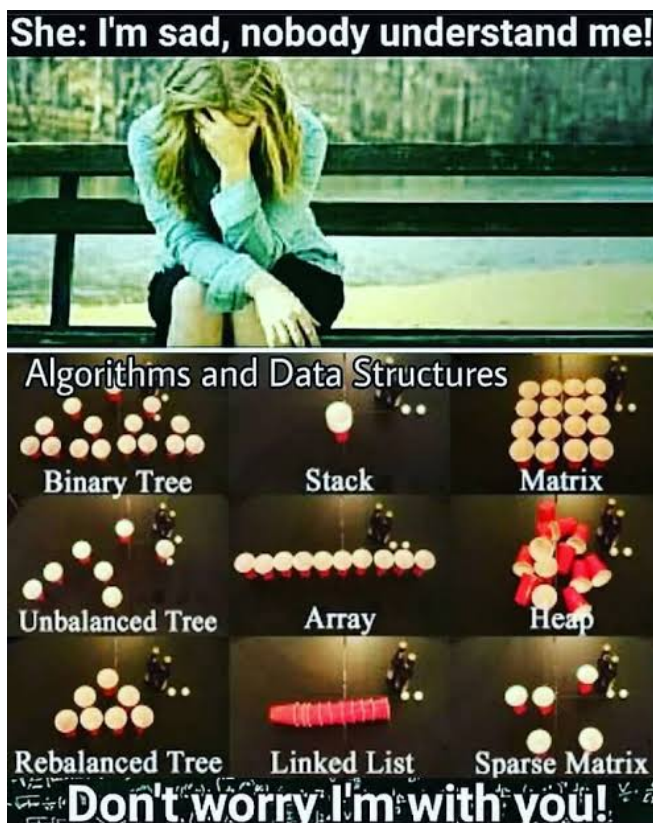
ساختمان داده روشی برای ذخیره، سازمان‌دهی و مدیریت داده‌ها در یک برنامه است؛ به گونه‌ای که بتوانیم عملیات موردنیاز را با روش مناسب و کارآمد روی آن‌ها انجام دهیم.

در علوم کامپیوتر، یک بیان مشهور وجود دارد:

ساختمان داده + الگوریتم = برنامه

البته این عبارت یک فرمول ریاضی دقیق نیست، بلکه راهی برای بیان یک ایده مهم است: یک برنامه خوب فقط مجموعه‌ای از دستورها نیست؛ بلکه باید هم روش حل مسئله و هم روش مناسب نگهداری داده‌ها را در نظر بگیرد.

در این فصل ابتدا با مفهوم کلی ساختمان‌های داده آشنا می‌شویم و سپس به یکی از مهم‌ترین و پایه‌ای‌ترین آن‌ها، یعنی آرایه (Array)، می‌پردازیم.

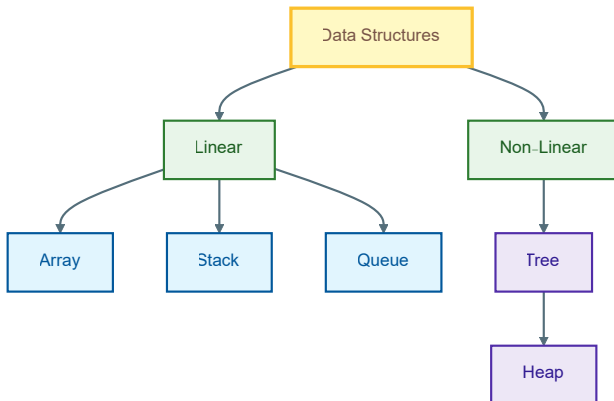


۲ ساختمان‌های داده

ساختمان داده (Data Structure) روشی سیستماتیک برای ذخیره و سازمان‌دهی داده‌ها در حافظه است. انتخاب ساختمان داده مناسب می‌تواند روی نحوه انجام عملیاتی مانند موارد زیر تأثیر بگذارد:

- دسترسی به داده
- جستجو
- درج داده
- حذف داده
- مرتب‌سازی
- پیمایش داده‌ها

برای مثال، اگر فقط تعداد محدودی داده داشته باشیم، شاید چند متغیر ساده کافی باشند. اما اگر با صدها، هزاران یا میلیون‌ها داده سروکار داشته باشیم، سازمان‌دهی آن‌ها اهمیت بسیار بیشتری پیدا می‌کند. ساختمان‌های داده را می‌توان از دیدهای مختلف دسته‌بندی کرد. یکی از دسته‌بندی‌های پایه، تقسیم آن‌ها به ساختمان‌های داده خطی و غیرخطی است.



در ساختمان‌های خطی (Linear)، عناصر معمولاً در قالب یک ترتیب خطی سازمان‌دهی می‌شوند؛ مانند آرایه، پشته و صف.

در ساختمان‌های غیرخطی (Non-Linear)، روابط میان داده‌ها می‌توانند شاخه‌ای یا سلسله‌مراتبی باشند؛ مانند درخت و ساختمان‌هایی که بر پایه آن ساخته می‌شوند.

در ادامه ابتدا ساختمان‌های اصلی را به‌طور خلاصه معرفی می‌کنیم و سپس تمرکز اصلی فصل را روی آرایه قرار می‌دهیم.

۳ ساختمان‌های داده خطی

در یک ساختمان خطی، عناصر به شکلی سازمان‌دهی می‌شوند که بتوان آن‌ها را در قالب یک ترتیب یا دنباله در نظر گرفت.

سه نمونه مهم از ساختمان‌های خطی عبارت‌اند از:

۱. آرایه (Array)

۲. پشته (Stack)

۳. صف (Queue)

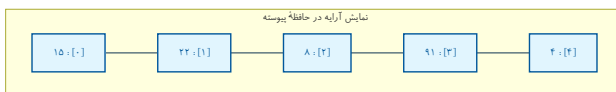
۳.۱. آرایه (Array)

آرایه یکی از پایه‌ای‌ترین ساختمان‌های داده است. آرایه به ما اجازه می‌دهد چندین مقدار را تحت یک نام واحد ذخیره کنیم. در زبان‌های سطح پایینی مانند C، آرایه‌ها معمولاً به صورت مجموعه‌ای از عناصر هم‌نوع در بخش‌های متوالی حافظه پیاده‌سازی می‌شوند. به این ویژگی **حافظه پیوسته (Contiguous Memory)** گفته می‌شود.

برای مثال:

$A = [15, 22, 8, 91, 4]$

می‌توان آرایه را به شکل زیر در حافظه تصور کرد:



یکی از ویژگی‌های مهم آرایه، امکان دسترسی مستقیم یا تصادفی (Random Access) به عناصر است. یعنی اگر اندیس یک عنصر را بدانیم، می‌توانیم مستقیماً به آن

دسترسی پیدا کنیم. در آرایه‌های معمول، این دسترسی در مرتبه زمانی $O(1)$ انجام می‌شود. از طرف دیگر، درج یا حذف یک عنصر در وسط یک آرایه ممکن است نیازمند جابه‌جایی تعدادی از عناصر باشد و در نتیجه هزینه‌ای در مرتبه $O(n)$ داشته باشد. در ادامه فصل، آرایه را به‌صورت مفصل بررسی خواهیم کرد.

۳.۲. پشته (Stack)

پشته (Stack) ساختمانی خطی است که از قانون:

LIFO = Last In, First Out

پیروی می‌کند؛ یعنی:

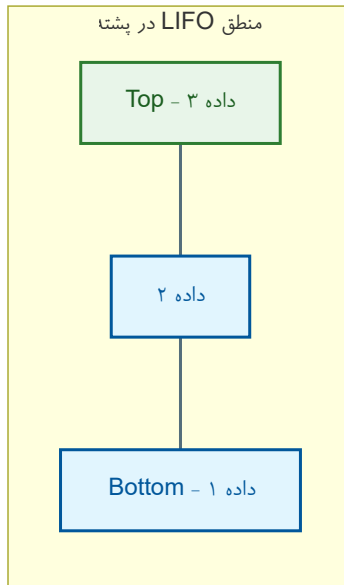
آخرین داده‌ای که وارد می‌شود، اولین داده‌ای است که خارج می‌شود.

می‌توان پشته را مانند تعدادی بشقاب روی هم تصور کرد. اگر بشقاب جدیدی روی بشقاب‌های قبلی قرار دهید، برای برداشتن آن نیز ابتدا همان بشقاب بالایی را برمی‌دارید.

دو عملیات اصلی پشته عبارت‌اند از:

– **Push**: اضافه کردن داده به بالای پشته

– **Pop**: برداشتن داده از بالای پشته



پشته در بخش‌های مختلف علوم کامپیوتر کاربرد دارد؛ از جمله:

- مدیریت فراخوانی توابع در Call Stack
- پیاده‌سازی Undo
- ارزیابی برخی عبارت‌های ریاضی
- الگوریتم‌های جستجو مانند DFS

۳.۳. صف (Queue)

صف (Queue) نیز یک ساختمان خطی است، اما برخلاف پشته از قانون:

FIFO = First In, First Out

پیروی می‌کند.

یعنی:

اولین داده‌ای که وارد صف می‌شود، اولین داده‌ای است که از آن خارج می‌شود.

این ساختمان را می‌توان مانند صف افراد در یک نانوایی تصور کرد.

دو عملیات اصلی صف عبارت‌اند از:

- **Enqueue**: اضافه کردن داده به انتهای صف

- **Dequeue**: خارج کردن داده از ابتدای صف



صف‌ها در مواردی مانند:

- زمان‌بندی پردازش‌ها
- بافرهای شبکه
- مدیریت درخواست‌ها
- الگوریتم BFS

کاربرد دارند.

۴ ساختمان‌های داده غیر خطی

در بعضی مسئله‌ها، رابطه بین داده‌ها به شکل یک خط یا دنباله ساده نیست. برای مثال، ساختمان فایل‌ها در یک سیستم عامل می‌تواند به شکل سلسله‌مراتبی باشد:

پوشه اصلی

└ پوشه A

| └ فایل ۱

| └ فایل ۲

└ پوشه B

 └ فایل ۳

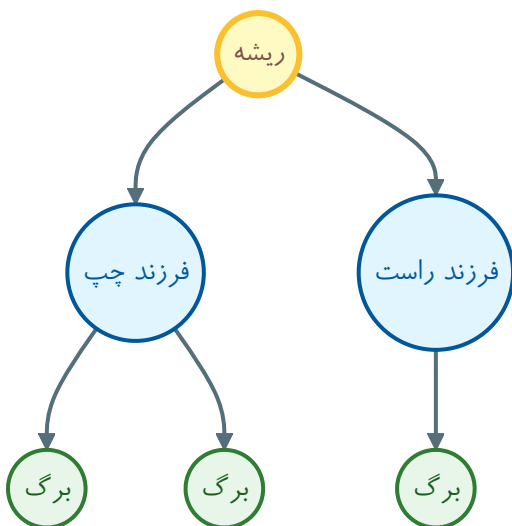
 └ پوشه C

برای چنین روابطی، ساختمان‌های غیرخطی مانند **درخت (Tree)** مناسب‌تر هستند.

۴.۱. درخت (Tree)

درخت یک ساختمان داده سلسله‌مراتبی است که از **گره‌ها (Nodes)** و ارتباط میان آن‌ها تشکیل می‌شود. مهم‌ترین مفاهیم درخت عبارت‌اند از:

- **Root** یا ریشه: بالاترین گره
- **Parent** یا والد: گره‌ای که گره دیگری از آن منشعب شده است
- **Child** یا فرزند: گره‌ای که زیر یک گره دیگر قرار دارد
- **Leaf** یا برگ: گره‌ای که فرزند ندارد
- **Edge** یا یال: ارتباط بین دو گره



درخت‌ها در موارد مختلفی استفاده می‌شوند، از جمله:

- ساختمان فایل‌ها
- درخت‌های تصمیم
- ساختمان‌های مورد استفاده در کامپایلرها
- درخت‌های نحوی
- برخی الگوریتم‌های جستجو

نکته مهم این است که هر درخت الزاماً جستجوی $O(\log n)$ ندارد. این ویژگی به نوع درخت و نحوه سازمان‌دهی آن بستگی دارد. بنابراین نباید صرفاً به دلیل درخت بودن یک ساختمان، انتظار زمان جستجوی لگاریتمی داشت.

۴.۲. هیپ (Heap)

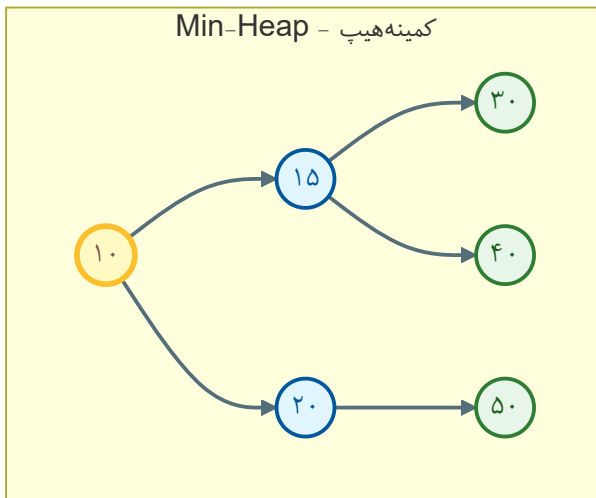
هیپ (Heap) نوع خاصی از درخت است که معمولاً به شکل یک درخت دودویی کامل (Complete Binary Tree) سازمان‌دهی می‌شود. دو نوع مهم هیپ عبارت‌اند از:

Max-Heap

در Max-Heap، مقدار هر گره والد بزرگ‌تر یا مساوی فرزندان خود است. در نتیجه، بزرگ‌ترین عنصر در ریشه قرار می‌گیرد.

Min-Heap

در Min-Heap، مقدار هر گره والد کوچک‌تر یا مساوی فرزندان خود است. در نتیجه، کوچک‌ترین عنصر در ریشه قرار می‌گیرد.



۵ آرایه؛ اولین ساختمان داده مهم

اکنون که تصویر کلی ساختمان‌های داده را دیدیم، به یکی از بنیادی‌ترین آن‌ها می‌رسیم:

آرایه (Array)

فرض کنید می‌خواهیم نمرات ۱۰۰۰ دانشجو را ذخیره کنیم. تعریف ۱۰۰۰ متغیر جداگانه کار بسیار نامناسبی خواهد بود. آرایه به ما اجازه می‌دهد این داده‌ها را تحت یک نام مشترک نگهداری کنیم:

```
scores
```

و هر مقدار را با استفاده از یک اندیس (Index) مشخص کنیم. می‌توان آرایه را مانند یک ردیف صندوق امانات تصور کرد؛ همه صندوق‌ها متعلق به یک مجموعه هستند، اما هر صندوق شماره مخصوص خودش را دارد.

۵.۱. آرایه و همگونی نوع داده

در بسیاری از زبان‌ها و در تعریف کلاسیک آرایه، عناصر آرایه از یک نوع داده هستند.

برای مثال:

```
A = [10, 20, 30, 40]
```

می‌تواند آرایه‌ای از اعداد صحیح باشد.

این ویژگی را همگونی (Homogeneity) می‌نامیم.

یعنی عناصر یک آرایه معمولاً از یک نوع داده هستند.

البته جزئیات این موضوع به زبان برنامه‌نویسی بستگی دارد؛ برای مثال، زبان‌هایی مانند Python در سطح زبان امکانات انعطاف‌پذیرتری برای نگهداری اشیای متفاوت در یک **list** دارند. بنابراین نباید ویژگی‌های آرایه در یک زبان را بدون توجه به زبان برنامه‌نویسی به همهٔ زبان‌ها تعمیم داد.

۵.۲ آرایهٔ یک‌بعدی

ساده‌ترین شکل آرایه، آرایهٔ یک‌بعدی (**One-Dimensional Array**) است. در این آرایه، عناصر در یک دنباله قرار می‌گیرند:

```
A = [10, 20, 30, 40, 50]
```

هر عنصر با یک اندیس مشخص می‌شود. در بسیاری از زبان‌های برنامه‌نویسی، اندیس‌گذاری از صفر شروع می‌شود. بنابراین برای آرایه‌ای با **n** عنصر:

```
First index = 0
```

```
Last index = n - 1
```

برای مثال، آرایه‌ای با پنج عنصر دارای اندیس‌های:

```
0, 1, 2, 3, 4
```

است.

می‌توان آن را به شکل زیر تصور کرد:

۴	۳	۲	۱	۰	Index
۵۰	۴۰	۳۰	۲۰	۱۰	Value

مثلاً:

`A[0] = 10`

`A[2] = 30`

`A[4] = 50`

۵.۳. رابطه آرایه و حلقه

آرایه‌ها و حلقه‌ها ارتباط بسیار نزدیکی با یکدیگر دارند. اگر بخواهیم تمام عناصر یک آرایه را پردازش کنیم، معمولاً از یک حلقه استفاده می‌کنیم. برای آرایه پنج‌عنصری:

`For i from 0 to 4:`

`Print A[i]`

در اینجا متغیر **i** هم‌زمان نقش شمارنده حلقه و اندیس آرایه را دارد. بنابراین یک الگوی بسیار مهم شکل می‌گیرد:

حلقه

↓

تغییر اندیس

↓

`A[i]` دسترسی به



پردازش عنصر

این الگو اساس بسیاری از الگوریتم‌های مربوط به آرایه‌هاست؛ از چاپ و جستجو گرفته تا مرتب‌سازی و محاسبات آماری.

۵.۴. دریافت و چاپ داده‌های یک آرایه

فرض کنید می‌خواهیم ۱۰۰۰ عدد را از کاربر دریافت کنیم و سپس آن‌ها را به همان ترتیبی که وارد شده‌اند چاپ کنیم. ابتدا به آرایه‌ای با ۱۰۰۰ خانه نیاز داریم.

شبه‌کد:

```
1. Define array A with 1000 elements.
2. For i from 0 to 999:
    Read A[i]
3. For i from 0 to 999:
    Print A[i]
```

در اینجا دو مرحله جداگانه داریم:

مرحله اول: دریافت از کاربر

A[0]

A[1]

A[2]

...
A[999]

مرحلهٔ دوم: چاپ

A[0]
A[1]
A[2]
...
A[999]

هر دو مرحله با یک حلقهٔ خطی انجام می‌شوند.

۵.۵. چاپ عناصر آرایه به صورت معکوس

یکی از تمرین‌های مهم برای یادگیری اندیس‌ها این است که عناصر آرایه را از آخر به اول چاپ کنیم.

فرض کنید:

A = [10, 20, 30, 40, 50]

خروجی موردنظر:

50
40

30

20

10

برای این کار چند روش وجود دارد.

روش اول: پیمایش معکوس

در این روش داده‌ها به شکل عادی ذخیره می‌شوند و هنگام چاپ، اندیس را از آخر به اول حرکت می‌دهیم.

1. Define array A[1000].
2. For i from 0 to 999:
 Read A[i]
3. For i from 999 down to 0:
 Print A[i]

. این روش معمولاً ساده‌ترین و خواناترین روش است

روش دوم: نگاشت اندیس

در این روش حلقهٔ چاپ صعودی است، اما اندیس واقعی را با یک رابطهٔ ریاضی محاسبه می‌کنیم:

i - آخرین اندیس

برای آرایه ۱۰۰۰ خانه‌ای:

```
1. Define array A[1000].
2. For i from 0 to 999:
    Read A[i]
3. For i from 0 to 999:
    Print A[999 - i]
```

مثلاً:

```
i = 0 → A[999]
i = 1 → A[998]
i = 2 → A[997]
...
```

روش سوم: ذخیره‌سازی معکوس

در این روش، داده‌ها از همان ابتدا در موقعیت معکوس ذخیره می‌شوند:

```
1. Define array A[1000].
2. For i from 0 to 999:
    Read temporary variable n
    A[999 - i] = n
3. For i from 0 to 999:
    Print A[i]
```

در این روش خود آرایه پس از دریافت داده‌ها به ترتیب معکوس قرار گرفته است.

۵.۷. مقایسه روش‌های چاپ معکوس

روش	نحوه ذخیره‌سازی	نحوه چاپ	ویژگی
اول	عادی	پیمایش نزولی	ساده و خوانا
دوم	عادی	<code>A[last-i]</code>	استفاده از نگاشت اندیس
سوم	معکوس	پیمایش صعودی	داده‌ها واقعاً به شکل معکوس ذخیره می‌شوند

نکته مهم این است که هر سه روش می‌توانند خروجی یکسانی تولید کنند، اما نحوه سازمان‌دهی داده در حافظه در آن‌ها متفاوت است.

۶. آرایهٔ دوبعدی / ماتریس

آرایهٔ یک‌بعدی برای داده‌هایی مناسب است که یک ترتیب خطی دارند. اما بسیاری از داده‌ها ساختمانی جدولی دارند.

برای مثال:

- نمرات چند دانشجو در چند درس
- جدول داده‌ها
- ماتریس‌های ریاضی
- صفحهٔ یک بازی
- پیکسل‌های یک تصویر

برای چنین داده‌هایی می‌توان از آرایهٔ دوبعدی (Two-Dimensional Array) استفاده کرد. آرایهٔ دوبعدی را می‌توان مانند یک جدول در نظر گرفت:

```
A[i][j]
```

که در آن:

- **i** معمولاً اندیس سطر
- **j** معمولاً اندیس ستون

است.

بنابراین:

```
A[3][5]
```

به خانه‌ای در سطر ۳ و ستون ۵ اشاره می‌کند. در زبان‌هایی که اندیس گذاری از صفر شروع می‌شود، `A[0][0]` اولین خانه است.

۶.۱. ساختمان آرایهٔ دوبعدی

فرض کنید آرایه‌ای به شکل زیر داریم:

`A[20][30]`

یعنی:

- ۲۰ سطر
- ۳۰ ستون

پس تعداد کل خانه‌ها برابر است با:

$$20 \times 30 = 600$$

محدودهٔ اندیس‌ها:

محدوده	مورد
0 تا 19	سطرها
0 تا 29	ستونها
20	تعداد سطرها
30	تعداد ستونها
600	تعداد کل خانه‌ها

بنابراین اولین خانه:

$A[0][0]$

و آخرین خانه:

$A[19][29]$

است.

مثال یک آرایهٔ دوبعدی

فرض کنید آرایهٔ زیر را داشته باشیم:

ستون			
	0	1	2
0	10	20	30
1	40	50	60
2	70	80	90

↑

سطر

در این حالت:

`A[0][0] = 10`

`A[0][1] = 20`

`A[0][2] = 30`

`A[1][0] = 40`

`A[1][1] = 50`

`A[1][2] = 60`

`A[2][0] = 70`

`A[2][1] = 80`

`A[2][2] = 90`

بنابراین:

`A[1][2]`

یعنی سطر شماره ۱ و ستون شماره ۲، و مقدار آن برابر است با:

60

۶.۲. آرایه دوبعدی و حلقه‌های تودرتو

برای پیمایش تمام خانه‌های یک آرایه دوبعدی، معمولاً از دو حلقه تودرتو (**Nested Loops**) استفاده می‌کنیم. حلقه بیرونی معمولاً بین سطرها حرکت می‌کند و حلقه درونی ستون‌های هر سطر را بررسی می‌کند.

برای $A[20][30]$:

```
FOR i = 0 TO 19:  
  FOR j = 0 TO 29:  
    Process A[i][j]  
  END FOR  
END FOR
```

ترتیب دسترسی به خانه‌ها به شکل زیر خواهد بود:

```
A[0][0]  
A[0][1]  
A[0][2]  
...  
A[0][29]  
  
A[1][0]  
A[1][1]  
...  
A[1][29]  
  
A[2][0]  
...
```

حلقه بیرونی برای سطرها و حلقه درونی برای ستون‌ها.

۷. آرایه‌های سه‌بعدی و چندبعدی

مفهوم آرایه را می‌توان به بیش از دو بعد نیز گسترش داد.

آرایه سه‌بعدی / تانسور

آرایه سه‌بعدی را می‌توان مانند مجموعه‌ای از چند جدول تصور کرد.

مثلاً:

```
C[page][row][column]
```

در اینجا سه اندیس داریم:

- صفحه
- سطر
- ستون

آرایه n-بعدی

به‌طور کلی می‌توان ساختمان‌هایی با ابعاد بیشتر نیز داشت:

```
D[dimension1][dimension2] ... [dimensionN]
```

در حوزه‌هایی مانند یادگیری ماشین و پردازش تصویر، مفهوم **Tensor** یا تانسور اهمیت زیادی دارد. تانسورها را می‌توان به‌صورت عمومی‌تر به‌عنوان داده‌هایی با چند محور یا بعد در نظر گرفت.

البته تانسور صرفاً معادل «آرایه چندبعدی» در همه زمینه‌ها نیست و در ریاضیات و یادگیری ماشین مفهوم دقیق‌تری دارد؛ اما برای شروع می‌توان ارتباط این دو مفهوم را به‌عنوان یک

مدل ذهنی در نظر گرفت.

مسئله ترکیبی: شمارش اعداد اول در آرایه ۳۰×۲۰

اکنون می‌توانیم چند مفهوم فصل‌های قبلی را در یک مسئله واحد ترکیب کنیم.

صورت مسئله

یک آرایه ۲۰×۳۰ از کاربر دریافت کنید و مشخص کنید چند عدد اول در این آرایه وجود دارد.

در این مسئله باید:

۱. یک آرایه دوبعدی داشته باشیم.
۲. تمام ۶۰۰ خانه را دریافت کنیم.
۳. تمام خانه‌ها را پیمایش کنیم.
۴. برای هر عدد بررسی کنیم که اول است یا خیر.
۵. تعداد اعداد اول را بشماریم.

الگوریتم اولیه تشخیص اعداد اول

در نسخه ساده‌تر، برای تشخیص اول بودن n ، مقسوم‌علیه‌های ممکن را از 2 تا $n/2$ بررسی می‌کنیم. ایده این است که اگر عددی غیر از 1 و خودش بتواند n را تقسیم کند، عدد اول نیست.

شبه‌کد:

Count Prime Numbers in 20x30 Array

```
1. START

2. DECLARE A[20][30]

3. FOR i = 0 TO 19:
    FOR j = 0 TO 29:
        READ A[i][j]
    END FOR
END FOR

4. SET NO = 0

5. FOR i = 0 TO 19:
    FOR j = 0 TO 29:

        SET prime = 1
        SET n = A[i][j]

        IF n < 2 THEN
            prime = 0

        ELSE
```

```

FOR k = 2 TO n/2:

    IF n % k == 0 THEN
        prime = 0
        BREAK
    END IF

END FOR

END IF

IF prime == 1 THEN
    NO = NO + 1
END IF

END FOR

END FOR

6. PRINT NO

7. END

```

الگوریتم بهینه شمارش اعداد اول

نسخه بهینه شده الگوریتم را می توان به صورت زیر نوشت:

Algorithm: Count Prime Numbers in 20x30 Array

1. START

2. DECLARE A[20][30]

3. FOR i = 0 TO 19:

 FOR j = 0 TO 29:

 READ A[i][j]

 END FOR

END FOR

4. SET NO = 0

5. FOR i = 0 TO 19:

 FOR j = 0 TO 29:

 SET n = A[i][j]

 SET prime = 1

 IF n < 2 THEN

 prime = 0

 ELSE

```

        SET limit = SQRT(n)

        FOR k = 2 TO limit:

            IF n % k == 0 THEN
                prime = 0
                BREAK
            END IF

        END FOR

        IF prime == 1 THEN
            NO = NO + 1
        END IF

    END FOR

6. PRINT NO

7. END

```

در این نسخه:

```
limit = SQRT(n)
```

حد بالای بررسی را مشخص می کند. سپس فقط تا همین مقدار پیش می رویم.

Peak Performance



۹. پیچیدگی زمانی

یکی از موضوعات مهم در علوم کامپیوتر، تحلیل مقدار محاسبات موردنیاز یک الگوریتم است. پیچیدگی زمانی (**Time Complexity**) نشان می‌دهد با افزایش اندازه ورودی، تعداد عملیات الگوریتم تقریباً با چه سرعتی افزایش پیدا می‌کند.

برای مثال، اگر یک آرایه 20×30 داشته باشیم:

$$20 \times 30 = 600$$

بنابراین برای پیمایش کامل آرایه باید ۶۰۰ خانه را بررسی کنیم.

اما اگر ابعاد آرایه متغیر باشند و تعداد سطرها را R و تعداد ستون‌ها را C در نظر بگیریم، تعداد خانه‌ها برابر است با:

$$R \times C$$

پس پیمایش کامل آرایه دارای پیچیدگی:

$$O(R \times C)$$

است.

حالا فرض کنید برای هر خانه، اول بودن یک عدد را نیز بررسی می‌کنیم. اگر برای تشخیص اول بودن، مقسوم‌علیه‌ها را تا $\sqrt{n}$ بررسی کنیم، برای هر خانه تقریباً $\sqrt{n}$ عملیات انجام می‌شود.

در نتیجه، پیچیدگی کلی الگوریتم تقریباً برابر است با:

$$O(R \times C \times \sqrt{n})$$

که در آن:

- تعداد سطرها R
- تعداد ستون‌ها C
- عددی که اول بودن آن بررسی می‌شود n

است.

چرا Big-O مهم است؟

فرض کنید دو الگوریتم داریم که یک مسئله را حل می‌کنند:

Algorithm A $\rightarrow O(n)$

Algorithm B $\rightarrow O(n^2)$

برای ورودی‌های کوچک ممکن است تفاوت زیادی احساس نشود؛ اما با بزرگ شدن n ، تعداد عملیات الگوریتم دوم بسیار سریع‌تر افزایش پیدا می‌کند.

به همین دلیل، در تحلیل الگوریتم معمولاً به جای شمارش دقیق تک‌تک عملیات، نحوه رشد تعداد عملیات نسبت به اندازه ورودی را بررسی می‌کنیم.

قواعد ساده تحلیل

چند الگوی بسیار رایج:

یک حلقه تا n :

```
for i = 1 to n
```

پیچیدگی:

$O(n)$

دو حلقه تو در تو تا n :

```
for i = 1 to n  
  for j = 1 to n
```

پیچیدگی:

$O(n^2)$

سه حلقه تو در تو:

$O(n^3)$

دو بخش مستقل پشت سر هم:

$O(n) + O(n^2)$

جمله غالب را نگه می‌داریم:

$O(n^2)$

حلقه‌ای که هر بار n را نصف می‌کند:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$$

پیچیدگی:

$$O(\log n)$$

چیت‌شیت پیچیدگی زمانی

پیچیدگی زمانی	اگر الگوریتم...
$O(1)$	فقط چند عملیات ثابت انجام دهد
$O(n)$	یک بار روی n داده پیمایش کند
$O(n)$	دو بار به‌صورت مستقل روی n داده پیمایش کند
$O(n^2)$	دو حلقه n تایی تو در تو داشته باشد
$O(n^3)$	سه حلقه n تایی تو در تو داشته باشد
$O(\log n)$	هر بار ورودی را نصف کند
$O(n \log n)$	برای هر عنصر، یک جستجوی دودویی انجام دهد
$O(n \log n)$	آرایه را مرتب‌سازی با Merge Sort کند
$O(n^2)$	هر جفت از عناصر را بررسی کند
$O(2^n)$	تمام زیرمجموعه‌های n عنصر را بررسی کند
$O(n!)$	تمام جایگشت‌های n عنصر را بررسی کند
$O(R \times C)$	آرایه $R \times C$ را کامل پیمایش کند

پیچیدگی زمانی	اگر الگوریتم...
$O(R \times C \times \sqrt{n})$	برای هر خانه $R \times C$ تا $\sqrt{n}$ بررسی انجام دهد

به صورت کلی، از نظر رشد تعداد عملیات معمولاً این ترتیب را داریم:

$O(1)$

↓

$O(\log n)$

↓

$O(n)$

↓

$O(n \log n)$

↓

$O(n^2)$

↓

$O(n^3)$

↓

$O(2^n)$

↓

$O(n!)$

نکته مهم: $O(\dots)$ معمولاً زمان دقیق اجرای برنامه بر حسب ثانیه نیست؛ بلکه نرخ

رشد هزینه الگوریتم را نسبت به اندازه ورودی توصیف می کند.

۱۰. الگوهای مهمی که در این فصل یاد گرفتیم

مسئله شمارش اعداد اول در آرایه دویبعدی، چندین الگوی بنیادی برنامه‌نویسی را هم‌زمان به کار می‌گیرد.

الگوی اول: پیمایش آرایه

```
FOR i:  
    process A[i]
```

الگوی دوم: پیمایش آرایه دویبعدی

```
FOR i:  
    FOR j:  
        process A[i][j]
```

الگوی سوم: شمارنده

```
count = 0  
  
if condition is True:  
    count = count + 1
```

الگوی چهارم: پرچم

```
flag = 1
```

```
If an undesirable condition is found:
```

```
    flag = 0
```

الگوی پنجم: توقف زودهنگام

```
If the result is determined:
```

```
    BREAK
```

ترکیب این الگوها بخش مهمی از حل مسئله در برنامه‌نویسی را تشکیل می‌دهد.

۱۱ جمع‌بندی فصل

در این فصل ابتدا با مفهوم کلی ساختمان داده آشنا شدیم و دیدیم که ساختمان داده روشی برای سازمان‌دهی و نگهداری داده‌ها در یک برنامه است. سپس ساختمان‌های داده را به‌طور کلی به دو گروه خطی و غیرخطی تقسیم کردیم و نمونه‌هایی مانند آرایه، پشته، صف و درخت را معرفی کردیم. در ادامه تمرکز اصلی را روی آرایه قرار دادیم و یاد گرفتیم که چگونه چند مقدار را تحت یک نام مشترک نگهداری می‌کند و عناصر آن با استفاده از اندیس قابل دسترسی هستند. همچنین با مفهوم اندیس‌گذاری از صفر، آخرین اندیس $N-1$ و دسترسی مستقیم به عناصر آشنا شدیم و دیدیم که حلقه‌ها ابزار اصلی پیمایش آرایه‌ها هستند. سپس آرایه‌های دوبعدی و چندبعدی را بررسی کردیم و یاد گرفتیم که در آرایه‌های دوبعدی معمولاً از یک حلقه برای پیمایش سطرها و حلقه‌ای دیگر برای پیمایش ستون‌ها استفاده می‌شود. در ادامه، الگوهایی مانند **Counter، Flag و Break** را در کنار آرایه‌ها به کار بردیم و با استفاده از عملگر $\%$ و بررسی مقسوم‌علیه‌ها، الگوریتم تشخیص عدد اول را بررسی کردیم. همچنین دیدیم که بررسی مقسوم‌علیه‌ها تا $\sqrt{n}$ می‌تواند نسبت به بررسی تا $n/2$ تعداد محاسبات را کاهش دهد و با مفهوم پیچیدگی زمانی به عنوان ابزاری برای تحلیل کارایی الگوریتم‌ها آشنا شدیم. در نهایت، دیدیم که ساختمان داده و الگوریتم مکمل یکدیگرند؛ ساختمان داده مشخص می‌کند داده‌ها چگونه سازمان‌دهی و نگهداری شوند و الگوریتم مشخص می‌کند چگونه روی آن‌ها عملیات انجام دهیم. در مسائل واقعی نیز معمولاً چندین مفهوم به‌صورت هم‌زمان ترکیب می‌شوند؛ همان‌طور که در مسئله‌ی پایانی فصل، آرایه‌ی دوبعدی، حلقه‌های تودرتو، شرط، **Flag، Counter، Break** و الگوریتم تشخیص عدد اول در کنار یکدیگر به کار گرفته شدند.

مرتب‌سازی و جستجو

فصل نهم



۱ مقدمه

یکی از مهم‌ترین کارهایی که روی داده‌ها انجام می‌دهیم، مرتب‌سازی (Sorting) و جستجو (Searching) است. فرض کنید آرایه‌ای از هزاران عدد، نام، شناسه یا رکورد در اختیار داریم. در بسیاری از مسائل، صرفاً نگهداری این داده‌ها کافی نیست؛ بلکه باید بتوانیم:

- آن‌ها را در یک ترتیب مشخص قرار دهیم؛
- یک مقدار خاص را در میان آن‌ها پیدا کنیم؛
- تعداد زیادی جستجو را با سرعت مناسب انجام دهیم؛
- و برای انجام این عملیات، الگوریتمی انتخاب کنیم که از نظر زمان و حافظه مناسب باشد.

مرتب‌سازی به معنای قرار دادن عناصر یک مجموعه در یک ترتیب مشخص، مانند ترتیب صعودی یا نزولی، است.

برای مثال:

آرایه اولیه

[7, 2, 9, 1, 5]

مرتب‌سازی ↓

[1, 2, 5, 7, 9]

مرتب‌سازی در بسیاری از مسائل دیگر نیز نقش پایه‌ای دارد. برای مثال، جستجوی دودویی

فقط زمانی می‌تواند از مزیت اصلی خود استفاده کند که داده‌ها (Binary Search) مرتب باشند. در این فصل ابتدا چند الگوریتم مهم مرتب‌سازی را بررسی می‌کنیم و سپس به دو الگوریتم مهم جستجو، یعنی جستجوی خطی و جستجوی دودویی، می‌رسیم.

الگوریتم‌های مرتب‌سازی بررسی شده عبارت‌اند از:

۱. مرتب‌سازی حبابی (Bubble Sort)

۲. مرتب‌سازی ادغامی (Merge Sort)

۳. مرتب‌سازی درجی (Insertion Sort)

۴. مرتب‌سازی هیپی (Heap Sort)

۵. مرتب‌سازی سریع (Quick Sort)

پس دو روش اصلی جستجو را بررسی خواهیم کرد:

۱. جستجوی خطی (Linear Search)

۲. جستجوی دودویی (Binary Search)

۲. مرتب‌سازی

۲.۱ مرتب‌سازی حبابی (Bubble Sort)

مرتب‌سازی حبابی (Bubble Sort) یکی از ساده‌ترین الگوریتم‌های مرتب‌سازی است و به همین دلیل برای یادگیری مفهوم مرتب‌سازی بسیار مناسب است. ایده اصلی آن بسیار ساده است:

عناصر مجاور را با یکدیگر مقایسه کن و اگر ترتیبشان اشتباه بود، آن‌ها را جابه‌جا کن.

فرض کنید آرایه زیر را داریم:

[5, 2, 8, 1]

ابتدا 5 و 2 را مقایسه می‌کنیم:

$5 > 2$

پس ترتیب آن‌ها اشتباه است و جابه‌جا می‌شوند:

[2, 5, 8, 1]

سپس 5 و 8 را مقایسه می‌کنیم:

$5 < 8$

پس نیازی به جابه‌جایی نیست.

در مرحله بعد 8 و 1 مقایسه می‌شوند:

$$8 > 1$$

بنابراین:

[2, 5, 1, 8]

اکنون بزرگ‌ترین عنصر، یعنی 8، به انتهای آرایه رسیده است.

به همین دلیل نام این الگوریتم **حبابی** است؛ زیرا عنصر بزرگ‌تر در هر پیمایش مانند حبابی که به سطح آب می‌آید، به سمت انتهای آرایه حرکت می‌کند.



۲.۱.۱ نسخه پایه Bubble Sort

فرض کنید آرایه‌ای با 100 عنصر داریم. در ساده‌ترین نسخه، 99 بار آرایه را پیمایش می‌کنیم و در هر پیمایش، عناصر مجاور را با یکدیگر مقایسه می‌کنیم.

Algorithm: Bubble Sort (Basic Version)

1. START

2. DECLARE A[100]

3. FOR i = 0 TO 99:

 READ A[i]

```

END FOR

4. FOR i = 1 TO 99:
    FOR j = 0 TO 98:
        IF A[j] > A[j+1] THEN
            SET temp = A[j]
            SET A[j] = A[j+1]
            SET A[j+1] = temp
        END IF
    END FOR
END FOR

5. FOR i = 0 TO 99:
    PRINT A[i]
END FOR

6. END

```

در این نسخه، در هر بار اجرای حلقه داخلی، 99 مقایسه انجام می‌شود و حلقه بیرونی نیز 99 بار اجرا می‌شود. بنابراین تعداد مقایسه‌ها در این نسخه برابر است با:

$$99 \times 99 = 9801$$

این مقدار برای آرایه‌ای با فقط ۱۰۰ عنصر نسبتاً زیاد است و نشان می‌دهد که می‌توان الگوریتم را بهتر کرد.

۲.۱.۲ بهینه‌سازی اول: کاهش محدوده مقایسه

در نسخه قبلی یک نکته مهم نادیده گرفته شده بود. پس از اولین پیمایش، بزرگ‌ترین عنصر حتماً به آخرین خانه رسیده است. بنابراین در پیمایش دوم دیگر نیازی نداریم آخرین خانه را دوباره بررسی کنیم. پس از پیمایش دوم، دو عنصر انتهایی مرتب هستند. به طور کلی، پس از پیمایش i ام، i عنصر انتهایی در جای درست خود قرار گرفته‌اند. بنابراین می‌توانیم محدوده حلقه داخلی را هر بار کوچک‌تر کنیم:

Algorithm: Bubble Sort (Optimized)

1. START

2. DECLARE A[100]

3. FOR $i = 0$ TO 99:

 READ A[i]

END FOR

4. FOR $i = 1$ TO 99:

 FOR $j = 0$ TO $99 - i$:

 IF $A[j] > A[j+1]$ THEN

 SWAP A[j] AND A[j+1]

 END IF

 END FOR

```
END FOR
```

```
5. FOR i = 0 TO 99:
```

```
    PRINT A[i]
```

```
END FOR
```

```
6. END
```

اکنون تعداد مقایسه‌ها برابر است با:

$$99 + 98 + 97 + \dots + 1$$

مجموع اعداد ۱ تا 99 برابر است با:

$$99 \times 100 / 2 = 4950$$

بنابراین تعداد مقایسه‌ها تقریباً نصف نسخه پایه شده است.

۲.۱.۳ بهینه‌سازی دوم: استفاده از پرچم

یک بهینه‌سازی مهم‌تر نیز وجود دارد. فرض کنید آرایه از ابتدا مرتب باشد:

```
[1, 2, 3, 4, 5]
```

در اولین پیمایش، هیچ دو عنصری نیاز به جابه‌جایی ندارند. اگر بتوانیم این موضوع را تشخیص دهیم، دیگر نیازی نیست پیمایش‌های بعدی را انجام دهیم. برای این کار از یک متغیر پرچم یا **Flag** استفاده می‌کنیم.

ایده:

- در ابتدای هر پیمایش، $m = 0$ قرار می‌دهیم.
- اگر حتی یک جابه‌جایی انجام شد، $m = 1$ می‌شود.
- اگر پیمایش تمام شد و m همچنان 0 بود، یعنی هیچ جابه‌جایی انجام نشده است.
- در نتیجه آرایه مرتب شده و می‌توان الگوریتم را متوقف کرد.

Algorithm: Bubble Sort (Flag Optimized)

```
1. START

2. DECLARE A[100]

3. FOR i = 0 TO 99:
    READ A[i]
END FOR

4. SET m = 1
5. SET i = 1

6. WHILE (i ≤ 99 AND m == 1):

    SET m = 0

    FOR j = 0 TO 99 - i:
```



```

        IF A[j] > A[j+1] THEN
            SWAP A[j] AND A[j+1]
            SET m = 1
        END IF

    END FOR

    SET i = i + 1

END WHILE

7. FOR i = 0 TO 99:
    PRINT A[i]
END FOR

8. END

```

در اینجا **m** در واقع وضعیت «آیا در این پیمایش جابه‌جایی رخ داده است؟» را نگهداری می‌کند.

۲.۱.۴. تحلیل Bubble Sort

عملکرد Bubble Sort به وضعیت اولیه آرایه نیز بستگی دارد.

بهترین حالت

اگر آرایه از ابتدا مرتب باشد و از نسخه دارای Flag استفاده کنیم، پس از اولین پیمایش هیچ جابه‌جایی رخ نمی‌دهد. بنابراین تقریباً:

$$n - 1$$

مقایسه انجام می‌شود.

برای $n = 100$:

99 مقایسه.

بدترین حالت

در بدترین حالت، مانند آرایه‌ای که کاملاً برعکس مرتب شده است، تعداد مقایسه‌های نسخه بهینه‌شده برابر است با:

$$(n - 1) + (n - 2) + \dots + 1$$

که برابر است با:

$$n(n - 1) / 2$$

و در نتیجه مرتبه زمانی آن:

$$O(n^2)$$

است.

جمع‌بندی

حالت	تعداد تقریبی مقایسه‌ها
بهترین حالت با Flag	$n - 1$
بدترین حالت	$n(n-1)/2$
پیچیدگی بدترین حالت	$O(n^2)$

نکته مهم این است که بهینه‌سازی Flag، مرتبه زمانی بدترین حالت را تغییر نمی‌دهد؛ فقط باعث می‌شود در ورودی‌های تقریباً مرتب یا کاملاً مرتب، الگوریتم زودتر متوقف شود.

۲.۲ مرتب‌سازی ادغامی (Merge Sort)

Merge Sort بر اساس روش تقسیم و حل (Divide and Conquer) کار می‌کند.

اگر دو بخش از آرایه از قبل مرتب باشند، می‌توان آن‌ها را با یک پیمایش خطی به شکلی بسیار ساده با یکدیگر ادغام کرد.

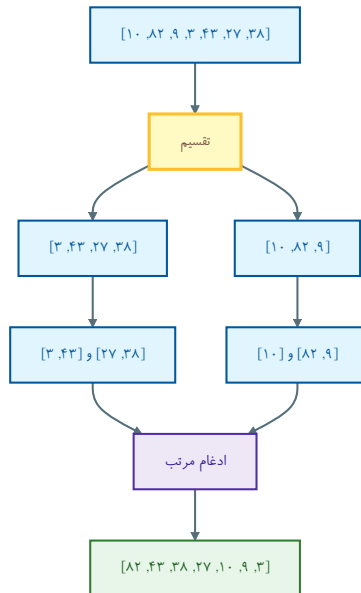
بنابراین به جای اینکه مستقیماً کل آرایه را مرتب کنیم:

۱. آرایه را به دو نیمه تقسیم می‌کنیم.
 ۲. هر نیمه را دوباره تقسیم می‌کنیم.
 ۳. این کار را ادامه می‌دهیم تا به بخش‌های تک‌عنصری برسیم.
 ۴. سپس بخش‌ها را به صورت مرتب با یکدیگر ادغام می‌کنیم.
- یک آرایه تک‌عنصری به طور ذاتی مرتب است. برای مثال:

[38, 27, 43, 3, 9, 82, 10]

به بخش‌های کوچک‌تر تقسیم می‌شود و سپس هنگام بازگشت، بخش‌ها با ترتیب صحیح با

یکدیگر ادغام می‌شوند.



۲.۲.۱ ادغام دو آرایه مرتب

فرض کنید دو آرایه مرتب داریم:

A = [...] با 20 عنصر

B = [...] با 30 عنصر

و می‌خواهیم آن‌ها را در آرایه C با ۵۰ عنصر ادغام کنیم. در هر مرحله، اولین عنصر استفاده‌نشده از A و اولین عنصر استفاده‌نشده از B را مقایسه می‌کنیم. هر کدام کوچک‌تر بود، وارد C می‌شود.

Merge Two Sorted Arrays

```
1. START

2. DECLARE A[20], B[30], C[50]

3. SET k = 0
4. SET h = 0

5. FOR i = 0 TO 49:

    IF k < 20 AND h < 30 THEN

        IF A[k] < B[h] THEN
            C[i] = A[k]
            k = k + 1
        ELSE
            C[i] = B[h]
            h = h + 1
        END IF

    ELSE IF k < 20 THEN

        C[i] = A[k]
```

```

        k = k + 1

    ELSE IF h < 30 THEN

        C[i] = B[h]
        h = h + 1

    END IF

END FOR

6. FOR i = 0 TO 49:
    PRINT C[i]
END FOR

7. END

```

سه حالت اصلی وجود دارد:

وضعیت	عملیات
هر دو آرایه هنوز عنصر دارند	عنصر کوچک‌تر را انتخاب کن
فقط A عنصر دارد	عنصر بعدی A را بردار
فقط B عنصر دارد	عنصر بعدی B را بردار

۲.۲.۲ الگوریتم کامل Merge Sort

مرتب‌سازی ادغامی از بازگشت (Recursion) استفاده می‌کند. شرط پایه این است که اگر محدوده فقط یک عنصر داشته باشد، دیگر کاری برای انجام دادن وجود ندارد.

Algorithm: Merge Sort

1. IF $\text{left} \geq \text{right}$ THEN
 RETURN
END IF
2. SET $\text{mid} = (\text{left} + \text{right}) / 2$
3. CALL MergeSort(A, left, mid)
4. CALL MergeSort(A, mid + 1, right)
5. CALL Merge(A, left, mid, right)

پس ابتدا نیمه چپ و نیمه راست را مرتب می‌کنیم و سپس دو نیمه مرتب‌شده را با یکدیگر ادغام می‌کنیم. تابع Merge می‌تواند به صورت زیر باشد:

Algorithm: Merge

1. CREATE temp array of size $(\text{right} - \text{left} + 1)$

```
2. SET i = left
3. SET j = mid + 1
4. SET k = 0

5. WHILE i ≤ mid AND j ≤ right:
```

```
    IF A[i] ≤ A[j] THEN
```

```
        temp[k] = A[i]
```

```
        i = i + 1
```

```
    ELSE
```

```
        temp[k] = A[j]
```

```
        j = j + 1
```

```
    END IF
```

```
    k = k + 1
```

```
END WHILE
```

```
6. WHILE i ≤ mid:
```

```
    temp[k] = A[i]
```

```
    i = i + 1
```

```
    k = k + 1
```

```
END WHILE
```



```
7. WHILE  $j \leq \text{right}$ :
```

```
    temp[k] = A[j]
```

```
     $j = j + 1$ 
```

```
     $k = k + 1$ 
```

```
END WHILE
```

```
8. COPY temp BACK TO A[left..right]
```

۲.۲.۳ تعداد مقایسه‌ها در Merge

اگر دو آرایه مرتب با طول‌های m و n را ادغام کنیم:

- در بهترین حالت ممکن است تقریباً $\min(m, n)$ مقایسه لازم باشد.
- در بدترین حالت، حداکثر $m+n-1$ مقایسه انجام می‌شود.
- در هر صورت، زمان ادغام از مرتبه $O(m+n)$ است.

نکته مهم این است که Merge Sort در هر سطح از تقسیم، در مجموع تقریباً n عنصر را پردازش می‌کند و تعداد سطوح نیز حدود $\log_2 n$ است. بنابراین پیچیدگی زمانی آن:

$$O(n \log n)$$

است.

برای مثال، اگر $n = 8$ باشد:

$$\log_2(8) = 3$$

و تعداد مقایسه‌ها در بدترین حالت دقیقاً $n \log_2 n$ نیست؛ این رابطه فقط مرتبه رشد را نشان می‌دهد. تعداد واقعی مقایسه‌ها به نحوه توزیع عناصر بستگی دارد.

۲.۲.۴. ویژگی‌های Merge Sort

ویژگی	توضیح
ایده اصلی	تقسیم، حل بازگشتی، ادغام
بهترین حالت	$O(n \log n)$
حالت متوسط	$O(n \log n)$
بدترین حالت	$O(n \log n)$
حافظه جانبی	معمولاً $O(n)$
پایداری	می‌تواند Stable باشد
نکته مهم	زمان بدترین حالت تضمین‌شده است

یکی از معایب اصلی Merge Sort این است که برای ادغام معمولاً به آرایه موقت نیاز دارد.

۲.۳ مرتب‌سازی درجی (Insertion Sort)

Insertion Sort را می‌توان با مرتب کردن کارت‌های بازی (پاسور) در دست مقایسه کرد. فرض کنید تعدادی کارت مرتب در دست داریم:

[2, 5, 8]

اکنون کارت 4 را دریافت می‌کنیم.

از سمت راست شروع می‌کنیم:

$$4 < 8$$

پس 8 را یک خانه به راست می‌بریم.

سپس:

$$4 < 5$$

پس 5 نیز یک خانه به راست منتقل می‌شود.

در نهایت:

$$4 > 2$$

بنابراین 4 بعد از 2 قرار می‌گیرد:

[2, 4, 5, 8]

این دقیقاً ایده Insertion Sort است.

۲.۳.۱ روش مناسب برای پیدا کردن جایگاه

می‌توان برای پیدا کردن محل مناسب عنصر جدید از دو جهت حرکت کرد.

روش اول: چپ به راست

ابتدا از ابتدای بخش مرتب‌شده حرکت می‌کنیم و جایگاه مناسب عنصر جدید را پیدا

می‌کنیم. سپس عناصر بعدی را یک خانه به راست منتقل می‌کنیم.

روش دوم: راست به چپ

از آخرین عنصر بخش مرتب‌شده شروع می‌کنیم.

تا زمانی که عنصر فعلی از مقدار جدید بزرگ‌تر است:

۱. آن را یک خانه به راست منتقل می‌کنیم.

۲. به عنصر قبلی می‌رویم.

وقتی به محل مناسب رسیدیم، مقدار جدید را در آن قرار می‌دهیم.

روش دوم از نظر پیاده‌سازی طبیعی‌تر است، زیرا جستجوی محل و شیف دادن عناصر را هم‌زمان انجام می‌دهد.

۲.۳.۲ نسخه چپ به راست

Algorithm: Insertion Sort (Left to Right)

1. START

2. DECLARE A[1000]

3. FOR i = 0 TO 999:

 READ n

 IF i == 0 THEN

```

        A[0] = n

    ELSE

        SET k = 0

        WHILE (k < i AND n > A[k]):
            k = k + 1
        END WHILE

        FOR j = i DOWNTO k + 1:
            A[j] = A[j-1]
        END FOR

        A[k] = n

    END IF

END FOR

4. FOR i = 0 TO 999:
    PRINT A[i]
END FOR

```

5. END

۲.۳.۳ نسخه راست به چپ

نسخه رایج‌تر و ساده‌تر:

Algorithm: Insertion Sort (Right to Left)

1. START

2. DECLARE A[1000]

3. READ A[0]

4. FOR i = 1 TO 999:

 READ n

 SET j = i - 1

 WHILE (j $\geq$ 0 AND A[j] > n):

 A[j+1] = A[j]

 j = j - 1

```
END WHILE
```

```
A[j+1] = n
```

```
END FOR
```

```
5. FOR i = 0 TO 999:
```

```
    PRINT A[i]
```

```
END FOR
```

```
6. END
```

در این الگوریتم، در هر مرحله بخش:

```
A[0] ... A[i-1]
```

مرتب است.

عنصر جدید **n** در جای مناسب خود قرار می‌گیرد و در نتیجه بخش:

```
A[0] ... A[i]
```

نیز مرتب خواهد بود.

۲.۳.۴. تحلیل Insertion Sort

بهترین حالت

اگر آرایه از قبل مرتب باشد، برای هر عنصر جدید فقط کافی است یک مقایسه انجام شود و شیف‌ت قابل‌توجهی رخ نمی‌دهد.

بنابراین:

$$O(n)$$

بدترین حالت

اگر آرایه کاملاً معکوس باشد، هر عنصر جدید باید تقریباً از کل بخش مرتب‌شده عبور کند. تعداد عملیات از مرتبه:

$$1 + 2 + 3 + \dots + (n-1)$$

است و در نتیجه:

$$O(n^2)$$

حالت متوسط

در حالت متوسط نیز پیچیدگی زمانی:

$$O(n^2)$$

است.

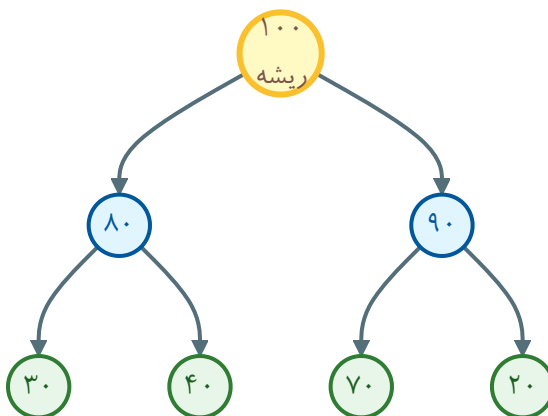
حالت	پیچیدگی
بهترین	$O(n)$
متوسط	$O(n^2)$
بدترین	$O(n^2)$

Insertion Sort در آرایه‌های کوچک یا داده‌هایی که از قبل تقریباً مرتب هستند می‌تواند عملکرد بسیار مناسبی داشته باشد.

۲.۴ مرتب‌سازی هیپی (Heap Sort)

Heap یک ساختار داده مبتنی بر درخت دودویی کامل (Complete Binary Tree) است. در **Max-Heap** مقدار هر گره پدر بزرگ‌تر یا مساوی فرزندان آن است.

برای مثال:



در این مثال، 100 بزرگ‌ترین مقدار است و در ریشه قرار دارد.

وقتی Heap را در قالب آرایه نمایش می‌دهیم، برای گره موجود در اندیس i معمولاً:

```
left = 2i + 1
```

```
right = 2i + 2
```

است.

این همان رابطه‌ای است که در الگوریتم **Heapify** استفاده می‌شود.

برای مرتب‌سازی صعودی با Max-Heap:

۱. آرایه را به Max-Heap تبدیل می‌کنیم.

۲. بزرگ‌ترین عنصر در خانه اول قرار دارد.

۳. عنصر اول را با آخرین عنصر بخش فعال آرایه جابه‌جا می‌کنیم.

۴. اندازه Heap را یک واحد کاهش می‌دهیم.

۵. Heap را دوباره بازسازی می‌کنیم.

۶. این کار را تا مرتب شدن کامل آرایه ادامه می‌دهیم.

۲.۴.۱ الگوریتم Heap Sort

Algorithm: Heap Sort

1. START

2. DECLARE $A[n]$

```

3. FOR i = 0 TO n-1:
    READ A[i]
END FOR

4. FOR i = (n/2) - 1 DOWNTO 0:
    CALL Heapify(A, n, i)
END FOR

5. FOR i = n - 1 DOWNTO 1:

    SWAP A[0] AND A[i]

    CALL Heapify(A, i, 0)

END FOR

6. FOR i = 0 TO n-1:
    PRINT A[i]
END FOR

7. END

```

۲.۴.۲ الگوریتم Heapify

وظیفه **Heapify** این است که در صورت نقض قانون **Heap**، عنصر مورد نظر را به

سمت پایین حرکت دهد تا ساختار Heap دوباره برقرار شود.

Algorithm: Heapify

1. SET largest = i
2. SET left = 2 * i + 1
3. SET right = 2 * i + 2
4. IF left < n AND A[left] > A[largest] THEN
 SET largest = left
END IF
5. IF right < n AND A[right] > A[largest] THEN
 SET largest = right
END IF
6. IF largest $\neq$ i THEN

 SWAP A[i] AND A[largest]

 CALL Heapify(A, n, largest)

END IF

از نظر زمانی در بهترین، متوسط و بدترین حالت دارای مرتبه Heap Sort

$O(n \log n)$

است و در پیاده‌سازی معمول آرایه‌ای، حافظه جانبی آن $O(1)$ است.

۲.۵ مرتب‌سازی سریع (Quick Sort)

Quick Sort نیز از روش **Divide and Conquer** استفاده می‌کند.

سه مرحله اصلی آن عبارت‌اند از:

۱. انتخاب یک عنصر به نام **محور (Pivot)**.
۲. انجام **افراز (Partition)** آرایه نسبت به محور.
۳. اجرای بازگشتی همین کار روی بخش‌های چپ و راست.

فرض کنید:

[7, 2, 1, 6, 8, 5, 3, 4]

و محور را 4 انتخاب کنیم.

پس از **Partition**، عناصر کوچک‌تر یا مساوی محور در سمت چپ و عناصر بزرگ‌تر در سمت راست قرار می‌گیرند:

[2, 1, 3] [4] [7, 6, 8, 5]

نکته مهم این است که پس از Partition، خود Pivot در جایگاه نهایی خود قرار گرفته است.

۲.۵.۱ الگوریتم Quick Sort

Algorithm: Quick Sort

1. IF $\text{left} \geq \text{right}$ THEN

 RETURN

END IF

2. SET $\text{pivot} = A[\text{right}]$

3. SET $i = \text{left} - 1$

4. FOR $j = \text{left}$ TO $\text{right} - 1$:

 IF $A[j] \leq \text{pivot}$ THEN

$i = i + 1$

 SWAP $A[i]$ AND $A[j]$

 END IF

```
END FOR

5. SWAP A[i+1] AND A[right]

6. SET pivotIndex = i + 1

7. CALL QuickSort(A, left, pivotIndex - 1)

8. CALL QuickSort(A, pivotIndex + 1, right)
```

برای آرایه‌ای با ۱۰۰۰ عنصر:

```
Algorithm: Quick Sort

1. START

2. DECLARE A[1000]

3. FOR i = 0 TO 999:
    READ A[i]
END FOR

4. CALL QuickSort(A, 0, 999)
```

```
5. FOR i = 0 TO 999:
```

```
    PRINT A[i]
```

```
END FOR
```

```
6. END
```

۲.۵.۲ تحلیل Quick Sort

عملکرد Quick Sort به نحوه انتخاب Pivot بستگی دارد. اگر Pivot ها هر بار تقریباً آرایه را به دو بخش متعادل تقسیم کنند، پیچیدگی متوسط الگوریتم:

$$O(n \log n)$$

خواهد بود. اما اگر انتخاب Pivot بسیار نامناسب باشد و هر بار یکی از بخش‌ها تقریباً خالی و بخش دیگر تقریباً به اندازه کل آرایه باشد، پیچیدگی می‌تواند به:

$$O(n^2)$$

برسد. بنابراین برخلاف Merge Sort. $O(n \log n)$ برای Quick Sort در همه ورودی‌ها تضمین شده نیست.

۲.۶ مفهوم پایداری در مرتب‌سازی

یکی از ویژگی‌های مهم بعضی الگوریتم‌های مرتب‌سازی، پایداری (Stability) است. فرض کنید اطلاعات دانشجویان را بر اساس نمره مرتب کنیم و دو دانشجو نمره یکسان داشته باشند:

Ali 18
Sara 18
Reza 15

اگر قبل از مرتب‌سازی Ali قبل از Sara بوده باشد، یک مرتب‌سازی پایدار ترتیب نسبی این دو را حفظ می‌کند:

Reza 15
Ali 18
Sara 18

بنابراین:

مرتب‌سازی پایدار، ترتیب نسبی عناصر دارای کلید یکسان را حفظ می‌کند.

این ویژگی زمانی اهمیت پیدا می‌کند که داده‌ها بیش از یک ویژگی داشته باشند.

۲.۷ مقایسه الگوریتم‌های مرتب‌سازی

الگوریتم	بهترین حالت	حالت متوسط	بدترین حالت	حافظه جانبی معمول	پایداری
Bubble Sort	$O(n)$ *	$O(n^2)$	$O(n^2)$	$O(1)$	بله
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	بله

الگوریتم	بهترین حالت	حالت متوسط	بدترین حالت	حافظه جانبی معمول	پایداری
Merge Sort	$O(n)$	$O(n)$	$O(n)$	$O(n)$	بله، در پیاده‌سازی مناسب
Heap Sort	$O(n)$	$O(n)$	$O(n)$	$O(1)$	خیر
Quick Sort	$O(n)$	$O(n)$	$O(n^2)$	معمولاً $O(\log n)$ برای عمق بازگشت متوسط	خیر، در پیاده‌سازی معمول

- در Bubble Sort این بهترین حالت به نسخه‌ای مربوط است که از Flag برای توقف زودهنگام استفاده می‌کند.

هیچ الگوریتمی در تمام شرایط به شکل مطلق «بهترین» نیست. انتخاب الگوریتم به عواملی مانند اندازه داده، میزان مرتب بودن اولیه، محدودیت حافظه، نیاز به پایداری و نحوه استفاده از داده‌ها بستگی دارد.

۳. جستجو

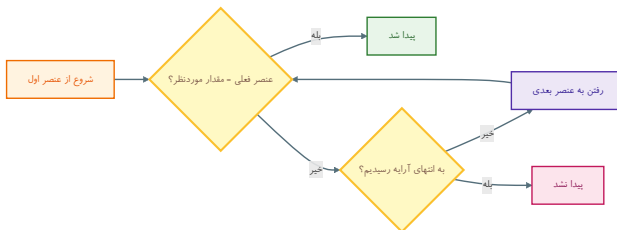
۳.۱ جستجوی خطی (Linear Search)

یکی از عملیات بسیار رایج در برنامه‌نویسی، پیدا کردن یک مقدار مشخص در میان مجموعه‌ای از داده‌ها است. فرض کنید آرایه زیر را داریم:

[17, 4, 91, 23, 8, 42, 5]

و می‌خواهیم بدانیم آیا 42 در آرایه وجود دارد یا خیر. ساده‌ترین راه این است که از اولین عنصر شروع کنیم و عناصر را یکی‌یکی بررسی کنیم. این روش **جستجوی خطی (Linear Search)** نام دارد. ایده آن را می‌توان این‌طور خلاصه کرد:

از ابتدا شروع کن. عناصر را یکی‌یکی بررسی کن و تا زمانی ادامه بده که مقدار مورد نظر را پیدا کنی یا به انتهای آرایه بررسی.



۳.۱.۱ ویژگی مهم Linear Search

مهم‌ترین ویژگی جستجوی خطی این است که به مرتب بودن آرایه نیاز ندارد. برای مثال:

[17, 4, 91, 23, 8, 42, 5]

با وجود نامرتب بودن، می‌توانیم 42 را با Linear Search پیدا کنیم. در مقابل، Binary Search که بعداً بررسی می‌کنیم، به مرتب بودن داده‌ها نیاز دارد.

۳.۱.۲ مزایا و محدودیت‌ها

مزایا	محدودیت‌ها
ساده است	در داده‌های بزرگ می‌تواند کند باشد
پایه‌سازی آسانی دارد	ممکن است تمام عناصر را بررسی کند
به مرتب بودن داده‌ها نیاز ندارد	برای جستجوهای بسیار زیاد ممکن است مناسب نباشد

۳.۱.۳ جستجوی خطی برای یک مقدار

فرض کنید آرایه‌ای با ۱۰۰۰ عنصر داریم و می‌خواهیم یک عدد مشخص را پیدا کنیم.

Algorithm: Linear Search

1. START
2. DECLARE A[1000]
3. FOR i = 0 TO 999:
 READ A[i]
END FOR
4. READ num

```
5. SET found = 0

6. FOR i = 0 TO 999:

    IF A[i] == num THEN

        SET found = 1
        SET position = i

        BREAK

    END IF

END FOR

7. IF found == 1 THEN
    PRINT "Found at position", position
ELSE
    PRINT "Not found"
END IF

8. END
```

فرض کنید:

```
A = [12, 7, 25, 4, 18, 31]
```

:9

```
num = 18
```

مقایسه‌ها:

```
12 == 18 → خیر
```

```
7 == 18 → خیر
```

```
25 == 18 → خیر
```

```
4 == 18 → خیر
```

```
18 == 18 → بله
```

به محض پیدا شدن 18، اجرای حلقه متوقف می‌شود.

۳.۱.۴. متغیر Flag در جستجوی خطی

در الگوریتم بالا از متغیری به نام `found` استفاده کردیم. این متغیر یک پرچم (**Flag**) است.

```
found = 0
```

یعنی:

هنوز مقدار مورد نظر پیدا نشده است.

:9

```
found = 1
```

یعنی:

مقدار مورد نظر پیدا شده است.

اگر چند جستجوی مستقل انجام دهیم، باید قبل از شروع هر جستجوی جدید دوباره:

```
found = 0
```

قرار دهیم. در غیر این صورت ممکن است وضعیت جستجوی قبلی به جستجوی جدید منتقل شود.

۳.۱.۵. جستجوی چند مقدار

فرض کنید آرایه‌ای شامل ۱۰۰۰ عنصر داریم و پس از آن ۱۰ مقدار مختلف دریافت می‌کنیم. برای هر مقدار باید یک جستجوی مستقل انجام شود.

Algorithm: Linear Search for Multiple Queries

1. START

```
2. DECLARE A[1000]

3. FOR k = 0 TO 999:
    READ A[k]
END FOR

4. FOR i = 1 TO 10:

    READ num

    SET found = 0

    FOR j = 0 TO 999:

        IF A[j] == num THEN

            SET found = 1

            PRINT "Found at position", j + 1

            BREAK

        END IF
```



```
END FOR
```

```
IF found == 0 THEN
```

```
    PRINT "Not found"
```

```
END IF
```

```
END FOR
```

```
5. END
```

ساختار کلی مسئله:

دریافت آرایه



شروع جستجوی اول



بررسی عناصر آرایه



شروع جستجوی دوم



بررسی عناصر آرایه



...



جستجوی دهم

این مثال نمونه‌ای از **حلقه‌های تودرتو** نیز هست؛ یک حلقه برای جستجوهای مختلف داریم و درون آن، حلقه دیگری برای پیمایش آرایه قرار گرفته است.

۳.۱.۶. تفاوت Position و Index

در الگوریتم قبلی از:

$$j + 1$$

استفاده کردیم.

دلیل این است که در بسیاری از زبان‌های برنامه‌نویسی، اندیس آرایه از صفر شروع می‌شود. مثلاً:

$$A = [10, 20, 30, 40, 50]$$

دارای اندیس‌های:

مقدار	Index
۱۰	۰
۲۰	۱
۳۰	۲
۴۰	۳
۵۰	۴

بنابراین 40 در:

```
index = 3
```

قرار دارد. اما اگر بخواهیم آن را برای کاربر به صورت «عنصر چهارم» نمایش دهیم:

```
position = 3 + 1 = 4
```

پس باید تفاوت دو مفهوم را به خاطر داشته باشیم:

– **Index**: اندیس داخلی آرایه، معمولاً از صفر.

– **Position**: شماره‌گذاری قابل نمایش برای کاربر، که می‌تواند از یک شروع شود.

۳.۱.۷. تعداد مقایسه‌ها در Linear Search

اگر آرایه n عنصر داشته باشد، تعداد مقایسه‌ها به محل قرارگیری مقدار مورد نظر بستگی دارد.

بهترین حالت

اگر مقدار در اولین عنصر باشد:

```
1 comparison
```

بدترین حالت

اگر مقدار در آخرین عنصر باشد یا اصلاً وجود نداشته باشد:

```
n comparisons
```

حالت متوسط

در یک تحلیل ساده، حدوداً:

$$n / 2$$

مقایسه انجام می‌شود.

حالت	تعداد تقریبی مقایسه
بهترین	1
متوسط	$n/2$
بدترین	n

برای $n = 1000$:

حالت	مقایسه
بهترین	1
متوسط	حدود 500
بدترین	1000

اگر ۱۰ جستجو داشته باشیم و هر ۱۰ مورد در بدترین حالت قرار بگیرند:

$$10 \times 1000 = 10000$$

مقایسه انجام می‌شود.

۳.۱.۸. شرط ادامه و نقش Break

گاهی حلقه جستجوی خطی را می‌توان با شرطی مانند زیر طراحی کرد:

```
j < n AND found == 0
```

یعنی:

تا زمانی که هنوز به انتهای آرایه نرسیده‌ایم و مقدار مورد نظر را پیدا نکرده‌ایم، جستجو را ادامه بده.

همچنین می‌توان هنگام پیدا شدن مقدار از **break** استفاده کرد:

```
IF A[j] == num THEN  
    found = 1  
    BREAK  
END IF
```

break باعث می‌شود همان حلقه‌ای که دستور در آن قرار دارد متوقف شود. البته اگر هدف پیدا کردن تمام محل‌های وقوع یک مقدار باشد، نباید با اولین پیدا شدن از حلقه خارج شویم.

۳.۲ جستجوی دودویی (Binary Search)

جستجوی خطی ساده است، اما در آرایه‌های بزرگ ممکن است مجبور شویم تعداد زیادی عنصر را بررسی کنیم. جستجوی دودویی (Binary Search) ایده متفاوتی دارد:

به جای بررسی تک‌تک عناصر، در هر مرحله تقریباً نیمی از محدوده جستجو را حذف کن.

اما برای استفاده از این روش یک شرط اساسی وجود دارد:

داده‌ها باید مرتب باشند.

برای مثال:

[3, 7, 12, 18, 25, 31, 42, 56]

برای Binary Search مناسب است.

اما:

[18, 3, 42, 7, 31, 12, 56, 25]

مرتب نیست و نمی‌توان به شکل معمول از اطلاعات ترتیب برای حذف نصف آرایه استفاده کرد.

۳.۲.۱ ایده اصلی Binary Search

فرض کنید می‌خواهیم 42 را در یک آرایه مرتب پیدا کنیم. ابتدا عنصر وسط را بررسی می‌کنیم. سه حالت ممکن است:

حالت اول: مقدار وسط برابر هدف است

```
A[mid] == 42
```

پس مقدار پیدا شده است.

حالت دوم: مقدار وسط بزرگ‌تر از هدف است

```
A[mid] > 42
```

چون آرایه مرتب است، تمام عناصر سمت راست `mid` نیز از `42` بزرگ‌تر یا مساوی خواهند بود. پس می‌توانیم نیمه راست را کنار بگذاریم.

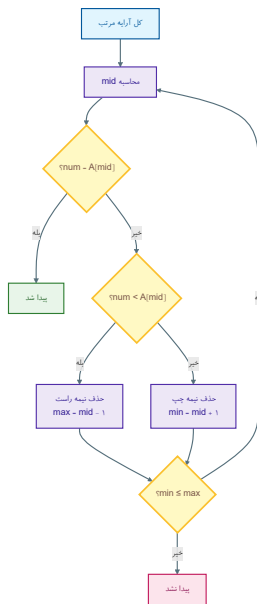
حالت سوم: مقدار وسط کوچک‌تر از هدف است

```
A[mid] < 42
```

پس مقدار مورد نظر نمی‌تواند در نیمه چپ باشد و نیمه چپ را کنار می‌گذاریم. بنابراین در هر مرحله تقریباً نصف فضای جستجو حذف می‌شود.

۳.۲.۲ تشبیه فرهنگ لغت

فرض کنید به دنبال یک کلمه در یک فرهنگ لغت بزرگ هستید. احتمالاً از صفحه اول شروع نمی‌کنید. کتاب را تقریباً از وسط باز می‌کنید و بررسی می‌کنید کلمه مورد نظر از کلمه‌ای که در آن صفحه قرار دارد، از نظر الفبایی جلوتر است یا عقب‌تر. سپس نصف نامربط کتاب را کنار می‌گذارید و همین کار را روی نیمه باقی‌مانده انجام می‌دهید. Binary Search دقیقاً همین ایده را روی آرایه پیاده می‌کند.



۳.۲.۳ متغیرهای `max` ، `min` و `mid`

برای مشخص کردن محدوده فعلی جستجو، سه متغیر اصلی داریم:

متغیر	مفهوم
<code>min</code>	ابتدای محدوده جستجو
<code>max</code>	انتهای محدوده جستجو
<code>mid</code>	عنصر وسط محدوده

برای آرایه ۱۰۰۰ عنصری:

```
min = 0
```



```
max = 999
```

سپس:

```
mid = (min + max) / 2
```

در محاسبات اندیسی، نتیجه باید به یک اندیس صحیح تبدیل شود.

مثلاً:

```
min = 0
```

```
max = 999
```

```
mid = (0 + 999) / 2
```

در تقسیم صحیح:

```
mid = 499
```

۳.۲.۴ قوانین Binary Search

پس از محاسبه `mid`، مقدار مورد نظر را با `A[mid]` مقایسه می‌کنیم.

نتیجه	شرط
مقدار پیدا شده است	<code>num == A[mid]</code>
نیمه راست حذف می‌شود	<code>num < A[mid]</code>

نتیجه	شرط
نیمه چپ حذف می‌شود	<code>num > A[mid]</code>
مقدار پیدا نشده است	<code>min > max</code>

اگر:

```
num < A[mid]
```

باشد:

```
max = mid - 1
```

و اگر:

```
num > A[mid]
```

باشد:

```
min = mid + 1
```

دلیل استفاده از `1-` و `1+` این است که خود `mid` قبلاً بررسی شده و می‌دانیم جواب نیست.

۳.۲.۵. الگوریتم کامل Binary Search

فرض کنید آرایه‌ای ۱۰۰۰ عنصری داریم و می‌خواهیم ۱۰ مقدار مختلف را در آن جستجو کنیم. ابتدا آرایه باید مرتب شود.

Algorithm: Binary Search

```
1. START

2. DECLARE A[1000]

3. FOR i = 0 TO 999:
    READ A[i]
END FOR

4. CALL Sort(A)

5. FOR query = 1 TO 10:

    READ num

    SET min = 0
    SET max = 999
    SET found = 0

    WHILE (min ≤ max AND found == 0):

        SET mid = (min + max) / 2
```

```

        IF num == A[mid] THEN

            PRINT "found at index", mid
            SET found = 1

        ELSE IF num < A[mid] THEN

            SET max = mid - 1

        ELSE

            SET min = mid + 1

        END IF

    END WHILE

    IF found == 0 THEN
        PRINT "not found"
    END IF

END FOR

6. END

```

در این الگوریتم، `Sort(A)` یک عملیات کلی است و می‌تواند با یکی از الگوریتم‌های مرتب‌سازی فصل انجام شود.

۳.۲.۶. اجرای مرحله به مرحله Binary Search

فرض کنید آرایه مرتب‌شده:

```
[3, 7, 12, 18, 25, 31, 42, 56, 63]
```

باشد و بخواهیم `42` را پیدا کنیم.

ابتدا:

```
min = 0  
max = 8
```

پس:

```
mid = (0 + 8) / 2  
     = 4
```

بنابراین:

```
A[4] = 25
```

از آنجا که:

```
42 > 25
```

می‌دانیم مقدار مورد نظر در نیمه راست قرار دارد.

پس:

```
min = mid + 1  
= 5
```

اکنون دیگر نیازی به بررسی عناصر اندیس‌های 0 تا 4 نداریم.

این نکته مهم‌ترین بخش Binary Search است:

الگوریتم نه فقط یک عنصر را بررسی می‌کند، بلکه با استفاده از مرتب بودن داده‌ها، تعداد زیادی از گزینه‌های غیرممکن را یک‌باره حذف می‌کند.

۳.۲.۷ شرط پایان Binary Search

شرط اصلی حلقه:

```
min ≤ max
```

است. تا زمانی که این شرط برقرار باشد، حداقل یک موقعیت ممکن برای پیدا شدن مقدار وجود دارد.

اما اگر:

```
min > max
```

شود، محدوده جستجو خالی شده است. در این حالت یعنی تمام موقعیت‌های ممکن بررسی شده‌اند و مقدار در آرایه وجود ندارد. در نسخه‌ای که از **found** نیز استفاده می‌کند.

شرط کامل:

```
min ≤ max AND found == 0
```

است.

۳.۲.۸ تعداد مقایسه‌ها در Binary Search

در جستجوی خطی، در هر مرحله حداکثر یک عنصر کنار گذاشته می‌شود. اما در Binary Search، تقریباً نصف محدوده در هر مرحله حذف می‌شود:



بنابراین تعداد مراحل متناسب با:

```
 $\log_2(n)$ 
```

است.

پیچیدگی زمانی جستجوی دودویی:

$$O(\log n)$$

است.

برای $n = 1000$:

$$\log_2(1000) \approx 9.97$$

بنابراین تعداد مراحل در حدود ۱۰ است؛ تعداد دقیق مقایسه‌ها به جزئیات پیاده‌سازی و اینکه دقیقاً چه عملیاتی را «مقایسه» حساب کنیم بستگی دارد.

۳.۲.۹ مقایسه ساده

حالت	Linear Search	Binary Search
بهترین	1	1
متوسط	حدود $n/2$	حدود $\log_2 n$
بدترین	n	حدود $\log_2 n$

برای $n = 1000$:

الگوریتم	بهترین	متوسط تقریبی	بدترین
Linear Search	1	500	1000
Binary Search	1	حدود 10	حدود 10-11

۳.۲.۱۰ هزینه مرتب‌سازی

یک نکته بسیار مهم وجود دارد. برای استفاده از **Binary Search**، آرایه باید مرتب باشد. اگر آرایه از ابتدا مرتب نباشد، ابتدا باید آن را مرتب کنیم. پس نمی‌توان فقط هزینه جستجو را بررسی کرد.

مثلاً:

```
آرایه نامرتب
↓
مرتب‌سازی
↓
Binary Search
↓
Binary Search
↓
Binary Search
↓
...
```

اگر فقط یک بار بخواهیم در یک آرایه نامرتب جستجو کنیم، ممکن است هزینه مرتب‌سازی بیشتر از سود حاصل از **Binary Search** باشد. اما اگر همان آرایه قرار باشد بارها جستجو شود، مرتب‌سازی یک بار انجام می‌شود و پس از آن می‌توان از **Binary Search** برای جستجوهایی متعدد استفاده کرد.

این مسئله یکی از نکات مهم در طراحی الگوریتم است:

سرعت یک عملیات را نباید جدا از هزینه آماده‌سازی داده‌ها بررسی کرد.

۳.۳ مقایسه نهایی Binary Search و Linear Search

ویژگی	Linear Search	Binary Search
روش	بررسی یکی یکی عناصر	حذف تقریباً نصف محدوده در هر مرحله
نیاز به مرتب بودن	ندارد	دارد
بهترین حالت	$O(1)$	$O(1)$
متوسط	$O(n)$	$O(\log n)$
بدترین حالت	$O(n)$	$O(\log n)$
سادگی پیاده سازی	بسیار ساده	کمی پیچیده تر
مناسب برای داده نامرتب	بله	خیر
مناسب برای جستجوهای متعدد در داده مرتب	ممکن است مناسب باشد	مزیت قابل توجه دارد

به زبان ساده:

Linear Search:

«یکی یکی بگرد تا پیدا کنی.»

[7] → [12] → [18] → [25] → [31] → [42]

Binary Search:

«وسط را بررسی کن و هر چیزی را که دیگر نمی‌تواند جواب باشد حذف کن.»

کل آرایه



نصف آرایه



یک‌چهارم



یک‌هشتم



...



مقدار مورد نظر

۴. جمع‌بندی فصل

در این فصل با دو گروه مهم از الگوریتم‌ها، یعنی مرتب‌سازی و جستجو، آشنا شدیم. در بخش مرتب‌سازی، الگوریتم‌های **Bubble Sort**، **Insertion Sort**، **Merge Sort** و **Quick Sort**، **Heap Sort** را بررسی کردیم و با تفاوت آن‌ها از نظر نحوه‌ی کار، پیچیدگی زمانی و نیاز به حافظه آشنا شدیم. دیدیم که الگوریتم‌های ساده‌ای مانند **Bubble Sort** و **Insertion Sort** در حالت کلی می‌توانند $O(n^2)$ باشند، در حالی که **Merge Sort** و **Heap Sort** زمان $O(n \log n)$ دارند و **Quick Sort** نیز در حالت متوسط $O(n \log n)$ و در بدترین حالت $O(n^2)$ است. سپس با مفهوم پایداری (**Stability**) آشنا شدیم؛ یعنی حفظ ترتیب نسبی عناصر دارای مقدار برابر. در بخش جستجو نیز **Linear Search** و **Binary Search** را بررسی کردیم؛ جستجوی خطی بدون نیاز به مرتب بودن داده‌ها عناصر را یکی‌یکی بررسی می‌کند و پیچیدگی آن $O(n)$ است، در حالی که جستجوی دودویی با حذف تقریباً نیمی از فضای جستجو در هر مرحله، به $O(\log n)$ می‌رسد و به مرتب بودن داده‌ها نیاز دارد. در نهایت، دیدیم که انتخاب الگوریتم مناسب فقط به درست بودن نتیجه محدود نمی‌شود و باید ویژگی‌هایی مانند اندازه و وضعیت داده‌ها، تعداد دفعات جستجو یا مرتب‌سازی، محدودیت حافظه و هزینه‌ی آماده‌سازی داده‌ها را نیز در نظر گرفت. بنابراین این فصل ما را از صرفاً پیاده‌سازی الگوریتم‌ها به سمت تحلیل هزینه و انتخاب آگاهانه‌ی الگوریتم متناسب با مسئله و داده هدایت کرد.

توابع

فصل دهم



۱ مقدمه

تا اینجا الگوریتم‌هایی را بررسی کرده‌ایم که مراحل مختلف حل مسئله را به صورت پشت‌سرهم اجرا می‌کنند. برای مثال، ممکن است الگوریتمی شامل دریافت داده، انجام محاسبات، بررسی یک شرط، تکرار چند دستور و در نهایت نمایش نتیجه باشد. اما هرچه برنامه بزرگ‌تر می‌شود، قرار دادن تمام دستورها در یک بخش از برنامه مشکلاتی ایجاد می‌کند. کد طولانی‌تر می‌شود، بخش‌هایی از آن ممکن است چندین بار تکرار شوند و پیدا کردن خطاها نیز دشوارتر خواهد شد.

فرض کنید در یک برنامه چند بار لازم باشد:

- دو عدد را با یکدیگر جمع کنیم؛
- بزرگ‌ترین مقدار را پیدا کنیم؛
- یک عدد را از نظر اول بودن بررسی کنیم؛
- یک آرایه را مرتب کنیم؛
- یا یک محاسبه مشخص را انجام دهیم.

اگر هر بار تمام دستورهای مربوط به این عملیات را از ابتدا بنویسیم، هم حجم برنامه افزایش پیدا می‌کند و هم احتمال خطا بیشتر می‌شود. راه‌حل این مشکل استفاده از تابع (Function) است. تابع را می‌توان بخشی مستقل از برنامه دانست که برای انجام یک وظیفه مشخص طراحی شده است. پس از تعریف تابع، هر زمان که به آن عملیات نیاز داشته باشیم، می‌توانیم آن را فراخوانی (Call) کنیم.

۲. تابع (function)

به زبان ساده:

تابع یک بخش مستقل از برنامه است که یک وظیفه مشخص را انجام می‌دهد و می‌توان آن را هر زمان که لازم باشد فراخوانی کرد.

برای مثال، ساختار کلی یک برنامه می‌تواند چنین باشد:

برنامه اصلی

```
|
|— دریافت داده
|
|— فراخوانی تابع تشخیص عدد اول
|       |
|       |— بررسی عدد
|
|— ادامه برنامه
|
|— فراخوانی دوباره تابع
```

در این ساختار، برنامه اصلی لازم نیست جزئیات الگوریتم تشخیص عدد اول را هر بار دوباره بنویسد؛ کافی است تابع مربوط به آن را فراخوانی کند.

۲.۱ چرا از تابع استفاده می‌کنیم؟

استفاده از توابع یکی از مهم‌ترین روش‌ها برای سازمان‌دهی برنامه‌هاست. چند دلیل اصلی برای استفاده از تابع وجود دارد.

۲.۱.۱ جلوگیری از تکرار کد

اگر یک عملیات چندین بار در برنامه مورد نیاز باشد، بهتر است آن عملیات یک بار در قالب تابع نوشته شود. برای مثال، فرض کنید چند عدد داریم و می‌خواهیم برای هر کدام بررسی کنیم که آیا عدد اول است یا خیر. به جای اینکه الگوریتم تشخیص عدد اول را چندین بار تکرار کنیم، یک تابع به نام `isPrime` تعریف می‌کنیم:

```
isPrime(7)
isPrime(12)
isPrime(97)
```

هر سه فراخوانی از یک الگوریتم واحد استفاده می‌کنند.

۲.۱.۲ تقسیم برنامه به بخش‌های کوچک‌تر

یک برنامه بزرگ را می‌توان به چند وظیفه کوچک‌تر تقسیم کرد:

برنامه

└ دریافت داده

└ مرتب‌سازی

└ جستجو

└ محاسبه

└ نمایش نتیجه

هر یک از این وظایف می‌تواند در قالب یک تابع جداگانه پیاده‌سازی شود. در نتیجه، به جای یک برنامه بسیار بزرگ، با مجموعه‌ای از بخش‌های کوچک‌تر و قابل فهم‌تر روبه‌رو هستیم.

۲.۱.۳. استفاده مجدد از کد

یک تابع را می‌توان چندین بار و با ورودی‌های مختلف فراخوانی کرد. برای مثال:

```
sum(3, 5)
sum(10, 20)
sum(100, 200)
```

هر سه فراخوانی از یک تابع استفاده می‌کنند، اما آرگومان‌های متفاوتی به آن می‌دهند.

۲.۱.۴. ساده‌تر شدن آزمایش و خطایی

وقتی برنامه به توابع کوچک‌تر تقسیم شده باشد، می‌توان هر بخش را جداگانه آزمایش کرد. برای مثال، اگر برنامه‌ای شامل توابع زیر باشد:

```
readData()
sortData()
search()
printResult()
```

و نتیجه جستجو اشتباه باشد، می‌توان ابتدا تابع `search` را بررسی کرد، بدون اینکه لازم باشد تمام برنامه را یک‌جا بررسی کنیم.

۲.۲. ساختار کلی یک تابع

یک تابع معمولاً شامل چند بخش اصلی است:

```
FUNCTION name(parameters)
```

دستورات تابع

```
RETURN result
```

```
END FUNCTION
```

برای مثال:

```
FUNCTION add(a, b)
```

```
result = a + b
```

```
RETURN result
```

```
END FUNCTION
```

این تابع دو ورودی به نام‌های **a** و **b** دریافت می‌کند. آن‌ها را با یکدیگر جمع می‌کند و نتیجه را برمی‌گرداند.

اگر بنویسیم:

```
x = add(3, 5)
```

تابع اجرا می‌شود:

$$3 + 5 = 8$$

و مقدار 8 به محل فراخوانی برگردانده می‌شود.

بنابراین:

$$x = 8$$

در این مثال، می‌توانیم وظیفه هر بخش را چنین در نظر بگیریم:

FUNCTION add(a, b)

↑

تعریف تابع

add(3, 5)

↑

فراخوانی

3 و 5

↑

آرگومان‌ها

8

↑

۲.۳ پارامتر و آرگومان

در هنگام کار با توابع، دو اصطلاح مهم وجود دارد:

- پارامتر (Parameter)
- آرگومان (Argument)

این دو مفهوم به یکدیگر مربوطاند، اما دقیقاً یکسان نیستند.

در تعریف تابع:

```
FUNCTION add(a, b)
```

متغیرهای **a** و **b** پارامتر هستند.

اما هنگام فراخوانی:

```
add(3, 5)
```

مقادیر **3** و **5** آرگومان هستند.

بنابراین:

مفهوم	محل استفاده	مثال
Parameter	هنگام تعریف تابع	a , b
Argument	هنگام فراخوانی تابع	5 , 3

می‌توان این تفاوت را به شکل زیر به خاطر سپرد:

```
FUNCTION add(a, b)
    ↑  ↑
    Parameters

add(3, 5)
    ↑  ↑
    Arguments
```

پارامتر مشخص می‌کند تابع چه نوع ورودی‌ای را دریافت می‌کند، در حالی که آرگومان مقدار واقعی‌ای است که هنگام اجرای تابع در اختیار آن قرار می‌گیرد.

۲.۴ مقدار بازگشتی و دستور `return`

بسیاری از توابع علاوه بر دریافت ورودی، یک مقدار را نیز به برنامه‌فراخواننده برمی‌گردانند. این کار معمولاً با دستور `return` انجام می‌شود.

برای مثال:

```
FUNCTION square(x)

    RETURN x * x

END FUNCTION
```

اگر بنویسیم:

```
result = square(5)
```

می‌توان اجرای آن را مرحله‌به‌مرحله چنین تصور کرد:

```
square(5)
```

↓

```
x = 5
```

↓

```
x × x
```

↓

```
5 × 5
```

↓

```
25
```

↓

```
return 25
```

↓

```
result = 25
```

بنابراین **return** دو نقش مهم دارد:

۱. یک مقدار را به محل فراخوانی برمی‌گرداند.
۲. اجرای همان فراخوانی تابع را به پایان می‌رساند.

برای مثال:

```
FUNCTION square(x)
```

```
    RETURN x * x
```

```
    PRINT "Hello"
```

```
END FUNCTION
```

دستور **PRINT** در این مثال اجرا نخواهد شد، زیرا اجرای تابع با رسیدن به **return** به پایان رسیده است.

۲.۵ تابع بدون مقدار بازگشتی

همه توابع الزاماً مقداری را به برنامه برنمی گردانند. گاهی هدف تابع فقط انجام یک عمل است؛ برای مثال نمایش یک پیام.

```
FUNCTION printHello()
```

```
    PRINT "Hello"
```

```
END FUNCTION
```

فراخوانی:

```
printHello()
```

باعث می‌شود:

```
Hello
```

نمایش داده شود.

در چنین حالتی تابع برای انجام یک عملیات استفاده شده است و مقدار مشخصی را به برنامه فراخواننده بر نمی‌گرداند. در بعضی زبان‌های برنامه‌نویسی برای چنین توابعی از مفهوم `void` استفاده می‌شود:

```
void printHello()
```

بنابراین به‌صورت ساده می‌توان دو حالت را در نظر گرفت:

نوع تابع	مثال	نتیجه
دارای مقدار بازگشتی	<code>square(5)</code>	25
بدون مقدار بازگشتی	<code>printHello()</code>	انجام یک عمل

توجه داشته باشید که شکل دقیق تعریف تابع و نحوه مشخص کردن نوع بازگشتی به زبان برنامه‌نویسی بستگی دارد.

۲.۶ فراخوانی تابع (Function Call)

وقتی تابعی را تعریف می‌کنیم، دستورات داخل آن لزوماً همان لحظه اجرا نمی‌شوند. تعریف تابع در واقع مشخص می‌کند که اگر این تابع فراخوانی شد، چه دستورهایایی باید اجرا شوند. برای اجرای آن باید تابع را فراخوانی کنیم.

برای مثال:


```
FUNCTION multiply(a, b)
```

```
    RETURN a * b
```

```
END FUNCTION
```

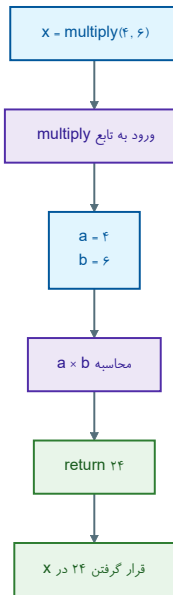
تعریف بالا به تنهایی حاصل ضرب را محاسبه نمی کند.

اما وقتی بنویسیم:

```
x = multiply(4, 6)
```

تابع فراخوانی می شود.

فرآیند را می توان به صورت زیر نمایش داد:



در اینجا کنترل اجرای برنامه موقتاً وارد تابع می‌شود. تابع کار خود را انجام می‌دهد و پس از پایان، کنترل به محل فراخوانی برمی‌گردد.

۲.۷ یک مثال کامل از تابع

فرض کنید می‌خواهیم تابعی بنویسیم که بزرگ‌تر بودن دو عدد را مشخص کند.

```
FUNCTION max(a, b)
```

```
  IF a > b THEN
```

```
    RETURN a
```

```
ELSE
    RETURN b
END IF

END FUNCTION
```

در برنامه اصلی می‌توانیم بنویسیم:

```
READ x
READ y

result = max(x, y)

PRINT result
```

اگر ورودی‌ها چنین باشند:

```
x = 12
y = 8
```

تابع به شکل زیر فراخوانی می‌شود:

```
max(12, 8)
```

درون تابع شرط زیر بررسی می‌شود:

```
12 > 8
```

چون شرط درست است، دستور زیر اجرا می‌شود:

```
RETURN 12
```

بنابراین مقدار `result` برابر `12` خواهد شد:

```
result = 12
```

مزیت این روش این است که برنامه اصلی لازم نیست جزئیات مقایسه دو عدد را بداند؛ فقط از تابع `max` استفاده می‌کند.

۲.۸ متغیرهای محلی (Local Variables)

متغیرهایی که درون یک تابع تعریف می‌شوند، معمولاً محلی (**Local**) هستند. برای مثال:

```
FUNCTION square(x)
```

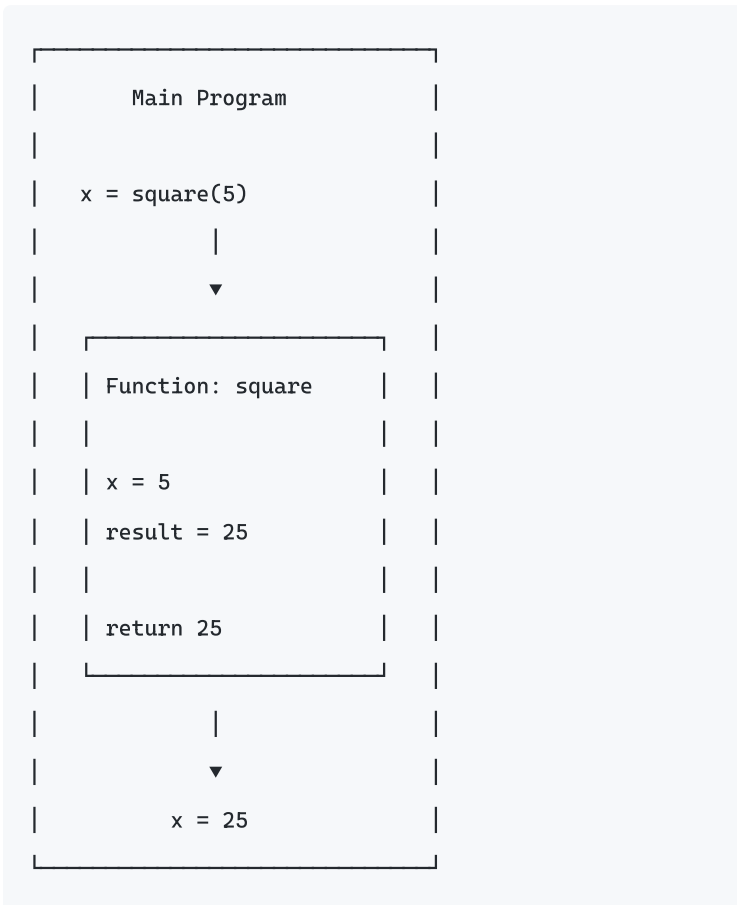
```
    result = x * x
```

```
    RETURN result
```

```
END FUNCTION
```

در اینجا `result` درون تابع تعریف شده است.

به صورت مفهومی، تابع محیطی برای اجرای متغیرها و دستورهای خودش دارد:



در این مثال، `result` متعلق به اجرای تابع است و برنامه اصلی معمولاً نمی‌تواند مستقیماً آن را به عنوان یک متغیر محلی همان تابع استفاده کند. این موضوع ما را به مفهوم **Scope** می‌رساند.

۲.۹ محدوده متغیرها (Scope)

Scope یا محدوده مشخص می‌کند یک متغیر در کدام بخش از برنامه قابل دسترسی است.

برای مثال:

```
FUNCTION test()
```

```
    x = 10
```

```
END FUNCTION
```

```
PRINT x
```

در بسیاری از زبان‌ها، **x** در اینجا خارج از محدوده‌ای است که در آن تعریف شده و بنابراین نمی‌توان آن را مستقیماً از بیرون تابع استفاده کرد. در مقابل، متغیری که در محدوده‌ای گسترده‌تر تعریف شده باشد، ممکن است از بخش‌های مختلف برنامه قابل دسترسی باشد. چنین متغیری معمولاً **متغیر سراسری (Global Variable)** نامیده می‌شود. برای برنامه‌های قابل نگهداری، معمولاً بهتر است داده‌ها تا حد امکان به صورت مشخص از طریق پارامترها وارد تابع شوند و از طریق مقدار بازگشتی خارج شوند، به جای اینکه توابع وابستگی زیادی به متغیرهای سراسری داشته باشند. نحوه دقیق **Scope** در زبان‌های مختلف می‌تواند متفاوت باشد، اما اصل کلی مهم است:

هر متغیر در یک محدوده مشخص از برنامه تعریف و قابل دسترسی است.

۲.۱۰ تبدیل یک الگوریتم به تابع

یکی از کاربردهای مهم تابع این است که الگوریتم‌هایی را که قبلاً یاد گرفته‌ایم، به بخش‌های قابل استفاده مجدد تبدیل کنیم. برای مثال، در فصل‌های قبل الگوریتم تشخیص عدد اول را بررسی کردیم. اکنون می‌توانیم آن را در قالب تابع قرار دهیم:

```
FUNCTION isPrime(n)

    IF n < 2 THEN
        RETURN 0
    END IF

    limit = Sqrt(n)

    FOR k = 2 TO limit:

        IF n % k == 0 THEN
            RETURN 0
        END IF

    END FOR

    RETURN 1
```

```
END FUNCTION
```

این تابع اگر عدد اول باشد 1 و اگر اول نباشد 0 برمی‌گرداند.

اکنون فرض کنید یک آرایهٔ دوبعدی داریم و می‌خواهیم تعداد اعداد اول آن را بشماریم:

```
NO = 0
```

```
FOR i = 0 TO 19:
```

```
    FOR j = 0 TO 29:
```

```
        IF isPrime(A[i][j]) == 1 THEN
```

```
            NO = NO + 1
```

```
        END IF
```

```
    END FOR
```

```
END FOR
```

```
PRINT NO
```

نکتهٔ مهم اینجاست که الگوریتم اصلی دیگر لازم نیست بداند چگونه اول بودن یک عدد

بررسی می‌شود.

فقط می‌گوید:


```
isPrime(A[i][j])
```

یعنی:

«بررسی کن آیا این مقدار اول است یا خیر.»

جزئیات مربوط به تقسیم بر مقسوم‌علیه‌ها داخل تابع `isPrime` قرار گرفته است.

۲.۱۱ تابع و مفهوم انتزاع (Abstraction)

این موضوع یکی از مفاهیم مهم علوم کامپیوتر یعنی انتزاع (**Abstraction**) را نشان می‌دهد. وقتی از یک تابع استفاده می‌کنیم، معمولاً لازم نیست جزئیات داخلی آن را بدانیم.

برای مثال:

```
isPrime(97)
```

برای استفاده از این تابع کافی است بدانیم که:

تابع یک عدد دریافت می‌کند و مشخص می‌کند که آن عدد اول است یا خیر.

لازم نیست در هر بار استفاده دوباره الگوریتم زیر را در ذهن مرور کنیم:

2

3

4

...

$\sqrt{n}$

و برای هر مقدار بررسی کنیم:

$n \% k$

این جزئیات یک بار داخل تابع نوشته شده‌اند.

بنابراین تابع یک رابط ساده برای استفاده از یک عملیات فراهم می‌کند و جزئیات پیاده‌سازی آن عملیات را از قسمت‌های دیگر برنامه جدا می‌کند. این ایده در برنامه‌های بزرگ بسیار مهم است؛ زیرا هر بخش از برنامه می‌تواند روی وظیفه خودش تمرکز کند.

۳. مفهوم بازگشت (Recursion)

اکنون که مفهوم تابع را می‌دانیم، می‌توانیم یک ویژگی جالب توابع را بررسی کنیم:

یک تابع می‌تواند خودش را فراخوانی کند.

تابعی که در جریان اجرای خودش دوباره همان تابع را فراخوانی می‌کند، تابع بازگشتی (Recursive Function) نام دارد.

برای مثال:

```
FUNCTION example(n)

    ...

    example(n - 1)

    ...

END FUNCTION
```

در اینجا تابع **example** درون خودش دوباره فراخوانی شده است.

در نگاه اول ممکن است این سؤال ایجاد شود:

اگر تابع خودش را دوباره صدا بزند، آیا این روند برای همیشه ادامه پیدا نمی‌کند؟

اگر تابع بازگشتی شرط توقف مناسبی نداشته باشد، بله؛ فراخوانی‌ها می‌توانند بدون پایان

ادامه پیدا کنند. به همین دلیل، یک تابع بازگشتی صحیح باید یک حالت پایه (Base Case) داشته باشد.

۳.۱ دو بخش اصلی تابع بازگشتی

یک تابع بازگشتی معمولاً از دو بخش اصلی تشکیل می‌شود:

حالت پایه (Base Case)

شرایطی که در آن دیگر نیازی به فراخوانی مجدد تابع وجود ندارد.

حالت بازگشتی (Recursive Case)

بخشی که تابع خودش را دوباره، معمولاً با یک مسئله کوچک‌تر یا ساده‌تر، فراخوانی می‌کند. ساختار کلی:

```
FUNCTION recursiveFunction(n)

    IF base condition THEN
        RETURN result
    END IF

    RETURN recursiveFunction(smaller problem)

END FUNCTION
```

به زبان ساده:

اگر مسئله به ساده‌ترین حالت رسید، پاسخ را بده؛ در غیر این صورت مسئله را کوچک‌تر کن و دوباره همین تابع را روی مسئله کوچک‌تر اجرا کن.

این ایده اساس بسیاری از الگوریتم‌های بازگشتی است.

۳.۲ مثال ساده: شمارش معکوس

برای درک بهتر، تابعی می‌نویسیم که از n تا صفر را چاپ کند:

```
FUNCTION countdown(n)

    IF n == 0 THEN
        PRINT 0
        RETURN
    END IF

    PRINT n

    countdown(n - 1)

END FUNCTION
```

اگر بنویسیم:

```
countdown(3)
```

مراحل اجرا به صورت مفهومی چنین خواهد بود:

countdown(3)

↓

PRINT 3

↓

countdown(2)

↓

PRINT 2

↓

countdown(1)

↓

PRINT 1

↓

countdown(0)

↓

PRINT 0

↓

پایان

خروجی:

3

2

1

```
0
```

در این مثال:

```
n == 0
```

حالت پایه است.

9:

```
countdown(n - 1)
```

حالت بازگشتی است.

دقت کنید که مقدار **n** در هر مرحله کاهش پیدا می‌کند:

```
3 → 2 → 1 → 0
```

بنابراین تابع به حالت پایه نزدیک می‌شود.

۳.۳ چرا Base Case ضروری است؟

اگر حالت پایه را حذف کنیم:

```
FUNCTION countdown(n)
```

```
    PRINT n
```

```
countdown(n - 1)
```

```
END FUNCTION
```

و تابع را با:

```
countdown(3)
```

فراخوانی کنیم، روندی مانند زیر شکل می‌گیرد:

```
3  
2  
1  
0  
-1  
-2  
-3  
...
```

هیچ شرطی وجود ندارد که اجرای تابع را متوقف کند. در نتیجه فراخوانی‌های جدید پشت سر هم ایجاد می‌شوند. بنابراین یک اصل بسیار مهم داریم:

هر تابع بازگشتی باید بتواند در نهایت به یک حالت پایه برسد.

وجود **Base Case** به‌تنهایی نیز کافی نیست؛ باید مسیر اجرای تابع واقعاً به آن حالت

برسد.

مثال کلاسیک: فاکتوریل

یکی از ساده‌ترین و معروف‌ترین مثال‌های بازگشت، محاسبهٔ فاکتوریل (**Factorial**) است. فاکتوریل عدد n به شکل زیر تعریف می‌شود:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$$

فاکتوریل را می‌توان به شکل بازگشتی نیز تعریف کرد:

$$n! = n \times (n-1)!$$

و حالت پایه:

$$0! = 1$$

این تعریف دقیقاً با ساختار یک تابع بازگشتی مطابقت دارد.

تابع بازگشتی فاکتوریل

```
FUNCTION factorial(n)
```

```
    IF n == 0 THEN
```

```
        RETURN 1
```

```
    END IF
```

```
RETURN n * factorial(n - 1)
```

```
END FUNCTION
```

اگر بنویسیم:

```
factorial(5)
```

ابتدا تابع نمی‌تواند مستقیماً نتیجه نهایی را محاسبه کند، زیرا برای محاسبه 5! به 4! نیاز دارد:

```
factorial(5)
```

```
= 5 × factorial(4)
```

سپس:

```
factorial(4)
```

```
= 4 × factorial(3)
```

و به همین شکل:

```
factorial(5)
```

```
= 5 × factorial(4)
```

```
= 5 × 4 × factorial(3)
```

$$= 5 \times 4 \times 3 \times \text{factorial}(2)$$

$$= 5 \times 4 \times 3 \times 2 \times \text{factorial}(1)$$

$$= 5 \times 4 \times 3 \times 2 \times 1 \times \text{factorial}(0)$$

اکنون به حالت پایه رسیده‌ایم:

$$\text{factorial}(0) = 1$$

بنابراین محاسبات از پایین به بالا انجام می‌شوند:

$$\text{factorial}(1) = 1 \times 1 = 1$$

$$\text{factorial}(2) = 2 \times 1 = 2$$

$$\text{factorial}(3) = 3 \times 2 = 6$$

$$\text{factorial}(4) = 4 \times 6 = 24$$

$$\text{factorial}(5) = 5 \times 24 = 120$$

پس:

```
factorial(5) = 120
```

درک این رفت و برگشت برای فهمیدن مفهوم **Call Stack** بسیار مهم است.

۴. پشته فراخوانی (Call Stack)

برای درک دقیق‌تر توابع بازگشتی باید با مفهوم **Call Stack** آشنا شویم. در هنگام اجرای یک برنامه، هر بار که تابعی فراخوانی می‌شود، اطلاعات مربوط به آن فراخوانی باید به شکلی نگهداری شود تا برنامه بعداً بتواند اجرای خود را از همان نقطه ادامه دهد. یکی از ساختارهایی که برای این منظور استفاده می‌شود **Stack** یا پشته است. هر بار که یک تابع فراخوانی می‌شود، یک وضعیت جدید روی **Stack** قرار می‌گیرد.

برای:

```
factorial(5)
```

ابتدا یک فراخوانی داریم:

```
factorial(5)
```

اما برای محاسبه آن باید `factorial(4)` را بدانیم:

```
factorial(5)
```

```
factorial(4)
```

سپس:

```
factorial(5)
```

```
factorial(4)
```

```
factorial(3)
```

بعد:

```
factorial(5)
factorial(4)
factorial(3)
factorial(2)
```

:9

```
factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)
```

در نهایت:

```
factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)
factorial(0)
```

در این مرحله `factorial(0)` به حالت پایه می‌رسد:

```
factorial(0) = 1
```

اکنون دیگر نیازی به ایجاد فراخوانی جدید نیست و **Stack** شروع به باز شدن (Unwinding) می‌کند. نتایج به ترتیب به فراخوانی‌های قبلی برمی‌گردند:

```
factorial(1) = 1
factorial(2) = 2
factorial(3) = 6
factorial(4) = 24
factorial(5) = 120
```

می‌توان این روند را به دو مرحله تقسیم کرد:

مرحلهٔ اول: ایجاد فراخوانی‌ها

$5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

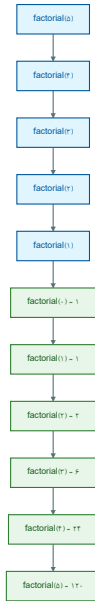
مرحلهٔ دوم: بازگشت نتایج

$1 \rightarrow 1 \rightarrow 2 \rightarrow 6 \rightarrow 24 \rightarrow 120$

بنابراین **recursion** را نباید فقط به صورت «تابعی که خودش را صدا می‌زند» دید. پشت صحنه، هر فراخوانی باید نگهداری شود تا پس از پایان فراخوانی داخلی، برنامه بتواند به فراخوانی قبلی برگردد.

۴.۱ تصویر مفهومی Call Stack

می‌توان روند فاکتوریل را چنین تصور کرد:



در نیمه اول، عمق **Stack** افزایش پیدا می‌کند. در نیمه دوم، فراخوانی‌ها یکی‌یکی پایان پیدا می‌کنند و نتایج به سطح قبلی برمی‌گردند.

۵. بازگشت در برابر تکرار (Recursion vs. Iteration)

بسیاری از مسائل بازگشتی را می‌توان با استفاده از حلقه‌ها نیز حل کرد. برای مثال، فاکتوریل را می‌توان بدون Recursion نوشت:

```
FUNCTION factorial(n)

    result = 1

    FOR i = 1 TO n:
        result = result * i
    END FOR

    RETURN result

END FUNCTION
```

نسخه بازگشتی:

```
FUNCTION factorial(n)

    IF n == 0 THEN
        RETURN 1
    END IF
```

```
RETURN n * factorial(n - 1)
```

```
END FUNCTION
```

هر دو روش می‌توانند نتیجه یکسانی تولید کنند.

روش	ایده
Iteration	استفاده از حلقه و تکرار یک بخش از کد
Recursion	فراخوانی مجدد یک تابع توسط خودش

Recursion به خودی خود به معنی سریع‌تر یا بهتر بودن نیست. در بعضی مسائل، تعریف بازگشتی مسئله بسیار طبیعی و قابل فهم است. در بعضی مسائل دیگر، استفاده از یک حلقه ساده‌تر و کم‌هزینه‌تر است. بنابراین هنگام حل مسئله باید به ساختار خود مسئله توجه کنیم. نه اینکه همیشه یکی از این دو روش را به صورت مطلق انتخاب کنیم.

۶. سه سؤال مهم هنگام طراحی یا حل تابع بازگشتی

هنگام نوشتن یک تابع بازگشتی، یا حل سوالات آن در کنکور یا امتحانات، سه سؤال اساسی باید از خودمان پرسیم.

سؤال اول: حالت پایه چیست؟

تابع دقیقاً در چه شرایطی باید متوقف شود؟

برای فاکتوریل:

```
n == 0
```

سؤال دوم: حالت بازگشتی چیست؟

تابع چگونه خودش را دوباره فراخوانی می‌کند؟

در فاکتوریل:

```
factorial(n - 1)
```

سؤال سوم: آیا مسئله در هر مرحله کوچک‌تر می‌شود؟

این سؤال بسیار مهم است.

در مثال فاکتوریل:

```
n
```

```
n - 1
```

$n - 2$

$n - 3$

...

0

در هر مرحله به **Base Case** نزدیک‌تر می‌شویم. اگر این روند وجود نداشته باشد، تابع ممکن است هرگز متوقف نشود. بنابراین یک تابع بازگشتی مناسب باید:

Base Case

+

Base Case حرکت به سمت

+

Recursive Call

را داشته باشد.

مثال دیگر: مجموع اعداد از ۱ تا n

می‌خواهیم مجموع اعداد 1 تا n را محاسبه کنیم.

برای مثال:

$$1 + 2 + 3 + 4 + 5 = 15$$

می‌توانیم این مسئله را به شکل بازگشتی تعریف کنیم:

```
sum(n) = n + sum(n - 1)
```

و حالت پایه را برابر قرار دهیم با:

```
sum(0) = 0
```

در نتیجه تابع:

```
FUNCTION sum(n)

    IF n == 0 THEN
        RETURN 0
    END IF

    RETURN n + sum(n - 1)

END FUNCTION
```

برای:

```
sum(5)
```

داریم:

```
sum(5)
= 5 + sum(4)
```

```
= 5 + 4 + sum(3)
= 5 + 4 + 3 + sum(2)
= 5 + 4 + 3 + 2 + sum(1)
= 5 + 4 + 3 + 2 + 1 + sum(0)
```

و چون:

```
sum(0) = 0
```

در نهایت:

```
5 + 4 + 3 + 2 + 1 + 0 = 15
```

پس:

```
sum(5) = 15
```

این مثال نشان می‌دهد که **recursion** الزاماً مخصوص فاکتوریل نیست؛ هر مسئله‌ای که بتوان آن را به نسخه کوچک‌تر خودش تبدیل کرد، ممکن است قابلیت بیان بازگشتی داشته باشد.

۷. خطای رایج: بازگشت بدون پیشرفت

یکی از خطاهای مهم در طراحی تابع بازگشتی این است که تابع خودش را فراخوانی کند، اما مسئله در هر مرحله کوچک‌تر نشود.

مثلاً:

```
FUNCTION bad(n)

    RETURN n + bad(n)

END FUNCTION
```

اگر بنویسیم:

```
bad(5)
```

تابع دوباره:

```
bad(5)
```

را فراخوانی می‌کند.

و این فراخوانی نیز دوباره:

```
bad(5)
```

را اجرا می‌کند. در نتیجه هیچ تغییری در مسئله ایجاد نشده است.

به عبارت دیگر، روند زیر شکل می گیرد:

5 → 5 → 5 → 5 → ...

و تابع هیچ گاه به یک حالت پایه نمی رسد. اگر تعداد فراخوانی های فعال بیش از ظرفیت **Stack** شود، برنامه ممکن است با خطایی مانند **Stack Overflow** مواجه شود. پس صرف اینکه یک تابع خودش را فراخوانی کند، به معنی درست بودن **recursion** نیست.

۸. رابطه تابع و تابع بازگشتی

تابع بازگشتی در واقع نوع خاصی از تابع است.

می‌توان رابطه آن‌ها را چنین نشان داد:

```
Function
|
├─ تابع معمولی
|
└─ تابع بازگشتی
    |
    └─ Base Case
        |
        └─ Recursive Case
```

بنابراین برای فهم درست Recursion، ابتدا باید مفاهیم زیر را به خوبی درک کرده باشیم:

```
Function
↓
Parameter
↓
Argument
↓
Function Call
```

↓

Return

↓

Scope

سپس می‌توانیم همین مفاهیم را برای تابعی به کار ببریم که خودش را دوباره فراخوانی می‌کند:

Recursive Function

↓

Base Case

+

Recursive Case

↓

Function Call

↓

Call Stack

↓

Stack Unwinding

↓

Result

۹. مقایسه مفاهیم اصلی تابع

مفهوم	توضیح
Function	بخش مستقلی از برنامه برای انجام یک وظیفه مشخص
Function Call	درخواست اجرای یک تابع
Parameter	متغیری که در تعریف تابع برای دریافت ورودی قرار می‌گیرد
Argument	مقدار واقعی‌ای که هنگام فراخوانی به تابع داده می‌شود
Return	بازگرداندن مقدار و پایان اجرای همان فراخوانی تابع
Local Variable	متغیری که در محدوده محلی، مانند یک تابع، تعریف شده است
Scope	محدوده‌ای که یک متغیر در آن قابل دسترسی است
Abstraction	استفاده از یک عملیات بدون نیاز به دانستن جزئیات پیاده‌سازی آن

۱۰. مقایسه مفاهیم بازگشت

مفهوم	توضیح
Recursion	فراخوانی یک تابع توسط خودش
Base Case	شرایطی که در آن بازگشت متوقف می‌شود
Recursive Case	بخشی که تابع خودش را دوباره فراخوانی می‌کند
Call Stack	ساختاری برای نگهداری وضعیت فراخوانی‌های فعال
Stack Unwinding	پایان یافتن فراخوانی‌های داخلی و بازگشت نتایج به فراخوانی‌های قبلی
Stack Overflow	وضعیتی که در اثر فراخوانی‌های بیش از ظرفیت Stack ایجاد می‌شود

۱۱. الگوریتم‌های نهایی فصل

تابع معمولی

```
FUNCTION add(a, b)

    result = a + b

    RETURN result

END FUNCTION
```

فراخوانی:

```
x = add(3, 5)
```

نتیجه:

```
x = 8
```

تابع تشخیص عدد اول

```
FUNCTION isPrime(n)
```

```

IF n < 2 THEN
    RETURN 0
END IF

limit = SQRT(n)

FOR k = 2 TO limit:

    IF n % k == 0 THEN
        RETURN 0
    END IF

END FOR

RETURN 1

END FUNCTION

```

این تابع یک نمونه عملی از انتزاع است: قسمت‌های دیگر برنامه فقط از `isPrime` استفاده می‌کنند و لازم نیست جزئیات الگوریتم تشخیص عدد اول را دوباره بنویسند.

تابع بازگشتی فاکتوریل

```

FUNCTION factorial(n)

```

```
IF n == 0 THEN
    RETURN 1
END IF

RETURN n * factorial(n - 1)

END FUNCTION
```

فراخوانی:

```
result = factorial(5)
```

نتیجه:

```
result = 120
```

۱۲. جمع‌بندی فصل

در این فصل از یک مفهوم پایه، یعنی تابع (Function)، به یکی از مفاهیم مهم الگوریتمی،

یعنی بازگشت (Recursion)، رسیدیم. ابتدا با ساختار کلی تابع و مفاهیمی مانند

Return, Parameter, Argument, Function Call, Processing آشنا

شدیم و دیدیم که پارامترها در تعریف تابع قرار می‌گیرند، آرگومان‌ها هنگام فراخوانی به

تابع داده می‌شوند و **return** می‌تواند یک مقدار را به محل فراخوانی برگرداند و اجرای

آن فراخوانی را پایان دهد. سپس با مفهوم **متغیرهای محلی**، استفاده مجدد از کد و انتزاع

(Abstraction) آشنا شدیم و دیدیم که تابع با جدا کردن یک وظیفه مشخص از سایر

بخش‌های برنامه، باعث کاهش تکرار کد و افزایش نظم و قابلیت استفاده مجدد می‌شود. در

ادامه، به تابع بازگشتی (Recursive Function) رسیدیم؛ تابعی که در جریان اجرای

خود دوباره خودش را فراخوانی می‌کند و برای درست کار کردن به دو بخش اصلی، یعنی

Base Case و **Recursive Case**، نیاز دارد. هر فراخوانی باید مسئله را به سمت

Base Case کوچک‌تر کند و فراخوانی‌های ایجادشده نیز در **Call Stack** نگهداری

می‌شوند. پس از رسیدن به **Base Case**، فرایند **Stack Unwinding** آغاز می‌شود

و فراخوانی‌ها به ترتیب معکوس پایان می‌یابند و نتایج به فراخوانی‌های قبلی بازمی‌گردند. در

نهایت، دیدیم که **Recursion** مفهومی جادویی یا کاملاً جدا از تابع نیست، بلکه همان

تابع معمولی است که در یکی از مراحل اجرای خود دوباره خودش را فراخوانی می‌کند؛

بنابراین نکته اصلی در طراحی یک الگوریتم بازگشتی، داشتن یک **حالت پایه مشخص** و

اطمینان از حرکت هر فراخوانی به سمت آن است. همچنین **Recursion** همیشه جایگزین

بهتری برای **Iteration** نیست و انتخاب میان این دو به ساختار و ماهیت مسئله بستگی

دارد.

زبان C

فصل یازدهم



۱. مقدمه

در فصل‌های قبلی، مفاهیم مختلف الگوریتم و برنامه‌نویسی را یاد گرفتیم. با متغیرها، انواع داده، عملگرها، شرط‌ها، حلقه‌ها، آرایه‌ها، توابع و بازگشت آشنا شدیم و الگوریتم‌های مختلف را با شبه‌کد و فلوجارت بیان کردیم. اما یک الگوریتم به‌خودی‌خود هنوز یک برنامه قابل اجرا نیست. برای تبدیل الگوریتم به یک برنامه واقعی، باید آن را با استفاده از یک زبان برنامه‌نویسی (Programming Language) پیاده‌سازی کنیم. در این فصل، زبان C را به‌عنوان زبان پیاده‌سازی مفاهیمی که تاکنون یاد گرفته‌ایم بررسی می‌کنیم.

هدف این فصل یادگیری تمام امکانات زبان C نیست. زبان C بسیار گسترده‌تر از مباحثی است که در اینجا بررسی می‌شوند. هدف این است که بتوانیم مفاهیم پایه‌ای برنامه‌سازی را که تاکنون یاد گرفته‌ایم، در شکل واقعی و نحوی زبان C بشناسیم و بنویسیم.

به عبارت دیگر:

تا اینجا یاد گرفتیم «الگوریتم چگونه کار می‌کند»؛ در این فصل یاد می‌گیریم «همان الگوریتم در زبان C چگونه نوشته می‌شود».

۲. زبان C چیست؟

C یک زبان برنامه‌نویسی عمومی و کامپایل‌شونده است که از دهه ۱۹۷۰ به یکی از زبان‌های مهم در آموزش و توسعه نرم‌افزار تبدیل شده است. C در حوزه‌های مختلفی استفاده شده است؛ از جمله:

- برنامه‌نویسی سیستم‌ها؛
- سیستم‌عامل‌ها؛
- نرم‌افزارهای سطح پایین؛
- سیستم‌های نهفته (Embedded Systems)؛

- برنامه‌های مهندسی؛
- و آموزش مفاهیم پایه برنامه‌نویسی.

یکی از ویژگی‌های مهم C این است که بسیاری از مفاهیم پایه برنامه‌نویسی را با ساختاری نسبتاً مستقیم نشان می‌دهد. برای مثال، مفاهیمی مانند:

متغیر

شرط

حلقه

آرایه

تابع

همگی در C شکل مشخص و نسبتاً ساده‌ای دارند.

۳. اولین برنامه در زبان C

یک برنامه بسیار ساده در C می‌تواند چنین باشد:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");

    return 0;
}
```

در نگاه اول ممکن است این برنامه شلوغ به نظر برسد، اما هر قسمت آن معنی مشخصی دارد. بیایید آن را قسمت به قسمت بررسی کنیم.

۳. ساختار کلی برنامه C

برنامه زیر از چند بخش تشکیل شده است:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");

    return 0;
}
```

مهم‌ترین قسمت‌ها عبارت‌اند از:

```
#include <stdio.h>
```

↓

دسترسی به امکانات ورودی/خروجی استاندارد

```
int main(void)
```

↓

تابع اصلی برنامه

```
{
```

```
...
```

```
}
```

↓

بدنه تابع

```
printf( ... )
```

↓

نمایش خروجی

```
return 0;
```

↓

main پایان موفق تابع

در C تقریباً همه دستورهای اجرایی داخل یک تابع قرار می گیرند.

مهم ترین تابع برنامه، تابع:

```
main
```

است.

۳.۱. تابع `main`

اجرای یک برنامه C معمولاً از تابع `main` آغاز می شود.

ساختار متداول آن:

```
int main(void)
{
    // دستورات
}
```

در اینجا:

- `int` نوع مقدار بازگشتی تابع است؛

- `main` نام تابع است؛

- `void` در این شکل نشان می‌دهد تابع آرگومان مستقیمی دریافت نمی‌کند؛

• `{ }` و `{ }` محدوده بدنه تابع را مشخص می‌کنند.

برای مثال:

```
int main(void)
{
    printf("Hello");
    return 0;
}
```

اجرای برنامه از `main` آغاز می‌شود و دستورات داخل آن به ترتیب اجرا می‌شوند.

۳.۲. آکولادها `{ }` و بلوک کد

در C از آکولادهای:

```
{  
}
```

برای مشخص کردن یک بلوک (Block) از دستورات استفاده می‌شود.

برای مثال:

```
if (x > 0)  
{  
    printf("Positive");  
}
```

دستوری که داخل آکولاد قرار دارد، متعلق به بدنه شرط است. همین مفهوم در تابع نیز وجود دارد:

```
int main(void)  
{  
    printf("Hello");  
    return 0;  
}
```

دستورهای بین `{` و `}` بدنه تابع `main` را تشکیل می‌دهند. آکولادها در ساختارهایی مانند موارد زیر بسیار مهم‌اند:

```
if ( ... )
```

```
{  
    ...  
}
```

```
for ( ... )  
{  
    ...  
}
```

```
while ( ... )  
{  
    ...  
}
```

:9

```
int function( ... )  
{  
    ...  
}
```

۳.۴. نقطه‌ویرگول ;

در C بیشتر دستورهای اجرایی با علامت:

```
;
```

تمام می‌شوند. برای مثال:

```
int x = 10;  
x = x + 5;  
printf("%d", x);
```

هر کدام یک دستور هستند. یکی از خطاهای بسیار رایج در ابتدای یادگیری C فراموش کردن `;` است.

مثلاً:

```
int x = 10
```

صحیح نیست.

شکل صحیح:

```
int x = 10;
```

اما نباید تصور کنیم که هر خط در C الزاماً باید با `;` تمام شود. ساختارهایی مانند:

```
if ( ... )  
{  
  
}
```

یا:


```
for ( ... )  
{  
}
```

خودشان به این شکل با **;** تمام نمی‌شوند.

۳.۵. کامنت‌ها در C

Comment یا توضیح بخشی از متن برنامه است که توسط کامپایلر به عنوان دستور اجرایی برنامه در نظر گرفته نمی‌شود. کامنت برای توضیح دادن کد به برنامه‌نویس استفاده می‌شود.

کامنت یک خطی

در C می‌توان از:

```
//
```

استفاده کرد.

مثلاً:

```
int x = 10; // مقدار اولیه
```

کامنت چندخطی

برای توضیحات چندخطی می‌توان از:

```
*/  
  
این یک  
کامنت چندخطی است  
  
/*
```

استفاده کرد. کامنت‌ها برای توضیح منطق برنامه مفید هستند، اما نباید جایگزین نام‌گذاری مناسب و ساختار واضح کد شوند.

۳.۶. متغیرها در C

همان‌طور که در فصل‌های قبل یاد گرفتیم، متغیر محلی برای ذخیره یک مقدار در برنامه است. در C هنگام تعریف متغیر باید نوع داده آن را مشخص کنیم.
ساختار کلی:

```
type variable;
```

برای مثال:

```
int age;
```

در اینجا:

```
int  
↓  
نوع داده
```

age

↓

نام متغیر

می‌توان هنگام تعریف، مقدار اولیه نیز تعیین کرد:

```
int age = 22;
```

۴. انواع داده اصلی در C

C انواع داده مختلفی دارد. مهم‌ترین انواع داده پایه که در مبانی برنامه‌سازی باید بشناسیم عبارت‌اند از:

int

float

double

char

void

و تعدادی نوع و modifier مرتبط مانند:

short

long

long long

signed

unsigned

۴.۱. نوع `int`

نوع:

```
int
```

برای ذخیرهٔ اعداد صحیح استفاده می‌شود.

مثلاً:

```
int age = 22;  
int score = 95;  
int temperature = -5;
```

مقادیر اعشاری برای `int` مناسب نیستند.

مثلاً:

```
int x = 3.14;
```

به این معنی نیست که `x` می‌تواند مقدار دقیق `3.14` را نگهداری کند.

۴.۲. نوع `float`

نوع:

```
float
```

برای ذخیرهٔ اعداد دارای بخش اعشاری استفاده می‌شود.

مثلاً:

```
float temperature = 36.5;  
float price = 19.99;
```

در C معمولاً عدد اعشاری دارای پسوند `f` را می‌توان به صورت صریح `float` در نظر گرفت:

```
float x = 3.14f;
```

۴.۳. نوع `double`

نوع:

```
double
```

نیز برای اعداد اعشاری استفاده می‌شود و معمولاً نسبت به `float` دقت بیشتری دارد.

مثلاً:

```
double pi = 3.141592653589793;
```

در بسیاری از محاسبات عددی معمول، `double` انتخاب رایجی برای اعداد اعشاری است. تفاوت دقیق اندازه و دقت این انواع به پیاده‌سازی و محیط مورد استفاده بستگی دارد، اما در سیستم‌های رایج:

float



دقت کمتر

double



دقت بیشتر

۴.۴. نوع char

نوع:

char

برای نگهداری یک کاراکتر استفاده می‌شود.

مثلاً:

```
char grade = 'A';  
char letter = 'x';
```

در C کاراکتر با تک‌نقل‌قول مشخص می‌شود:

'A'

نه:

```
"A"
```

این دو مفهوم یکسان نیستند.

```
'A'
```

↓

یک character

```
"A"
```

↓

یک string

رشته‌ها را بعداً بررسی خواهیم کرد.

۴.۵. نوع `void`

`void` به معنی «بدون نوع مقدار» است و در چند موقعیت مهم دیده می‌شود.

مثلاً:

```
void printHello(void)
{
    printf("Hello");
}
```

این تابع مقدار بازگشتی ندارد.

همچنین:

```
int main(void)
```

در اینجا `void` مربوط به ورودی‌های تابع `main` است و به این معنی است که این شکل از `main` آرگومان دریافت نمی‌کند.

۴.۶. انواع `long`، `short` و `long long`

C اجازه می‌دهد در کنار انواع صحیح از modifierهایی مانند `short` و `long` استفاده کنیم.

مثلاً:

```
short int x;  
long int y;  
long long int z;
```

یا شکل کوتاه‌تر:

```
short x;  
long y;  
long long z;
```

این انواع برای نمایش دامنه‌های متفاوت اعداد صحیح استفاده می‌شوند.

مثلاً:


```
long long population;
```

می‌تواند برای مقادیر صحیح بسیار بزرگ‌تر از محدوده معمول `int` مناسب باشد. اندازه دقیق این انواع به استاندارد و پیاده‌سازی C و محیط اجرایی بستگی دارد؛ بنابراین نباید بدون مشخص کردن محیط، همیشه یک اندازه بایتی ثابت را به همه سیستم‌ها نسبت داد.

۴.۷. `signed` و `unsigned`

برای انواع صحیح می‌توان مشخص کرد که مقدار اجازه منفی بودن داشته باشد یا نه. مثلاً:

```
signed int x;  
unsigned int y;
```

`unsigned` یعنی متغیر فقط برای مقادیر نامنفی در نظر گرفته می‌شود.

برای مثال:

```
unsigned int age;
```

اگر متغیری قرار است هیچ‌گاه مقدار منفی نداشته باشد، `unsigned` می‌تواند از نظر نوع داده معنای مشخصی ایجاد کند. با این حال، استفاده از `unsigned` باید آگاهانه باشد؛ زیرا رفتار محاسبات مربوط به مقادیر `unsigned` با اعداد صحیح علامت‌دار متفاوت است.

۴.۸. جدول کلی انواع داده

نوع	کاربرد کلی
char	یک کاراکتر
int	عدد صحیح
short	نوع صحیح با حداقل اندازه معمول کوتاه‌تر
long	نوع صحیح با دامنه گسترده‌تر
long long	نوع صحیح با دامنه بزرگ‌تر
float	عدد اعشاری با دقت معمول کمتر
double	عدد اعشاری با دقت معمول بیشتر
void	نشان‌دهنده نبود مقدار/نوع در کاربردهای مشخص

۵. نام‌گذاری متغیرها

برای مثال:

```
int studentAge;
```

نام متغیر باید از قواعد زبان C پیروی کند. نام متغیر می‌تواند شامل حروف، اعداد و `_` باشد، اما نمی‌تواند با عدد شروع شود.

صحیح:

```
age
studentAge
score1
total_sum
```

نادرست:

```
1age
```

همچنین کلمات رزرو شدهٔ زبان C را نمی‌توان به‌عنوان نام عادی متغیر استفاده کرد.

برای مثال:

```
int  
if  
else  
return
```

کلمات کلیدی هستند.

۶. حساس بودن C به حروف بزرگ و کوچک

C یک زبان **Case-Sensitive** است.

یعنی:

```
age
```

:9

```
Age
```

:9

AGE

سه نام متفاوت هستند.

همین موضوع درباره نام توابع نیز صدق می‌کند:

printf

با:

Printf

یکسان نیست.

۷. انتساب در C

در C برای قرار دادن مقدار در یک متغیر از:

=

استفاده می‌کنیم.

مثلاً:

```
int x;
```

```
x = 10;
```

یا:

```
int x = 10;
```

نباید `=` را با `==` اشتباه گرفت.

```
x = 10;
```

یعنی مقدار `10` را به `x` نسبت بده.

اما:

```
x == 10
```

یعنی بررسی کن آیا `x` برابر `10` است یا خیر. این تفاوت یکی از مهم‌ترین نکات زبان C است.

۸. عملگرهای ریاضی

عملگرهای اصلی ریاضی در C عبارت‌اند از:

+	جمع
-	تفریق
*	ضرب
/	تقسیم
%	باقیمانده

مثلاً:

```
int a = 10;
int b = 3;

int sum = a + b;
int difference = a - b;
int product = a * b;
int quotient = a / b;
int remainder = a % b;
```

در اینجا:

```
sum      = 13
difference = 7
product   = 30
quotient  = 3
remainder = 1
```

۸.۱. تقسیم صحیح و اعشاری

یکی از نکات مهم C تفاوت تقسیم صحیح و اعشاری است. اگر هر دو طرف تقسیم از نوع صحیح باشند:

```
int x = 7 / 2;
```

نتیجه:

3

است، نه 3.5.

زیرا تقسیم در اینجا تقسیم صحیح است.

اما:

```
double x = 7.0 / 2.0;
```

نتیجه:

3.5

خواهد بود. بنابراین نوع عملوندها روی نتیجه محاسبه تأثیر می‌گذارد.

۸.۲. عملگر باقیمانده %

عملگر % باقیمانده تقسیم صحیح را محاسبه می‌کند. مثلاً:

```
10 % 3
```

برابر است با:

1

زیرا:

$$10 = 3 \times 3 + 1$$

یکی از کاربردهای بسیار مهم % تشخیص زوج و فرد است:

```
if (n % 2 == 0)
{
    printf("Even");
}
```

اگر باقی‌مانده تقسیم عدد بر 2 صفر باشد، عدد زوج است.

۸.۳. افزایش و کاهش

C عملگرهای مخصوصی برای یک واحد افزایش یا کاهش دادن مقدار دارد:

++

--

مثلاً:

```
x++;
```

معادل افزایش x به اندازه یک است.

9:

```
x--;
```


یک واحد از x کم می‌کند. این عملگرها در حلقه‌ها بسیار پرکاربرد هستند.

۸.۴. پیشوند و پسوند

عملگر $++$ می‌تواند قبل یا بعد از متغیر قرار بگیرد:

```
++x;
```

یا:

```
x++;
```

در یک دستور ساده، هر دو مقدار x را یک واحد افزایش می‌دهند. اما در یک عبارت بزرگ‌تر، زمان انجام افزایش اهمیت پیدا می‌کند.

مثلاً:

```
int x = 5;  
int y = x++;
```

در اینجا مقدار فعلی x ابتدا در y قرار می‌گیرد و سپس x افزایش می‌یابد.

بنابراین:

```
y = 5  
x = 6
```

در مقابل:

```
int x = 5;
int y = ++x;
```

ابتدا `x` افزایش می‌یابد و سپس مقدار جدید استفاده می‌شود:

```
x = 6
y = 6
```

۹. عملگرهای مقایسه‌ای

برای مقایسهٔ مقادیر از عملگرهای زیر استفاده می‌شود:

عملگر	معنی
<code>==</code>	برابر
<code>!=</code>	نا برابر
<code><</code>	بزرگ‌تر
<code>></code>	کوچک‌تر
<code><=</code>	بزرگ‌تر یا مساوی
<code>=></code>	کوچک‌تر یا مساوی

مثلاً:

```
if (x > 10)
{
    printf("Large");
}
```

```
}
```

۱۰. عملگرهای منطقی

عملگرهای منطقی اصلی:

```
&&
```

```
||
```

```
!
```

هستند.

AND

```
&&
```

یعنی هر دو شرط باید درست باشند.

```
if (age ≥ 18 && age ≤ 30)
{
    ...
}
```

OR

```
||
```

یعنی حداقل یکی از شرط‌ها درست باشد.

```
if (x == 0 || y == 0)
{
    ...
}
```

NOT

```
!
```

مقدار منطقی را معکوس می‌کند.

```
if (!found)
{
    ...
}
```

۱۱. مقدار منطقی در C

C به‌طور سنتی مفهوم truth value را بر اساس مقدار عددی بیان می‌کند:

```
0
```

به‌عنوان false در نظر گرفته می‌شود.

مقدار غیرصفر:

هر مقدار غیر از صفر

به عنوان `true` در نظر گرفته می‌شود.

مثلاً:

```
if (x)
{
    printf("True");
}
```

اگر `x` صفر نباشد، شرط برقرار است. در C مدرن می‌توان از نوع `_Bool` و هدر:

```
#include <stdbool.h>
```

و نام‌های:

```
bool
true
false
```

نیز استفاده کرد.

۱۲. تقدم عملگرها

همان‌طور که در ریاضیات ترتیب انجام عملیات اهمیت دارد، در C نیز عملگرها تقدم مشخصی دارند.

برای مثال:

```
int x = 2 + 3 * 4;
```

ابتدا ضرب انجام می‌شود:

$$3 \times 4 = 12$$

و سپس:

$$2 + 12 = 14$$

اگر بخواهیم ترتیب را صریح کنیم، از پرانتز استفاده می‌کنیم:

```
int x = (2 + 3) * 4;
```

که نتیجه:

20

است.

در برنامه‌های پیچیده بهتر است برای واضح‌تر کردن منظور، از پرانتز استفاده کنیم.

۱۳. تبدیل نوع داده (Type Casting)

گاهی لازم است یک مقدار را از یک نوع به نوع دیگر تبدیل کنیم. مثلاً:

```
int a = 7;
int b = 2;
```

عبارت:

```
a / b
```

تقسیم صحیح انجام می‌دهد. اگر بخواهیم نتیجه اعشاری داشته باشیم، می‌توانیم از casting استفاده کنیم:

```
double result = (double)a / b;
```

در اینجا:

```
(double)a
```

یعنی مقدار **a** را برای این عبارت به **double** تبدیل کن.

نتیجه:

```
3.5
```

خواهد بود.

۱۴. ورودی و خروجی در C

یکی از مهم‌ترین بخش‌های برنامه‌نویسی، ارتباط برنامه با کاربر است. به‌طور کلی:

Input



پردازش



Output

در C برای ورودی و خروجی استاندارد معمولاً از کتابخانه:

`stdio.h`

استفاده می‌کنیم.

`stdio` مخفف:

Standard Input/Output

است. برای استفاده از امکانات این کتابخانه می‌نویسیم:

```
#include <stdio.h>
```

۱۴.۱. خروجی با `printf`

تابع:

`printf`

برای چاپ اطلاعات در خروجی استاندارد استفاده می‌شود.

مثلاً:

```
printf("Hello");
```

یا:

```
printf("Hello World!");
```

می‌توان متن را با متغیر ترکیب کرد:

```
int age = 22;  
  
printf("%d", age);
```

خروجی:

```
22
```

۱۴.۲. Format Specifier

وقتی می‌خواهیم مقدار یک متغیر را با `printf` نمایش دهیم، باید به تابع بگوییم نوع داده‌ی موردنظر چیست. این کار با **Format Specifier** انجام می‌شود.

چند مورد مهم:

کاربرد معمول	Specifier
<code>int</code>	<code>%d</code>

کاربرد معمول	Specifier
<code>unsigned int</code>	<code>u%</code>
نمایش عدد اعشاری	<code>f%</code>
<code>char</code>	<code>c%</code>
رشته C	<code>s%</code>
<code>long</code>	<code>ld%</code>
<code>long long</code>	<code>lld%</code>
نمایش عدد صحیح در مبنای ۱۶	<code>x%</code>
نمایش عدد صحیح در مبنای ۸	<code>o%</code>

مثلاً:

```
int age = 22;
printf("%d", age);
```

۱۴.۳. چاپ چند مقدار

می‌توان چند متغیر را در یک `printf` نمایش داد:

```
int age = 22;
float height = 1.85;

printf("Age: %d\n", age);
```

```
printf("Height: %f\n", height);
```

همچنین:

```
printf("Age = %d, Height = %f\n", age, height);
```

ترتیب آرگومان‌ها باید با format specifierها هماهنگ باشد.

۱۴.۴. کاراکترهای ویژه در رشته‌ها

در رشته‌های C از بعضی escape sequenceها استفاده می‌کنیم.

مهم‌ترین آن‌ها:

معنی	Escape
رفتن به خط بعد	n\
Tab	t\
چاپ \	\\
چاپ "	"\
چاپ '	'\
کاراکتر پایان رشته	0\

مثلاً:

```
printf("Hello\nWorld");
```

خروجی:

```
Hello  
World
```

۱۴.۵. ورودی با `scanf`

برای دریافت ورودی از کاربر می‌توان از:

```
scanf
```

استفاده کرد.

مثلاً:

```
int age;  
  
scanf("%d", &age);
```

اگر کاربر عددی وارد کند، مقدار آن در `age` قرار می‌گیرد.

ساختار کلی:

```
scanf("format", &variable);
```

۱۴.۶. چرا در `scanf` از `&` استفاده می‌کنیم؟

در مثال:

```
scanf("%d", &age);
```

علامت:

&

قبل از `age` قرار گرفته است.

برای درک مقدماتی، می‌توان گفت `scanf` باید محل قرار دادن مقدار ورودی را بداند. `age&` آدرس متغیر `age` را در اختیار تابع قرار می‌دهد. در این فصل لازم نیست وارد جزئیات کامل `Pointer`ها شویم؛ اما دانستن این نکته ضروری است:

در `scanf` برای دریافت مقدار و قرار دادن آن داخل یک متغیر معمولی، معمولاً آدرس آن متغیر را با `&` در اختیار `scanf` قرار می‌دهیم.

برای مثال:

```
int x;  
scanf("%d", &x);
```

اما در مورد بعضی انواع داده، مانند آرایه‌های کاراکتری که به‌عنوان رشته استفاده می‌شوند، شکل استفاده متفاوت است.

۱۴.۷. دریافت چند ورودی

می‌توان چند مقدار را نیز با یک `scanf` دریافت کرد:

```
int a, b;

scanf("%d %d", &a, &b);
```

مثلاً اگر کاربر وارد کند:

```
10 20
```

آنگاه:

```
a = 10
b = 20
```

خواهند بود.

۱۴.۸. ورودی اعشاری

برای یک متغیر `float`:

```
float x;

scanf("%f", &x);
```

برای `double` در `scanf` معمولاً از:

```
double x;
```

```
scanf("%lf", &x);
```

استفاده می‌شود. این تفاوت یکی از نکات مهم format specifierها در C است.

۱۴.۹. جدول مهم scanf

نوع داده	Format معمول در scanf
int	d%
unsigned int	u%
float	f%
double	lf%
char	c%
long	ld%
long long	lld%

توجه کنید که format specifier مناسب به نوع متغیر بستگی دارد. استفاده اشتباه از آن‌ها می‌تواند باعث رفتار نادرست برنامه شود.

۱۵. یک برنامه کامل با ورودی و خروجی

اکنون می‌توانیم یک برنامه کامل ساده بنویسیم:

```
#include <stdio.h>
```

```

int main(void)
{
    int a;
    int b;

    printf("Enter two numbers: ");

    scanf("%d %d", &a, &b);

    printf("Sum = %d\n", a + b);

    return 0;
}

```

روند برنامه:

a و b دریافت
 ↓
 محاسبه $a + b$
 ↓
 نمایش نتیجه

مثلاً:

Input:

10 20

Output:

Sum = 30

۱۶. شرط `if`

ساختار شرطی در C:

```
if (condition)
{
    // دستورها
}
```

مثلاً:

```
if (x > 0)
{
    printf("Positive");
}
```

اگر شرط درست باشد، دستورات داخل بلوک اجرا می‌شوند.

۱۶.۱ `if ... else`

اگر بخواهیم برای حالت درست و نادرست دو رفتار متفاوت داشته باشیم:

```
if (condition)
{
    // حالت درست
}
else
{
    // حالت نادرست
}
```

مثلاً:

```
if (x ≥ 0)
{
    printf("Non-negative");
}
else
{
    printf("Negative");
}
```

۱۶.۲. else if

برای چند حالت:

```
if (condition1)
{
    ...
}
else if (condition2)
{
    ...
}
else
{
    ...
}
```

مثلاً:

```
if (score ≥ 90)
{
    printf("A");
}
else if (score ≥ 80)
{
    printf("B");
}
else if (score ≥ 70)
```

```

{
    printf("C");
}
else
{
    printf("D");
}

```

شرط‌ها از بالا به پایین بررسی می‌شوند. به محض اینکه یکی از آن‌ها برقرار باشد، آن بخش اجرا می‌شود و زنجیره همان **if** تمام می‌شود.

۱۶.۳. شرط‌های تو در تو

یک شرط می‌تواند داخل شرط دیگری قرار گیرد:

```

if (x > 0)
{
    if (x % 2 == 0)
    {
        printf("Positive even");
    }
}

```

در اینجا ابتدا مثبت بودن بررسی می‌شود و سپس زوج بودن.

۱۷. switch

در شرایطی که یک متغیر با چند مقدار مشخص مقایسه می‌شود، می‌توان از `switch` استفاده کرد.

ساختار:

```
switch (x)
{
    case 1:
        ...
        break;

    case 2:
        ...
        break;

    default:
        ...
}
```

مثلاً:

```
int day;

scanf("%d", &day);

switch (day)
```

```

{
    case 1:
        printf("Saturday");
        break;

    case 2:
        printf("Sunday");
        break;

    default:
        printf("Invalid");
}

```

۱۷.۱. نقش break در switch

وجود **break** معمولاً باعث می‌شود اجرای **switch** پس از اجرای **case** مربوطه متوقف شود.

مثلاً:

```

switch (x)
{
    case 1:
        printf("One");
        break;
}

```

```

    case 2:
        printf("Two");
        break;
}

```

اگر `x` برابر `1` باشد، پس از چاپ `One` از `switch` خارج می‌شویم. اگر `break` را حذف کنیم، ممکن است اجرای برنامه وارد `case` های بعدی نیز بشود؛ این رفتار را **fall-through** می‌نامند.

۱۸. حلقه `for`

ساختار کلی:

```

for (initialization; condition; update)
{
    // body
}

```

مثلاً:

```

for (int i = 0; i < 5; i++)
{
    printf("%d\n", i);
}

```

مقادیر:

```
0  
1  
2  
3  
4
```

چاپ می‌شوند.

سه قسمت `for` :

```
initialization
```

↓

```
condition
```

↓

```
body
```

↓

```
update
```

↓

```
condition
```

۱۸.۱. تبدیل حلقه‌ شبیه‌ کد به C

اگر در شبه‌ کد داشته باشیم:


```
FOR i = 1 TO 10
    PRINT i
END FOR
```

می‌توان آن را در C به شکل زیر نوشت:

```
for (int i = 1; i ≤ 10; i++)
{
    printf("%d\n", i);
}
```

تفاوت مهم این است که در C باید دقیقاً مشخص کنیم شرط حلقه چه زمانی برقرار است.

۱۸.۲. حلقه `while`

ساختار:

```
while (condition)
{
    // body
}
```

مثلاً:

```
int i = 1;
```

```
while (i ≤ 5)
{
    printf("%d\n", i);
    i++;
}
```

تا زمانی که شرط درست باشد، حلقه ادامه پیدا می‌کند.

۱۸.۳. حلقهٔ `do ... while`

در این ساختار ابتدا بدنه اجرا می‌شود و بعد شرط بررسی می‌شود:

```
do
{
    // body
}
while (condition);
```

مثلاً:

```
int i = 1;

do
{
```

```

        printf("%d\n", i);

        i++;
    }
    while (i ≤ 5);

```

تفاوت مهم:

```

while
    ↓
اول شرط
بعد بدنه

do-while
    ↓
اول بدنه
بعد شرط

```

بنابراین **do-while** بدنه خود را حداقل یک بار اجرا می‌کند.

۱۸.۴. مقایسه سه حلقه

حلقه	شرط چه زمانی بررسی می‌شود؟	حداقل یک بار اجرا؟
for	قبل از هر تکرار	خیر
while	قبل از هر تکرار	خیر

حلقه	شرط چه زمانی بررسی می‌شود؟	حداقل یک بار اجرا؟
do-while	بعد از اجرای بدنه	بله

۱۸.۵ break در حلقه‌ها

break باعث خروج فوری از نزدیک‌ترین حلقه‌ای می‌شود که دستور داخل آن قرار دارد. مثلاً:

```
for (int i = 0; i < 10; i++)
{
    if (i == 5)
    {
        break;
    }

    printf("%d\n", i);
}
```

خروجی:

```
0
1
2
3
4
```

وقتی `i` به 5 برسد، `break` اجرا می‌شود و حلقه تمام می‌شود.

۱۸.۶. `continue`

`continue` با `break` متفاوت است. `continue` حلقه را کاملاً متوقف نمی‌کند؛ بلکه ادامه همان تکرار را کنار می‌گذارد و به مرحله بعدی حلقه می‌رود.

مثلاً:

```
for (int i = 0; i < 5; i++)
{
    if (i == 2)
    {
        continue;
    }

    printf("%d\n", i);
}
```

خروجی:

```
0
1
3
4
```

عدد 2 چاپ نمی‌شود، اما حلقه ادامه پیدا می‌کند.

۱۸.۷. حلقه‌های تو در تو

در C می‌توان یک حلقه را داخل حلقه دیگر قرار داد. مثلاً:

```
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 3; j++)
    {
        printf("%d %d\n", i, j);
    }
}
```

حلقه داخلی برای هر بار اجرای حلقه بیرونی اجرا می‌شود. این ساختار برای کار با آرایه‌های دوبعدی و مسائل جدولی بسیار مهم است.

۱۹. آرایه در C

برای تعریف آرایه‌ای از `int`:

```
int numbers[5];
```

یعنی آرایه‌ای شامل ۵ عنصر از نوع `int`. اندیس‌گذاری در C از صفر شروع می‌شود:

```
numbers[0]
numbers[1]
```

```
numbers[2]
numbers[3]
numbers[4]
```

بنابراین اگر آرایه ۵ عنصر داشته باشد، آخرین اندیس:

4

است.

۱۹.۱. مقداردهی آرایه

می‌توان آرایه را هنگام تعریف مقداردهی کرد:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

در نتیجه:

```
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50
```

همچنین در برخی موارد می‌توان اندازه را از روی مقدارهای اولیه مشخص کرد:

```
int numbers[] = {10, 20, 30, 40, 50};
```

۱۹.۲. پیمایش آرایه

برای دسترسی به همهٔ عناصر یک آرایه معمولاً از حلقه استفاده می‌کنیم:

```
int numbers[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++)
{
    printf("%d\n", numbers[i]);
}
```

ساختار کلی:

```
i = 0
↓
numbers[0]

i = 1
↓
numbers[1]

...
```



```
i = 4
```

↓

```
numbers[4]
```

۱۹.۳. دریافت آرایه از کاربر

مثلاً برای دریافت پنج عدد:

```
int numbers[5];

for (int i = 0; i < 5; i++)
{
    scanf("%d", &numbers[i]);
}
```

در هر مرحله یک عنصر دریافت می‌شود.

۱۹.۴. پیدا کردن بزرگ‌ترین عنصر آرایه

یکی از الگوریتم‌های پایه‌ای:

```
int numbers[5] = {12, 7, 25, 4, 18};
```

```

int max = numbers[0];

for (int i = 1; i < 5; i++)
{
    if (numbers[i] > max)
    {
        max = numbers[i];
    }
}

printf("%d", max);

```

منطق الگوریتم همان چیزی است که قبلاً در شبه‌کد یاد گرفتیم:

```

اولین عنصر = max
↓
بررسی عنصر بعدی
↓
اگر بزرگ‌تر بود:
    max = عنصر
↓
ادامه تا پایان آرایه

```

۱۹۵. جستجوی خطی در C

الگوریتم Linear Search را می‌توان به شکل زیر پیاده‌سازی کرد:

```
int numbers[5] = {10, 20, 30, 40, 50};

int target = 30;
int found = 0;

for (int i = 0; i < 5; i++)
{
    if (numbers[i] == target)
    {
        found = 1;
        break;
    }
}

if (found)
{
    printf("Found");
}
else
{
    printf("Not found");
}
```

در اینجا:

```
found = 0
```

یعنی هنوز عنصر پیدا نشده است.

اگر مقدار موردنظر پیدا شود:

```
found = 1
```

می‌شود.

۲۰. آرایهٔ دوبعدی

آرایهٔ دوبعدی در C:

```
int matrix[3][4];
```

یعنی آرایه‌ای با:

سطر 3

ستون 4

برای دسترسی:

```
matrix[row][column]
```

استفاده می‌کنیم.

مثلاً:

```
matrix[0][0]
```

عنصر سطر اول و ستون اول است.

۲۰.۱. پیمایش آرایه دوبعدی

برای پیمایش کامل:

```
for (int i = 0; i < 3; i++)
{
    for (int j = 0; j < 4; j++)
    {
        printf("%d ", matrix[i][j]);
    }

    printf("\n");
}
```

حلقه بیرونی معمولاً سطرها را کنترل می‌کند و حلقه داخلی ستون‌ها را.

۲۱. رشته‌ها در C

C نوع داده‌ای به نام `string` به معنای یک نوع پایه مستقل مانند `int` ندارد. رشته‌ها معمولاً با آرایه‌ای از `char` نمایش داده می‌شوند.

مثلاً:

```
char name[] = "Ali";
```

در حافظه، رشته به همراه یک کاراکتر پایان‌دهنده به نام **Null Character** ذخیره می‌شود:

```
'A' 'l' 'i' '\0'
```

کاراکتر:

```
'\0'
```

پایان رشته را مشخص می‌کند.

۲۱.۱. تفاوت String و Character

این تفاوت بسیار مهم است. یک کاراکتر:

```
char c = 'A';
```

اما یک رشته:

```
char name[] = "Ali";
```

پس:

'A'

↓

یک character

"Ali"

↓

یک string

۲۱.۲. چاپ رشته

می‌توان رشته را با `%s` چاپ کرد:

```
char name[] = "Ali";
```

```
printf("%s", name);
```

خروجی:

Ali

در این حالت برای `printf` خود نام آرایه کاراکتری به شکل مورد انتظار آن تابع استفاده می‌شود و نیازی به نوشتن `name&` نداریم.

۲۱.۳. دریافت یک کاراکتر

برای دریافت یک کاراکتر می‌توان از `c%` استفاده کرد:

```
char c;  
  
scanf(" %c", &c);
```

فاصله قبل از `c%` در این الگو می‌تواند باعث شود `whitespace`های قبلی، مانند `newline` باقی‌مانده از ورودی قبلی، نادیده گرفته شوند.

۲۱.۴. دریافت رشته

برای ورودی رشته روش‌های مختلفی وجود دارد.

برای مثال:

```
char name[50];  
  
scanf("%49s", name);
```

این روش یک رشته را تا اولین فاصله دریافت می‌کند. برای دریافت خط کامل شامل فاصله، تابع‌هایی مانند:

```
fgets
```

روش مناسب‌تری هستند.

مثلاً:


```
char name[50];
```

```
fgets(name, sizeof(name), stdin);
```

این موضوع اهمیت دارد، زیرا `scanf("%s", ...)` و `fgets(...)` رفتار یکسانی ندارند.

۲۲. توابع در C

توابعی که در فصل قبل به صورت مفهومی یاد گرفتیم، در C نیز وجود دارند. ساختار کلی:

```
return_type function_name(parameters)
{
    // body

    return value;
}
```

مثلاً:

```
int add(int a, int b)
{
    return a + b;
}
```

در اینجا:

`int`

↓

نوع بازگشتی

`add`

↓

نام تابع

`int a, int b`

↓

پارامترها

۲۲.۱. فراخوانی تابع در C

پس از تعریف تابع:

```
int add(int a, int b)
{
    return a + b;
}
```

می‌توانیم آن را فراخوانی کنیم:

```
int result = add(3, 5);
```

در اینجا:

```
a = 3
```

```
b = 5
```

و تابع:

```
8
```

را برمی گرداند.

بنابراین:

```
result = 8
```

۲۲.۲. تابع void

اگر تابع قرار نیست مقدار بازگرداند:

```
void printHello(void)
{
    printf("Hello");
}
```

فراخوانی:

```
printHello();
```

تابع کاری انجام می‌دهد اما مقدار بازگشتی ندارد.

۲۲.۳. پارامترهای تابع

برای مثال:

```
int square(int x)
{
    return x * x;
}
```

در اینجا **x** یک پارامتر است.

فراخوانی:

```
int result = square(5);
```

در این فراخوانی، **5** آرگومان است.

۲۲.۴. چند پارامتر

یک تابع می‌تواند چند پارامتر داشته باشد:

```
int max(int a, int b)
{
    if (a > b)
    {
        return a;
    }
}
```

```

    }

    return b;
}

```

فراخوانی:

```
int result = max(10, 20);
```

۲۲.۵. Prototype یا اعلان تابع

اگر تابع را بعد از `main` تعریف کنیم، معمولاً باید قبل از استفاده، اعلان آن را در اختیار کامپایلر قرار دهیم.

مثلاً:

```

#include <stdio.h>

int add(int a, int b);

int main(void)
{
    int result = add(3, 5);

    printf("%d\n", result);
}

```

```

    return 0;
}

int add(int a, int b)
{
    return a + b;
}

```

خط:

```
int add(int a, int b);
```

یک **Function Prototype** است. این اعلان به کامپایلر می‌گوید تابعی به نام `add` وجود دارد که دو `int` دریافت می‌کند و یک `int` برمی‌گرداند.

۲۲.۶ Definition و Declaration

دو اصطلاح مهم:

Declaration

اعلام می‌کند یک موجودیت وجود دارد و اطلاعات کلی آن چیست.

مثلاً:

```
int add(int a, int b);
```

Definition

پیاده‌سازی واقعی تابع را ارائه می‌کند:

```
int add(int a, int b)
{
    return a + b;
}
```

این تفاوت در برنامه‌های بزرگ اهمیت زیادی پیدا می‌کند.

۲۲.۷ Scope در C

مثلاً:

```
int main(void)
{
    int x = 10;

    printf("%d", x);

    return 0;
}
```

x در محدودهٔ بلوک `main` تعریف شده است. همچنین می‌توان متغیرهای محلی را داخل بلوک‌های دیگر داشت:

```

if (x > 0)
{
    int y = 20;
    printf("%d", y);
}

```

y در همان محدودهٔ بلوک تعریف شده است.

۲۲.۸. متغیر سراسری

اگر متغیری خارج از توابع تعریف شود، می‌تواند در محدودهٔ گسترده‌تری قرار گیرد.

مثلاً:

```

int counter = 0;

int main(void)
{
    counter++;
    return 0;
}

```

این متغیر یک **Global Variable** است. در برنامه‌های بزرگ، استفادهٔ بی‌دلیل از متغیرهای سراسری می‌تواند وابستگی بین قسمت‌های مختلف برنامه را افزایش دهد؛ بنابراین بهتر است داده‌ها تا حد امکان با ساختار مشخص بین توابع منتقل شوند.

۲۲.۹. تابع بازگشتی در C

تابع بازگشتی در C نیز دقیقاً همان مفهوم فصل قبل را دارد. مثلاً فاکتوریل:

```
int factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }

    return n * factorial(n - 1);
}
```

فراخوانی:

```
int result = factorial(5);
```

مراحل:

```
factorial(5)
  ↓
5 × factorial(4)
  ↓
5 × 4 × factorial(3)
  ↓
```

```

5 × 4 × 3 × factorial(2)
    ↓
5 × 4 × 3 × 2 × factorial(1)
    ↓
5 × 4 × 3 × 2 × 1 × factorial(0)
    ↓
factorial(0) = 1

```

و سپس نتایج به عقب برمی گردند.

۳۰. عملگرهای بیتی در C

C به طور مستقیم امکان کار با بیت‌ها را فراهم می‌کند.

عملگرهای مهم:

&	AND
	OR
^	XOR
~	NOT
<<	Left Shift
>>	Right Shift

برای مثال:

```
int a = 5;
```

```
int b = 3;
```

```
int x = a & b;
```

اگر اعداد را باینری در نظر بگیریم:

5 = 0101

3 = 0011

AND

0101

0011

—

0001

بنابراین:

5 & 3 = 1

۳۰.۱ Shift

عملگر:

<<

بیت‌ها را به سمت چپ جابه‌جا می‌کند.

مثلاً:

```
int x = 3 << 1;
```

به‌صورت مفهومی:

0011

↓

0110

که برابر 6 است.

عملگر:

>>

برای جابه‌جایی به سمت راست استفاده می‌شود. رفتار دقیق Shift به نوع داده و به‌خصوص signed یا unsigned بودن آن وابسته است؛ بنابراین برای محاسبات بیتی جدی باید این تفاوت‌ها را در نظر گرفت.

۳۱. const

اگر بخواهیم مقدار یک متغیر پس از مقداردهی تغییر نکند، می‌توانیم از:

```
const
```

استفاده کنیم.

مثلاً:

```
const int DAYS = 7;
```

پس از آن نباید مقدار آن را تغییر دهیم:

```
DAYS = 10;
```

این کار معتبر نیست.

`const` برای مشخص کردن مقادیری که قرار نیست تغییر کنند مفید است.

۳۲. `define#`

یکی دیگر از امکانات آشنای C. پیش‌پردازنده است.

مثلاً:

```
#define PI 3.141592653589793
```

بعد می‌توان نوشت:

```
double area = PI * r * r;
```

`define#` قبل از مرحله اصلی کامپایل توسط **Preprocessor** پردازش می‌شود.

امروزه برای ثابت‌های دارای نوع مشخص، در بسیاری از موقعیت‌ها `const` نیز گزینه مناسبی است.

۳۳. `include#`

قبلاً دیدیم:

```
#include <stdio.h>
```

این دستور به **Preprocessor** می‌گوید محتوای موردنیاز هدر مشخص‌شده را در مرحلهٔ پیش‌پردازش در دسترس قرار دهد.

برای مثال، اعلام و امکانات مربوط به توابع استاندارد ورودی و خروجی در `stdio.h` قرار دارند.

بدون در نظر گرفتن این هدر، استفادهٔ صحیح از توابعی مانند `printf` و `scanf` ممکن نیست.

۳۴. کتابخانه و Header

یک کتابخانه مجموعه‌ای از امکانات آماده است که می‌توان از آن‌ها استفاده کرد.

برای مثال:

```
#include <stdio.h>
```

برای امکانات ورودی و خروجی استاندارد است.

یکی دیگر:

```
#include <math.h>
```

برای بسیاری از توابع ریاضی است.

مثلاً:

sqrt

pow

از توابع شناخته‌شدهٔ ریاضی هستند.

۳۴.۱. تابع sqrt

برای استفاده از sqrt معمولاً:

```
#include <math.h>
```

را اضافه می‌کنیم.

مثلاً:

```
double x = sqrt(25.0);
```

نتیجه:

5

است. در بعضی محیط‌های کامپایل، نحوهٔ لینک کردن کتابخانهٔ ریاضی نیز اهمیت دارد، هرچند این جزئیات به محیط کامپایل مربوط است.

۳۵. الگوریتم تشخیص عدد اول در C

اکنون یکی از الگوریتم‌هایی را که در فصل‌های قبلی داشتیم، به C تبدیل کنیم:

```

#include <stdio.h>
#include <math.h>

int isPrime(int n)
{
    if (n < 2)
    {
        return 0;
    }

    int limit = (int)sqrt(n);

    for (int k = 2; k ≤ limit; k++)
    {
        if (n % k == 0)
        {
            return 0;
        }
    }

    return 1;
}

```

و استفاده:


```

int main(void)
{
    int n;

    scanf("%d", &n);

    if (isPrime(n))
    {
        printf("Prime\n");
    }
    else
    {
        printf("Not prime\n");
    }

    return 0;
}

```

۳۶.. تبدیل شبه کد به C

یکی از مهم‌ترین مهارت‌ها در برنامه‌سازی این است که بتوانیم یک الگوریتم را از شکل انتزاعی به کد واقعی تبدیل کنیم.

مثلاً شبه کد:

```
READ n

IF n % 2 == 0 THEN
    PRINT "Even"
ELSE
    PRINT "Odd"
END IF
```

تبدیل به C:

```
#include <stdio.h>

int main(void)
{
    int n;

    scanf("%d", &n);

    if (n % 2 == 0)
    {
        printf("Even\n");
    }
    else
    {
```

```

        printf("Odd\n");
    }

    return 0;
}

```

ساختار منطقی تغییر نکرده است. فقط نحوه بیان آن از شبه‌کد به **syntax** زبان **C** تبدیل شده است.

۳۶.۱. تبدیل حلقه به **C**

شبه‌کد:

```

FOR i = 1 TO 10
    PRINT i
END FOR

```

C:

```

for (int i = 1; i ≤ 10; i++)
{
    printf("%d\n", i);
}

```

یا:

```

WHILE  $i \leq 10$ 
    PRINT  $i$ 
     $i = i + 1$ 
END WHILE

```

در C:

```

while ( $i \leq 10$ )
{
    printf("%d\n",  $i$ );
     $i++$ ;
}

```

۳۶.۲. تبدیل آرایه به C

شبه کد:

```

FOR  $i = 0$  TO  $n-1$ 
    READ  $A[i]$ 
END FOR

```

در C:

```

for ( $\text{int } i = 0; i < n; i++$ )
{

```

```
scanf("%d", &A[i]);  
}
```

و برای چاپ:

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", A[i]);  
}
```

۳۷. یک برنامه کامل تر

اکنون برنامه‌ای می‌نویسیم که:

۱. تعداد عناصر را دریافت کند؛
۲. عناصر آرایه را دریافت کند؛
۳. مجموع را محاسبه کند؛
۴. بزرگ‌ترین مقدار را پیدا کند؛
۵. نتیجه را نمایش دهد.

```
#include <stdio.h>  
  
int main(void)  
{  
    int n;
```

```
printf("Enter number of elements: ");  
scanf("%d", &n);  
  
int numbers[n];  
  
for (int i = 0; i < n; i++)  
{  
    scanf("%d", &numbers[i]);  
}  
  
int sum = 0;  
int max = numbers[0];  
  
for (int i = 0; i < n; i++)  
{  
    sum += numbers[i];  
  
    if (numbers[i] > max)  
    {  
        max = numbers[i];  
    }  
}
```

```

printf("Sum = %d\n", sum);
printf("Maximum = %d\n", max);

return 0;
}

```

۳۷.۲. بهتر کردن برنامه با تابع

می‌توانیم برنامه قبلی را به چند تابع تقسیم کنیم.

مثلاً تابع محاسبهٔ مجموع:

```

int sumArray(int A[], int n)
{
    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += A[i];
    }

    return sum;
}

```

و تابع پیدا کردن بیشینه:

```

int maxArray(int A[], int n)
{
    int max = A[0];

    for (int i = 1; i < n; i++)
    {
        if (A[i] > max)
        {
            max = A[i];
        }
    }

    return max;
}

```

سپس برنامه اصلی می‌تواند ساده‌تر شود:

```

int sum = sumArray(numbers, n);
int max = maxArray(numbers, n);

```

این دقیقاً همان مفهومی است که درباره **Abstraction** و تقسیم برنامه به وظایف کوچک‌تر یاد گرفتیم.

۳۸. یک برنامه نهایی ترکیبی

برنامه زیر چند مفهوم اصلی این فصل را با هم ترکیب می‌کند:


```

#include <stdio.h>

int isPrime(int n)
{
    if (n < 2)
    {
        return 0;
    }

    for (int i = 2; i * i ≤ n; i++)
    {
        if (n % i == 0)
        {
            return 0;
        }
    }

    return 1;
}

int main(void)
{
    int n;

```

```

printf("Enter n: ");
scanf("%d", &n);

for (int i = 2; i ≤ n; i++)
{
    if (isPrime(i))
    {
        printf("%d ", i);
    }
}

return 0;
}

```

اگر کاربر وارد کند:

20

خروجی:

2 3 5 7 11 13 17 19

در این برنامه:

- `stdio.h` برای ورودی و خروجی استفاده شده است؛

- `main` نقطه شروع برنامه است؛

- `n` یک متغیر صحیح است؛
- `scanf` ورودی را دریافت می‌کند؛
- `for` تکرار را انجام می‌دهد؛
- `if` تصمیم‌گیری می‌کند؛
- `%` برای بررسی بخش‌پذیری استفاده شده است؛
- `isPrime` یک تابع است؛
- `return` نتیجه تابع را برمی‌گرداند؛
- `printf` خروجی را نمایش می‌دهد.

این دقیقاً همان الگوریتمی است که قبلاً در قالب شبه‌کد بررسی کردیم، اما اکنون به یک برنامه واقعی C تبدیل شده است.

۳۹. کامپایل برنامه C

برنامه‌ای که با C نوشته‌ایم مستقیماً توسط پردازنده اجرا نمی‌شود. در یک روند معمول، کد منبع توسط **Compiler** پردازش می‌شود.

به صورت ساده:

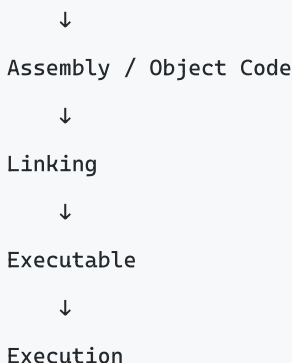
Source Code



Preprocessing



Compilation



مثلاً فایل:

```
program.c
```

کد منبع C است. کامپایلر آن را به شکل‌های میانی و در نهایت برنامه قابل اجرا تبدیل می‌کند.

۴۰. خطاهای Syntax و خطاهای منطقی

همه خطاها یکسان نیستند.

خطای نحوی (Syntax Error)

مثلاً:

```
int x = 10
```

اگر `;` لازم باشد و وجود نداشته باشد، کامپایلر می‌تواند خطا بدهد.

خطای منطقی (Logic Error)

ممکن است برنامه کاملاً صحیح کامپایل شود اما جواب اشتباه بدهد.

مثلاً:

```
int area = length + width;
```

اگر هدف محاسبه مساحت مستطیل باشد، دستور از نظر **syntax** صحیح است، اما الگوریتم اشتباه است.

جواب صحیح:

```
int area = length * width;
```

بنابراین:

کامپایل شدن برنامه به معنی درست بودن منطق آن نیست.

۴۱. تفاوت الگوریتم و زبان C

یک الگوریتم ممکن است به شکل زیر نوشته شود:

```
READ n
```

```
IF n > 0 THEN
```

```
    PRINT "Positive"
```

```
ELSE
    PRINT "Non-positive"
END IF
```

اما زبان C همان مفهوم را به شکل زیر بیان می‌کند:

```
int n;

scanf("%d", &n);

if (n > 0)
{
    printf("Positive\n");
}
else
{
    printf("Non-positive\n");
}
```

الگوریتم مستقل از زبان است. C فقط یکی از روش‌های بیان و اجرای آن الگوریتم است. بنابراین باید این دو مهارت را از هم جدا کنیم:

حل مسئله

↓

طراحی الگوریتم



پیاده‌سازی با زبان برنامه‌نویسی

اگر الگوریتم اشتباه باشد، استفاده از **syntax** صحیح C آن را درست نمی‌کند.

۴۱. جدول تطبیق شبه‌کد و C

مفهوم	شبه‌کد	C
تعریف متغیر	<code>x = 10</code>	<code>int x = 10</code>
ورودی	<code>READ x</code>	<code>scanf("%d", &x)</code>
خروجی	<code>PRINT x</code>	<code>printf("%d", x)</code>
شرط	<code>IF</code>	<code>if</code>
در غیر این صورت	<code>ELSE</code>	<code>else</code>
چند شرط	<code>ELSE IF</code>	<code>else if</code>
حلقه شمارشی	<code>FOR</code>	<code>for</code>
حلقه شرطی	<code>WHILE</code>	<code>while</code>
حلقه حداقل یک‌بار	<code>DO WHILE</code>	<code>do ... while</code>
توقف حلقه	<code>BREAK</code>	<code>break</code>
رد کردن تکرار	<code>CONTINUE</code>	<code>continue</code>
تابع	<code>FUNCTION</code>	<code>return_type function()</code>
بازگرداندن مقدار	<code>RETURN</code>	<code>return</code>
آرایه	<code>A[i]</code>	<code>A[i]</code>

مفهوم	شبه‌کد	C
آرایهٔ دوبعدی	<code>A[i][j]</code>	<code>A[i][j]</code>
مقایسه	<code>=</code> یا <code>==</code>	<code>==</code>
انتساب	<code>=</code>	<code>=</code>
باقیمانده	<code>%</code>	<code>%</code>

نکتهٔ مهم این است که بعضی شبه‌کدها از نمادگذاری‌های متفاوتی استفاده می‌کنند؛ بنابراین جدول بالا یک تطبیق مفهومی است، نه یک قانون جهانی برای تمام انواع شبه‌کد.

۴۲. برنامهٔ کامل از ابتدا تا انتها

در پایان، برنامه‌ای می‌نویسیم که بسیاری از مفاهیم این کتاب را در کنار هم قرار دهد.
هدف برنامه:

۱. دریافت تعداد اعداد؛
۲. دریافت آرایه؛
۳. محاسبهٔ مجموع؛
۴. پیدا کردن بزرگ‌ترین مقدار؛
۵. بررسی اعداد اول؛
۶. نمایش نتایج.

```
#include <stdio.h>

int isPrime(int n)
{
```



```

    if (n < 2)
    {
        return 0;
    }

    for (int i = 2; i * i ≤ n; i++)
    {
        if (n % i == 0)
        {
            return 0;
        }
    }

    return 1;
}

int sumArray(int A[], int n)
{
    int sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += A[i];
    }
}

```

```

        return sum;
    }

    int maxArray(int A[], int n)
    {
        int max = A[0];

        for (int i = 1; i < n; i++)
        {
            if (A[i] > max)
            {
                max = A[i];
            }
        }

        return max;
    }

    int main(void)
    {
        int n;

        printf("Enter number of elements: ");

```

```

scanf("%d", &n);

int A[n];

printf("Enter elements:\n");

for (int i = 0; i < n; i++)
{
    scanf("%d", &A[i]);
}

int sum = sumArray(A, n);
int max = maxArray(A, n);

printf("Sum = %d\n", sum);
printf("Maximum = %d\n", max);

printf("Prime numbers:\n");

for (int i = 0; i < n; i++)
{
    if (isPrime(A[i]))
    {
        printf("%d ", A[i]);
    }
}

```

```

    }

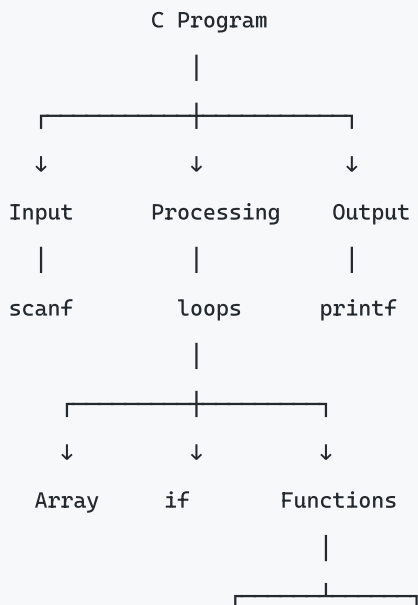
}

printf("\n");

return 0;
}

```

این برنامه نمونه خوبی برای جمع‌بندی مباحث است، زیرا چند مفهوم مختلف را هم‌زمان به کار می‌گیرد:



↓ ↓
 isPrime sumArray
 |
 return

۴۳. از کد تا الگوریتم

در این کتاب، هدف فقط حفظ کردن syntax زبان C نیست. برای مثال، اگر برنامه زیر را ببینیم:

```
for (int i = 0; i < n; i++)
{
    if (A[i] > max)
    {
        max = A[i];
    }
}
```

باید بتوانیم از روی آن منطق الگوریتم را تشخیص دهیم:

برای هر عنصر آرایه

بزرگتر بود max اگر عنصر از

را برابر آن عنصر قرار بده max

و برعکس، اگر الگوریتم زیر را ببینیم:

از 0 تا $n-1$ برای i :

بزرگتر بود $\max$ از $A[i]$ اگر

```
max = A[i]
```

باید بتوانیم آن را به C تبدیل کنیم. این توانایی بسیار مهم‌تر از حفظ کردن چند خط کد است.

۴۴. آنچه باید از زبان C در مبانی برنامه‌سازی بدانیم

در پایان این فصل، انتظار می‌رود ساختار و کاربرد مفاهیم زیر را بشناسیم:

ساختار پایه

```
#include <stdio.h>

int main(void)
{
    return 0;
}
```

متغیرها

```
int x;
float y;
```

```
double z;  
char c;
```

مقداردهی

```
int x = 10;
```

ورودی

```
scanf("%d", &x);
```

خروجی

```
printf("%d", x);
```

شرط

```
if (x > 0)  
{  
    ...  
}  
else  
{  
    ...  
}
```

```
}
```

چند شرط

```
if ( ... )  
{  
    ...  
}  
else if ( ... )  
{  
    ...  
}  
else  
{  
    ...  
}
```

انتخاب چندحالت

```
switch (x)  
{  
    case 1:  
        ...  
        break;  
}
```



```
    default:
        ...
}
```

for حلقة

```
for (int i = 0; i < n; i++)
{
    ...
}
```

while حلقة

```
while (condition)
{
    ...
}
```

do-while حلقة

```
do
{
    ...
}
```

```
}  
while (condition);
```

توقف و رد کردن تکرار

```
break;  
continue;
```

آرایه

```
int A[10];
```

آرایهٔ دوبعدی

```
int A[10][20];
```

تابع

```
int add(int a, int b)  
{  
    return a + b;  
}
```

تابع بدون مقدار بازگشتی

```
void printHello(void)
{
    printf("Hello");
}
```

تابع بازگشتی

```
int factorial(int n)
{
    if (n == 0)
    {
        return 1;
    }

    return n * factorial(n - 1);
}
```

۴۵. برگهٔ تقلب زبان C

مفهوم	Syntax در C
عدد صحیح	<code>;int x</code>
عدد اعشاری	<code>;double x</code>
کاراکتر	<code>;char c</code>
ثابت	<code>;const int x = 10</code>

مفهوم	Syntax در C
مقداردهی	<code>x = 10;</code>
جمع	<code>a + b</code>
تفریق	<code>a - b</code>
ضرب	<code>a * b</code>
تقسیم	<code>a / b</code>
باقیمانده	<code>a % b</code>
افزایش	<code>++x</code>
کاهش	<code>--x</code>
برابر	<code>==</code>
نابرابر	<code>!=</code>
بزرگ‌تر	<code><</code>
کوچک‌تر	<code>></code>
بزرگ‌تر یا مساوی	<code><=</code>
کوچک‌تر یا مساوی	<code>>=</code>
AND	<code>&&</code>
OR	<code> </code>
NOT	<code>!</code>
Bitwise AND	<code>&</code>
Bitwise OR	<code> </code>

مفهوم	Syntax در C
Bitwise XOR	<code>^</code>
Left Shift	<code>>></code>
Right Shift	<code><<</code>
شرط	<code>if</code>
در غیر این صورت	<code>else</code>
چند شرط	<code>else if</code>
انتخاب چندحالتی	<code>switch</code>
خروج از حلقه	<code>break</code>
رد کردن تکرار	<code>continue</code>
حلقه شمارشی	<code>for</code>
حلقه شرطی	<code>while</code>
حلقه حداقل یک بار	<code>do ... while</code>
آرایه	<code>;int A[10]</code>
عنصر آرایه	<code>A[i]</code>
آرایه دوبعدی	<code>A[i][j]</code>
تابع	<code>(...)int f</code>
بازگرداندن مقدار	<code>return</code>
تابع بدون مقدار	<code>(...)void f</code>
ورودی صحیح	<code>scanf("%d", &x)</code>

مفهوم	C در Syntax
خروجی صحیح	<code>printf("%d", x)</code>
ورودی <code>float</code>	<code>scanf("%f", &x)</code>
ورودی <code>double</code>	<code>scanf("%lf", &x)</code>
خروجی اعشاری	<code>printf("%f", x)</code>
کاراکتر	<code>c%</code>
رشته	<code>s%</code>
هدر ورودی/خروجی	<code><include <stdio.h#</code>
تعریف ثابت پیش پردازنده	<code>define#</code>
کامنت یک خطی	<code>//</code>
کامنت چندخطی	<code>/* ... */</code>

۴۶. جمع بندی نهایی فصل

در این فصل، زبان **C** را نه به عنوان مجموعه‌ای از دستورهای جداگانه، بلکه به عنوان ابزاری برای پیاده‌سازی الگوریتم‌ها و حل مسئله بررسی کردیم. ابتدا با ساختار پایه یک برنامه **C** و عناصر اصلی آن مانند `main` و کتابخانه‌ها آشنا شدیم و سپس به ترتیب مفاهیمی مانند متغیرها، انواع داده، عملگرها، ورودی و خروجی، شرطها، حلقه‌ها، آرایه‌ها، رشته‌ها، توابع، **Scope** و **Recursion** را بررسی کردیم. در ادامه دیدیم که الگوریتم‌هایی که پیش تر با شبهه‌کد طراحی می‌کردیم، می‌توانند از مسیر الگوریتم → شبهه‌کد → زبان **C** → کامپایل → برنامه‌اجرائی به برنامه‌های واقعی تبدیل شوند. همچنین به این نکته مهم رسیدیم که **Syntax** هدف نهایی برنامه‌نویسی نیست؛ نشانه‌هایی مانند `;`، `{ }`، `()`، `[ ]`، `&&` و `||` تنها ابزارهایی برای بیان مفاهیم برنامه‌نویسی هستند و هدف اصلی،

توانایی تبدیل یک مسئله به تحلیل مسئله، طراحی الگوریتم، انتخاب ساختار مناسب، پیاده‌سازی، آزمایش و رفع خطا است. بنابراین اگر مسئله نیاز به تصمیم‌گیری داشته باشد از شرط، برای تکرار از حلقه، برای نگهداری مجموعه‌ای از داده‌ها از آرایه، برای انجام یک وظیفه مستقل و قابل استفاده مجدد از تابع و در مسائل دارای ساختار مناسب از **Recursion** استفاده می‌کنیم. در نهایت، دیدیم که ارتباط میان حل مسئله، الگوریتم، ساختار برنامه و زبان C مهم‌تر از حفظ کردن دستورهای زبان است؛ به‌طوری که در پایان این فصل، دانشجو دیگر فقط مجموعه‌ای از Syntax‌های C را نمی‌شناسد، بلکه می‌داند هر یک از این ابزارها برای پیاده‌سازی کدام مفهوم الگوریتمی به کار می‌روند. بنابراین C در این مرحله هدف نیست؛ ابزار است و مبانی برنامه‌سازی زمانی معنا پیدا می‌کند که از حفظ کردن Syntax فراتر رفته و به حل مسئله با کامپیوتر برسیم.

